

Universal Serial Bus (USB)

USB Overview

Overview

Specifications and Features

Overview

USB Device Configuration

Overview

USB Device Programming Guide

Overview

USB Device Classes

Overview

CDC ACM Class

Overview

HID Class

Overview

MSC SCSI Class

Overview

Vendor Class

Overview

USB API

API Documentation

USB Device API

USB Device ACM API

sl_usbd_cdc_acm_line_coding_t

baudrate

parity

stop_bits

data_bits

sl_usbd_cdc_acm_callbacks_t

enable

disable

line_control_changed

line_coding_changed

sl_usbd_cdc_acm_init

sl_usbd_cdc_acm_create_instance

sl_usbd_cdc_acm_add_to_configuration

sl_usbd_cdc_acm_is_enabled

sl_usbd_cdc_acm_read

sl_usbd_cdc_acm_read_async

sl_usbd_cdc_acm_write
sl_usbd_cdc_acm_write_async
sl_usbd_cdc_acm_get_line_control_state
sl_usbd_cdc_acm_get_line_coding
sl_usbd_cdc_acm_set_line_coding
sl_usbd_cdc_acm_set_line_state_event
sl_usbd_cdc_acm_clear_line_state_event
SL_USBD_CDC_ACM_NBR_NONE
SL_USBD_CDC_ACM_PARITY_NONE
SL_USBD_CDC_ACM_PARITY_ODD
SL_USBD_CDC_ACM_PARITY_EVEN
SL_USBD_CDC_ACM_PARITY_MARK
SL_USBD_CDC_ACM_PARITY_SPACE
SL_USBD_CDC_ACM_STOP_BIT_1
SL_USBD_CDC_ACM_STOP_BIT_1_5
SL_USBD_CDC_ACM_STOP_BIT_2
SL_USBD_CDC_ACM_CTRL_BREAK
SL_USBD_CDC_ACM_CTRL_RTS
SL_USBD_CDC_ACM_CTRL_DTR
SL_USBD_CDC_ACM_STATE_DCD
SL_USBD_CDC_ACM_STATE_DSR
SL_USBD_CDC_ACM_STATE_BREAK
SL_USBD_CDC_ACM_STATE_RING
SL_USBD_CDC_ACM_STATE_FRAMING
SL_USBD_CDC_ACM_STATE_PARITY
SL_USBD_CDC_ACM_STATE_OVERUN
SL_USBD_CDC_ACM_CALL_MGMT_DEV
SL_USBD_CDC_ACM_CALL_MGMT_DATA_CCI_DCI
SL_USBD_CDC_ACM_CALL_MGMT_DATA_OVER_DCI

USB Device CDC API

sl_usbd_cdc_subclass_driver_t
 enable
 disable
 mgmt_req
 notify_cmpl
 fnct_descriptor
 fnct_get_descriptor_size
sl_usbd_cdc_async_function_t
sl_usbd_cdc_init
sl_usbd_cdc_create_instance
sl_usbd_cdc_add_to_configuration
sl_usbd_cdc_is_enabled
sl_usbd_cdc_add_data_interface
sl_usbd_cdc_read_data

sl_usbd_cdc_read_data_async
sl_usbd_cdc_write_data
sl_usbd_cdc_write_data_async
sl_usbd_cdc_notify_host
SL_USBD_CDC_NBR_NONE
SL_USBD_CDC_DATA_IF_NBR_NONE
SL_USBD_CDC_SUBCLASS_RSVD
SL_USBD_CDC_SUBCLASS_DLCM
SL_USBD_CDC_SUBCLASS_ACM
SL_USBD_CDC_SUBCLASS_TCM
SL_USBD_CDC_SUBCLASS_MCCM
SL_USBD_CDC_SUBCLASS_CAPICM
SL_USBD_CDC_SUBCLASS_ENCM
SL_USBD_CDC_SUBCLASS_ATM
SL_USBD_CDC_SUBCLASS_WHCM
SL_USBD_CDC_SUBCLASS_DEV_MGMT
SL_USBD_CDC_SUBCLASS_MDLM
SL_USBD_CDC_SUBCLASS_OBEX
SL_USBD_CDC_SUBCLASS_EEM
SL_USBD_CDC_SUBCLASS_NCM
SL_USBD_CDC_SUBCLASS_VENDOR
SL_USBD_CDC_COMM_PROTOCOL_NONE
SL_USBD_CDC_COMM_PROTOCOL_AT_V250
SL_USBD_CDC_COMM_PROTOCOL_AT_PCCA_101
SL_USBD_CDC_COMM_PROTOCOL_AT_PCCA_101_ANNEX
SL_USBD_CDC_COMM_PROTOCOL_AT_GSM_7_07
SL_USBD_CDC_COMM_PROTOCOL_AT_3GPP_27_07
SL_USBD_CDC_COMM_PROTOCOL_AT_TIA_CDMA
SL_USBD_CDC_COMM_PROTOCOL_EEM
SL_USBD_CDC_COMM_PROTOCOL_EXT
SL_USBD_CDC_COMM_PROTOCOL_VENDOR
SL_USBD_CDC_DATA_PROTOCOL_NONE
SL_USBD_CDC_DATA_PROTOCOL_NTB
SL_USBD_CDC_DATA_PROTOCOL_PHY
SL_USBD_CDC_DATA_PROTOCOL_HDLC
SL_USBD_CDC_DATA_PROTOCOL_TRANS
SL_USBD_CDC_DATA_PROTOCOL_Q921M
SL_USBD_CDC_DATA_PROTOCOL_Q921
SL_USBD_CDC_DATA_PROTOCOL_Q921TM
SL_USBD_CDC_DATA_PROTOCOL_COMPRES
SL_USBD_CDC_DATA_PROTOCOL_Q9131
SL_USBD_CDC_DATA_PROTOCOL_V24
SL_USBD_CDC_DATA_PROTOCOL_CAPI

SL_USBD_CDC_DATA_PROTOCOL_HOST
SL_USBD_CDC_DATA_PROTOCOL_CDC
SL_USBD_CDC_DATA_PROTOCOL_VENDOR
SL_USBD_CDC_REQ_SEND_ENCAP_COMM
SL_USBD_CDC_REQ_GET_ENCAP_RESP
SL_USBD_CDC_REQ_SET_COMM_FEATURE
SL_USBD_CDC_REQ_GET_COMM_FEATURE
SL_USBD_CDC_REQ_CLR_COMM_FEATURE
SL_USBD_CDC_REQ_SET_AUX_LINE_STATE
SL_USBD_CDC_REQ_SET_HOOK_STATE
SL_USBD_CDC_REQ_PULSE_SETUP
SL_USBD_CDC_REQ_SEND_PULSE
SL_USBD_CDC_REQ_SET_PULSE_TIME
SL_USBD_CDC_REQ_RING_AUX_JACK
SL_USBD_CDC_REQ_SET_LINE_CODING
SL_USBD_CDC_REQ_GET_LINE_CODING
SL_USBD_CDC_REQ_SET_CTRL_LINE_STATE
SL_USBD_CDC_REQ_SEND_BREAK
SL_USBD_CDC_REQ_SET_RINGER_PARAMS
SL_USBD_CDC_REQ_GET_RINGER_PARAMS
SL_USBD_CDC_REQ_SET_OPERATION_PARAMS
SL_USBD_CDC_REQ_GET_OPERATION_PARAMS
SL_USBD_CDC_REQ_SET_LINE_PARAMS
SL_USBD_CDC_REQ_GET_LINE_PARAMS
SL_USBD_CDC_REQ_DIAL_DIGITS
SL_USBD_CDC_REQ_SET_UNIT_PARAM
SL_USBD_CDC_REQ_GET_UNIT_PARAM
SL_USBD_CDC_REQ_CLR_UNUT_PARAM
SL_USBD_CDC_REQ_GET_PROFILE
SL_USBD_CDC_REQ_SET_ETHER_MULTI_FILTER
SL_USBD_CDC_REQ_SET_ETHER_PWR_MGT_FILTER
SL_USBD_CDC_REQ_GET_ETHER_PWR_MGT_FILTER
SL_USBD_CDC_REQ_SET_ETHER_PKT_FILTER
SL_USBD_CDC_REQ_GET_ETHER_STAT
SL_USBD_CDC_REQ_SET_ATM_DATA_FMT
SL_USBD_CDC_REQ_GET_ATM_DATA_FMT
SL_USBD_CDC_REQ_GET_ATM_DEV_STAT
SL_USBD_CDC_REQ_SET_ATM_DFLT_VC
SL_USBD_CDC_REQ_GET_ATM_VC_STAT
SL_USBD_CDC_REQ_GET_NTB_PARAM2
SL_USBD_CDC_REQ_GET_NET_ADDR
SL_USBD_CDC_REQ_SET_NET_ADDR
SL_USBD_CDC_REQ_GET_NTB_FMT

SL_USBD_CDC_REQ_SET_NTBFMT
SL_USBD_CDC_REQ_GET_NTBFINPUT_SIZE
SL_USBD_CDC_REQ_SET_NTBFINPUT_SIZE
SL_USBD_CDC_REQ_GET_MAX_DATAGRAM_SIZE
SL_USBD_CDC_REQ_SET_MAX_DATAGRAM_SIZE
SL_USBD_CDC_REQ_GET_CRC_MODE
SL_USBD_CDC_REQ_SET_CRC_MODE
SL_USBD_CDC_DESC_TYPE_CS_IF
SL_USBD_CDC_DESC_TYPE_CS_EP
SL_USBD_CDC_DESC_SUBTYPE_HEADER
SL_USBD_CDC_DESC_SUBTYPE_CALL_MGMT
SL_USBD_CDC_DESC_SUBTYPE_ACM
SL_USBD_CDC_DESC_SUBTYPE_DLM
SL_USBD_CDC_DESC_SUBTYPE_TEL_RINGER
SL_USBD_CDC_DESC_SUBTYPE_TEL_CALL_LINE
SL_USBD_CDC_DESC_SUBTYPE_UNION
SL_USBD_CDC_DESC_SUBTYPE_COUNTRY_SEL
SL_USBD_CDC_DESC_SUBTYPE_TEL_OPERA
SL_USBD_CDC_DESC_SUBTYPE_USB_TERM
SL_USBD_CDC_DESC_SUBTYPE_NET_CH_TERM
SL_USBD_CDC_DESC_SUBTYPE_PROTOCOL_UNIT
SL_USBD_CDC_DESC_SUBTYPE_EXT_UNIT
SL_USBD_CDC_DESC_SUBTYPE_MULTI_CH_MGMT
SL_USBD_CDC_DESC_SUBTYPE_CAPI_CTRL
SL_USBD_CDC_DESC_SUBTYPE_ETHER_NET
SL_USBD_CDC_DESC_SUBTYPE_ATM_NET
SL_USBD_CDC_DESC_SUBTYPE_WIRELESS_CTRL
SL_USBD_CDC_DESC_SUBTYPE_MOBILE
SL_USBD_CDC_DESC_SUBTYPE_DETAIL
SL_USBD_CDC_DESC_SUBTYPE_DEV_MGMT
SL_USBD_CDC_DESC_SUBTYPE_OBEX
SL_USBD_CDC_DESC_SUBTYPE_CMD_SET
SL_USBD_CDC_DESC_SUBTYPE_CMD_SET_DETAIL
SL_USBD_CDC_DESC_SUBTYPE_TCM
SL_USBD_CDC_DESC_SUBTYPE_OBEX_SERV
SL_USBD_CDC_DESC_SUBTYPE_NCM
SL_USBD_CDC_DESC_SUBTYPE_VENDOR
SL_USBD_CDC_NOTIFICATION_HEADER

USB Device Core API

sl_usbd_device_config_t
 vendor_id
 product_id
 device_bcd
 manufacturer_str_ptr

- product_str_ptr
- serial_nbr_str_ptr
- lang_id
- sl_usbd_setup_req_t
 - bmRequestType
 - bRequest
 - wValue
 - wIndex
 - wLength
- sl_usbd_class_driver_t
 - enable
 - disable
 - alt_settings_update
 - endpoint_state_update
 - interface_descriptor
 - interface_get_descriptor_size
 - endpoint_descriptor
 - endpoint_get_descriptor_size
 - interface_req
 - class_req
 - vendor_req
- sl_usbd_device_speed_t
- sl_usbd_device_state_t
- sl_usbd_bus_event_t
- sl_usbd_config_event_t
- sl_usbd_async_function_t
- sl_usbd_core_init
- sl_usbd_core_start_device
- sl_usbd_core_stop_device
- sl_usbd_core_get_device_state
- sl_usbd_core_get_device_speed
- sl_usbd_core_set_device_self_power
- sl_usbd_core_get_device_configuration
- sl_usbd_core_get_device_frame_number
- sl_usbd_core_add_configuration
- sl_usbd_core_add_interface
- sl_usbd_core_add_alt_interface
- sl_usbd_core_add_interface_group
- sl_usbd_core_get_device_descriptor
- sl_usbd_core_get_configuration_descriptor
- sl_usbd_core_get_string_descriptor
- sl_usbd_core_add_string
- sl_usbd_core_get_string_index

sl_usbd_core_write_08b_to_descriptor_buf
sl_usbd_core_write_16b_to_descriptor_buf
sl_usbd_core_write_24b_to_descriptor_buf
sl_usbd_core_write_32b_to_descriptor_buf
sl_usbd_core_write_buf_to_descriptor_buf
sl_usbd_core_write_control_sync
sl_usbd_core_read_control_sync
sl_usbd_core_add_bulk_endpoint
sl_usbd_core_read_bulk_sync
sl_usbd_core_read_bulk_async
sl_usbd_core_write_bulk_sync
sl_usbd_core_write_bulk_async
sl_usbd_core_add_interrupt_endpoint
sl_usbd_core_read_interrupt_sync
sl_usbd_core_read_interrupt_async
sl_usbd_core_write_interrupt_sync
sl_usbd_core_write_interrupt_async
sl_usbd_core_endpoint_write_zlp
sl_usbd_core_endpoint_read_zlp
sl_usbd_core_abort_endpoint
sl_usbd_core_endpoint_read_complete
sl_usbd_core_endpoint_write_complete
sl_usbd_core_endpoint_write_complete_extended
sl_usbd_core_stall_endpoint
sl_usbd_core_is_endpoint_stalled
sl_usbd_core_get_max_endpoint_packet_size
sl_usbd_core_get_max_phy_endpoint_number
sl_usbd_core_get_max_open_endpoint_number
sl_usbd_on_bus_event
sl_usbd_on_config_event
SL_USBD_FRAME_NBR_MAX
SL_USBD_FRAME_NBR_MASK
SL_USBD_MICROFRAME_NBR_MASK
SL_USBD_DEV_ATTRIB_SELF_POWERED
SL_USBD_DEV_ATTRIB_REMOTE_WAKEUP
SL_USBD_DESC_LEN_DEV
SL_USBD_DESC_LEN_DEV_QUAL
SL_USBD_DESC_LEN_HDR
SL_USBD_DESC_LEN_CFG
SL_USBD_DESC_LEN_OTHER_SPD_CFG
SL_USBD_DESC_LEN_IF
SL_USBD_DESC_LEN_IF_ASSOCIATION
SL_USBD_DESC_LEN_EP

SL_USBD_DESC_LEN_OTG
SL_USBD_REQ_DIR_MASK
SL_USBD_REQ_DIR_HOST_TO_DEVICE
SL_USBD_REQ_DIR_DEVICE_TO_HOST
SL_USBD_REQ_DIR_BIT
SL_USBD_REQ_TYPE_MASK
SL_USBD_REQ_TYPE_STANDARD
SL_USBD_REQ_TYPE_CLASS
SL_USBD_REQ_TYPE_VENDOR
SL_USBD_REQ_TYPE_RESERVED
SL_USBD_REQ_RECIPIENT_MASK
SL_USBD_REQ_RECIPIENT_DEVICE
SL_USBD_REQ_RECIPIENT_INTERFACE
SL_USBD_REQ_RECIPIENT_ENDPOINT
SL_USBD_REQ_RECIPIENT_OTHER
SL_USBD_REQ_GET_STATUS
SL_USBD_REQ_CLEAR_FEATURE
SL_USBD_REQ_SET_FEATURE
SL_USBD_REQ_SET_ADDRESS
SL_USBD_REQ_GET_DESCRIPTOR
SL_USBD_REQ_SET_DESCRIPTOR
SL_USBD_REQ_GET_CONFIGURATION
SL_USBD_REQ_SET_CONFIGURATION
SL_USBD_REQ_GET_INTERFACE
SL_USBD_REQ_SET_INTERFACE
SL_USBD_REQ_SYNCH_FRAME
SL_USBD_DESC_TYPE_DEVICE
SL_USBD_DESC_TYPE_CONFIGURATION
SL_USBD_DESC_TYPE_STRING
SL_USBD_DESC_TYPE_INTERFACE
SL_USBD_DESC_TYPE_ENDPOINT
SL_USBD_DESC_TYPE_DEVICE_QUALIFIER
SL_USBD_DESC_TYPE_OTHER_SPEED_CONFIGURATION
SL_USBD_DESC_TYPE_INTERFACE_POWER
SL_USBD_DESC_TYPE_OTG
SL_USBD_DESC_TYPE_IAD
SL_USBD_LANG_ID_ARABIC_SAUDI_ARABIA
SL_USBD_LANG_ID_CHINESE_TAIWAN
SL_USBD_LANG_ID_ENGLISH_US
SL_USBD_LANG_ID_ENGLISH_UK
SL_USBD_LANG_ID_FRENCH
SL_USBD_LANG_ID_GERMAN
SL_USBD_LANG_ID_GREEK

SL_USBD_LANG_ID_ITALIAN
SL_USBD_LANG_ID_PORTUGUESE
SL_USBD_LANG_ID_SANSKRIT
SL_USBD_CLASS_CODE_USE_IF_DESC
SL_USBD_CLASS_CODE_AUDIO
SL_USBD_CLASS_CODE_CDC_CONTROL
SL_USBD_CLASS_CODE_HID
SL_USBD_CLASS_CODE_PHYSICAL
SL_USBD_CLASS_CODE_IMAGE
SL_USBD_CLASS_CODE_PRINTER
SL_USBD_CLASS_CODE_MASS_STORAGE
SL_USBD_CLASS_CODE_HUB
SL_USBD_CLASS_CODE_CDC_DATA
SL_USBD_CLASS_CODE_SMART_CARD
SL_USBD_CLASS_CODE_CONTENT_SECURITY
SL_USBD_CLASS_CODE_VIDEO
SL_USBD_CLASS_CODE_PERSONAL_HEALTHCARE
SL_USBD_CLASS_CODE_DIAGNOSTIC_DEVICE
SL_USBD_CLASS_CODE_WIRELESS_CONTROLLER
SL_USBD_CLASS_CODE_MISCELLANEOUS
SL_USBD_CLASS_CODE_APPLICATION_SPECIFIC
SL_USBD_CLASS_CODE_VENDOR_SPECIFIC
SL_USBD_SUBCLASS_CODE_USE_IF_DESC
SL_USBD_SUBCLASS_CODE_USE_COMMON_CLASS
SL_USBD_SUBCLASS_CODE_VENDOR_SPECIFIC
SL_USBD_PROTOCOL_CODE_USE_IF_DESC
SL_USBD_PROTOCOL_CODE_USE_IAD
SL_USBD_PROTOCOL_CODE_VENDOR_SPECIFIC
SL_USBD_FEATURE_SEL_ENDPOINT_HALT
SL_USBD_FEATURE_SEL_DEVICE_REMOTE_WAKEUP
SL_USBD_FEATURE_SEL_TEST_MODE
SL_USBD_MAX_BUS_PWR_LIMIT_mA
SL_USBD_ENDPOINT_TYPE_MASK
SL_USBD_ENDPOINT_TYPE_CTRL
SL_USBD_ENDPOINT_TYPE_ISOC
SL_USBD_ENDPOINT_TYPE_BULK
SL_USBD_ENDPOINT_TYPE_INTR
SL_USBD_ENDPOINT_TYPE_SYNC_MASK
SL_USBD_ENDPOINT_TYPE_SYNC_NONE
SL_USBD_ENDPOINT_TYPE_SYNC_ASYNC
SL_USBD_ENDPOINT_TYPE_SYNC_ADAPTIVE
SL_USBD_ENDPOINT_TYPE_SYNC_SYNC
SL_USBD_ENDPOINT_TYPE_USAGE_MASK

SL_USBD_ENDPOINT_TYPE_USAGE_DATA
SL_USBD_ENDPOINT_TYPE_USAGE_FEEDBACK
SL_USBD_ENDPOINT_TYPE_USAGE_IMPLICIT_FEEDBACK
SL_USBD_ENDPOINT_NBR_MASK
SL_USBD_ENDPOINT_MAX_NBR
SL_USBD_ENDPOINT_DIR_MASK
SL_USBD_ENDPOINT_DIR_BIT
SL_USBD_ENDPOINT_DIR_OUT
SL_USBD_ENDPOINT_DIR_IN
SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_1
SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_2
SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_3
SL_USBD_ENDPOINT_MAX_INTERVAL_VAL
SL_USBD_MICROSOFT_COMPAT_ID_NULL
SL_USBD_MICROSOFT_COMPAT_ID_RNDIS
SL_USBD_MICROSOFT_COMPAT_ID_PTP
SL_USBD_MICROSOFT_COMPAT_ID_MTP
SL_USBD_MICROSOFT_COMPAT_ID_XUSB20
SL_USBD_MICROSOFT_COMPAT_ID_BLUETOOTH
SL_USBD_MICROSOFT_COMPAT_ID_WINUSB
SL_USBD_MICROSOFT_COMPAT_ID_NONE
SL_USBD_MICROSOFT_SUBCOMPAT_ID_NULL
SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_1_1
SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_1_2
SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_EDR
SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_SZ
SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_EXPAND_SZ
SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_BINARY
SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_DWORD_LITTLE_ENDIAN
SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_DWORD_BIG_ENDIAN
SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_LINK
SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_MULTI_SZ
SL_USBD_ENDPOINT_INFO_TYPE_CTRL
SL_USBD_ENDPOINT_INFO_TYPE_ISOC
SL_USBD_ENDPOINT_INFO_TYPE_BULK
SL_USBD_ENDPOINT_INFO_TYPE_INTR
SL_USBD_ENDPOINT_INFO_DIR_OUT
SL_USBD_ENDPOINT_INFO_DIR_IN
SL_USBD_CONFIG_NBR_NONE
SL_USBD_CONFIG_NBR_SPD_BIT
SL_USBD_INTERFACE_NBR_NONE
SL_USBD_ALT_INTERFACE_NBR_NONE
SL_USBD_INTERFACE_GROUP_NBR_NONE

SL_USBD_ENDPOINT_ADDR_NONE
SL_USBD_ENDPOINT_PHY_NONE
SL_USBD_ENDPOINT_NBR_NONE
SL_USBD_CLASS_NBR_NONE
USBD_CFG_STR_EN
USBD_CFG_MS_OS_DESC_EN
USBD_CFG_URB_EXTRA_EN
SL_USBD_URB_MAX_NBR
USBD_CFG_HS_EN
USBD_CFG_EP_ISOC_EN
USBD_CFG_OPTIMIZE_SPD
SL_USBD_CORE_EVENT_BUS_NBR
SL_USBD_CORE_EVENT_BUS_NBR_TOTAL
SL_USBD_CORE_EVENT_ASYNC_NBR
SL_USBD_CORE_EVENT_NBR_TOTAL
SL_USBD_FRAME_NBR_GET
SL_USBD_MICROFRAME_NBR_GET
SL_USBD_FRAME_NBR_DIFF_GET
SL_USBD_ENDPOINT_PHY_TO_ADDR
SL_USBD_ENDPOINT_ADDR_TO_PHY
SL_USBD_ENDPOINT_ADDR_TO_LOG
SL_USBD_ENDPOINT_PHY_TO_LOG
SL_USBD_ENDPOINT_LOG_TO_ADDR_IN
SL_USBD_ENDPOINT_LOG_TO_ADDR_OUT
SL_USBD_ENDPOINT_IS_IN

USB Device HID API

sl_usbd_hid_callbacks_t
 enable
 disable
 get_report_desc
 get_phy_desc
 set_output_report
 get_feature_report
 set_feature_report
 get_protocol
 set_protocol
sl_usbd_hid_country_code_t
sl_usbd_hid_async_function_t
sl_usbd_hid_init
sl_usbd_hid_create_instance
sl_usbd_hid_add_to_configuration
sl_usbd_hid_is_enabled
sl_usbd_hid_write_sync
sl_usbd_hid_read_sync

sl_usbd_hid_read_async
sl_usbd_hid_write_async
SL_USBD_HID_EXTERN
SL_USBD_HID_SUBCLASS_NONE
SL_USBD_HID_SUBCLASS_BOOT
SL_USBD_HID_PROTOCOL_NONE
SL_USBD_HID_PROTOCOL_KBD
SL_USBD_HID_PROTOCOL_MOUSE
SL_USBD_HID_ITEM_LONG
SL_USBD_HID_ITEM_TYPE_MAIN
SL_USBD_HID_ITEM_TYPE_GLOBAL
SL_USBD_HID_ITEM_TYPE_LOCAL
SL_USBD_HID_ITEM_TYPE_RESERVED
SL_USBD_HID_MAIN_INPUT
SL_USBD_HID_MAIN_OUTPUT
SL_USBD_HID_MAIN_COLLECTION
SL_USBD_HID_MAIN_FEATURE
SL_USBD_HID_MAIN_ENDCOLLECTION
SL_USBD_HID_MAIN_CONSTANT
SL_USBD_HID_MAIN_DATA
SL_USBD_HID_MAIN_VARIABLE
SL_USBD_HID_MAIN_ARRAY
SL_USBD_HID_MAIN_RELATIVE
SL_USBD_HID_MAIN_ABSOLUTE
SL_USBD_HID_MAIN_WRAP
SL_USBD_HID_MAIN_NOWRAP
SL_USBD_HID_MAIN_NONLINEAR
SL_USBD_HID_MAIN_LINEAR
SL_USBD_HID_MAIN_NOPREFERRED
SL_USBD_HID_MAIN_PREFERREDSTATE
SL_USBD_HID_MAIN_NULLSTATE
SL_USBD_HID_MAIN_NONULLPOSITION
SL_USBD_HID_MAIN_VOLATILE
SL_USBD_HID_MAIN_NONVOLATILE
SL_USBD_HID_MAIN_BUFFEREDBYTES
SL_USBD_HID_MAIN_BITFIELD
SL_USBD_HID_COLLECTION_PHYSICAL
SL_USBD_HID_COLLECTION_APPLICATION
SL_USBD_HID_COLLECTION_LOGICAL
SL_USBD_HID_COLLECTION_REPORT
SL_USBD_HID_COLLECTION_NAMEDARRAY
SL_USBD_HID_COLLECTION_USAGESWITCH
SL_USBD_HID_COLLECTION_USAGEMODIFIER

SL_USBD_HID_GLOBAL_USAGE_PAGE
SL_USBD_HID_GLOBAL_LOG_MIN
SL_USBD_HID_GLOBAL_LOG_MAX
SL_USBD_HID_GLOBAL_PHY_MIN
SL_USBD_HID_GLOBAL_PHY_MAX
SL_USBD_HID_GLOBAL_UNIT_EXPONENT
SL_USBD_HID_GLOBAL_UNIT
SL_USBD_HID_GLOBAL_REPORT_SIZE
SL_USBD_HID_GLOBAL_REPORT_ID
SL_USBD_HID_GLOBAL_REPORT_COUNT
SL_USBD_HID_GLOBAL_PUSH
SL_USBD_HID_GLOBAL_POP
SL_USBD_HID_LOCAL_USAGE
SL_USBD_HID_LOCAL_USAGE_MIN
SL_USBD_HID_LOCAL_USAGE_MAX
SL_USBD_HID_LOCAL_DESIGNATOR_INDEX
SL_USBD_HID_LOCAL_DESIGNATOR_MIN
SL_USBD_HID_LOCAL_DESIGNATOR_MAX
SL_USBD_HID_LOCAL_STRING_INDEX
SL_USBD_HID_LOCAL_STRING_MIN
SL_USBD_HID_LOCAL_STRING_MAX
SL_USBD_HID_LOCAL_DELIMITER
SL_USBD_HID_BIAS_NOT_APPLICABLE
SL_USBD_HID_BIAS_RIGHT_HAND
SL_USBD_HID_BIAS_LEFT_HAND
SL_USBD_HID_BIAS_BOTH_HANDS
SL_USBD_HID_BIAS_EITHER_HAND
SL_USBD_HID_DESIGNATOR_NONE
SL_USBD_HID_DESIGNATOR_HAND
SL_USBD_HID_DESIGNATOR_EYEBALL
SL_USBD_HID_DESIGNATOR_EYEBROW
SL_USBD_HID_DESIGNATOR_EYELID
SL_USBD_HID_DESIGNATOR_EAR
SL_USBD_HID_DESIGNATOR_NOSE
SL_USBD_HID_DESIGNATOR_MOUTH
SL_USBD_HID_DESIGNATOR_UPPER_LIP
SL_USBD_HID_DESIGNATOR_LOWER_LIP
SL_USBD_HID_DESIGNATOR_JAW
SL_USBD_HID_DESIGNATOR_NECK
SL_USBD_HID_DESIGNATOR_UPPER_ARM
SL_USBD_HID_DESIGNATOR_ELBOW
SL_USBD_HID_DESIGNATOR_FOREARM
SL_USBD_HID_DESIGNATOR_WRIST

SL_USBD_HID_DESIGNATOR_PALM
SL_USBD_HID_DESIGNATOR_THUMB
SL_USBD_HID_DESIGNATOR_INDEX_FINGER
SL_USBD_HID_DESIGNATOR_MIDDLE_FINGER
SL_USBD_HID_DESIGNATOR_RING_FINGER
SL_USBD_HID_DESIGNATOR_LITTLE_FINGER
SL_USBD_HID_DESIGNATOR_HEAD
SL_USBD_HID_DESIGNATOR_SHOULDER
SL_USBD_HID_DESIGNATOR_HIP
SL_USBD_HID_DESIGNATOR_WAIST
SL_USBD_HID_DESIGNATOR_THIGH
SL_USBD_HID_DESIGNATOR_KNEE
SL_USBD_HID_DESIGNATOR_CALF
SL_USBD_HID_DESIGNATOR_ANKLE
SL_USBD_HID_DESIGNATOR_FOOT
SL_USBD_HID_DESIGNATOR_HEEL
SL_USBD_HID_DESIGNATOR_BALL_OF_FOOT
SL_USBD_HID_DESIGNATOR_BIG_TOE
SL_USBD_HID_DESIGNATOR_SECOND_TOE
SL_USBD_HID_DESIGNATOR_THIRD_TOE
SL_USBD_HID_DESIGNATOR_FOURTH_TOE
SL_USBD_HID_DESIGNATOR_LITTLE_TOE
SL_USBD_HID_DESIGNATOR_BROW
SL_USBD_HID_DESIGNATOR_CHEEK
SL_USBD_HID_QUALIFIER_NOT_APPLICABLE
SL_USBD_HID_QUALIFIER_RIGHT
SL_USBD_HID_QUALIFIER_LEFT
SL_USBD_HID_QUALIFIER_BOTH
SL_USBD_HID_QUALIFIER_EITHER
SL_USBD_HID_QUALIFIER_CENTER
SL_USBD_HID_USAGE_PAGE_UNDEFINED
SL_USBD_HID_USAGE_PAGE_GENERIC_DESKTOP_CONTROLS
SL_USBD_HID_USAGE_PAGE_SIMULATION_CONTROLS
SL_USBD_HID_USAGE_PAGE_VR_CONTROLS
SL_USBD_HID_USAGE_PAGE_SPORT_CONTROLS
SL_USBD_HID_USAGE_PAGE_GAME_CONTROLS
SL_USBD_HID_USAGE_PAGE_GENERIC_DEVICE_CONTROLS
SL_USBD_HID_USAGE_PAGE_KEYBOARD
SL_USBD_HID_USAGE_PAGE_LEDS
SL_USBD_HID_USAGE_PAGE_BUTTON
SL_USBD_HID_USAGE_PAGE_ORDINAL
SL_USBD_HID_USAGE_PAGE_TELEPHONY
SL_USBD_HID_USAGE_PAGE_CONSUMER

SL_USBD_HID_USAGE_PAGE_DIGITIZER
SL_USBD_HID_USAGE_PAGE_PID_PAGE
SL_USBD_HID_USAGE_PAGE_UNICODE
SL_USBD_HID_USAGE_PAGE_ALPHANUMERIC_DISPLAY
SL_USBD_HID_USAGE_PAGE_MEDICAL_INSTRUMENTS
SL_USBD_HID_USAGE_PAGE_MONITOR_0
SL_USBD_HID_USAGE_PAGE_MONITOR_1
SL_USBD_HID_USAGE_PAGE_MONITOR_2
SL_USBD_HID_USAGE_PAGE_MONITOR_3
SL_USBD_HID_USAGE_PAGE_POWER_0
SL_USBD_HID_USAGE_PAGE_POWER_1
SL_USBD_HID_USAGE_PAGE_POWER_2
SL_USBD_HID_USAGE_PAGE_POWER_3
SL_USBD_HID_USAGE_PAGE_BAR_CODE_SCANNER_PAGE
SL_USBD_HID_USAGE_PAGE_SCALE_PAGE
SL_USBD_HID_USAGE_PAGE_MSR_DEVICES
SL_USBD_HID_USAGE_PAGE_POINT_OF_SALE_PAGES
SL_USBD_HID_USAGE_PAGE_CAMERA_CONTROL_PAGE
SL_USBD_HID_USAGE_PAGE_ARCADE_PAGE
SL_USBD_HID_CP_POINTER
SL_USBD_HID_CA_MOUSE
SL_USBD_HID_CA_JOYSTICK
SL_USBD_HID_CA_GAME_PAD
SL_USBD_HID_CA_KEYBOARD
SL_USBD_HID_CA_KEYPAD
SL_USBD_HID_CA_MULTI_AXIS_CONTROLLER
SL_USBD_HID_DV_X
SL_USBD_HID_DV_Y
SL_USBD_HID_DV_Z
SL_USBD_HID_DV_WHEEL
SL_USBD_HID_CA_SYSTEM_CONTROL
USB Device MSC API
sl_usbd_msc_subclass_driver_t
 enable
 disable
 process_command
 read_data
 write_data
sl_usbd_msc_init
sl_usbd_msc_create_instance
sl_usbd_msc_add_to_configuration
sl_usbd_msc_lun_add
sl_usbd_msc_is_enabled
SL_USBD_MSC_LU_NBR_INVALID

SL_USBD_MSC_SUBCLASS_CODE_RBC
SL_USBD_MSC_SUBCLASS_CODE_SFF_8020i
SL_USBD_MSC_SUBCLASS_CODE_MMC_2
SL_USBD_MSC_SUBCLASS_CODE_QIC_157
SL_USBD_MSC_SUBCLASS_CODE_UFI
SL_USBD_MSC_SUBCLASS_CODE_SFF_8070i
SL_USBD_MSC_SUBCLASS_CODE_SCSI
SL_USBD_MSC_PROTOCOL_CODE_CTRL_BULK_INTR_CMD_INTR
SL_USBD_MSC_PROTOCOL_CODE_CTRL_BULK_INTR
SL_USBD_MSC_PROTOCOL_CODE_BULK_ONLY

USB Device MSC SCSI API

sl_usbd_msc_scsi_callbacks_t

enable

disable

host_eject

sl_usbd_msc_scsi_lun_api

init

open

close

get_capacity

read

write

sl_usbd_msc_scsi_lun_info

scsi_lun_api_ptr

vendor_id_ptr

product_id_ptr

product_revision_level

is_read_only

sl_usbd_msc_scsi_lun

lun_index

vendor_id_str

product_id_str

product_revision_level

is_read_only

is_removable

class_nbr

lu_nbr

scsi_lun_api_ptr

data_ptr

blocks_nbr

block_size

lb_addr

lb_count

sense_key

[asc](#)[opened_flag](#)[present_flag](#)[sl_usbd_msc_scsi_lb_qty_t](#)[sl_usbd_msc_scsi_lba_t](#)[sl_usbd_msc_scsi_init](#)[sl_usbd_msc_scsi_create_instance](#)[sl_usbd_msc_scsi_add_to_configuration](#)[sl_usbd_msc_scsi_lun_add](#)[sl_usbd_msc_scsi_lun_attach](#)[sl_usbd_msc_scsi_lun_detach](#)[sl_usbd_msc_scsi_lun_get_capacity](#)[sl_usbd_msc_scsi_is_enabled](#)

[USB Device Vendor API](#)

[sl_usbd_vendor_callbacks_t](#)[enable](#)[disable](#)[setup_req](#)[sl_usbd_vendor_async_function_t](#)[sl_usbd_vendor_init](#)[sl_usbd_vendor_create_instance](#)[sl_usbd_vendor_add_to_configuration](#)[sl_usbd_vendor_is_enabled](#)[sl_usbd_vendor_abort_read_bulk](#)[sl_usbd_vendor_abort_write_bulk](#)[sl_usbd_vendor_abort_read_interrupt](#)[sl_usbd_vendor_abort_write_interrupt](#)[sl_usbd_vendor_read_bulk_sync](#)[sl_usbd_vendor_write_bulk_sync](#)[sl_usbd_vendor_read_bulk_async](#)[sl_usbd_vendor_write_bulk_async](#)[sl_usbd_vendor_read_interrupt_sync](#)[sl_usbd_vendor_write_interrupt_sync](#)[sl_usbd_vendor_read_interrupt_async](#)[sl_usbd_vendor_write_interrupt_async](#)

[API Documentation](#)

[USB Device Troubleshooting](#)

[Overview](#)

[Microsoft Windows OS USB Host](#)

[Overview](#)

Overview

USB Device

USB is one of the most successful communication interfaces in the history of computer systems and is the de facto standard for connecting computer peripherals. Silicon Labs USB Device stack is a USB device module designed specifically for embedded systems. Built from the ground up with Silicon Labs' quality, scalability, and reliability, it has gone through a rigorous validation process to comply with the USB 2.0 specification. This documentation describes how to initialize, start, and use Silicon Labs USB Device stack. It explains the various configuration values and their uses. It also includes an overview of the technology, types of configuration possibilities, implementation procedures, and examples of typical usage for every available class.

To help you understand the USB concepts quickly, the documentation features many examples of USB with basic functions. These examples will provide you with a framework that allows you to build devices quickly. These examples include:

- USB-to-serial adapter (Communications Device Class)
- Mouse or keyboard (Human Interface Device Class)
- Removable storage device (Mass Storage Class)
- Custom device (Vendor Class)

The following is the overview of the documentation sections:

- [Specifications and Features](#)
- [USB Device Configuration](#)
- [USB Device Programming Guide](#)
- [USB Device Classes](#)
 - [CDC ACM Class](#)
 - [HID Class](#)
 - [MSC SCSI Class](#)
 - [Vendor Class](#)
- [USB Device Troubleshooting](#)
- [Microsoft Windows OS USB Host](#)

Overview

Specifications

- Complies with the "Universal Serial Bus specification revision 2.0"
- Implements the "Interface Association Descriptor Engineering Change Notice (ECN)"
- Transfer types
 - Control
 - Bulk
 - Interrupt
- USB classes
 - Communication Device Class (CDC)
 - Abstract Control Model (ACM)
 - Human Interface Device (HID)
 - Mass Storage Class (MSC)
 - Vendor-specific class framework

Features

- Scalable to include only required features to minimize memory footprint
- Supports Full-speed (12 Mbit/s)
- Supports composite (multi-function) devices
- Supports multi-configuration devices
- Supports USB power-saving functionalities (device suspend and resume)
- Complete integration of Mass Storage Class into Micrium OS File System module
- Developed with CMSIS-RTOS2 abstraction layer so that it can work with different OSes. Silicon Labs GSDK comes with FreeRTOS and Micrium OS ports.

Overview

USB Device Configuration

This section discusses how to configure Silicon Labs USB Device. There are three groups of configuration parameters, as follows:

- [USB Device Core Configuration](#)
- [USB Device Information Configuration](#)
- [USB Device Hardware Configuration](#)

USB Device Core Configuration

Silicon Labs USB Device can be configured at compile time via a set of #defines located in the `sl_usbd_core_config.h` file. USB Device uses #defines when possible because they allow code and data sizes to be scaled at compile time based on which features are enabled. This allows the read-only memory (ROM) and random-access memory (RAM) footprints of Silicon Labs USB Device to be adjusted based on your application's requirements.

Recommended: Start the configuration process with the default values (highlighted in **bold**).

The sections below are organized based on the order in the template configuration file, `sl_usbd_core_config.h`.

- [Core Configuration](#)
- [Classes Configuration](#)

Core Configuration

Table - USB Device Core Configuration Constants

Constant	Description	Default Value
SL_USBD_TASK_STACK_SIZE	Configures the stack size in bytes of USB core task	4096
SL_USBD_TASK_PRIORITY	Configures the priority of USB core task. This is a CMSIS-RTOS2 priority .	osPriorityHigh
SL_USBD_AUTOSTART_DEVICE	If enabled, the USB device will be automatically started once the kernel is started and that USB core task is scheduled for the first time. If disabled, your application will need to call <code>sl_usbd_core_start_device()</code> when ready to be detected by the USB host.	1
SL_USBD_CONFIGURATION_QUANTITY	The total number of configurations that will be added via the <code>sl_usbd_add_configuration()</code> function.	1

Constant	Description	Default Value
SL_USBD_INTERFACE_QUANTITY	The total number of USB interfaces to be added for all your configurations. This greatly depends on the class(es) used. For more information on how many interfaces a class instance requires, refer to the section "Resource Needs from core" of your class(es).	10
SL_USBD_ALT_INTERFACE_QUANTITY	The total number of USB alternate interfaces to be added for all your configurations. This greatly depends on the class(es) used. This value must always be equal or greater than SL_USBD_INTERFACE_QUANTITY . For more information on how many alternate interfaces a class instance requires, refer to the section "Resource Needs from Core" of your class(es).	10
SL_USBD_INTERFACE_GROUP_QUANTITY	The total number of USB interface groups that will be added for all your configurations. This greatly depends on the class(es) used. For more information on how many interface groups requires a class instance, refer to the section "Resource Needs from Core" of your class(es).	2
SL_USBD_DESCRIPTOR_QUANTITY	The total number of Endpoint descriptors that will be added for all your configurations. This greatly depends on the class(es) used. For more information on how many endpoint descriptors a class instance requires, refer to "Number of endpoints" in the section "Resource Needs from Core" of your class(es). Note that the control endpoints don't need to be considered here.	20
SL_USBD_STRING_QUANTITY	The total number of USB strings. Setting the quantity to zero will disable the feature. Disabling this will cause the device to not store any USB description strings passed from the application. This means the host will be unable to retrieve the description strings (such as manufacturer and product name).	30
SL_USBD_OPEN_ENDPOINTS_QUANTITY	The total number of opened endpoints per configuration. A device requires at least two opened endpoints for control transfers, but you must also add the endpoints of the class(es) used. For more information on how many opened endpoints a class instance requires, refer to "Number of endpoints" in the section "Resource Needs from Core" of your class(es).	20

Classes Configuration

Classes have specific compile-time configurations. Refer to [USB Device Classes](#) for more information.

USB Device Information Configuration

The sl_usbd_device_config.h configuration file regroups compile-time #define-s to set basic information regarding your device, such as Vendor/Product ID, device strings, etc.

The table below describes each information configuration define available in this configuration file.

Table - USB Device Information Configuration Defines

Constant	Description
SL_USBD_DEVICE_VENDOR_ID	Your vendor identification number as delivered by the USB Implementers Forum. For more information on how you can obtain a vendor ID, see http://www.usb.org/developers/vendor/ .
SL_USBD_DEVICE_PRODUCT_ID	Your product identification number.
SL_USBD_DEVICE_RELEASE_NUMBER	Release number of your device.

Constant	Description
SL_USBD_DEVICE_MANUFACTURER_STRING	String describing the manufacturer of your device. This configuration is ignored when the configuration <code>SL_USBD_STRING_QUANTITY</code> is set to 0.
SL_USBD_DEVICE_PRODUCT_STRING	String describing your product. This configuration is ignored when the configuration <code>SL_USBD_STRING_QUANTITY</code> is set to 0.
SL_USBD_DEVICE_SERIAL_NUMBER_STRING	String containing the serial number of your device. This configuration is ignored when the configuration <code>SL_USBD_STRING_QUANTITY</code> is set to 0.
SL_USBD_DEVICE_LANGUAGE_ID	Identification number of the language of your device's strings. Possible values are:
	- <code>SL_USBD_LANG_ID_ARABIC_SAUDI_ARABIA</code>
	- <code>SL_USBD_LANG_ID_CHINESE_TAIWAN</code>
	- <code>SL_USBD_LANG_ID_ENGLISH_US</code>
	- <code>SL_USBD_LANG_ID_ENGLISH_UK</code>
	- <code>SL_USBD_LANG_ID_FRENCH</code>
	- <code>SL_USBD_LANG_ID_GERMAN</code>
	- <code>SL_USBD_LANG_ID_GREEK</code>
	- <code>SL_USBD_LANG_ID_ITALIAN</code>
	- <code>SL_USBD_LANG_ID_PORTUGUESE</code>
	- <code>SL_USBD_LANG_ID_SANSKRIT</code>
	This configuration is ignored when the configuration <code>SL_USBD_STRING_QUANTITY</code> is set to 0.

USB Device Hardware Configuration

Depending on the Silicon Labs device you are using, you will possibly have the GPIO pin and port to configure for USB VBUS Sense signal. The configuration defines are in the `sl_usbd_hardware_config.h` header file.

Constant	Description
SL_USBD_DRIVER_VBUS_SENSE_PORT	GPIO Port for the USB VBUS Sense signal on your board.
SL_USBD_DRIVER_VBUS_SENSE_PIN	GPIO Pin for the USB VBUS Sense signal on your board.

Overview

USB Device Programming Guide

This section explains how to use the USB Device module.

Initial Setup of USB Device Module

This section describes the basic steps required to initialize the USB Device module and to add, prepare, and start a device.

- [Initializing the USB Device Module](#)
 - [Initializing the USB Device Core](#)
 - [Initializing the Class\(es\)](#)
- [Adding Your USB Device](#)
- [Building Your USB Device](#)
 - [Adding Configuration\(s\)](#)
 - [Adding USB Function\(s\)](#)
- [Starting Your USB Device](#)
- [Event Hook Functions](#)

Initializing the USB Device Module

Initializing the USB Device Core

Begin by initializing the USB device module core by calling the function `sl_usbd_core_init()`.

The example below shows calling `sl_usbd_core_init()`.

Example - Calling `sl_usbd_core_init()`

```
sl_status_t status;

status = sl_usbd_core_init();
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Initializing the Class(es)

After the USB device module core has been initialized, you must initialize each class you intend to use. See section "Programming Guide" of your class(es) for more information.

Building Your USB Device

Adding Configuration(s)

After you successfully initialize your device, you can start adding the USB functions to it, starting with a new configuration. A device must have at least one configuration. To add configuration(s), call the function `sl_usbd_core_add_configuration()` function. This function must be called for each configuration you want to add.

The example below shows how to add a Full-speed.

Example - Adding Configuration(s) to Your Device

```
sl_status_t status;
uint8_t config_nbr_fs;

/* Adding a full-speed configuration to the device. */
status = sl_usbd_core_add_configuration(0, /* No special attributes to the configuration. */
    100u, /* Max power consumption: 100mA. */
    SL_USBD_DEVICE_SPEED_FULL, /* Full-Speed configuration. */
    "Config Add Example Full-Speed config",
    &config_nbr_fs);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Adding USB Function(s)

After you have successfully added at least one configuration to your device, you can add the interfaces and endpoints to your device. Each USB class has its own needs in terms of interface and endpoints type, quantity, and other parameters. Silicon Labs USB Device adds interfaces and endpoints in the classes it offers.

From your application, you can instantiate a USB class and add it to a configuration. For more information on the concept of USB device class instances, see [USB Device Classes](#). Note that you can instantiate and add many different class instances to a configuration to create a multi-function (composite) device.

The example below shows how to create a class instance and add it to a configuration.

Example - Adding a Class Instance to Your Device

```
sl_status_t status;
uint8_t class_nbr;

/* Create an instance of the class you want to use.*/
/* Note that depending on the class this function can have more arguments. */
status = sl_usbd_<class>_create_instance(&class_nbr);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}

/* Add the class instance to the Full-Speed configuration. */
status = sl_usbd_<class>_add_to_configuration(class_nbr, /* Class number returned by sl_usbd_<class>_create_instance. */
    config_nbr_fs); /* Configuration number returned by sl_usbd_core_add_configuration(). */
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Starting Your USB Device

By default, the device will be started automatically by the USB device core task after the device initialization is completed and the kernel started. To control when the device is started and becomes visible by the USB host, use the configuration define `SL_USBD_AUTO_START_USB_DEVICE` to disable the auto-start feature. When disabled, after you have built/prepared your device, you can start it and make it visible to the USB host by calling the function `sl_usbd_core_start_device()`.

The example below shows how to start your device using the `sl_usbd_core_start_device()` function.

Example - Starting Your Device


```
sl_status_t status;

status = sl_usbd_core_start_device();
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Event Hook Functions

The USB Device core module offers two weak hook functions that you can re-defined in your application. Their purpose is to notify when bus and configuration events occur.

Table - USB Event Hook Functions

Event	Description	Function Signature
Bus	Called when USB Bus event occurs	void sl_usbd_on_bus_event(sl_usbd_bus_event_t event);
Configuration	Called when USB Configuration event occurs	void sl_usbd_on_config_event(sl_usbd_config_event_t event, uint8_t config_nbr);

Example - Event Hook Functions

```
void sl_usbd_on_bus_event(sl_usbd_bus_event_t event) {
    switch (event) {
        case SL_USBD_EVENT_BUS_CONNECT:
            // called when USB cable is inserted in a host controller
            break;

        case SL_USBD_EVENT_BUS_DISCONNECT:
            // called when USB cable is removed from a host controller
            break;

        case SL_USBD_EVENT_BUS_RESET:
            // called when the host sends reset command
            break;

        case SL_USBD_EVENT_BUS_SUSPEND:
            // called when the host sends suspend command
            break;

        case SL_USBD_EVENT_BUS_RESUME:
            // called when the host sends wake up command
            break;

        default:
            break;
    }
}

void sl_usbd_on_config_event(sl_usbd_config_event_t event, uint8_t config_nbr) {
    switch (event) {
        case SL_USBD_EVENT_CONFIG_SET:
            // called when the host sets a configuration
            break;

        case SL_USBD_EVENT_CONFIG_UNSET:
            // called when a configuration is unset
            break;

        default:
            break;
    }
}
```

Overview

USB Device Classes

The USB classes available in Silicon Labs USB Device share some common characteristics. This section explains these characteristics and their interactions with the core layer.

For more information on a specific class, see the following:

- [CDC ACM Class](#)
- [HID Class](#)
- [MSC SCSI Class](#)
- [Vendor Class](#)

About Class Instances

The USB classes available in USB Device implement the concept of class instances. A class instance represents one function within a device. The function can be described by one interface or by a group of interfaces and belongs to a specific class.

Each USB class implementation has some configurations and functions in common, based on the concept of class instance. The common configurations and functions are presented in the table below. In the column title 'Constants or Function', the placeholder XXXX can be replaced by the name of the class: CDC, HID, MSC, CDC_ACM or VENDOR (Vendor for function names).

Table - Constants and Functions Related to the Concept of Multiple Class Instances

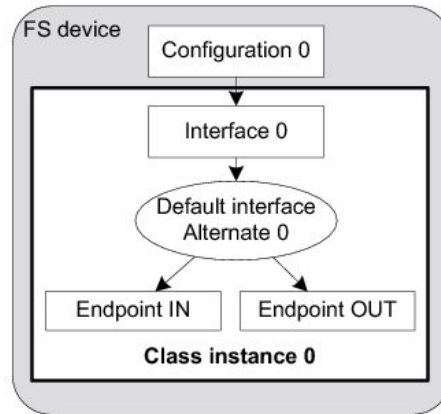
Constant or Function	Description
SL_USBD_XXXX_CLASS_INSTANCE_QUANTITY	Configures the maximum number of class instances.
SL_USBD_XXXX_CONFIGURATION_QUANTITY	Configures the maximum number of configurations. During the class initialization, a created class instance will be added to one or more configurations.
slusbd_XXXX_create_instance()	Creates a new class instance.
slusbd_XXXX_add_to_configuration()	Adds an existing class instance to the specified device configuration.

In terms of code implementation, the class will declare a local global variable that contains a class control structure. This class control structure is associated with one class instance and will contain specific information to manage the class instance.

The following figures show several case scenarios. Each figure includes a code example that corresponds to the case scenario.

[Figure - Multiple Class Instances - FS Device \(1 Configuration with 1 Interface\)](#) represents a typical USB device. The device is Full-Speed (FS) and contains a single configuration. The function of the device is described by one interface composed of a pair of endpoints for data communication. One class instance is created and will allow you to manage the entire interface with its associated endpoint.

Figure - Multiple Class Instances - FS Device (1 Configuration with 1 Interface)



The code corresponding to [Figure - Multiple Class Instances - FS Device \(1 Configuration with 1 Interface\)](#) is shown in the example below.

Example - Multiple Class Instances - FS Device (1 Configuration with 1 Interface)

```

sl_status_t status;
uint8_t class_0;

void app_usbd_XXXX_enable(uint8_t class_nbr)
{
    /* Handle Class enable event. */
}

void app_usbd_XXXX_disable(uint8_t class_nbr)
{
    /* Handle Class disable event. */
}

sl_usbd_XXXX_callbacks_t class_callbacks = {
    .enable = app_usbd_XXXX_enable,
    .disable = app_usbd_XXXX_disable
};

status = sl_usbd_XXXX_init();
if (status != SL_STATUS_OK) {
    /* $$$$ Handle the error. */
}

status = sl_usbd_XXXX_create_instance(&class_callbacks,
                                     &class_0);
if (status != SL_STATUS_OK) {
    /* $$$$ Handle the error. */
}

status = sl_usbd_XXXX_add_to_configuration(class_0, config_0);
if (status != SL_STATUS_OK) {
    /* $$$$ Handle the error. */
}

```

(1) Every class offers a set of callback functions for device connect/disconnect events and for class specific events. The callback structure object is passed as argument when creating the class instance with `sl_usbd_XXXX_create_instance()` function.

(1) Initialize the class. All internal variables, structures, and class ports will be initialized. Note that the `Init()` function in some classes may take other arguments.

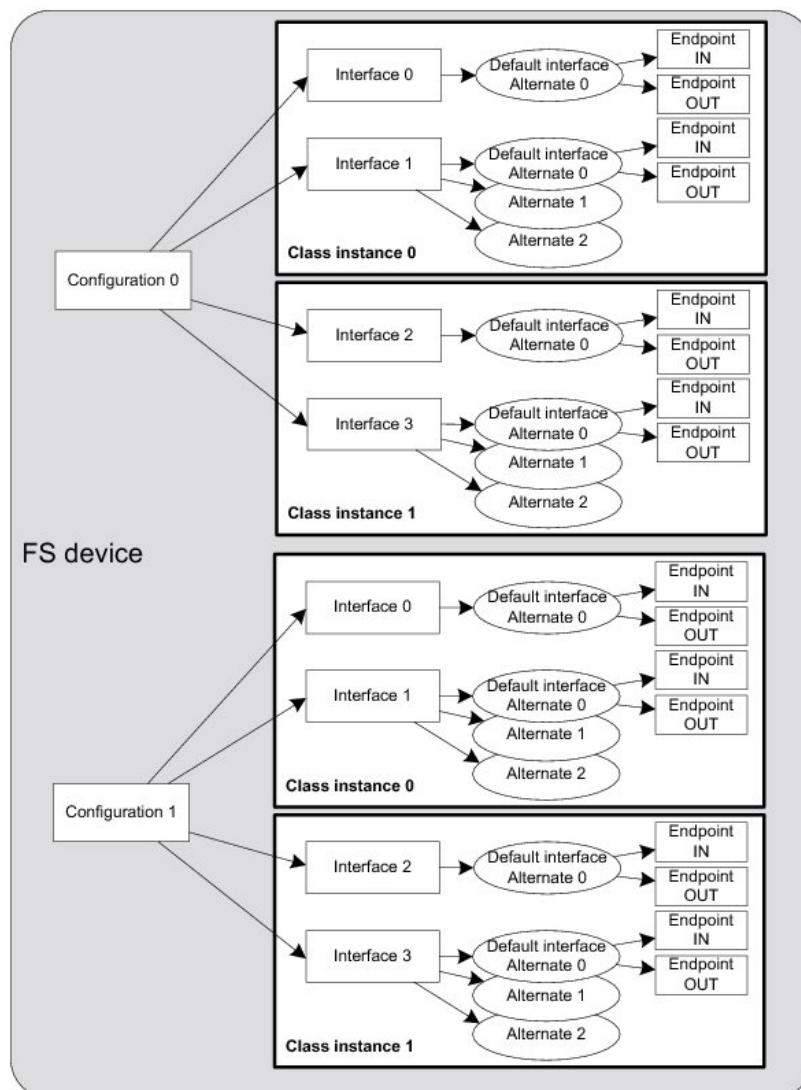
(2) Create the class instance, which is `class_0`. The function `sLusbd_XXXX_create_instance()` allocates a class control structure associated with `class_0`. Depending on the class, `sLusbd_XXXX_create_instance()` may have additional parameters aside from the class number that represent class-specific information stored in the class control structure.

(3) Add the class instance, `class_0`, to the specified configuration number, `config_0`. `sLusbd_XXXX_add_to_configuration()` will create the interface 0 and its associated IN and OUT endpoints. As a result, the class instance encompasses the interface 0 and its endpoints. Any communication done on the interface 0 will use the class instance number, `class_0`.

Figure - Multiple Class Instances - FS Device (2 Configurations and Multiple Interfaces) represents a more complex example. A full-speed device is composed of two configurations. The device has two functions which belong to the same class, but each function is described by two interfaces and has a pair of bidirectional endpoints.

In this example, two class instances are created. Each class instance is associated with a group of interfaces as opposed to Figure - Multiple Class Instances - FS Device (1 Configuration with 1 Interface) and Figure - Multiple Class Instances - FS Device (2 Configurations and Multiple Interfaces) where the class instance was associated with a single interface.

Figure - Multiple Class Instances - FS Device (2 Configurations and Multiple Interfaces)



The code corresponding to Figure - Multiple Class Instances - FS Device (2 Configurations and Multiple Interfaces) is shown in the example below. The error handling is omitted for clarity.

Example - Multiple Class Instances - FS Device (2 Configurations and Multiple Interfaces)

```
sl_status_t status;
uint8_t class_0;
uint8_t class_1;

status = sl_usbd_XXXX_init(); (1)

status = sl_usbd_XXXX_create_instance(&class_0); (2)
status = sl_usbd_XXXX_create_instance(&class_1); (3)

status = sl_usbd_XXXX_add_to_configuration(class_0, cfg_0); (4)
status = sl_usbd_XXXX_add_to_configuration(class_1, cfg_0); (5)

status = sl_usbd_XXXX_add_to_configuration(class_0, cfg_1); (6)
status = sl_usbd_XXXX_add_to_configuration(class_1, cfg_1); (6)
```

- (1) Initialize the class. Any internal variables, structures, and class ports will be initialized.
- (2) Create the class instance, `class_0`. The function `sl_usbd_XXXX_create_instance()` allocates a class control structure associated with `class_0`.
- (3) Create the class instance, `class_1`. The function `sl_usbd_XXXX_create_instance()` allocates another class control structure associated with `class_1`.
- (4) Add the class instance, `class_0`, to the configuration, `cfg_0`. `sl_usbd_XXXX_add_to_configuration()` will create the interface 0, interface 1, alternate interfaces, and the associated IN and OUT endpoints. The class instance number, `class_0`, will be used for any data communication on interface 0 or interface 1.
- (5) Add the class instance, `class_1`, to the configuration, `cfg_0`. `sl_usbd_XXXX_add_to_configuration()` will create the interface 2, interface 3 and their associated IN and OUT endpoints. The class instance number, `class_1`, will be used for any data communication on interface 2 or interface 3.
- (6) Add the same class instances, `class_0` and `class_1`, to the other configuration, `cfg_1`.

Every class defines structure of type `sl_usbd_XXXX_callbacks_t`. Its purpose is to give every class a set of callback functions to be called when event occurs. Two callback functions are present in each class. They are presented in the table bellow.

Table - Common Class Callback Functions

Fields	Description	Function Signature
.enable	Called when the USB class instance is enabled successfully.	void app_usbd_XXXX_enable(uint8_t class_nbr);
.disable	Called when the USB class instance is disabled.	void app_usbd_XXXX_disable(uint8_t class_nbr);

Overview

USB Device CDC ACM Class

- [USB Device CDC Base Class Overview](#)
- [USB Device CDC ACM Class Resource Needs from Core](#)
- [USB Device CDC ACM Subclass Overview](#)
- [USB Device CDC ACM Class Configuration](#)
- [USB Device CDC ACM Class Programming Guide](#)

This section describes the Communications Device Class (CDC) class and the associated CDC subclass supported by Silicon Labs' USB Device stack. Silicon Labs USB-Device currently supports the Abstract Control Model (ACM) subclass, which is commonly used for serial emulation.

CDC includes various telecommunication and networking devices. Telecommunication devices encompass analog modems, analog and digital telephones, ISDN terminal adapters, etc. For example, networking devices contain ADSL and cable modems, Ethernet adapters, and hubs. CDC defines a framework to encapsulate existing communication services standards, such as V.250 (for modems over telephone network) and Ethernet (for local area network devices), using a USB link. A communication device is in charge of device management, call management when needed, and data transmission.

CDC defines seven major groups of devices. Each group belongs to a model of communication, which may include several subclasses. Each group of devices has its own specification document besides the CDC base class. The seven groups are:

- Public Switched Telephone Network (PSTN), devices including voiceband modems, telephones, and serial emulation devices.
- Integrated Services Digital Network (ISDN) devices, including terminal adaptors and telephones.
- Ethernet Control Model (ECM) devices, including devices supporting the IEEE 802 family (ex.: cable and ADSL modems, WiFi adapters).
- Asynchronous Transfer Mode (ATM) devices, including ADSL modems and other devices connected to ATM networks (workstations, routers, LAN switches).
- Wireless Mobile Communications (WMC) devices, including multi-function communications handset devices used to manage voice and data communications.
- Ethernet Emulation Model (EEM) devices which exchange Ethernet-framed data.
- Network Control Model (NCM) devices, including high-speed network devices (High Speed Packet Access modems, Line Terminal Equipment)

The CDC and the associated subclass implementation complies with the following specifications:

- *Universal Serial Bus, Class Definitions for Communications Devices, Revision 1.2*, November 3 2010.
- *Universal Serial Bus, Communications, Subclass for PSTN Devices, Revision 1.2*, February 9 2007.

USB Device CDC Base Class Overview

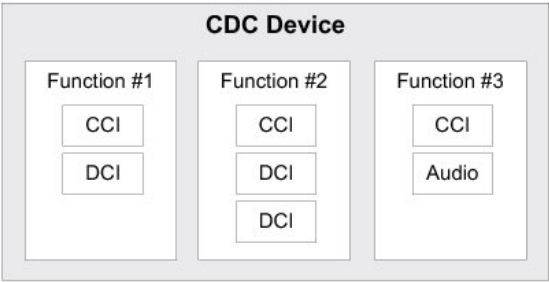
A CDC device is composed of the following interfaces to implement communication capability:

- **Communications Class Interface (CCI)** is responsible for the device management and optionally the call management. The device management enables the general configuration and control of the device and the notification of events to the host. The call management enables calls establishment and termination. Call management might be multiplexed through a DCI. A CCI is mandatory for all CDC devices. It identifies the CDC function by specifying the communication model supported by the CDC device. The interface(s) following the CCI can be any defined USB class interface, such as Audio or a vendor-specific interface. The vendor-specific interface is represented specifically by a DCI.
- **Data Class Interface (DCI)** is responsible for data transmission. Data transmitted and/or received do not follow a specific format. Data could be raw data from a communication line, data following a proprietary format, etc. All the DCIs following the CCI can be seen as subordinate interfaces.

A CDC device must have at least one CCI and zero or more DCIs. One CCI and any subordinate DCI together provide a feature to the host. This capability is also referred to as a function. In a CDC composite device, you could have several

functions. Therefore, the device would be composed of several sets of CCI and DCI(s) as shown in [Figure - CDC Composite Device](#).

Figure - CDC Composite Device



A CDC device is likely to use the following combination of endpoints:

- A pair of control IN and OUT endpoints called the default endpoint.
- An optional bulk or interrupt IN endpoint.
- A pair of bulk or isochronous IN and OUT endpoints. Note that Silicon Labs USB device stack does not currently support isochronous endpoints.

The table below shows the usage of the different endpoints and by which interface of the CDC they are used.

Table - CDC Endpoint Usage

Endpoint	Direction	Interface	Usage
Control IN	Device-to-host	CCI	Standard requests for enumeration, class-specific requests, device management, and optionally call management.
Control OUT	Host-to-device	CCI	Standard requests for enumeration, class-specific requests, device management, and optionally call management.
Interrupt or bulk IN	Device-to-host	CCI	Events notification, such as ring detect, serial line status, network status.
Bulk or isochronous IN	Device-to-host	DCI	Raw or formatted data communication.
Bulk or isochronous OUT	Host-to-device	DCI	Raw or formatted data communication.

Most communication devices use an interrupt endpoint to notify the host of events. Isochronous endpoints should not be used for data transmission when a proprietary protocol relies on data retransmission in case of USB protocol errors. Isochronous communication can inherently lose data since it has no retry mechanisms.

The seven major models of communication encompass several subclasses. A subclass describes the way the device should use the CCI to handle the device management and call management. The table below shows all the possible subclasses and the communication model they belong to.

Table - CDC Subclasses

Subclass	Communication Model	Example of Devices Using This Subclass
Direct Line Control Model	PSTN	Modem devices directly controlled by the USB host

Subclass	Communication Model	Example of Devices Using This Subclass
Abstract Control Model	PSTN	Serial emulation devices, modem devices controlled through a serial command set
Telephone Control Model	PSTN	Voice telephony devices
Multi-Channel Control Model	ISDN	Basic rate terminal adaptors, primary rate terminal adaptors, telephones
CAPI Control Model	ISDN	Basic rate terminal adaptors, primary rate terminal adaptors, telephones
Ethernet Networking Control Model	ECM	DOC-SIS cable modems, ADSL modems that support PPPoE emulation, Wi-Fi adaptors (IEEE 802.11-family), IEEE 802.3 adaptors
ATM Networking Control Model	ATM	ADSL modems
Wireless Handset Control Model	WMC	Mobile terminal equipment connecting to wireless devices
Device Management	WMC	Mobile terminal equipment connecting to wireless devices
Mobile Direct Line Model	WMC	Mobile terminal equipment connecting to wireless devices
OBEX	WMC	Mobile terminal equipment connecting to wireless devices
Ethernet Emulation Model	EEM	Devices using Ethernet frames as the next layer of transport. Not intended for routing and Internet connectivity devices
Network Control Model	NCM	IEEE 802.3 adaptors carrying high-speed data bandwidth on network

USB Device CDC ACM Class Resource Needs from Core

Each time you add a CDC ACM class instance to a USB configuration via a call to the function `slUsbd_cdc_acm_add_to_configuration()`, the following resources will be allocated from the core.

Resource	Quantity
Interfaces	2
Alternate interfaces	2
Endpoints	3
Interface groups	1

Note that those numbers are per configuration. When setting up your `SL_USBD_INTERFACE_QUANTITY`, `SL_USBD_ALT_INTERFACE_QUANTITY`, `SL_USBD_INTERFACE_GROUP_QUANTITY` and `SL_USBD_DESCRIPTOR_QUANTITY` configuration values, don't forget to take into account on how many configurations the class will be added. For the `SL_USBD_OPEN_ENDPOINTS_QUANTITY` configuration value, since endpoints are opened only when a configuration is set by the host, you just need to take into account the number of needed endpoints for a class instance.

USB Device CDC ACM Subclass Overview

The CDC base class is composed of a Communications Class Interface (CCI) and Data Class Interface (DCI), which is discussed in detail in [USB Device CDC Base Class Overview](#). This section discusses a CCI of type ACM. It consists of a default endpoint for the management element and an interrupt endpoint for the notification element. A pair of bulk endpoints is used to carry unspecified data over the DCI.

The ACM subclass is used by two types of communication devices:

- Devices supporting AT commands (for instance, voiceband modems).
- Serial emulation devices which are also called Virtual COM port devices.

There are several subclass-specific requests for the ACM subclass. They allow you to control and configure the device. The complete list and description of all ACM requests can be found in the specification “*Universal Serial Bus, Communications, Subclass for PSTN Devices, revision 1.2, February 9, 2007*”, section 6.2.2.

From this list, Silicon Labs’ ACM subclass supports the following:

Table - ACM Requests Supported by Silicon Labs

Subclass Request	Description
SetCommFeature	The host sends this request to control the settings for a given communications feature. Not used for serial emulation.
GetCommFeature	The host sends this request to get the current settings for a given communications feature. Not used for serial emulation.
ClearCommFeature	The host sends this request to clear the settings for a given communications feature. Not used for serial emulation.
SetLineCoding	The host sends this request to configure the ACM device settings: baud rate, number of stop bits, parity type and number of data bits. For a serial emulation, this request is sent automatically by a serial terminal each time you configure the serial settings for an open virtual COM port.
GetLineCoding	The host sends this request to get the current ACM settings (baud rate, stop bits, parity, data bits). For a serial emulation, serial terminals send this request automatically during virtual COM port opening.
SetControlLineState	The host sends this request to control the carrier for half-duplex modems and indicate that Data Terminal Equipment (DTE) is ready or not. In the serial emulation case, the DTE is a serial terminal. For a serial emulation, certain serial terminals allow you to send this request with the controls set.
SetBreak	The host sends this request to generate an RS-232 style break. For a serial emulation, certain serial terminals allow you to send this request.

Silicon Labs’ ACM subclass uses the interrupt IN endpoint to notify the host about the current *serial line state*. The serial line state is a bitmap informing the host about:

- Data discarded because of overrun
- Parity error
- Framing error
- State of the ring signal detection
- State of break detection mechanism
- State of transmission carrier
- State of receiver carrier detection

Silicon Labs’ ACM subclass implementation complies with the following specification:

- *Universal Serial Bus, Communications, Subclass for PSTN Devices, revision 1.2, February 9, 2007.*

USB Device CDC ACM Class Configuration

This section discusses how to configure the CDC ACM Class (Communication Device Class, Abstract Control Model). There are two groups of configuration parameters:

- [USB Device CDC ACM Class Application Specific Configurations](#)
- [USB Device CDC ACM Class Instance Configurations](#)

USB Device CDC ACM Class Application Specific Configurations

- [CDC Base Class](#)
- [ACM Subclass](#)

CDC Base Class

First, to use the Silicon Labs USB device CDC class module, you will need to adjust the CDC compile-time configuration #define-s according to your application needs. They are regrouped inside the `sl_usbd_core_config.h` header file under the CDC section. Their purpose is to inform the USB device module about how many USB CDC objects to allocate.

The table below describes each configuration field available in this configuration structure.

Table - USB Device CDC Configuration Defines

Configuration Name	Description	Default Value
SL_USBD_CDC_CLASS_INSTANCE_QUANTITY	Number of class instances you will allocate via a call to the function <code>sl_usbd_cdc_acm_create_instance()</code> .	2
SL_USBD_CDC_CONFIGURATION_QUANTITY	Number of configurations. ACM class instances can be added to one or more configurations via a call to the <code>sl_usbd_cdc_acm_add_to_configuration()</code> .	1
SL_USBD_CDC_DATA_INTERFACE_QUANTITY	Total number of data interfaces (DCI) for all the CDC functions. Each CDC ACM function added via a call to the function <code>sl_usbd_cdc_acm_create_instance()</code> will add a data interface.	2

ACM Subclass

The ACM subclass has one compile-time configuration shown in the table below.

Table - USB Device CDC ACM Configuration Define

Configuration Name	Description	Default Value
SL_USBD_CDC_ACM_SUBCLASS_INSTANCE_QUANTITY	Configures the number of subclass instances you will allocate via a call to the function <code>sl_usbd_cdc_acm_create_instance()</code> .	2

USB Device CDC ACM Class Instance Configurations

This section defines the configurations related to the CDC ACM serial class instances.

- [Class Instance Creation](#)
 - [line state interval](#)
 - [call mgmt capabilities](#)
 - [p_acm_callbacks](#)

Class Instance Creation

To create a CDC ACM serial class instance, call the function `sl_usbd_cdc_acm_create_instance()` . This function takes three configuration arguments, as described here.

`line_state_interval`

This is the interval (in milliseconds) that your CDC ACM serial class instance will report the line state notifications to the host. This value must be a power of two (1, 2, 4, 8, 16, etc).

`call_mgmt_capabilities`

Call Management Capabilities bitmap. Possible values of the bitmap are as follows:

Value (bit)	Description
SL_USBD_ACM_SERIAL_CALL_MGMT_DEV	Device handles call management itself.
SL_USBD_ACM_SERIAL_CALL_MGMT_DATA_CCI_DCI	Device can send/receive call management information over a Data Class interface.

p_acm_callbacks

`p_acm_callbacks` is a pointer to a structure of type `slUsbd_cdc_acm_callbacks_t`. Its purpose is to give the CDC ACM Class a set of callback functions to be called when a CDC ACM event occurs.

Not all callbacks are mandatory and a null pointer (`NULL`) can be passed in the callbacks structure variable when the callback is not needed.

The table below describes each configuration field available in this configuration structure.

Table - **sl_usbd_cdc_acm_callbacks_t** Configuration Structure

Fields	Description	Function Signature
.enable	Called when the USB class instance is enabled successfully.	<code>void app_usbd_cdc_acm_enable(uint8_t subclass_nbr);</code>
.disable	Called when the USB class instance is disabled.	<code>void app_usbd_cdc_acm_disable(uint8_t subclass_nbr);</code>
.line_control_changed	Called when a line control change is received.	<code>void app_usbd_cdc_acm_line_control_changed(uint8_t subclass_nbr, uint8_t event, uint8_t event_chngd);</code>
line_coding_changed	Called when a line coding change is received.	<code>bool app_usbd_cdc_acm_line_coding_changed(uint8_t subclass_nbr, subclass_nbr, sl_usbd_cdc_acm_line_coding_t *p_line_coding</code>

See section [Registering Event Notification Callbacks](#) for callback functions example.

USB Device CDC ACM Class Programming Guide

This section explains how to use the CDC Abstract Control Model class.

- [Initializing the USB Device CDC ACM Class](#)
- [Adding a USB Device CDC ACM Class Instance to Your Device](#)
- [Communicating Using the CDC ACM Class](#)

Initializing the USB Device CDC ACM Class

To add CDC ACM class functionality to your device, you must first initialize the CDC base class and the ACM subclass by calling the functions `slUsbd_cdc_init()` and `sl_usbd_cdc_acm_init()`.

The example below shows how to call `slUsbd_cdc_init()` and `sl_usbd_cdc_acm_init()` using default arguments.

Example - Initialization of CDC ACM Class

```
sl_status_t status;

status = slUsbd_cdc_init();
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}

status = sl_usbd_cdc_acm_init();
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Adding a USB Device CDC ACM Class Instance to Your Device

To add CDC ACM class functionality to your device, you must create an instance, then add it to your device's configuration(s).

- [Creating a CDC ACM Class Instance](#)
- [Adding the CDC ACM Class Instance to Your Device's Configuration\(s\)](#)
- [Registering Event Notification Callbacks](#)

Creating a CDC ACM Class Instance

Create a CDC ACM class instance by calling the function `sl_usbd_cdc_acm_create_instance()`.

The example below shows how to create a CDC ACM class instance via `sl_usbd_cdc_acm_create_instance()`.

Example - Creating a CDC ACM Function via `sl_usbd_cdc_acm_create_instance()`

```
uint8_t    subclass_nbr;
sl_status_t status;

status = sl_usbd_cdc_acm_create_instance(64u,
                                          SL_USBD_ACM_SERIAL_CALL_MGMT_DATA_CCLDCI | SL_USBD_ACM_SERIAL_CALL_MGMT_DEV,
                                          NULL,
                                          &subclass_nbr);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Adding the CDC ACM Class Instance to Your Device's Configuration(s)

After you have created a CDC ACM class instance, you can add it to a configuration by calling the function `sl_usbd_cdc_acm_add_to_configuration()`.

The example below shows how to call `sl_usbd_cdc_acm_add_to_configuration()`.

Example - Call to USB ACM `sl_usbd_cdc_acm_add_to_configuration()`

```
sl_status_t status;

status = sl_usbd_cdc_acm_add_to_configuration(subclass_nbr,           (1)
                                              config_nbr_fs);         (2)
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

(1) Class number to add to the configuration returned by `sl_usbd_cdc_acm_create_instance()`.

(2) Configuration number (here adding it to a Full-Speed configuration).

Registering Event Notification Callbacks

The CDC ACM Serial class can notify your application of any changes in line control or coding via notification callback functions. A callback functions structure can be passed as argument during the ACM instance creation. Note that those callbacks are optional. [Example - CDC ACM Callbacks Registration](#) illustrates the use of the callback registration functions. [Example - CDC ACM Callbacks Implementation](#) shows an example of implementation of the callback functions.

Example - CDC ACM Callbacks Registration

```
uint8_t    subclass_nbr;
sl_status_t status;

sl_usbd_cdc_acm_callbacks_t sli_usbd_cdc_acm_callbacks = {
    app_usbd_cdc_acm_connect,
    app_usbd_cdc_acm_disconnect,
    app_usbd_cdc_acm_line_control_changed,
    app_usbd_cdc_acm_line_coding_changed,
};

status = sl_usbd_cdc_acm_create_instance(64u,
                                         SL_USBD_ACM_SERIAL_CALL_MGMT_DATA_CCLDCI | SL_USBD_ACM_SERIAL_CALL_MGMT_DEV,
                                         &sli_usbd_cdc_acm_callbacks,
                                         &subclass_nbr);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Example - CDC ACM Callbacks Implementation

```
bool app_usbd_cdc_acm_line_coding_changed (uint8_t          subclass_nbr,
                                           sl_usbd_cdc_acm_line_coding_t *p_line_coding)
{
    uint32_t baudrate_new;
    uint8_t  parity_new;
    uint8_t  stop_bits_new;
    uint8_t  data_bits_new;

    /* TODO: Apply new line coding.*/
    baudrate_new = p_line_coding->BaudRate;
    parity_new   = p_line_coding->Parity;
    stop_bits_new = p_line_coding->StopBits;
    data_bits_new = p_line_coding->DataBits;

    return (true);           (1)
}

void app_usbd_cdc_acm_line_control_changed (uint8_t  subclass_nbr,
                                           uint8_t  event,
                                           uint8_t  event_changed)
{
    bool rts_state;
    bool rts_state_changed;
    bool dtr_state;
    bool dtr_state_changed;
    bool brk_state;
    bool brk_state_changed;

    /* TODO: Apply new line control. */
    rts_state      = ((event & SL_USBD_CDC_ACM_CTRL_RTS) != 0) ? true : false;
    rts_state_changed = ((event_changed & SL_USBD_CDC_ACM_CTRL_RTS) != 0) ? true : false;
    dtr_state      = ((event & SL_USBD_CDC_ACM_CTRL_DTR) != 0) ? true : false;
    dtr_state_changed = ((event_changed & SL_USBD_CDC_ACM_CTRL_DTR) != 0) ? true : false;
    brk_state      = ((event & SL_USBD_CDC_ACM_CTRL_BREAK) != 0) ? true : false;
    brk_state_changed = ((event_changed & SL_USBD_CDC_ACM_CTRL_BREAK) != 0) ? true : false;
}
```

(1) It is important to return false to this function if the line coding applying failed. Otherwise, return true.

Communicating Using the CDC ACM Class

- [Serial Status](#)
 - [Line Coding](#)
 - [Line Control](#)

- [Line State](#)
- [Subclass Instance Communication](#)

Serial Status

Line Coding

The USB host controls the line coding (baud rate, parity, etc) of the CDC ACM device. When necessary, the application is responsible for setting the line coding.

There are two functions provided to retrieve and set the current line coding, as described in the table below.

Table - CDC ACM Line Coding Functions

Function	Description
<code>slUsbd_cdc_acm_get_line_coding()</code>	Your application can get the current line coding settings set either from the host with <code>SetLineCoding</code> requests or with the function <code>slUsbd_cdc_acm_set_line_coding()</code> .
<code>slUsbd_cdc_acm_set_line_coding()</code>	Your application can set the line coding. The host can retrieve the settings with the <code>GetLineCoding</code> request.

Line Control

The USB host controls the line control (RTS and DTR pins, break signal, and so on) of the CDC ACM device. When necessary, your application is responsible for applying the line controls.

A function is provided to retrieve and set the current line controls, as are described in the table below.

Table - CDC ACM Line Control Functions

Function	
<code>slUsbd_cdc_acm_get_line_control_state()</code>	Your application can get the current control line state set by the host with the <code>SetControlLineState</code> request.

Line State

The USB host retrieves the line state at a regular interval. Your application must update the line state each time it changes. When necessary, your application is responsible for setting the line state.

Two functions are provided to retrieve and set the current line controls, as described in the table below.

Table - CDC ACM Line State Functions

Function	
<code>slUsbd_cdc_acm_set_line_state_event()</code>	Your application can set any line state event(s). While setting the line state, an interrupt IN transfer is sent to the host to inform about it a change in the serial line state.
<code>slUsbd_cdc_acm_clear_line_state_event()</code>	Application can clear two events of the line state: transmission carrier and receiver carrier detection. All the other events are self-cleared by the ACM serial emulation subclass.

Subclass Instance Communication

Silicon Labs' ACM subclass offers the following functions to communicate with the host. For more details about the functions' parameters, see the [CDC ACM Subclass Functions](#) reference.

Function name	Operation
<code>slUsbd_cdc_acm_read()</code>	Receives data from host through a bulk OUT endpoint. This function is blocking.
<code>slUsbd_cdc_acm_write()</code>	Sends data to host through a bulk IN endpoint. This function is blocking.

Table - CDC ACM Communication API Summary

`sl_usbd_cdc_acm_read()` and `sl_usbd_cdc_acm_write()` provide synchronous communication, which means that the transfer is blocking. In other words, upon calling the function, the application blocks until the transfer is complete with or without an error. A timeout can be specified to avoid waiting forever. The example below shows a read and write example that receives data from the host using the bulk OUT endpoint and sends data to the host using the bulk IN endpoint.

Listing - Serial Read and Write Example

```

__ALIGNED(4) uint8_t rx_buf[2];
__ALIGNED(4) uint8_t tx_buf[2];
uint32_t          xfer_len;
sl_status_t       status;

status = sl_usbd_cdc_acm_read(subclass_nbr,           (1)
                               rx_buf,                 (2)
                               2u,                     (3)
                               0u,
                               &xfer_len);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}

status = sl_usbd_cdc_acm_write(subclass_nbr,          (1)
                                tx_buf,                (4)
                                2u,
                                0u,                   (3)
                                &xfer_len);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}

```

(1) The class instance number created with `sl_usbd_cdc_acm_create_instance()` provides an internal reference to the ACM subclass to route the transfer to the proper bulk OUT or IN endpoint.

(2) Your application must ensure that the buffer provided to the function is large enough to accommodate all the data. Otherwise, synchronization issues might happen.

(3) To avoid an infinite blocking situation, specify a timeout expressed in milliseconds. A value of '0' makes the application task wait forever.

(4) The application provides the initialized transmit buffer.

Overview

USB Device HID Class

- [USB Device HID Class Overview](#)
- [USB Device HID Class Resource Needs from Core](#)
- [USB Device HID Class Configuration](#)
- [USB Device HID Class Programming Guide](#)
- [HID Periodic Input Reports Task](#)

This section describes the Human Interface Device (HID) class supported by Silicon Labs USB Device.

The HID class encompasses devices used by humans to control computer operations, such as keyboards, mice, pointing devices, and game devices.

The HID class can also be used in a composite device that contains controls such as knobs, switches, buttons, and sliders. For example, mute and volume controls in an audio headset are controlled by the HID function of the headset. HID class can exchange data for any purpose using only control and interrupt transfers.

The HID class is one of the oldest and most widely-used USB classes. All the major host operating systems provide a native driver to manage HID devices, which is why a variety of vendor-specific devices work with the HID class. This class also includes various types of output items such as LEDs, audio, tactile feedback, etc.

The HID implementation complies with the following specifications:

- *Device Class Definition for Human Interface Devices (HID), 6/27/01, Version 1.11.*
- *Universal Serial Bus HID Usage Tables, 10/28/2004, Version 1.12.*

USB Device HID Class Overview

Overview

A HID device is composed of the following endpoints:

- A pair of control IN and OUT endpoints called the default endpoint
- An interrupt IN endpoint
- An optional interrupt OUT endpoint

The table below describes the usage of the different endpoints:

Table - HID Class Endpoints Usage

Endpoint	Direction	Usage
Control IN	Device-to-host	Standard requests for enumeration, class-specific requests, and data communication (Input, Feature reports sent to the host with GET_REPORT request).
Control OUT	Host-to-device	Standard requests for enumeration, class-specific requests and data communication (Output, Feature reports received from the host with SET_REPORT request).
Interrupt IN	Device-to-host	Data communication (Input and Feature reports).
Interrupt OUT	Host-to-device	Data communication (Output and Feature reports).

Report

A host and a HID device exchange data using reports. A report contains formatted data giving information about controls and other physical entities of the HID device. A control is manipulable by the user and operates an aspect of the device. For example, a control can be a button on a mouse or a keyboard, a switch, etc. Other entities inform the user about the state of certain device's features. For example, LEDs on a keyboard notify the user about the caps lock on, the numeric keypad active, etc.

The format and the use of a report data is understood by the host by analyzing the content of a *Report descriptor*. Analyzing the content is done by a parser. The Report descriptor describes data provided by each control in a device. It is composed of *items* which are pieces of information about the device and consist of a 1-byte prefix and variable-length data. For more details about the item format, refer to “*Device Class Definition for Human Interface Devices (HID) Version 1.11*”, section 5.6 and 6.2.2.

There are three principal types of items:

- *Main item* defines or groups certain types of data fields.
- *Global item* describes data characteristics of a control.
- *Local item* describes data characteristics of a control.

Each item type is defined by different functions. An item function can also be called a tag. An item function can be seen as a sub-item that belongs to one of the three principal item types. The table below provides a brief overview of the item's functions in each item type. For a complete description of the items in each category, see “*Device Class Definition for Human Interface Devices (HID) Version 1.11*”, section 6.2.2.

Table - Item's Function Description for Each Item Type

Item Type	Item Function	Description
Main	Input	Describes information about the data provided by one or more physical controls.
Main	Output	Describes data sent to the device.
Main	Feature	Describes device configuration information sent to or received from the device which influences the overall behavior of the device or one of its components.
Main	Collection	Group related items (Input, Output or Feature).
Main	End of Collection	Closes a collection.

Item Type	Item Function	Description
Global	Usage Page	Identifies a function available within the device.
Global	Logical Minimum	Defines the lower limit of the reported values in logical units.
Global	Logical Maximum	Defines the upper limit of the reported values in logical units.
Global	Physical Minimum	Defines the lower limit of the reported values in physical units, that is the Logical Minimum expressed in physical units.
Global	Physical Maximum	Defines the upper limit of the reported values in physical units, that is the Logical Maximum expressed in physical units.
Global	Unit Exponent	Indicates the unit exponent in base 10. The exponent ranges from -8 to +7.
Global	Unit	Indicates the unit of the reported values. For instance, length, mass, temperature units, etc.
Global	Report Size	Indicates the size of the report fields in bits.
Global	Report ID	Indicates the prefix added to a particular report.
Global	Report Count	Indicates the number of data fields for an item.
Global	Push	Places a copy of the global item state table on the CPU stack.
Global	Pop	Replaces the item state table with the last structure from the stack.
Local	Usage	Represents an index to designate a specific Usage within a Usage Page. It indicates the vendor's suggested use for a specific control or group of controls. A usage supplies information to an application developer about what a control is actually measuring.
Local	Usage Minimum	Defines the starting usage associated with an array or bitmap.
Local	Usage Maximum	Defines the ending usage associated with an array or bitmap.
Local	Designator Index	Determines the body part used for a control. Index points to a designator in the Physical descriptor.
Local	Designator Minimum	Defines the index of the starting designator associated with an array or bitmap.
Local	Designator Maximum	Defines the index of the ending designator associated with an array or bitmap.
Local	String Index	String index for a String descriptor. It allows a string to be associated with a particular item or control.
Local	String Minimum	Specifies the first string index when assigning a group of sequential strings to controls in an array or bitmap.
Local	String Maximum	Specifies the last string index when assigning a group of sequential strings to controls in an array or bitmap.
Local	Delimiter	Defines the beginning or end of a set of local items.

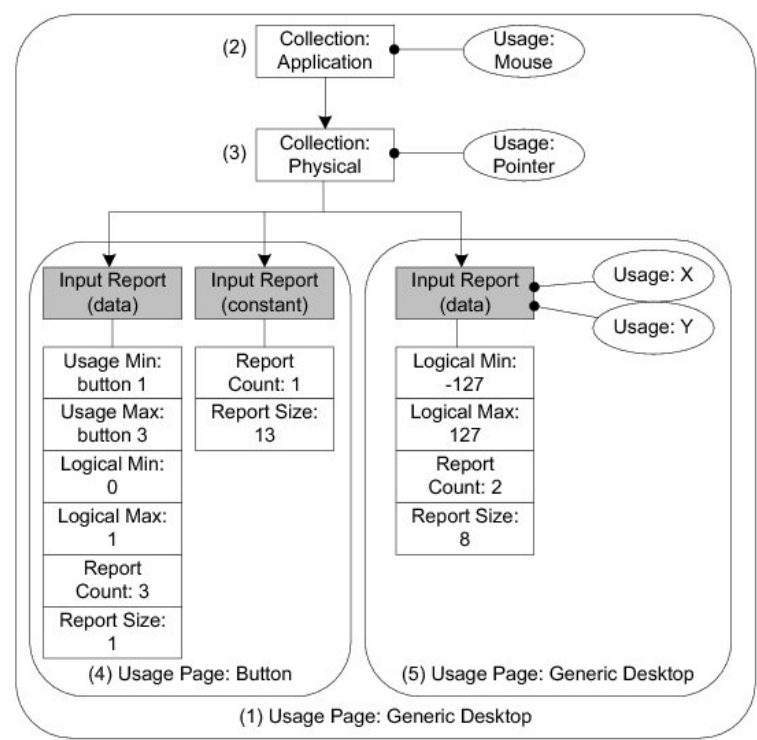
A control's data must define at least the following items:

- Input, Output or Feature Main items
- Usage Local item
- Usage Page Global item
- Logical Minimum Global item
- Logical Maximum Global item
- Report Size Global item

Report Count Global item

The table below shows the representation of a Mouse Report descriptor content from a host HID parser perspective. The mouse has three buttons (left, right, and wheel). The code presented in the example below is a code implementation corresponding to this mouse Report descriptor representation.

Figure - Report Descriptor Content from a Host HID Parser View



- (1) The *Usage Page* item function specifies the general function of the device. In this example, the HID device belongs to a generic desktop control.
- (2) The *Collection Application* groups Main items that have a common purpose and may be familiar to applications. In the diagram, the group is composed of three Input Main items. For this collection, the suggested use for the controls is a mouse as indicated by the *Usage* item.
- (3) Nested collections may be used to give more details about the use of a single control or group of controls to applications. In this example, the *Collection Physical*, nested into the *Collection Application*, is composed of the same three Input items forming the *Collection Application*. The *Collection Physical* is used for a set of data items that represent data points collected at one geometric point. In the example, the suggested use is a pointer as indicated by the *Usage* item. Here the pointer usage refers to the mouse position coordinates and the system software will translate the mouse coordinates in movement of the screen cursor.
- (4) Nested usage pages are also possible and give more details about a certain aspect within the general function of the device. In this case, two Inputs items are grouped and correspond to the buttons of the mouse. One Input item defines the three buttons of the mouse (right, left and wheel) in terms of number of data fields for the item (*Report Count* item), size of a data field (*Report Size* item) and possible values for each data field (*Usage Minimum* and *Maximum*, *Logical Minimum* and *Maximum* items). The other Input item is a 13-bit constant allowing the Input report data to be aligned on a byte boundary. This Input item is used only for padding purpose.
- (5) Another nested usage page referring to a generic desktop control is defined for the mouse position coordinates. For this usage page, the Input item describes the data fields corresponding to the x- and y-axis as specified by the two *Usage* items.

After analyzing the previous mouse Report descriptor content, the host's HID parser is able to interpret the Input report data sent by the device with an interrupt IN transfer or in response to a GET_REPORT request. The Input report data corresponding to the mouse Report descriptor shown in [Figure - Report Descriptor Content from a Host HID Parser View](#) is

shown in the table below. The total size of the report data is 4 bytes. Different types of reports may be sent over the same endpoint. For the purpose of distinguishing the different types of reports, a 1-byte report ID prefix is added to the data report. If a report ID was used in the example of the mouse report, the total size of the report data would be 5 bytes.

Table - Input Report Sent to Host and Corresponding to the State of a 3-Buttons Mouse

Bit Offset	Bit Count	Description
0	1	Button 1 (left button).
1	1	Button 2 (right button).
2	1	Button 3 (wheel button).
3	13	Not used.
16	8	Position on axis X.
24	8	Position on axis Y.

A Physical descriptor indicates the part or parts of the body intended to activate a control or controls. An application may use this information to assign a functionality to the control of a device. A Physical descriptor is an optional class-specific descriptor and most devices have little gain for using it. Refer to “Device Class Definition for Human Interface Devices (HID) Version 1.11” section 6.2.3 for more details about this descriptor.

USB Device HID Class Resource Needs from Core

Each time you add a HID class instance to a USB configuration via a call to the function `sl_usbd_hid_add_to_configuration()` , the following resources will be allocated from the core.

Resource	Quantity
Interfaces	1
Alternate interfaces	1
Endpoints	1 (2 if interrupt OUT endpoint is enabled)
Interface groups	0

Note that those numbers are per configuration. When setting up your `SL_USBD_INTERFACE_QUANTITY` , `SL_USBD_ALT_INTERFACE_QUANTITY` , `SL_USBD_INTERFACE_GROUP_QUANTITY` and `SL_USBD_DESCRIPTOR_QUANTITY` configuration values, don't forget to take into account on how many configurations the class will be added. For the `SL_USBD_OPEN_ENDPOINTS_QUANTITY` configuration value, since endpoints are opened only when a configuration is set by the host, you just need to take into account the number of needed endpoints for a class instance.

USB Device HID Class Configuration

Two groups of configuration parameters are used to configure the HID class:

- [USB Device HID Class Application-Specific Configurations](#)
- [USB Device HID Class Instance Configurations](#)

USB Device HID Class Application-Specific Configurations

First, to use the Silicon Labs USB device HID class module, adjust the HID compile-time configuration defines according to your application needs. They are regrouped inside the `sl_usbd_core_config.h` header file under the HID section. They can be divided into two sections, the quantity configurations and the HID task configurations. The quantity configurations purpose is to inform the USB device module about how many USB HID objects to allocate.

The table below describes each configuration define.

Table - USB Device HID Configuration Defines

Configuration Name	Description	Default Value
SL_USBD_HID_CLASS_INSTANCE_QUANTITY	Number of class instances you will allocate via a call to the function <code>slUsbd_hid_create_instance()</code> .	2
SL_USBD_HID_CONFIGURATION_QUANTITY	Number of configurations. HID class instances can be added to one or more configurations via a call to the function <code>slUsbd_hid_add_to_configuration()</code> .	1
SL_USBD_HID_REPORT_ID_QUANTITY	Configures the total number of report IDs to allocate.	2
SL_USBD_HID_PUSH_POP_ITEM_QUANTITY	Configures the total number of Push/Pop items to allocate.	0
SL_USBD_HID_TIMER_TASK_STACK_SIZE	The timer task handles all the timer-based HID operations. This configuration allows you to set the stack size (in number of bytes).	2048
SL_USBD_HID_TIMER_TASK_PRIORITY	Priority of HID task. his is a CMSIS-RTOS2 priority .	osPriorityHigh

USB Device HID Class Instance Configurations

- [Class Instance Creation](#)
 - [subclass](#)
 - [protocol](#)
 - [country_code](#)
 - [interval_in](#) and [interval_out](#)
 - [p_hid_callback](#)
- [HID Class Report Descriptor Example](#)

This section defines the configurations related to the HID class instances.

Class Instance Creation

Creating a HID class instance is done by calling the function `slUsbd_hid_create_instance()` , which takes several configuration arguments that are described below.

subclass

Code of the HID subclass. Possible values are:

- `SL_USBD_HID_SUBCLASS_NONE`
- `SL_USBD_HID_SUBCLASS_BOOT`

A HID device that uses the boot subclass must use standard report formats. For more information on the subclass codes, see section 4.2 of HID specification revision 1.11.

protocol

Protocol used by the HID device. Possible values are:

- `SL_USBD_HID_PROTOCOL_NONE`
- `SL_USBD_HID_PROTOCOL_KBD`
- `SL_USBD_HID_PROTOCOL_MOUSE`

If your HID function is a mouse, the protocol should be set to `SL_USBD_HID_PROTOCOL_MOUSE` . If it is a keyboard, it should be set to `SL_USBD_HID_PROTOCOL_KBD` . Otherwise, the protocol should be set to `SL_USBD_HID_PROTOCOL_NONE` . For more information on the subclass codes, see section 4.3 of HID specification revision 1.11.

country_code

ID of the country code. Possible values are:

- `SL_USBD_HID_COUNTRY_CODE_NOT_SUPPORTED`
- `SL_USBD_HID_COUNTRY_CODE_ARABIC`

- SL_USBD_HID_COUNTRY_CODE_BELGIAN
- SL_USBD_HID_COUNTRY_CODE_CANADIAN_BILINGUAL
- SL_USBD_HID_COUNTRY_CODE_CANADIAN_FRENCH
- SL_USBD_HID_COUNTRY_CODE_CZECH_REPUBLIC
- SL_USBD_HID_COUNTRY_CODE_DANISH
- SL_USBD_HID_COUNTRY_CODE_FINNISH
- SL_USBD_HID_COUNTRY_CODE_FRENCH
- SL_USBD_HID_COUNTRY_CODE_GERMAN
- SL_USBD_HID_COUNTRY_CODE_GREEK
- SL_USBD_HID_COUNTRY_CODE_HEBREW
- SL_USBD_HID_COUNTRY_CODE_HUNGARY
- SL_USBD_HID_COUNTRY_CODE_INTERNATIONAL
- SL_USBD_HID_COUNTRY_CODE_ITALIAN
- SL_USBD_HID_COUNTRY_CODE_JAPAN_KATAKANA
- SL_USBD_HID_COUNTRY_CODE_KOREAN
- SL_USBD_HID_COUNTRY_CODE_LATIN_AMERICAN
- SL_USBD_HID_COUNTRY_CODE_NETHERLANDS_DUTCH
- SL_USBD_HID_COUNTRY_CODE_NORWEGIAN
- SL_USBD_HID_COUNTRY_CODE_PERSIAN_FARSI
- SL_USBD_HID_COUNTRY_CODE_POLAND
- SL_USBD_HID_COUNTRY_CODE_PORTUGUESE
- SL_USBD_HID_COUNTRY_CODE_RUSSIA
- SL_USBD_HID_COUNTRY_CODE_SLOVAKIA
- SL_USBD_HID_COUNTRY_CODE_SPANISH
- SL_USBD_HID_COUNTRY_CODE_SWEDISH
- SL_USBD_HID_COUNTRY_CODE_SWISS_FRENCH
- SL_USBD_HID_COUNTRY_CODE_SWISS_GERMAN
- SL_USBD_HID_COUNTRY_CODE_SWITZERLAND
- SL_USBD_HID_COUNTRY_CODE_TAIWAN
- SL_USBD_HID_COUNTRY_CODE_TURKISH_Q
- SL_USBD_HID_COUNTRY_CODE_UK
- SL_USBD_HID_COUNTRY_CODE_US
- SL_USBD_HID_COUNTRY_CODE_YUGOSLAVIA
- SL_USBD_HID_COUNTRY_CODE_TURKISH_F

The country code identifies which country the hardware is localized for. Most hardware is not localized and therefore this value would be SL_USBD_HID_COUNTRY_CODE_NOT_SUPPORTED (0). However, keyboards may use the field to indicate the language of the key caps.

For more information on the country codes, see section 6.2.1 of HID specification revision 1.11.

interval_in and interval_out

interval_in and interval_out represent the polling interval of the IN interrupt endpoint and the OUT interrupt endpoint.

This represents the polling interval of the endpoint, in milliseconds. Setting this value depends on how frequently your device is susceptible to generate a new report for the host. For instance, if a report is generated every 16 milliseconds, the interval should be 16 or less.

The value must be a power of 2 (1, 2, 4, 8, 16, etc.).

interval_out value is ignored if ctrl_rd_en is set to true.

p_hid_callback

p_hid_callback is a pointer to a structure of type slusb_hid_callbacks_t. Its purpose is to give the HID Class a set of callback functions to be called when a HID event occurs.

Not all callbacks are mandatory and a null pointer (`NULL`) can be passed in the callbacks structure variable when the callback is not needed.

The table below describes each configuration field available in this configuration structure.

Table - **sl_usbd_hid_callbacks_t** Configuration Structure

Fields	Description	Function Signature
<code>.enable</code>	Called when the USB class instance is enabled successfully.	<code>void app_usbd_hid_enable(uint8_t class_nbr);</code>
<code>.disable</code>	Called when the USB class instance is disabled.	<code>void app_usbd_hid_disable(uint8_t class_nbr);</code>
<code>.get_report_desc</code>	Called during the HID Instance creation to pass your report descriptor. For each of your HID functions, you must provide a report descriptor. The report descriptor indicates to the host how the periodic report that will be sent by the device should be parsed. Writing your own report descriptor can be challenging, and that is why there are some resources to help. This is the only mandatory callback function.	<code>void app_usbd_hid_get_report_desc(uint8_t class_nbr, const uint8_t *p_report_ptr, uint16_t report_len);</code>
<code>.get_phy_desc</code>	Called during the HID Instance creation to pass your physical descriptor. The physical descriptor is a descriptor that provides information about the specific part or parts of the human body that are activating a control or controls. For more information on physical descriptors, see section 6.2.3 of HID specification revision 1.11. The physical descriptor is optional and most of the time ignored. The buffer passed here can be set to <code>NULL</code> and the length set to 0.	<code>void app_usbd_hid_get_phy_desc(uint8_t class_nbr, const uint8_t *p_report_ptr, uint16_t report_len);</code>
<code>.set_output_report</code>	Called when the host sets a report as described in your report descriptor (when it sends a report).	<code>void app_usbd_hid_set_output_report(uint8_t class_nbr, uint8_t report_id, uint8_t *p_report_buf, uint16_t report_len);</code>
<code>.get_feature_report</code>	Called when the host requests a feature report as described in your report descriptor.	<code>void app_usbd_hid_get_feature_report(uint8_t class_nbr, uint8_t report_id, uint8_t *p_report_buf, uint16_t report_len);</code>
<code>.set_feature_report</code>	Called when the host sets a feature report as described in your report descriptor.	<code>void app_usbd_hid_set_feature_report(uint8_t class_nbr, uint8_t report_id, uint8_t *p_report_buf, uint16_t report_len);</code>
<code>.get_protocol</code>	Retrieves current active protocol.	<code>void app_usbd_hid_get_protocol(uint8_t class_nbr, uint8_t *p_protocol);</code>
<code>.set_protocol</code>	Sets current active protocol.	<code>void app_usbd_hid_set_protocol(uint8_t class_nbr, uint8_t protocol);</code>

HID Class Report Descriptor Example

Silicon Labs' HID class sample application provides an example of a report descriptor for a simple mouse. The example below shows a mouse report descriptor.

Example - Mouse Report Descriptor

```
static uint8_t app_usbd_hid_report_desc[] = {
    SL_USBD_HID_GLOBAL_USAGE_PAGE + 1, SL_USBD_HID_USAGE_PAGE_GENERIC_DESKTOP_CONTROLS,
```

```

SL_USBD_HID_LOCAL_USAGE      +1, SL_USBD_HID_CA_MOUSE, (3)
SL_USBD_HID_MAIN_COLLECTION  +1, SL_USBD_HID_COLLECTION_APPLICATION, (4)
SL_USBD_HID_LOCAL_USAGE      +1, SL_USBD_HID_CP_POINTER, (5)
SL_USBD_HID_MAIN_COLLECTION  +1, SL_USBD_HID_COLLECTION_PHYSICAL, (6)
SL_USBD_HID_GLOBAL_USAGE_PAGE +1, SL_USBD_HID_USAGE_PAGE_BUTTON, (7)
SL_USBD_HID_LOCAL_USAGE_MIN   +1, 0x01,
SL_USBD_HID_LOCAL_USAGE_MAX   +1, 0x03,
SL_USBD_HID_GLOBAL_LOG_MIN    +1, 0x00,
SL_USBD_HID_GLOBAL_LOG_MAX    +1, 0x01,
SL_USBD_HID_GLOBAL_REPORT_COUNT +1, 0x03,
SL_USBD_HID_GLOBAL_REPORT_SIZE +1, 0x01,
SL_USBD_HID_MAIN_INPUT        +1, SL_USBD_HID_MAIN_DATA |
                               SL_USBD_HID_MAIN_VARIABLE |
                               SL_USBD_HID_MAIN_ABSOLUTE,
SL_USBD_HID_GLOBAL_REPORT_COUNT +1, 0x01, (8)
SL_USBD_HID_GLOBAL_REPORT_SIZE +1, 0x0D,
SL_USBD_HID_MAIN_INPUT        +1, SL_USBD_HID_MAIN_CONSTANT, (9)
SL_USBD_HID_GLOBAL_USAGE_PAGE +1, SL_USBD_HID_USAGE_PAGE_GENERIC_DESKTOP_CONTROLS,
SL_USBD_HID_LOCAL_USAGE      +1, SL_USBD_HID_DV_X,
SL_USBD_HID_LOCAL_USAGE      +1, SL_USBD_HID_DV_Y,
SL_USBD_HID_GLOBAL_LOG_MIN    +1, 0x81,
SL_USBD_HID_GLOBAL_LOG_MAX    +1, 0x7F,
SL_USBD_HID_GLOBAL_REPORT_SIZE +1, 0x08,
SL_USBD_HID_GLOBAL_REPORT_COUNT +1, 0x02,
SL_USBD_HID_MAIN_INPUT        +1, SL_USBD_HID_MAIN_DATA |
                               SL_USBD_HID_MAIN_VARIABLE |
                               SL_USBD_HID_MAIN_RELATIVE,
SL_USBD_HID_MAIN_ENDCOLLECTION, (10)
SL_USBD_HID_MAIN_ENDCOLLECTION                               (11));

```

(1) The table representing a mouse report descriptor is initialized in such way that each line corresponds to a short item. The latter is formed from a 1-byte prefix and a 1-byte data. Refer to “Device Class Definition for Human Interface Devices (HID) Version 1.11”, sections 5.3 and 6.2.2.2 for more details about short items format. This table content corresponds to the mouse report descriptor content viewed by a host HID parser in [Figure - Report Descriptor Content from a Host HID Parser View](#).

(2) The Generic Desktop Usage Page is used.

(3) Within the Generic Desktop Usage Page, the usage tag suggests that the group of controls is for controlling a mouse. A mouse collection typically consists of two axes (X and Y) and one, two, or three buttons.

(4) The mouse collection is started.

(5) Within the mouse collection, a usage tag suggests more specifically that the mouse controls belong to the pointer collection. A pointer collection is a collection of axes that generates a value to direct, indicate, or point user intentions to an application.

(6) The pointer collection is started.

(7) The Buttons Usage Page defines an Input item composed of three 1-bit fields. Each 1-bit field represents the mouse's button 1, 2 and 3 respectively and can return a value of 0 or 1.

(8) The Input Item for the Buttons Usage Page is padded with 13 other bits.

(9) Another Generic Desktop Usage Page is indicated for describing the mouse position with the axes X and Y. The Input item is composed of two 8-bit fields whose value can be between -127 and 127.

(10) The pointer collection is closed.

(11) The mouse collection is closed.

USB.org HID Page

The USB Implementers Forum (USB-IF) provides a tool called "HID Descriptor Tool" along with other information on the report descriptor format. See <http://www.usb.org/developers/hidpage/> for more information.

USB Device HID Class Programming Guide

This section explains how to use the HID class.

- [Initializing the USB Device HID Class](#)
- [Adding a USB Device HID Class Instance to your Device](#)
- [Communicating using the USB Device HID Class](#)

Initializing the USB Device HID Class

To add HID Class functionality to your device, you must first initialize the class by calling the function `sl_usbd_hid_init()`.

The example below shows how to call `sl_usbd_hid_init()` using default arguments. For more information on the configuration arguments to pass to `sl_usbd_hid_init()`, see [USB Device HID Class Application Specific Configurations](#).

Example - Calling `sl_usbd_hid_init()`

```
sl_status_t status;

status = sl_usbd_hid_init();
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Adding a USB Device HID Class Instance to Your Device

To add HID class functionality to your device, you must create an instance, then add it to your device's configuration(s).

Creating a HID Class Instance

Create a HID class instance by calling the function `sl_usbd_hid_create_instance()`.

The example below shows how to create a simple mouse function via `sl_usbd_hid_create_instance()` using default arguments. For more information on the configuration arguments to pass to `sl_usbd_hid_create_instance()`, see [USB Device HID Class Instance Configurations](#).

Example - Adding a Mouse Function via `sl_usbd_hid_create_instance()`

```

/* Global constants. */
static const uint8_t app_usbd_hid_mouse_report_desc[] = {
    SL_USBD_HID_GLOBAL_USAGE_PAGE + 1, SL_USBD_HID_USAGE_PAGE_GENERIC_DESKTOP_CONTROLS,
    SL_USBD_HID_LOCAL_USAGE + 1, SL_USBD_HID_CA_MOUSE,
    SL_USBD_HID_MAIN_COLLECTION + 1, SL_USBD_HID_COLLECTION_APPLICATION,
    SL_USBD_HID_LOCAL_USAGE + 1, SL_USBD_HID_CP_POINTER,
    SL_USBD_HID_MAIN_COLLECTION + 1, SL_USBD_HID_COLLECTION_PHYSICAL,
    SL_USBD_HID_GLOBAL_USAGE_PAGE + 1, SL_USBD_HID_USAGE_PAGE_BUTTON,
    SL_USBD_HID_LOCAL_USAGE_MIN + 1, 0x01,
    SL_USBD_HID_LOCAL_USAGE_MAX + 1, 0x03,
    SL_USBD_HID_GLOBAL_LOG_MIN + 1, 0x00,
    SL_USBD_HID_GLOBAL_LOG_MAX + 1, 0x01,
    SL_USBD_HID_GLOBAL_REPORT_COUNT + 1, 0x03,
    SL_USBD_HID_GLOBAL_REPORT_SIZE + 1, 0x01,
    SL_USBD_HID_MAIN_INPUT + 1, SL_USBD_HID_MAIN_DATA | SL_USBD_HID_MAIN_VARIABLE | SL_USBD_HID_MAIN_ABSOLUTE,
    SL_USBD_HID_GLOBAL_REPORT_COUNT + 1, 0x01,
    SL_USBD_HID_GLOBAL_REPORT_SIZE + 1, 0x0D,
    SL_USBD_HID_MAIN_INPUT + 1, SL_USBD_HID_MAIN_CONSTANT,
    SL_USBD_HID_GLOBAL_USAGE_PAGE + 1, SL_USBD_HID_USAGE_PAGE_GENERIC_DESKTOP_CONTROLS,
    SL_USBD_HID_LOCAL_USAGE + 1, SL_USBD_HID_DV_X,
    SL_USBD_HID_LOCAL_USAGE + 1, SL_USBD_HID_DV_Y,
    SL_USBD_HID_GLOBAL_LOG_MIN + 1, 0x81,
    SL_USBD_HID_GLOBAL_LOG_MAX + 1, 0x7F,
    SL_USBD_HID_GLOBAL_REPORT_SIZE + 1, 0x08,
    SL_USBD_HID_GLOBAL_REPORT_COUNT + 1, 0x02,
    SL_USBD_HID_MAIN_INPUT + 1, SL_USBD_HID_MAIN_DATA | SL_USBD_HID_MAIN_VARIABLE | SL_USBD_HID_MAIN_RELATIVE,
    SL_USBD_HID_MAIN_END_COLLECTION,
    SL_USBD_HID_MAIN_END_COLLECTION
};

/* Local variables.*/
uint8_t class_nbr;
sl_status_t status;

sl_usbd_hid_callbacks_t app_usbd_hid_callbacks = {
    NULL,
    NULL,
    app_usbd_hid_get_report_desc,
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
    NULL,
    NULL
};

void app_usbd_hid_get_report_desc(uint8_t class_nbr,
    const uint8_t **p_report_ptr,
    uint16_t *p_report_len)
{
    (void)class_nbr;

    *p_report_ptr = app_usbd_hid_mouse_report_desc;
    *p_report_len = sizeof(app_usbd_hid_mouse_report_desc);
}

status = sl_usbd_hid_create_instance(SL_USBD_HID_SUBCLASS_BOOT,
    SL_USBD_HID_PROTOCOL_MOUSE,
    SL_USBD_HID_COUNTRY_CODE_NOT_SUPPORTED,
    Ex_USBD_HID_Mouse_ReportDesc,
    sizeof(Ex_USBD_HID_Mouse_ReportDesc),
    2u,
    2u,
    true,
    &app_usbd_hid_callbacks,
    &class_nbr);

```

```
/* An error occurred. Error handling should be added here. */}
```

Adding the HID Class Instance to Your Device's Configuration(s)

After you have created a HID class instance, you can add it to a configuration by calling the function `sl_usbd_hid_add_to_configuration()` .

The example below shows how to call `sl_usbd_hid_add_to_configuration()` .

Example - Calling `sl_usbd_hid_add_to_configuration()`

```
sl_status_t status;

sl_usbd_hid_add_to_configuration(class_nbr,      (1)
                               config_nbr_fs);  (2)
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

- (1) Class number to add to the configuration returned by `sl_usbd_hid_create_instance()`.
- (2) Configuration number (here adding it to a Full-Speed configuration).

Communicating Using the USB Device HID Class

- [Class Instance Communication](#)
- [Synchronous Communication](#)

Class Instance Communication

The HID class offers the following functions to communicate with the host.

Table - HID Communication API Summary

Function name	Operation
sl_usbd_hid_read_sync()	Receives data from the host through interrupt OUT endpoint. This function is blocking.
sl_usbd_hid_write_sync()	Sends data to the host through interrupt IN endpoint. This function is blocking.

Synchronous Communication

Synchronous communication means that the transfer is blocking. Upon the function call, the applications blocks until the transfer is completed with or without an error. A timeout can be specified to avoid waiting forever.

The example below shows a read and write that receives data from the host using the interrupt OUT endpoint and sends data to the host using the interrupt IN endpoint.

Example - Synchronous HID Read and Write

```
__ALIGNED(4) uint8_t rx_buf[2];
__ALIGNED(4) uint8_t tx_buf[2];
uint32_t          xfer_len;
sl_status_t       status;

status = sl_usbd_hid_read_sync(class_nbr,      (1)
                               (void *)rx_buf, (2)
                               2u,              (3)
                               0u,
                               &xfer_len);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

```

}

status =sl_usbd_hid_write_sync(class_nbr,(1)(void *)tx_buf,(4)2u,0u,(3)&xfer_len);if(status != SL_STATUS_OK){/* An error occurred. Error handling
should be added here. */}

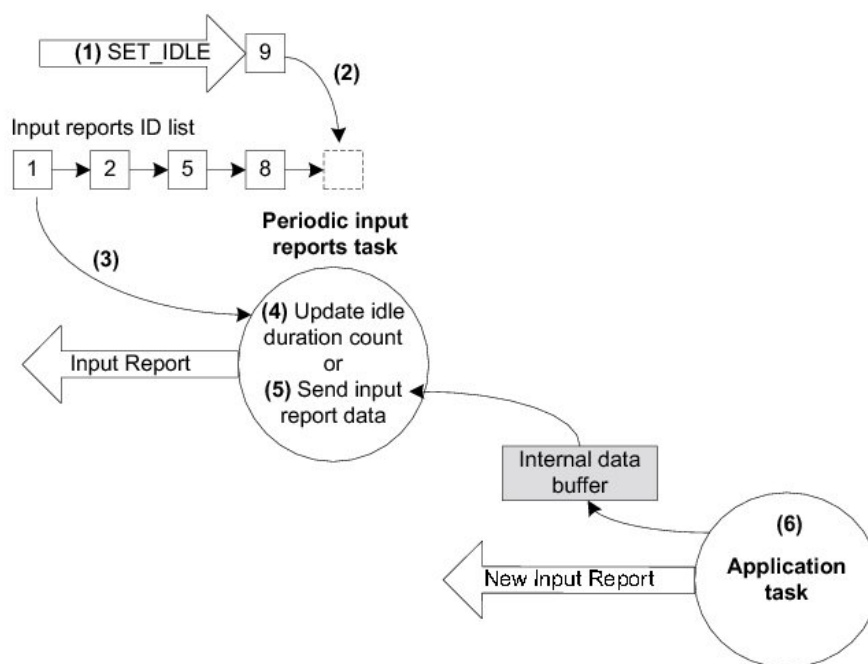
```

- (1) The class instance number created from `sl_usbd_hid_create_instance()` provides an internal reference for the HID class to route the transfer to the proper interrupt OUT or IN endpoint.
- (2) The application must ensure that the buffer provided to the function is large enough to accommodate all the data. Otherwise, synchronization issues might happen. Internally, the read operation is done either with the control endpoint or with the interrupt endpoint, depending on the control read flag set when calling `sl_usbd_hid_create_instance()`.
- (3) To avoid an infinite blocking situation, a timeout expressed in milliseconds can be specified. A value of '0' makes the application task wait forever.
- (4) The application provides the initialized transmit buffer.

HID Periodic Input Reports Task

To save bandwidth, the host has the ability to silence reports from an interrupt IN endpoint by limiting the reporting frequency. To do so, the host must send the SET_IDLE request. The HID class implemented by Silicon Labs contains an internal task that respects the reporting frequency limitation that you can apply to one or several input reports. [Figure - Periodic Input Reports Task](#) shows the functioning of the periodic input reports tasks.

Figure - Periodic Input Reports Task



- (1) The device receives a SET_IDLE request. This request specifies an idle duration for a given report ID. For more details about the SET_IDLE request, see “Device Class Definition for Human Interface Devices (HID) Version 1.11”, section 7.2.4. A report ID allows you to distinguish among the different types of reports sent from the same endpoint.
- (2) A report ID structure (allocated during the HID class initialization phase) is updated with the idle duration. An idle duration counter is initialized with the idle duration value. The report ID structure is inserted at the end of a linked list containing input reports ID structures. The idle duration value is expressed in 4-ms unit which gives a range of 4 to 1020 ms. If the idle duration is less than the polling interval of the interrupt IN endpoint, the reports are generated at the polling interval.
- (3) Every 4 ms, the periodic input report task browses the input reports ID list. For each input report ID, the task performs one of two possible operations. The duration of the task period matches the 4-ms unit used for the idle duration. If no

SET_IDLE requests have been sent by the host, the input reports ID list is empty and the task has nothing to process. The task processes only report IDs that are different from 0 and with an idle duration greater than 0.

(4) For a given input report ID, the task verifies if the idle duration has elapsed. If the idle duration has not elapsed, the counter is decremented and no input report is sent to the host.

(5) If the idle duration has elapsed (that is, the idle duration counter has reached zero), an input report is sent to the host by calling the `sl_usbd_hid_write_sync()` function via the interrupt IN endpoint.

(6) The input report data sent by the task comes from an internal data buffer allocated for each input report described in the Report descriptor. An application task can call the `sl_usbd_hid_write_sync()` function to send an input report. After sending the input report data, `sl_usbd_hid_write_sync()` updates the internal buffer associated to an input report ID with the data just sent. Then, the periodic input reports task always sends the same input report data after each idle duration elapsed and until the application task updates the data in the internal buffer. There is some locking mechanism to avoid corruption of the input report ID data in the event of a modification happening at the exact time of transmission done by the periodic input report task.

Overview

USB Device MSC Class

- [USB Device MSC Class Overview](#)
- [USB Device MSC Class Resource Needs from Core](#)
- [USB Device MSC Class Configuration](#)
- [USB Device MSC Class Programming Guide](#)
- [USB Device MSC Class Storage Drivers](#)

This section describes the mass storage device class (MSC) supported by Silicon Labs USB Device. MSC is a protocol that enables the transfer of information between a USB device and a host. The information being transferred is anything that can be stored electronically, such as executable programs, source code, documents, images, configuration data, or other text or numeric data. The USB device appears as an external storage medium to the host, enabling the transfer of files via drag and drop.

A file system defines how the files are organized in the storage media. The USB mass storage class specification does not require any particular file system to be used on conforming devices. Instead, it provides a simple interface to read and write sectors of data using the Small Computer System Interface (SCSI) transparent command set. As such, operating systems may treat the USB drive like a hard drive, and can format it with any file system they like.

The USB mass storage device class supports two transport protocols, as follows:

- Bulk-Only Transport (BOT)
- Control/Bulk/Interrupt (CBI) Transport (used only for floppy disk drives)

The mass storage device class implements the SCSI transparent command set using the BOT protocol only, which signifies that only bulk endpoints will be used to transmit data and status information. The MSC implementation supports multiple logical units.

The MSC implementation is in compliance with the following specifications:

- *Universal Serial Bus Mass Storage Class Specification Overview*, Revision 1.3 Sept. 5, 2008.
- *Universal Serial Bus Mass Storage Class Bulk-Only Transport*, Revision 1.0 Sept. 31, 1999.

USB Device MSC Class Overview

- [Protocol](#)
- [Endpoints](#)
- [Class Requests](#)
- [Small Computer System Interface \(SCSI\)](#)

Protocol

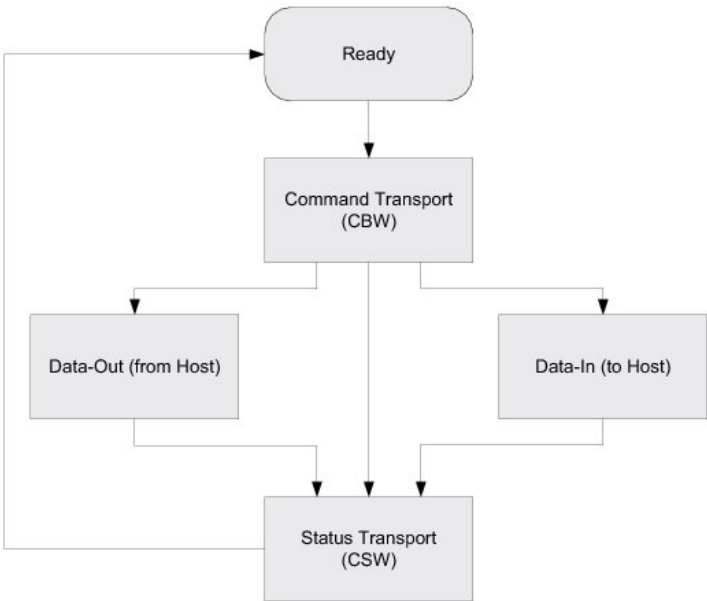
In this section, we will discuss the Bulk-Only Transport (BOT) protocol of the Mass Storage Class. The Bulk-Only Transport protocol has three stages:

- The Command Transport
- The Data Transport
- The Status Transport

Mass storage commands are sent by the host through a structure called the Command Block Wrapper (CBW). For commands requiring a data transport stage, the host will attempt to send or receive the exact number of bytes from the device as specified by the length and flag fields of the CBW. After the data transport stage, the host attempts to receive a Command Status Wrapper (CSW) from the device that details the status of the command as well as any data residue (if

any). For commands that do not include a data transport stage, the host attempts to receive the CSW directly after CBW is sent. The protocol is detailed in [Figure - MSC Protocol](#).

Figure - MSC Protocol



Endpoints

On the device side, in compliance with the BOT specification, the MSC is composed of the following endpoints:

- A pair of control IN and OUT endpoints called default endpoint.
- A pair of bulk IN and OUT endpoints.

The table below indicates the different usages of the endpoints.

Table - MSC Endpoint Usage

Endpoint	Direction	Usage
Control IN	Device to Host	Enumeration and MSC class-specific requests
Control OUT	Host to Device	Enumeration and MSC class-specific requests
Bulk IN	Device to Host	Send CSW and data
Bulk OUT	Host to Device	Receive CBW and data

Class Requests

There are two defined control requests for the MSC BOT protocol. These requests and their descriptions are detailed in the table below.

Table - Mass Storage Class Requests

Class Requests	Description
Bulk-Only Mass Storage Reset	This request is used to reset the mass storage device and its associated interface. This request readies the device to receive the next command block.

Class Requests	Description
Get Max LUN	This request is used to return the highest logical unit number (LUN) supported by the device. For example, a device with LUN 0 and LUN 1 will return a value of 1. A device with a single logical unit will return 0 or stall the request. The maximum value that can be returned is 15.

Small Computer System Interface (SCSI)

At the programming interface level, the MSC device implements one of the standard storage-media communication protocols, like SCSI and SFF-8020i (ATAPI). The "Programming Interface" specifies which protocol is implemented, and helps the host operating system to load the suitable device driver for communicating with the USB storage device. SCSI is the most common protocol used with USB MSC storage devices. We provide an implementation for MSC SCSI subclass that our GSDK users can use out of the box.

SCSI is a set of standards for handling communication between computers and peripheral devices. These standards include commands, protocols, electrical interfaces and optical interfaces. Storage devices that use other hardware interfaces, such as USB, use SCSI commands for obtaining device/host information and controlling the device's operation and transferring blocks of data in the storage media.

SCSI commands cover a vast range of device types and functions and as such, devices need a subset of these commands. In general, the following commands are necessary for basic communication:

- INQUIRY
- READ CAPACITY(10)
- READ(10)
- REQUEST SENSE
- TEST UNIT READY
- WRITE(10)

USB Device MSC Class Resource Needs from Core

Each time you add an MSC class instance to a USB configuration via the function `sl_usbd_msc_add_to_configuration()`, the following resources will be allocated from the core.

Resource	Quantity
Interfaces	1
Alternate interfaces	1
Endpoints	2
Interface groups	0

Note that those numbers are per configuration. When setting up your `SL_USBD_INTERFACE_QUANTITY`, `SL_USBD_ALT_INTERFACE_QUANTITY`, `SL_USBD_INTERFACE_GROUP_QUANTITY` and `SL_USBD_DESCRIPTOR_QUANTITY` configuration values, don't forget to take into account on how many configurations the class will be added. For the `SL_USBD_OPEN_ENDPOINTS_QUANTITY` configuration value, since endpoints are opened only when a configuration is set by the host, you just need to take into account the number of needed endpoints for a class instance.

USB Device MSC Class Configuration

Two groups of configuration parameters are used to configure the MSC class:

- [USB Device MSC Class Application-Specific Configurations](#)
- [USB Device MSC Class Logical Unit Configuration](#)

USB Device MSC Class Application-Specific Configurations

- [Class Compile-Time Configurations](#)
- [Class Instance Creation](#)

Class Compile-Time Configurations

Silicon Labs USB Device MSC class and SCSI subclass are configurable at compile time via #defines located in the `sl_usbd_core_config.h` file.

Table - Generic Configuration Constants

Configuration Name	Description	Default Value
SL_USBD_MSC_CLASS_INSTANCE_QUANTITY	Number of class instances you will allocate via a call to the function <code>sl_usbd_msc_scsi_create_instance()</code> .	2
SL_USBD_MSC_CONFIGURATION_QUANTITY	Number of configuration to which a class instance can be added via a call to the function <code>sl_usbd_msc_scsi_add_to_configuration()</code> .	1
SL_USBD_MSC_LUN_QUANTITY	Number of logical units per class instance that you will add via a call to the function <code>sl_usbd_msc_scsi_lun_add()</code> .	2
SL_USBD_MSC_SCSI_64_BIT_LBA_EN	Enables or disables support for Logical Block Address (LBA) of 64 bits.	0
SL_USBD_MSC_DATA_BUFFER_SIZE	Size of data buffer per class instance in bytes	512

Class Instance Creation

Creating a USB Device MSC SCSI class instance is done by calling the `sl_usbd_msc_scsi_create_instance()` function. This function takes one configuration argument that is described below.

p_scsi_callbacks

`p_scsi_callbacks` is a pointer to a configuration structure of type `sl_usbd_msc_scsi_callbacks_t` . In addition to the common usb device class callbacks connect/disconnect, it provides the MSC class with a set of optional callback functions that are called when an event occurs on the logical unit. A null pointer (`NULL`) can be passed to this argument if no callbacks are needed.

The table below describes each configuration field available in this configuration structure.

Table - `sl_usbd_msc_scsi_callbacks_t` Configuration Structure

Fields	Description	Function Signature
<code>.enable</code>	Called when the USB class instance is enabled successfully.	<code>void app_usbd_msc_scsi_enable(uint8_t class_nbr);</code>
<code>.disable</code>	Called when the USB class instance is disabled.	<code>void app_usbd_msc_scsi_disable(uint8_t class_nbr);</code>
<code>.host_eject</code>	Function called when a logical unit is ejected from the host.	<code>void app_usbd_msc_scsi_host_eject(uint8_t class_nbr, uint8_t lu_nbr);</code>

USB Device MSC Class Logical Unit Configuration

Adding a logical unit to an MSC class instance is done by calling the function `sl_usbd_msc_lun_add()` . This function takes one configuration argument that is described below.

p_lu_info

`p_lu_info` is a pointer to a structure of type `sl_usbd_msc_scsi_lun_info_t` . Its purpose is to provide the information on the logical unit to the MSC class.

The table below describes each configuration field available in this configuration structure.

Table - `sl_usbd_msc_scsi_lun_info_t` Configuration Structure

Field	Description
<code>.scsi_lun_api_ptr</code>	Pointer to the media driver API that will handle this logical unit. See USB Device MSC Class Storage Drivers for more information on storage drivers.
<code>.vendor_id_ptr</code>	Pointer to a string that contains the vendor identification of the logical unit. The maximum length of the string is 8 characters.
<code>.product_id_ptr</code>	Pointer to a string that contains the product identification of the logical unit. The maximum length of the string is 16 characters.
<code>.product_revision_level</code>	Product revision level.
<code>.is_read_only</code>	Flag that indicates if the logical unit should be seen as read only from the point of view of the host (<code>true</code>) or not (<code>false</code>).

USB Device MSC Class Programming Guide

This section explains how to use the MSC class.

- [Initializing the USB Device MSC Class](#)
- [Adding a USB Device MSC SCSI Class Instance to Your Device](#)
- [USB Device MSC Class Logical Unit Handling](#)

Initializing the USB Device MSC Class

To add MSC SCSI class functionality to your device, first initialize the MSC base class and the SCSI subclass by calling the function `sl_usbd_msc_init()` and `sl_usbd_msc_scsi_init()`.

The example below shows how to call `sl_usbd_msc_init()` and `sl_usbd_msc_scsi_init()`.

Example - Calling `sl_usbd_msc_init()` and `sl_usbd_msc_scsi_init()`

```
sl_status_t status;

status = sl_usbd_msc_init();
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}

status = sl_usbd_msc_scsi_init();
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Adding a USB Device MSC SCSI Class Instance to Your Device

To add MSC SCSI class functionality to your device, first create an instance, then add it to your device's configuration(s). You must add at least one logical unit to your instance.

Creating an MSC SCSI Class Instance

Create a MSC SCSI class instance by calling the function `sl_usbd_msc_scsi_create_instance()`.

The example below shows how to call `sl_usbd_msc_scsi_create_instance()` using default arguments. For more information on the configuration arguments to pass to `sl_usbd_msc_scsi_create_instance()`, see [USB Device MSC Class Application Specific Configurations](#).

Example - Calling `sl_usbd_msc_scsi_create_instance()`

```
uint8_t    class_nbr;
sl_status_t status;
```

```
sl_usbd_msc_scsi_callbacks_t app_usbd_msc_scsi_callbacks = {enable = NULL, disable = NULL, host_eject = NULL
};

status = sl_usbd_msc_scsi_create_instance(&app_usbd_msc_scsi_callbacks, 0 & class_nbr); if (status != SL_STATUS_OK) { /* An error occurred. Error
handling should be added here. */ }
```

Adding the MSC Class Instance to Your Device's Configuration(s)

After you have created an MSC class instance, you can add it to a configuration by calling the function `sl_usbd_msc_add_to_configuration()`.

The example below shows how to call `sl_usbd_msc_scsi_add_to_configuration()` using default arguments.

Example - Calling `sl_usbd_msc_scsi_add_to_configuration()`

```
sl_status_t status;

status = sl_usbd_msc_scsi_add_to_configuration(class_nbr,          (1)
                                              config_nbr_fs);      (2)
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

(1) Class number to add to the configuration returned by `sl_usbd_msc_scsi_create_instance()`.

(2) Configuration number (here adding it to a Full-Speed configuration).

USB Device MSC Class Logical Unit Handling

- [Adding a Logical Unit](#)
- [Attaching/Detaching a Storage Medium](#)

Adding a Logical Unit

When adding a logical unit to your MSC SCSI class instance, it must be bound to a storage medium (RAMDisk, SD card, flash memory, etc). The MSC class uses a storage driver to communicate with storage media. This driver will need to be supplied when adding the logical unit.

The example below shows how to add a logical unit via `sl_usbd_msc_scsi_lun_add()`.

Example - Adding a Logical Unit via `sl_usbd_msc_scsi_lun_add()`

```
sl_usbd_msc_scsi_lun_t    *lu_object_ptr = NULL;
sl_usbd_msc_scsi_lun_info_t lu_info;
sl_status_t              status;

lu_info.sl_usbd_msc_scsi_lun_api_t = &app_usbd_scsi_storage_block_device_api;
lu_info.vendor_id_ptr              = "Silicon Labs";
lu_info.product_id_ptr             = "block device example";
lu_info.product_revision_level     = 0x1000u;
lu_info.is_read_only               = false;

status = sl_usbd_msc_scsi_lun_add(class_nbr,
                                  &lu_info,
                                  &lu_object_ptr);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Attaching/Detaching a Storage Medium

After the logical unit has been added, a storage medium must be attached to be available from the host side.

The MSC class offers two functions to control the storage media association to the logical unit: `sl_usbd_msc_scsi_lun_attach()` and `sl_usbd_msc_scsi_lun_detach()`. These functions allow you to emulate the removal of a storage device in order to re-gain access from the embedded application if necessary.

The example below shows how to use the function `sl_usbd_msc_scsi_lun_attach()` and `sl_usbd_msc_scsi_lun_detach()`.

Example - Media Attach/Detach

```
sl_status_t status;

status = sl_usbd_msc_scsi_lun_attach(lu_object_ptr);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}

... (1)

status = sl_usbd_msc_scsi_lun_detach(lu_object_ptr);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}

... (2)

status = sl_usbd_msc_scsi_lun_attach(lu_object_ptr);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}

... (3)
```

(1) From this moment, if the MSC device is connected to a host, the storage media is accessible.

(2) If the MSC device is connected to a host, the media will now appear as unavailable. At this moment, operations can be performed on the media from the embedded application.

(3) Again, if the MSC device is connected to the host, the storage media will appear as connected.

USB Device MSC Class Storage Drivers

The USB Device MSC Class needs a storage driver to communicate with a storage medium. For the moment, Silicon Labs doesn't offer drivers.

The driver API is defined by typedef `sl_usbd_msc_scsi_lun_api_t`. Your `sl_usbd_msc_scsi_lun_api_t` variable must be included to your `sl_usbd_msc_scsi_lun_info_t` variable, passed as argument when you add a logical unit with `sl_usbd_msc_scsi_lun_add()`. See section [USB Device MSC SCSI API](#) for more details on the structures.

The storage driver implementation can be as simple as an array of sectors in RAM. Typical sector size (i.e., block size) is 512 for mass storage devices, and 2048 for CD-ROMs.

Overview

USB Device Vendor Class

- [USB Device Vendor Class Overview](#)
- [USB Device Vendor Class Resource Needs from Core](#)
- [USB Device Vendor Class Configuration](#)
- [USB Device Vendor Class Programming Guide](#)

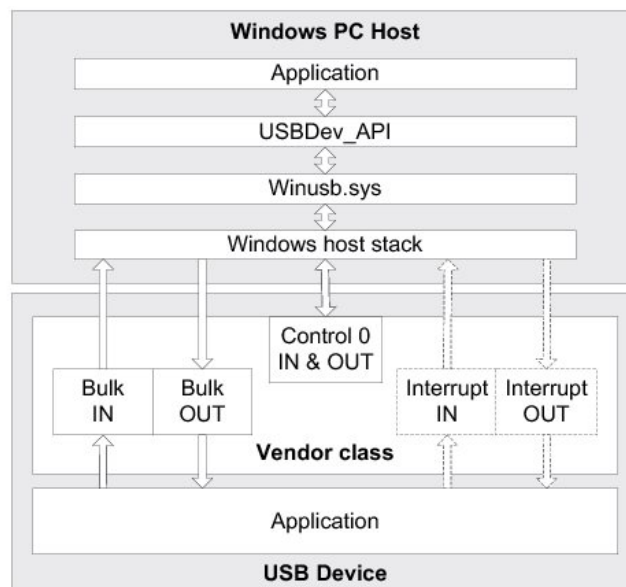
The Vendor class allows you to build vendor-specific devices that can implement a proprietary protocol. It relies on a pair of bulk endpoints to transfer data between the host and the device. Bulk transfers are convenient for transferring large amounts of unstructured data and provide a reliable exchange of data by using an error detection and retry mechanism.

In addition to bulk endpoints, the Vendor class can also use an optional pair of interrupt endpoints. Any operating system (OS) can work with the Vendor class provided that the OS has a driver to handle the Vendor class. Depending on the OS, the driver can be native or vendor-specific. For instance, under Microsoft Windows®, your application interacts with the WinUSB driver provided by Microsoft to communicate with the vendor device.

USB Device Vendor Class Overview

[Figure - General Architecture Between Windows Host and Vendor Class](#) shows the general architecture between the host and the device using the Vendor class. In this example, the host operating system is MS Windows.

Figure - General Architecture Between MS Windows Host and Vendor Class



On the MS Windows side, the application communicates with the vendor device by interacting with a USB library. Libraries, like libusb, offer an API to manage a device and its associated pipes, and to communicate with the device through control, bulk and interrupt endpoints.

On the device side, the Vendor class is composed of the following endpoints:

- A pair of control IN and OUT endpoints called the default endpoint.

- A pair of bulk IN and OUT endpoints.
- A pair of interrupt IN and OUT endpoints. This pair is optional.

The table below indicates the usage of the different endpoints:

Table - Vendor Class Endpoints Usage

Endpoint	Direction	Usage
Control IN	Device-to-host	Standard requests for enumeration and vendor-specific requests.
Control OUT	>Host-to-device	Standard requests for enumeration and vendor-specific requests.
Bulk IN	Device-to-host	Raw data communication. Data can be structured according to a proprietary protocol.
Bulk OUT	Host-to-device	Raw data communication. Data can be structured according to a proprietary protocol.
Interrupt IN	Device-to-host	Raw data communication or notification. Data can be structured according to a proprietary protocol.
Interrupt OUT	Host-to-device	Raw data communication or notification. Data can be structured according to a proprietary protocol.

The device application can use bulk and interrupt endpoints to send or receive data to or from the host. It can only use the default endpoint to decode vendor-specific requests sent by the host. The standard requests are managed internally by the Core layer of Silicon Labs USB Device.

USB Device Vendor Class Resource Needs from Core

Each time you add a vendor class instance to a configuration via the function `sl_usbd_vendor_add_to_configuration()`, the following resources will be allocated from the core.

Resource	Quantity
Interfaces	1
Alternate interfaces	1
Endpoints	2 (4 if you enabled interrupt endpoints)
Interface groups	0

Note that those numbers are per configuration. When setting up your `SL_USBD_INTERFACE_QUANTITY`, `SL_USBD_ALT_INTERFACE_QUANTITY`, `SL_USBD_INTERFACE_GROUP_QUANTITY` and `SL_USBD_DESCRIPTOR_QUANTITY` configuration values, don't forget to take into account on how many configurations the class will be added. For the `SL_USBD_OPEN_ENDPOINTS_QUANTITY` configuration value, since endpoints are opened only when a configuration is set by the host, you just need to take into account the number of needed endpoints for a class instance.

USB Device Vendor Class Configuration

Two groups of configuration parameters are used to configure the Vendor class:

- [USB Device Vendor Class Application-Specific Configurations](#)
- [USB Device Vendor Class Instance Configurations](#)

USB Device Vendor Class Application-Specific Configurations

First, to use the Silicon Labs USB device Vendor class module, adjust the Vendor compile-time configuration defines according to your application needs. They are regrouped inside the `sl_usbd_core_config.h` header file under the Vendor section. The quantity configurations purpose is to inform the USB device module about how many USB Vendor objects to allocate.

The table below describes each configuration define.

Table - USB Device Vendor Configuration Defines

Configuration Name	Description	Default Value
SL_USBD_VENDOR_CLASS_INSTANCE_QUANTITY	Number of class instances you will allocate via a call to the function <code>sLusbd_vendor_create_instance()</code> .	2
SL_USBD_VENDOR_CONFIGURATION_QUANTITY	Number of configurations. Vendor class instances can be added to one or more configurations via a call to the function <code>sLusbd_vendor_add_to_configuration()</code> .	1

USB Device Vendor Class Instance Configurations

This section defines the configurations related to the Vendor class instances.

- [Class Instance Creation](#)
 - [intr_en](#)
 - [interval](#)
 - [p_vendor_callbacks](#)

Class Instance Creation

Creating a Vendor class instance is done by calling the function `sLusbd_vendor_create_instance()` , which takes three configuration arguments that are described below.

intr_en

Boolean that indicates if a pair of interrupt endpoints should be added or not.

Value	Description
true	A pair of IN/OUT endpoints will be added and made available to the embedded application.
false	No interrupt endpoint will be added. Only a pair of Bulk IN/OUT endpoint will be available.

interval

- If you set `intr_en` to `true` , you can specify the interrupt endpoints polling interval (in milliseconds).
- If you set `intr_en` to `false` , you can set `interval` to 0 as it will be ignored by the class.

p_vendor_callbacks

`p_vendor_callbacks` is a pointer to a callback functions structure variable. that you can specify to handle the class specific control requests. If you don't use any class specific requests or needs enable/disable notification, you can set this to `NULL` .

The example below provides the expected signature of your class specific requests handler.

Example - Signature of Class-Specific Request Function

```
void app_usbd_vendor_req_handle(uint8_t class_nbr, (1)
                                const sLusbd_setup_req_t *p_setup_req); (2)

sLusbd_vendor_callbacks_t app_usbd_vendor_callback_functions =
{
    .enable = NULL,
    .disable = NULL,
    .setup_req = app_usbd_vendor_req_handle,
};
```

(1) Vendor class instance number.

(2) Pointer to a received setup request from host.

USB Device Vendor Class Programming Guide

This section explains how to use the Vendor class.

- [Initializing the USB Device Vendor Class](#)
- [Adding a USB Device Vendor Class Instance to your Device](#)
- [Communicating using the USB Device Vendor Class](#)

Initializing the USB Device Vendor Class

To add a Vendor Class functionality to your device, first initialize the class by calling the function `USBD_Vendor_Init()`.

The example below shows how to call `sl_usbd_vendor_init()`.

Example - Calling `sl_usbd_vendor_init()`

```
sl_status_t status;

status = sl_usbd_vendor_init();
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

Adding a USB Device Vendor Class Instance to your Device

To add vendor class functionality to your device, you must first create an instance, then add it to your device's configuration(s).

- [Creating a Vendor Class Instance](#)
- [Adding the Vendor Class Instance to Your Device's Configuration\(s\)](#)

Creating a Vendor Class Instance

Create a Vendor class instance by calling the function `sl_usbd_vendor_create_instance()`.

The example below shows how to call `sl_usbd_vendor_create_instance()` using default arguments. For more information about the configuration arguments to pass to `sl_usbd_vendor_create_instance()`, see [USB Device Vendor Class Instance Configurations](#).

Example - Calling `sl_usbd_vendor_create_instance()`

```
uint8_t class_nbr;
sl_status_t status;

status = sl_usbd_vendor_create_instance(false, (1)
                                         0u, (2)
                                         app_usbd_vendor_callback_functions, (3)
                                         &class_nbr);
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

(1) No Interrupt endpoints with this class instance.

(2) Interval is ignored since Interrupt endpoints are disabled.

(3) Callback function that is part of your application that handles vendor-specific class requests. See [Communicating using the USB Device Vendor Class](#) for more information.

Adding the Vendor Class Instance to Your Device's Configuration(s)

After you have created a vendor class instance, you can add it to a configuration by calling the function `USBD_Vendor_ConfigAdd()`.

The example below shows how to call `sl_usbd_vendor_add_to_configuration()` using default arguments.

Example - Calling `sl_usbd_vendor_add_to_configuration()`

```
sl_status_t status;

status = sl_usbd_vendor_add_to_configuration(class_nbr,           (1)
                                           config_nbr_fs);       (2)
if (status != SL_STATUS_OK) {
    /* An error occurred. Error handling should be added here. */
}
```

- (1) Class number to add to the configuration returned by `sl_usbd_vendor_create_instance()`.
- (2) Configuration number (here adding it to a Full-Speed configuration).

Communicating Using the USB Device Vendor Class

- [General](#)
- [Synchronous Communication](#)
- [Asynchronous Communication](#)
- [Vendor Request](#)

General

The Vendor class offers the following functions to communicate with the host. For more details about the parameters of the function, see [USB Device Vendor API](#).

Table - Vendor Communication API Summary

Function name	Operation
<code>sl_usbd_vendor_read_bulk_sync()</code>	Receives data from host through bulk OUT endpoint. This function is blocking.
<code>sl_usbd_vendor_write_bulk_sync()</code>	Sends data to host through bulk IN endpoint. This function is blocking.
<code>sl_usbd_vendor_read_bulk_async()</code>	Receives data from host through bulk OUT endpoint. This function is non-blocking.
<code>sl_usbd_vendor_write_bulk_async()</code>	Sends data to host through bulk IN endpoint. This function is non-blocking.
<code>sl_usbd_vendor_read_interrupt_sync()</code>	Receives data from host through interrupt OUT endpoint. This function is blocking.
<code>sl_usbd_vendor_write_interrupt_sync()</code>	Sends data to host through interrupt IN endpoint. This function is blocking.
<code>sl_usbd_vendor_read_interrupt_async() ()</code>	Receives data from host through interrupt OUT endpoint. This function is non-blocking.
<code>sl_usbd_vendor_write_interrupt_async() ()</code>	Sends data to host through interrupt IN endpoint. This function is non-blocking.

The vendor requests are also another way to communicate with the host. When managing vendor requests sent by the host, the application can receive or send data from or to the host using the control endpoint; you will need to provide an application callback passed as a parameter of `sl_usbd_vendor_create_instance()`.

Synchronous Communication

Synchronous communication means that the transfer is blocking. When a function is called, the application blocks until the transfer completes with or without an error. A timeout can be specified to avoid waiting forever.

The example below shows a read and write that receives data from the host using the bulk OUT endpoint and sends data to the host using the bulk IN endpoint.

Example - Synchronous Bulk Read and Write

```

__ALIGNED(4) uint8_t rx_buf[2];
__ALIGNED(4) uint8_t tx_buf[2];
uint32_t          xfer_len;
sl_status_t       status;

status = sl_usbd_vendor_read_bulk_sync(class_nbr,           (1)
                                       (void *)&rx_buf[0], (2)
                                       2u,
                                       0u,                  (3)
                                       &xfer_len);

if (status != SL_STATUS_OK) {
    /* $$$$ Handle the error. */
}

status = sl_usbd_vendor_write_bulk_sync(class_nbr,          (1)
                                       (void *)&tx_buf[0], (4)
                                       2u,
                                       0u,                  (3)
                                       false,                (5)
                                       &xfer_len);

if (status != SL_STATUS_OK) {
    /* $$$$ Handle the error. */
}

```

- (1) The class instance number created with `sl_usbd_vendor_create_instance()` provides an internal reference to the Vendor class to route the transfer to the proper bulk OUT or IN endpoint.
- (2) The application must ensure that the buffer provided to the function is large enough to accommodate all the data. Otherwise, synchronization issues might happen.
- (3) In order to avoid an infinite blocking situation, a timeout expressed in milliseconds can be specified. A value of '0' makes the application task wait forever.
- (4) The application provides the initialized transmit buffer.
- (5) If this flag is set to `true`, and the transfer length is multiple of the endpoint maximum packet size, the device stack will send a zero-length packet to the host to signal the end of the transfer.

The use of interrupt endpoint communication functions, `sl_usbd_vendor_read_interrupt_sync()` and `sl_usbd_vendor_write_interrupt_sync()`, is similar to bulk endpoint communication functions presented in [Example - Synchronous Bulk Read and Write](#).

Asynchronous Communication

Asynchronous communication means that the transfer is non-blocking. When a function is called, the application passes the transfer information to the device stack and does not block. Other application processing can be done while the transfer is in progress over the USB bus. Once the transfer has completed, a callback function is called by the device stack to inform the application about the transfer completion. The example below shows asynchronous read and write.

Example - Asynchronous Bulk Read and Write

```

void app_usbd_vendor_comm (uint8_t class_nbr)
{
    __ALIGNED(4) uint8_t rx_buf[2];
    __ALIGNED(4) uint8_t tx_buf[2];
    sl_status_t status;

    status = sl_usbd_vendor_read_bulk_async(class_nbr,           (1)
                                             (void *)&rx_buf[0], (2)
                                             2u,
                                             app_usbd_vendor_rx_completed, (3)
                                             NULL); (4)
    if (status != SL_STATUS_OK) {
        /* $$$$ Handle the error. */
    }
    status = sl_usbd_vendor_write_bulk_async(class_nbr,          (1)
                                              (void *)&tx_buf[0], (5)
                                              2u,
                                              app_usbd_vendor_tx_completed, (3)
                                              NULL, (4)
                                              false); (6)
    if (status != SL_STATUS_OK) {
        /* $$$$ Handle the error. */
    }
}

static void app_usbd_vendor_rx_completed(uint8_t class_nbr, (3)
                                          void *p_buf,
                                          uint32_t buf_len,
                                          uint32_t xfer_len,
                                          void *p_callback_arg,
                                          sl_status_t status)
{
    if (status != SL_STATUS_OK) {
        /* $$$$ Do some processing. */
    } else {
        /* $$$$ Handle the error. */
    }
}

static void app_usbd_vendor_tx_completed(uint8_t class_nbr, (3)
                                          void *p_buf,
                                          uint32_t buf_len,
                                          uint32_t xfer_len,
                                          void *p_callback_arg,
                                          sl_status_t status)
{
    if (status != SL_STATUS_OK) {
        /* $$$$ Do some processing. */
    } else {
        /* $$$$ Handle the error. */
    }
}

```

(1) The class instance number provides an internal reference to the Vendor class to route the transfer to the proper bulk OUT or IN endpoint.

(2) The application must ensure that the buffer provided is large enough to accommodate all the data. Otherwise, there may be synchronization issues.

(3) The application provides a callback function pointer passed as a parameter. Upon completion of the transfer, the device stack calls this callback function so that the application can finalize the transfer by analyzing the transfer result. For instance, on completion of a read operation, the application might perform processing on the received data. Upon write completion, the application can indicate if the write was successful and how many bytes were sent.

- (4) An argument associated with the callback can be also passed. Then in the callback context, some private information can be retrieved.
- (5) The application provides the initialized transmit buffer.
- (6) If this flag is set to `true`, and the transfer length is a multiple of the endpoint maximum packet size, the device stack will send a zero-length packet to the host to signal the end of transfer.

The use of interrupt endpoint communication functions, `sl_usbd_vendor_read_interrupt_async()` and `sl_usbd_vendor_write_interrupt_async()`, is similar to the bulk endpoint communication functions presented in [Example - Asynchronous Bulk Read and Write](#).

Vendor Request

The USB 2.0 specification defines three types of requests: standard, class, and vendor. All standard requests are handled directly by the core layer, while class requests are managed by the proper associated class.

Vendor requests can be processed by the vendor class. To process vendor requests, you must provide an application callback as a parameter of `sl_usbd_vendor_create_instance()`. After a vendor request is received by the USB device, it must be decoded properly. The example below shows vendor request decoding.

Certain requests may be required to receive from or send to the host during the data stage of a control transfer. If no data stage is present, you only have to decode the Setup packet. This example shows the three types of data stage management: no data, data OUT and data IN.

Example - Vendor Request Decoding

```

#define APP_VENDOR_REQ_NO_DATA          0x01u
#define APP_VENDOR_REQ_RECEIVE_DATA_FROM_HOST  0x02u
#define APP_VENDOR_REQ_SEND_DATA_TO_HOST      0x03u

#define APP_VENDOR_REQ_DATA_BUF_SIZE        50u

static uint8_t app_vendor_req_buf[APP_VENDOR_REQ_DATA_BUF_SIZE];

static bool app_usbd_vendor_req (uint8_t          class_nbr,
                                const sl_usbd_setup_req_t *p_setup_req) (1)
{
    bool    valid;
    sl_status_t status;
    uint16_t req_len;
    uint32_t xfer_len;

    (void)class_nbr;

    switch(p_setup_req->bRequest) { (2)
    case APP_VENDOR_REQ_NO_DATA: (3)
        valid = true;
        break;

    case APP_VENDOR_REQ_RECEIVE_DATA_FROM_HOST: (4)
        req_len = p_setup_req->wLength;
        if (req_len > APP_VENDOR_REQ_DATA_BUF_SIZE) {
            // Not enough room to receive data.
            return (false);
        }

        // Receive data via Control OUT EP.
        // Wait transfer completion forever.
        status = sl_usbd_core_read_control_sync((void *)&app_vendor_req_buf[0u],
                                                req_len,
                                                0u,
                                                &xfer_len);

        if (status != SL_STATUS_OK) {
            valid = false;
        } else {
            valid = true;
        }
        break;

    case APP_VENDOR_REQ_SEND_DATA_TO_HOST: (5)
        req_len = APP_VENDOR_REQ_DATA_BUF_SIZE;

        // Fill buf with a pattern.
        Mem_Set((void *)&AppVendorReqBuf[0u],
                'A',
                req_len);

        // Send data via Control IN EP.
        // Wait transfer completion forever.
        status = sl_usbd_core_write_control_sync((void *)&app_vendor_req_buf[0u],
                                                req_len,
                                                0u,
                                                false,
                                                &xfer_len);

        if (status != SL_STATUS_OK) {
            valid = DEF_FAIL;
        } else {
            valid = DEF_OK;
        }
        break;

    default: (6)
        // Request is not supported.

```

```

    valid = true; break;

    case APP_VENDOR_REQ_RECEIVE_DATA_FROM_HOST: (4)
        req_len = p_setup_req->wLength; if(req_len > APP_VENDOR_REQ_DATA_BUF_SIZE){ // Not enough room to receive data. return(false); } // Receive data via Control OUT EP. // Wait transfer completion forever.
        status = sl_usbd_core_read_control_sync((void *)&app_vendor_req_buf[0u], req_len, 0u, &xfer_len); if(status != SL_STATUS_OK){
            valid = false; } else{
            valid = true; } break;

    case APP_VENDOR_REQ_SEND_DATA_TO_HOST: (5)
        req_len = APP_VENDOR_REQ_DATA_BUF_SIZE; // Fill buf with a pattern. Mem_Set((void *)&AppVendorReqBuf[0u], 'A', req_len); // Send data via Control IN EP. // Wait transfer completion forever.
        status = sl_usbd_core_write_control_sync((void *)&app_vendor_req_buf[0u], req_len, 0u, false, &xfer_len); if(status != SL_STATUS_OK){
            valid = DEF_FAIL; } else{
            valid = DEF_OK; } break;

    default: (6) // Request is not supported.
        valid = DEF_FAIL; break; } return(valid); }

```

(1) The core will pass the Setup packet content to your application. The structure `sl_usbd_setup_req_t` contains the same fields as defined by the USB 2.0 specification (refer to section "9.3 USB Device Requests" of the specification for more details):

```

typedef struct {
    uint8_t bmRequestType; /* Characteristics of request. */
    uint8_t bRequest; /* Specific request. */
    uint16_t wValue; /* Varies according to request. */
    uint16_t wIndex; /* Varies according to request; typically used as index. */
    uint16_t wLength; /* Transfer length if data stage present. */
} sl_usbd_setup_req_t;

```

(2) Determine the request. You may use a switch statement if you are using different requests. In this example, there are three different requests corresponding to the three types of the data stage: `APP_VENDOR_REQ_NO_DATA`, `APP_VENDOR_REQ_RECEIVE_DATA_FROM_HOST`, and `APP_VENDOR_REQ_SEND_DATA_TO_HOST`.

(3) If no data stage is present, you only need to decode the other fields. The presence of a data stage or not is indicated by the field `wLength` being non-null or null.

(4) If the host sends data to the device, you must call the function `sl_usbd_core_read_control_sync()`. The buffer provided should be able to contain up to `wLength` bytes. If any error occurs, return `false` to the core that will stall the status stage of the control transfer, indicating to the host that the request cannot be processed. `true` is returned in case of success.

(5) If the host receives data from the device, you must call the function `sl_usbd_core_write_control_sync()`. If any error occurs, return `false` to the core that will stall the status stage of the control transfer, indicating to the host that the request cannot be processed. `true` is returned in case of success.

(6) In this example, all requests not recognized are marked by returning `false` to the core. This one will stall the data or status stage of the control transfer indicating to the host that the request is not supported.

The host sends vendor requests through a host vendor application. USB libraries, such as `libusb`, can be used to help you develop your custom host vendor application.

API Documentation

API Documentation

List of modules	Description
USB Device API	USB Device API
USB Device ACM API	USB Device ACM API
USB Device CDC API	USB Device CDC API
USB Device Core API	USB Device Core API
USB Device HID API	USB Device HID API
USB Device MSC API	USB Device MSC API
USB Device MSC SCSI API	USB Device MSC SCSI API
USB Device Vendor API	USB Device Vendor API

USB Device API

USB Device API

USB Device API.

Modules

[USB Device ACM API](#)

[USB Device CDC API](#)

[USB Device Core API](#)

[USB Device HID API](#)

[USB Device MSC API](#)

[USB Device MSC SCSI API](#)

[USB Device Vendor API](#)

USB Device ACM API

USB Device ACM API

USB Device CDC ACM API.

Modules

[sl_usbd_cdc_acm_line_coding_t](#)

[sl_usbd_cdc_acm_callbacks_t](#)

Functions

sl_status_t	sl_usbd_cdc_acm_init (void) GLOBAL VARIABLES.
sl_status_t	sl_usbd_cdc_acm_create_instance (uint16_t line_state_interval, uint16_t call_mgmt_capabilities, sl_usbd_cdc_acm_callbacks_t *p_acm_callbacks, uint8_t *p_subclass_nbr) Add a new instance of the CDC ACM serial emulation subclass.
sl_status_t	sl_usbd_cdc_acm_add_to_configuration (uint8_t subclass_nbr, uint8_t config_nbr) Add a CDC ACM subclass class instance into USB device configuration.
sl_status_t	sl_usbd_cdc_acm_is_enabled (uint8_t subclass_nbr, bool *p_enabled) Get the CDC ACM serial emulation subclass enable state.
sl_status_t	sl_usbd_cdc_acm_read (uint8_t subclass_nbr, uint8_t *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len) Receive data on the CDC ACM serial emulation subclass.
sl_status_t	sl_usbd_cdc_acm_read_async (uint8_t subclass_nbr, uint8_t *p_buf, uint32_t buf_len, sl_usbd_cdc_async_function_t async_fnct, void *p_async_arg) Receive data on the CDC ACM serial emulation subclass asynchronously.
sl_status_t	sl_usbd_cdc_acm_write (uint8_t subclass_nbr, uint8_t *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len) Send data on the CDC ACM serial emulation subclass.
sl_status_t	sl_usbd_cdc_acm_write_async (uint8_t subclass_nbr, uint8_t *p_buf, uint32_t buf_len, sl_usbd_cdc_async_function_t async_fnct, void *p_async_arg) Send data on the CDC ACM serial emulation subclass asynchronously.
sl_status_t	sl_usbd_cdc_acm_get_line_control_state (uint8_t subclass_nbr, uint8_t *p_line_ctrl) Return the state of control lines.
sl_status_t	sl_usbd_cdc_acm_get_line_coding (uint8_t subclass_nbr, sl_usbd_cdc_acm_line_coding_t *p_line_coding) Get the current state of the line coding.
sl_status_t	sl_usbd_cdc_acm_set_line_coding (uint8_t subclass_nbr, sl_usbd_cdc_acm_line_coding_t *p_line_coding) Set a new line coding.
sl_status_t	sl_usbd_cdc_acm_set_line_state_event (uint8_t subclass_nbr, uint8_t events) Set a line state event(s).

sl_status_t [sl_usbd_cdc_acm_clear_line_state_event](#)(uint8_t subclass_nbr, uint8_t events)
Clear a line state event(s).

Macros

- #define [SL_USBD_CDC_ACM_NBR_NONE](#) 255u
MODULE.
- #define [SL_USBD_CDC_ACM_PARITY_NONE](#) 0u
PORT SETTINGS DEFINES.
- #define [SL_USBD_CDC_ACM_PARITY_ODD](#) 1u
- #define [SL_USBD_CDC_ACM_PARITY_EVEN](#) 2u
- #define [SL_USBD_CDC_ACM_PARITY_MARK](#) 3u
- #define [SL_USBD_CDC_ACM_PARITY_SPACE](#) 4u
- #define [SL_USBD_CDC_ACM_STOP_BIT_1](#) 0u
- #define [SL_USBD_CDC_ACM_STOP_BIT_1_5](#) 1u
- #define [SL_USBD_CDC_ACM_STOP_BIT_2](#) 2u
- #define [SL_USBD_CDC_ACM_CTRL_BREAK](#) 0x01u
LINE EVENTS FLAGS DEFINES.
- #define [SL_USBD_CDC_ACM_CTRL_RTS](#) 0x02u
- #define [SL_USBD_CDC_ACM_CTRL_DTR](#) 0x04u
- #define [SL_USBD_CDC_ACM_STATE_DCD](#) 0x01u
- #define [SL_USBD_CDC_ACM_STATE_DSR](#) 0x02u
- #define [SL_USBD_CDC_ACM_STATE_BREAK](#) 0x04u
- #define [SL_USBD_CDC_ACM_STATE_RING](#) 0x08u
- #define [SL_USBD_CDC_ACM_STATE_FRAMING](#) 0x10u
- #define [SL_USBD_CDC_ACM_STATE_PARITY](#) 0x20u
- #define [SL_USBD_CDC_ACM_STATE_OVERUN](#) 0x40u
- #define [SL_USBD_CDC_ACM_CALL_MGMT_DEV](#) 0x01u
CALL MANAGEMENT CAPABILITIES.
- #define [SL_USBD_CDC_ACM_CALL_MGMT_DATA_CCI_DCI](#) 0x02u
- #define [SL_USBD_CDC_ACM_CALL_MGMT_DATA_OVER_DCI](#) (0x02u | 0x01u)

Function Documentation

sl_usbd_cdc_acm_init

sl_status_t sl_usbd_cdc_acm_init (void)

GLOBAL VARIABLES.

Parameters

N/A		
-----	--	--

MACROS GLOBAL CONSTANTS FUNCTION PROTOTYPES CDC ACM FUNCTIONS

Initialize the CDC ACM serial emulation subclass.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 170 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

sl_usbd_cdc_acm_create_instance

```
sl_status_t sl_usbd_cdc_acm_create_instance (uint16_t line_state_interval, uint16_t call_mgmt_capabilities,
sl_usbd_cdc_acm_callbacks_t *p_acm_callbacks, uint8_t *p_subclass_nbr)
```

Add a new instance of the CDC ACM serial emulation subclass.

Parameters

N/A	line_state_interval	Line state notification interval in milliseconds (value must be a power of 2).
N/A	call_mgmt_capabilities	Call Management Capabilities bitmap. OR'ed of the following flags: • SL_USBD_CDC_ACM_CALL_MGMT_DEV Device handles call management itself. • SL_USBD_CDC_ACM_CALL_MGMT_DATA_CCI_DCI Device can send/receive call management information over a Data Class interface.
N/A	p_acm_callbacks	Optional pointers to callback functions to be called on various events.
N/A	p_subclass_nbr	Param to variable that will receive CDC ACM serial emulation subclass instance number.

Returns

- Return SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 201 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

sl_usbd_cdc_acm_add_to_configuration

```
sl_status_t sl_usbd_cdc_acm_add_to_configuration (uint8_t subclass_nbr, uint8_t config_nbr)
```

Add a CDC ACM subclass class instance into USB device configuration.

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
N/A	config_nbr	Configuration index to add new test class interface to.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 215 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

sl_usbd_cdc_acm_is_enabled

```
sl_status_t sl_usbd_cdc_acm_is_enabled (uint8_t subclass_nbr, bool *p_enabled)
```

Get the CDC ACM serial emulation subclass enable state.

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
N/A	p_enabled	Boolean to a variable that will receive enable status. The variable is set to true, CDC ACM serial emulation is enabled. The variable is set to false, CDC ACM serial emulation is not enabled.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 229 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

sl_usbd_cdc_acm_read

```
sl_status_t sl_usbd_cdc_acm_read (uint8_t subclass_nbr, uint8_t *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len)
```

Receive data on the CDC ACM serial emulation subclass.

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
N/A	p_buf	Pointer to the destination buffer to receive data.
N/A	buf_len	Number of octets to receive.
N/A	timeout	Timeout, in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will receive transfer length. If no errors are returned, the variable is set to number of octets received, otherwise it is set to 0.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 249 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

sl_usbd_cdc_acm_read_async

```
sl_status_t sl_usbd_cdc_acm_read_async (uint8_t subclass_nbr, uint8_t *p_buf, uint32_t buf_len, sl_usbd_cdc_async_function_t async_fnct, void *p_async_arg)
```

Receive data on the CDC ACM serial emulation subclass asynchronously.

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
N/A	p_buf	Pointer to the destination buffer to receive data.
N/A	buf_len	Number of octets to receive.
N/A	async_fnct	Function that will be invoked upon completion of receive operation.
N/A	p_async_arg	Pointer to the argument that will be passed as parameter of 'async_fnct'.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 270 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

sl_usbd_cdc_acm_write

```
sl_status_t sl_usbd_cdc_acm_write (uint8_t subclass_nbr, uint8_t *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len)
```

Send data on the CDC ACM serial emulation subclass.

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
N/A	p_buf	Pointer to the buffer of data that will be transmitted.
N/A	buf_len	Number of octets to transmit.
N/A	timeout	Timeout in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will receive transfer length. If no errors are returned, the variable is set to number of octets transmitted, otherwise it is set to 0.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 293 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

sl_usbd_cdc_acm_write_async

```
sl_status_t sl_usbd_cdc_acm_write_async (uint8_t subclass_nbr, uint8_t *p_buf, uint32_t buf_len, sl_usbd_cdc_async_function_t async_fnct, void *p_async_arg)
```

Send data on the CDC ACM serial emulation subclass asynchronously.

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
N/A	p_buf	Pointer to the buffer of data that will be transmitted.
N/A	buf_len	Number of octets to transmit.
N/A	async_fnct	Function that will be invoked upon completion of transfer operation.
N/A	p_async_arg	Pointer to the argument that will be passed as parameter of 'async_fnct'.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 314 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

sl_usbd_cdc_acm_get_line_control_state

```
sl_status_t sl_usbd_cdc_acm_get_line_control_state (uint8_t subclass_nbr, uint8_t *p_line_ctrl)
```

Return the state of control lines.

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
N/A	p_line_ctrl	Pointer to the variable that will be set with the state of the control lines, encoded in bit fields.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 330 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

sl_usbd_cdc_acm_get_line_coding

```
sl_status_t sl_usbd_cdc_acm_get_line_coding (uint8_t subclass_nbr, sl_usbd_cdc_acm_line_coding_t *p_line_coding)
```

Get the current state of the line coding.

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
N/A	p_line_coding	Pointer to the structure where the current line coding will be stored.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 342 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

sl_usbd_cdc_acm_set_line_coding

```
sl_status_t sl_usbd_cdc_acm_set_line_coding (uint8_t subclass_nbr, sl_usbd_cdc_acm_line_coding_t *p_line_coding)
```

Set a new line coding.

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
N/A	p_line_coding	Pointer to the structure that contains the new line coding.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 354 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

sl_usbd_cdc_acm_set_line_state_event

```
sl_status_t sl_usbd_cdc_acm_set_line_state_event (uint8_t subclass_nbr, uint8_t events)
```

Set a line state event(s).

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
-----	--------------	--

N/A	events	Line state event(s) to set. OR'ed of the following flags: - SL_USBD_CDC_ACM_STATE_DCD - SL_USBD_CDC_ACM_STATE_DSR - SL_USBD_CDC_ACM_STATE_BREAK - SL_USBD_CDC_ACM_STATE_RING - SL_USBD_CDC_ACM_STATE_FRAMING - SL_USBD_CDC_ACM_STATE_PARITY - SL_USBD_CDC_ACM_STATE_OVERUN
-----	--------	--

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 373 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

sl_usbd_cdc_acm_clear_line_state_event

```
sl_status_t sl_usbd_cdc_acm_clear_line_state_event (uint8_t subclass_nbr, uint8_t events)
```

Clear a line state event(s).

Parameters

N/A	subclass_nbr	CDC ACM serial emulation subclass instance number.
N/A	events	Line state event(s) set to be cleared. OR'ed of the following flags (see Note #1) : - SL_USBD_CDC_ACM_STATE_DCD Set DCD signal (Rx carrier). - SL_USBD_CDC_ACM_STATE_DSR Set DSR signal (Tx carrier).

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) USB PSTN spec ver 1.20 states: "For the irregular signals like break, the incoming ring signal, or the overrun error state, this will reset their values to zero and again will not send another notification until their state changes." The irregular events are automatically cleared by the ACM serial emulation subclass.

Definition at line 393 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

Macro Definition Documentation

SL_USBD_CDC_ACM_NBR_NONE

```
#define SL_USBD_CDC_ACM_NBR_NONE
```

Value:

```
255u
```

MODULE.

INCLUDE FILES DEFINES

Definition at line 55 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

SL_USBD_CDC_ACM_PARITY_NONE

```
#define SL_USBD_CDC_ACM_PARITY_NONE
```

Value:

```
0u
```

PORT SETTINGS DEFINES.

Definition at line 61 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`**SL_USBD_CDC_ACM_PARITY_ODD**

```
#define SL_USBD_CDC_ACM_PARITY_ODD
```

Value:

```
1u
```

Definition at line 62 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`**SL_USBD_CDC_ACM_PARITY_EVEN**

```
#define SL_USBD_CDC_ACM_PARITY_EVEN
```

Value:

```
2u
```

Definition at line 63 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`**SL_USBD_CDC_ACM_PARITY_MARK**

```
#define SL_USBD_CDC_ACM_PARITY_MARK
```

Value:

```
3u
```

Definition at line 64 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`**SL_USBD_CDC_ACM_PARITY_SPACE**

```
#define SL_USBD_CDC_ACM_PARITY_SPACE
```

Value:

```
4u
```

Definition at line 65 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`**SL_USBD_CDC_ACM_STOP_BIT_1**


```
#define SL_USBD_CDC_ACM_STOP_BIT_1
```

Value:

```
0u
```

Definition at line 67 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_STOP_BIT_1_5

```
#define SL_USBD_CDC_ACM_STOP_BIT_1_5
```

Value:

```
1u
```

Definition at line 68 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_STOP_BIT_2

```
#define SL_USBD_CDC_ACM_STOP_BIT_2
```

Value:

```
2u
```

Definition at line 69 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_CTRL_BREAK

```
#define SL_USBD_CDC_ACM_CTRL_BREAK
```

Value:

```
0x01u
```

LINE EVENTS FLAGS DEFINES.

Definition at line 75 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_CTRL_RTS

```
#define SL_USBD_CDC_ACM_CTRL_RTS
```

Value:

```
0x02u
```

Definition at line 76 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_CTRL_DTR

```
#define SL_USBD_CDC_ACM_CTRL_DTR
```

Value:

```
0x04u
```

Definition at line 77 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_STATE_DCD

```
#define SL_USBD_CDC_ACM_STATE_DCD
```

Value:

```
0x01u
```

Definition at line 79 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_STATE_DSR

```
#define SL_USBD_CDC_ACM_STATE_DSR
```

Value:

```
0x02u
```

Definition at line 80 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_STATE_BREAK

```
#define SL_USBD_CDC_ACM_STATE_BREAK
```

Value:

```
0x04u
```

Definition at line 81 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_STATE_RING

```
#define SL_USBD_CDC_ACM_STATE_RING
```

Value:

```
0x08u
```

Definition at line 82 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_STATE_FRAMING

```
#define SL_USBD_CDC_ACM_STATE_FRAMING
```

Value:

```
0x10u
```

Definition at line 83 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_STATE_PARITY

```
#define SL_USBD_CDC_ACM_STATE_PARITY
```

Value:

```
0x20u
```

Definition at line 84 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_STATE_OVERUN

```
#define SL_USBD_CDC_ACM_STATE_OVERUN
```

Value:

```
0x40u
```

Definition at line 85 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_CALL_MGMT_DEV

```
#define SL_USBD_CDC_ACM_CALL_MGMT_DEV
```

Value:

```
0x01u
```

CALL MANAGEMENT CAPABILITIES.

Note(s) : (1) See 'USB, Communications Class, Subclass Specification for PSTN Devices, Revision 1.2, February 9 2007', section '5.3.1 Call Management Functional Descriptor' for more details about the Call Management capabilities.

Definition at line 95 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_CALL_MGMT_DATA_CCI_DCI

```
#define SL_USBD_CDC_ACM_CALL_MGMT_DATA_CCI_DCI
```

Value:

```
0x02u
```

Definition at line 96 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

SL_USBD_CDC_ACM_CALL_MGMT_DATA_OVER_DCI

```
#define SL_USBD_CDC_ACM_CALL_MGMT_DATA_OVER_DCI
```

Value:

(0x02u | 0x01u)

Definition at line 97 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

sl_usbd_cdc_acm_line_coding_t

DATA TYPES.

line coding data type

Public Attributes

uint32_t	baudrate
uint8_t	parity
uint8_t	stop_bits
uint8_t	data_bits

Public Attribute Documentation

baudrate

uint32_t sl_usbd_cdc_acm_line_coding_t::baudrate

Definition at line 111 of file protocol/usb/inc/slUsbd_class_cdc_acm.h

parity

uint8_t sl_usbd_cdc_acm_line_coding_t::parity

Definition at line 112 of file protocol/usb/inc/slUsbd_class_cdc_acm.h

stop_bits

uint8_t sl_usbd_cdc_acm_line_coding_t::stop_bits

Definition at line 113 of file protocol/usb/inc/slUsbd_class_cdc_acm.h

data_bits

uint8_t sl_usbd_cdc_acm_line_coding_t::data_bits

Definition at line 114 of file protocol/usb/inc/slUsbd_class_cdc_acm.h

sl_usbd_cdc_acm_callbacks_t

CDC ACM CALLBACKS.

CDC ACM Callbacks

Public Attributes

void(*	enable	Callback for enable event.
void(*	disable	Callback for disable event.
void(*	line_control_changed	Callback for line control changed event.
bool(*	line_coding_changed	Callback for line coding change event.

Public Attribute Documentation

enable

```
void(* sl_usbd_cdc_acm_callbacks_t::enable) (uint8_t subclass_nbr)
```

Callback for enable event.

Definition at line 122 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

disable

```
void(* sl_usbd_cdc_acm_callbacks_t::disable) (uint8_t subclass_nbr)
```

Callback for disable event.

Definition at line 124 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

line_control_changed

```
void(* sl_usbd_cdc_acm_callbacks_t::line_control_changed) (uint8_t subclass_nbr, uint8_t event, uint8_t event_chngd)
```

Callback for line control changed event.

Definition at line 126 of file `protocol/usb/inc/sl_usbd_class_cdc_acm.h`

line_coding_changed

```
bool(* sl_usbd_cdc_acm_callbacks_t::line_coding_changed) (uint8_t subclass_nbr, sl_usbd_cdc_acm_line_coding_t
```

```
*p_line_coding)
```

Callback for line coding change event.

Definition at line 130 of file protocol/usb/inc/sl_usbd_class_cdc_acm.h

USB Device CDC API

USB Device CDC API

USB Device CDC API.

Modules

[sl_usbd_cdc_subclass_driver_t](#)

Typedefs

```
typedef void(* sl\_usbd\_cdc\_async\_function\_t)(uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void
*p_callback_arg, sl_status_t status)
Async comm callback.
```

Functions

sl_status_t	sl_usbd_cdc_init (void) FUNCTION PROTOTYPES.
sl_status_t	sl_usbd_cdc_create_instance (uint8_t subclass, sl_usbd_cdc_subclass_driver_t *p_subclass_drv, void *p_subclass_arg, uint8_t protocol, bool notify_en, uint16_t notify_interval, uint8_t *p_class_nbr) Add a new instance of the CDC class.
sl_status_t	sl_usbd_cdc_add_to_configuration (uint8_t class_nbr, uint8_t config_nbr) Add a CDC instance into the USB device configuration.
sl_status_t	sl_usbd_cdc_is_enabled (uint8_t class_nbr, bool *p_enabled) Get the CDC class enable state.
sl_status_t	sl_usbd_cdc_add_data_interface (uint8_t class_nbr, bool isoc_en, uint8_t protocol, uint8_t *p_if_nbr) Add a data interface class to the CDC communication interface class.
sl_status_t	sl_usbd_cdc_read_data (uint8_t class_nbr, uint8_t data_if_nbr, uint8_t *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len) Receive data on the CDC data interface.
sl_status_t	sl_usbd_cdc_read_data_async (uint8_t class_nbr, uint8_t data_if_nbr, uint8_t *p_buf, uint32_t buf_len, sl_usbd_cdc_async_function_t async_fnct, void *p_async_arg) Receive data on the CDC data interface asynchronously.
sl_status_t	sl_usbd_cdc_write_data (uint8_t class_nbr, uint8_t data_if_nbr, uint8_t *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len) Send data on the CDC data interface.
sl_status_t	sl_usbd_cdc_write_data_async (uint8_t class_nbr, uint8_t data_if_nbr, uint8_t *p_buf, uint32_t buf_len, sl_usbd_cdc_async_function_t async_fnct, void *p_async_arg) Send data on the CDC data interface asynchronously.
sl_status_t	sl_usbd_cdc_notify_host (uint8_t class_nbr, uint8_t notification, uint16_t value, uint8_t *p_buf, uint16_t data_len, bool *p_result) Send a communication interface class notification to the host.

Macros

```

#define SL_USBD_CDC_NBR_NONE 255u
MODULE.

#define SL_USBD_CDC_DATA_IF_NBR_NONE 255u

#define SL_USBD_CDC_SUBCLASS_RSVD 0x00u
COMMUNICATIONS CLASS SUBCLASS CODES DEFINES.

#define SL_USBD_CDC_SUBCLASS_DLCM 0x01u

#define SL_USBD_CDC_SUBCLASS_ACM 0x02u

#define SL_USBD_CDC_SUBCLASS_TCM 0x03u

#define SL_USBD_CDC_SUBCLASS_MCCM 0x04u

#define SL_USBD_CDC_SUBCLASS_CAPICM 0x05u

#define SL_USBD_CDC_SUBCLASS_ENCM 0x06u

#define SL_USBD_CDC_SUBCLASS_ATM 0x07u

#define SL_USBD_CDC_SUBCLASS_WHCM 0x08u

#define SL_USBD_CDC_SUBCLASS_DEV_MGMT 0x09u

#define SL_USBD_CDC_SUBCLASS_MDLM 0x0Au

#define SL_USBD_CDC_SUBCLASS_OBEX 0x0Bu

#define SL_USBD_CDC_SUBCLASS_EEM 0x0Cu

#define SL_USBD_CDC_SUBCLASS_NCM 0x0Du

#define SL_USBD_CDC_SUBCLASS_VENDOR 0xFEu

#define SL_USBD_CDC_COMM_PROTOCOL_NONE 0x00u
COMMUNICATION CLASS PROTOCOL CODES DEFINES.

#define SL_USBD_CDC_COMM_PROTOCOL_AT_V250 0x01u

#define SL_USBD_CDC_COMM_PROTOCOL_AT_PCCA_101 0x02u

#define SL_USBD_CDC_COMM_PROTOCOL_AT_PCCA_101_ANNEX 0x03u

#define SL_USBD_CDC_COMM_PROTOCOL_AT_GSM_7_07 0x04u

#define SL_USBD_CDC_COMM_PROTOCOL_AT_3GPP_27_07 0x05u

#define SL_USBD_CDC_COMM_PROTOCOL_AT_TIA_CDMA 0x06u

#define SL_USBD_CDC_COMM_PROTOCOL_EEM 0x07u

#define SL_USBD_CDC_COMM_PROTOCOL_EXT 0xFEu

#define SL_USBD_CDC_COMM_PROTOCOL_VENDOR 0xFFu

#define SL_USBD_CDC_DATA_PROTOCOL_NONE 0x00u
DATA INTERFACE CLASS PROTOCOL CODES DEFINES.

```

```

#define SL_USBD_CDC_DATA_PROTOCOL_NTB 0x01u
#define SL_USBD_CDC_DATA_PROTOCOL_PHY 0x30u
#define SL_USBD_CDC_DATA_PROTOCOL_HDLC 0x31u
#define SL_USBD_CDC_DATA_PROTOCOL_TRANS 0x32u
#define SL_USBD_CDC_DATA_PROTOCOL_Q921M 0x50u
#define SL_USBD_CDC_DATA_PROTOCOL_Q921 0x51u
#define SL_USBD_CDC_DATA_PROTOCOL_Q921TM 0x52u
#define SL_USBD_CDC_DATA_PROTOCOL_COMPRES 0x90u
#define SL_USBD_CDC_DATA_PROTOCOL_Q9131 0x91u
#define SL_USBD_CDC_DATA_PROTOCOL_V24 0x92u
#define SL_USBD_CDC_DATA_PROTOCOL_CAPI 0x93u
#define SL_USBD_CDC_DATA_PROTOCOL_HOST 0xFDu
#define SL_USBD_CDC_DATA_PROTOCOL_CDC 0xFEu
#define SL_USBD_CDC_DATA_PROTOCOL_VENDOR 0xFFu
#define SL_USBD_CDC_REQ_SEND_ENCAP_COMM 0x00u
MANAGEMENT ELEMENTS REQUESTS DEFINES.
#define SL_USBD_CDC_REQ_GET_ENCAP_RESP 0x01u
#define SL_USBD_CDC_REQ_SET_COMM_FEATURE 0x02u
#define SL_USBD_CDC_REQ_GET_COMM_FEATURE 0x03u
#define SL_USBD_CDC_REQ_CLR_COMM_FEATURE 0x04u
#define SL_USBD_CDC_REQ_SET_AUX_LINE_STATE 0x10u
#define SL_USBD_CDC_REQ_SET_HOOK_STATE 0x11u
#define SL_USBD_CDC_REQ_PULSE_SETUP 0x12u
#define SL_USBD_CDC_REQ_SEND_PULSE 0x13u
#define SL_USBD_CDC_REQ_SET_PULSE_TIME 0x14u
#define SL_USBD_CDC_REQ_RING_AUX_JACK 0x15u
#define SL_USBD_CDC_REQ_SET_LINE_CODING 0x20u
#define SL_USBD_CDC_REQ_GET_LINE_CODING 0x21u
#define SL_USBD_CDC_REQ_SET_CTRL_LINE_STATE 0x22u
#define SL_USBD_CDC_REQ_SEND_BREAK 0x23u
#define SL_USBD_CDC_REQ_SET_RINGER_PAARMS 0x30u
#define SL_USBD_CDC_REQ_GET_RINGER_PARAMS 0x31u

```

```

#define SL_USBD_CDC_REQ_SET_OPERATION_PARAMS 0x32u
#define SL_USBD_CDC_REQ_GET_OPERATION_PARAMS 0x33u
#define SL_USBD_CDC_REQ_SET_LINE_PARAMS 0x34u
#define SL_USBD_CDC_REQ_GET_LINE_PARAMS 0x35u
#define SL_USBD_CDC_REQ_DIAL_DIGITS 0x36u
#define SL_USBD_CDC_REQ_SET_UNIT_PARAM 0x37u
#define SL_USBD_CDC_REQ_GET_UNIT_PARAM 0x38u
#define SL_USBD_CDC_REQ_CLR_UNUT_PARAM 0x39u
#define SL_USBD_CDC_REQ_GET_PROFILE 0x3Au
#define SL_USBD_CDC_REQ_SET_ETHER_MULTI_FILTER 0x40u
#define SL_USBD_CDC_REQ_SET_ETHER_PWR_MGT_FILTER 0x41u
#define SL_USBD_CDC_REQ_GET_ETHER_PWR_MGT_FILTER 0x42u
#define SL_USBD_CDC_REQ_SET_ETHER_PKT_FILTER 0x43u
#define SL_USBD_CDC_REQ_GET_ETHER_STAT 0x44u
#define SL_USBD_CDC_REQ_SET_ATM_DATA_FMT 0x50u
#define SL_USBD_CDC_REQ_GET_ATM_DATA_FMT 0x51u
#define SL_USBD_CDC_REQ_GET_ATM_DEV_STAT 0x52u
#define SL_USBD_CDC_REQ_SET_ATM_DFLT_VC 0x53u
#define SL_USBD_CDC_REQ_GET_ATM_VC_STAT 0x54u
#define SL_USBD_CDC_REQ_GET NTB_PARAM2 0x80u
#define SL_USBD_CDC_REQ_GET_NET_ADDR 0x81u
#define SL_USBD_CDC_REQ_SET_NET_ADDR 0x82u
#define SL_USBD_CDC_REQ_GET NTB_FMT 0x83u
#define SL_USBD_CDC_REQ_SET NTB_FMT 0x84u
#define SL_USBD_CDC_REQ_GET NTB_INPUT_SIZE 0x85u
#define SL_USBD_CDC_REQ_SET NTB_INPUT_SIZE 0x86u
#define SL_USBD_CDC_REQ_GET_MAX_DATAGRAM_SIZE 0x87u
#define SL_USBD_CDC_REQ_SET_MAX_DATAGRAM_SIZE 0x88u
#define SL_USBD_CDC_REQ_GET_CRC_MODE 0x89u
#define SL_USBD_CDC_REQ_SET_CRC_MODE 0x8Au
#define SL_USBD_CDC_DESC_TYPE_CS_IF 0x24u

```

CDC FUNCTIONAL DESCRIPTOR TYPE DEFINES.

```

#define SL_USBD_CDC_DESC_TYPE_CS_EP 0x25u

#define SL_USBD_CDC_DESC_SUBTYPE_HEADER 0x00u
CDC FUNCTIONAL DESCRIPTOR SUBTYPE DEFINES.

#define SL_USBD_CDC_DESC_SUBTYPE_CALL_MGMT 0x01u

#define SL_USBD_CDC_DESC_SUBTYPE_ACM 0x02u

#define SL_USBD_CDC_DESC_SUBTYPE_DLM 0x03u

#define SL_USBD_CDC_DESC_SUBTYPE_TEL_RINGER 0x04u

#define SL_USBD_CDC_DESC_SUBTYPE_TEL_CALL_LINE 0x05u

#define SL_USBD_CDC_DESC_SUBTYPE_UNION 0x06u

#define SL_USBD_CDC_DESC_SUBTYPE_COUNTRY_SEL 0x07u

#define SL_USBD_CDC_DESC_SUBTYPE_TEL_OPERA 0x08u

#define SL_USBD_CDC_DESC_SUBTYPE_USB_TERM 0x09u

#define SL_USBD_CDC_DESC_SUBTYPE_NET_CH_TERM 0x0Au

#define SL_USBD_CDC_DESC_SUBTYPE_PROTOCOL_UNIT 0x0Bu

#define SL_USBD_CDC_DESC_SUBTYPE_EXT_UNIT 0x0Cu

#define SL_USBD_CDC_DESC_SUBTYPE_MULTI_CH_MGMT 0x0Du

#define SL_USBD_CDC_DESC_SUBTYPE_CAPI_CTRL 0x0Eu

#define SL_USBD_CDC_DESC_SUBTYPE_ETHER_NET 0x0Fu

#define SL_USBD_CDC_DESC_SUBTYPE_ATM_NET 0x10u

#define SL_USBD_CDC_DESC_SUBTYPE_WIRELESS_CTRL 0x11u

#define SL_USBD_CDC_DESC_SUBTYPE_MOBILE DLM 0x12u

#define SL_USBD_CDC_DESC_SUBTYPE_DETAIL 0x13u

#define SL_USBD_CDC_DESC_SUBTYPE_DEV_MGMT 0x14u

#define SL_USBD_CDC_DESC_SUBTYPE_OBEX 0x15u

#define SL_USBD_CDC_DESC_SUBTYPE_CMD_SET 0x16u

#define SL_USBD_CDC_DESC_SUBTYPE_CMD_SET_DETAIL 0x17u

#define SL_USBD_CDC_DESC_SUBTYPE_TCM 0x18u

#define SL_USBD_CDC_DESC_SUBTYPE_OBEX_SERV 0x19u

#define SL_USBD_CDC_DESC_SUBTYPE_NCM 0x1Au

#define SL_USBD_CDC_DESC_SUBTYPE_VENDOR 0xFEu

#define SL_USBD_CDC_NOTIFICATION_HEADER 8u
CDC NOTIFICATION DEFINES.

```

Typedef Documentation

sl_usbd_cdc_async_function_t

```
typedef void(* sl_usbd_cdc_async_function_t) (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void *p_callback_arg, sl_status_t status) (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void *p_callback_arg, sl_status_t status)
```

Async comm callback.

Definition at line 262 of file protocol/usb/inc/sl_usbd_class_cdc.h

Function Documentation

sl_usbd_cdc_init

```
sl_status_t sl_usbd_cdc_init (void)
```

FUNCTION PROTOTYPES.

Parameters

N/A		
-----	--	--

CDC FUNCTIONS

Initializes CDC class.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 288 of file protocol/usb/inc/sl_usbd_class_cdc.h

sl_usbd_cdc_create_instance

```
sl_status_t sl_usbd_cdc_create_instance (uint8_t subclass, sl_usbd_cdc_subclass_driver_t *p_subclass_drv, void *p_subclass_arg, uint8_t protocol, bool notify_en, uint16_t notify_interval, uint8_t *p_class_nbr)
```

Add a new instance of the CDC class.

Parameters

N/A	subclass	Communication class subclass subcode (see Note #1).
N/A	p_subclass_drv	Pointer to the CDC subclass driver.
N/A	p_subclass_arg	Pointer to the CDC subclass driver argument.
N/A	protocol	Communication class protocol code.
N/A	notify_en	Notification enabled : true Enable CDC class notifications. false Disable CDC class notifications.
N/A	notify_interval	Notification interval in milliseconds (must be a power of 2).
N/A	p_class_nbr	Pointer to a variable that will receive class instance number. The variable is set to CDC class instance number if no errors are returned., otherwise it is set to SL_USBD_CDC_NBR_NONE.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Communication class subclass codes are defined in 'sl_usbd_class_cdc.h' 'SL_USBD_CDC_SUBCLASS_XXXX'.

Definition at line 316 of file protocol/usb/inc/sl_usbd_class_cdc.h

sl_usbd_cdc_add_to_configuration

```
sl_status_t sl_usbd_cdc_add_to_configuration (uint8_t class_nbr, uint8_t config_nbr)
```

Add a CDC instance into the USB device configuration.

Parameters

N/A	class_nbr	Class instance number.
N/A	config_nbr	Configuration index to which to add the new CDC class interface.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 333 of file protocol/usb/inc/sl_usbd_class_cdc.h

sl_usbd_cdc_is_enabled

```
sl_status_t sl_usbd_cdc_is_enabled (uint8_t class_nbr, bool *p_enabled)
```

Get the CDC class enable state.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_enabled	Pointer to a boolean variable that will receive enable state. The variable is set to true if CDC class is enabled. The variable is set to false if CDC class is not enabled.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 347 of file protocol/usb/inc/sl_usbd_class_cdc.h

sl_usbd_cdc_add_data_interface

```
sl_status_t sl_usbd_cdc_add_data_interface (uint8_t class_nbr, bool isoc_en, uint8_t protocol, uint8_t *p_if_nbr)
```

Add a data interface class to the CDC communication interface class.

Parameters

N/A	class_nbr	Class instance number.
N/A	isoc_en	Data interface isochronous enable (see Note #1) : • true Data interface uses isochronous EPs. • false Data interface uses bulk EPs.
N/A	protocol	Data interface protocol code.

N/A	p_if_nbr	Pointer to a variable that will receive data interface number.
-----	----------	--

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) The value of 'isoc_en' must be false. Isochronous EPs are not supported.

Definition at line 367 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

sl_usbd_cdc_read_data

```
sl_status_t sl_usbd_cdc_read_data (uint8_t class_nbr, uint8_t data_if_nbr, uint8_t *p_buf, uint32_t buf_len, uint16_t timeout,
uint32_t *p_xfer_len)
```

Receive data on the CDC data interface.

Parameters

N/A	class_nbr	Class instance number.
N/A	data_if_nbr	CDC data interface number.
N/A	p_buf	Pointer to the destination buffer to receive data.
N/A	buf_len	Number of octets to receive.
N/A	timeout	Timeout in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will receive transfer length. If no errors are returned, the variable is set to number of octets received, otherwise it is set to 0.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 391 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

sl_usbd_cdc_read_data_async

```
sl_status_t sl_usbd_cdc_read_data_async (uint8_t class_nbr, uint8_t data_if_nbr, uint8_t *p_buf, uint32_t buf_len,
sl_usbd_cdc_async_function_t async_fnct, void *p_async_arg)
```

Receive data on the CDC data interface asynchronously.

Parameters

N/A	class_nbr	Class instance number.
N/A	data_if_nbr	CDC data interface number.
N/A	p_buf	Pointer to the destination buffer to receive data.
N/A	buf_len	Number of octets to receive.
N/A	async_fnct	Function that will be invoked upon completion of receive operation.
N/A	p_async_arg	Pointer to the argument that will be passed as parameter of 'async_fnct'.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 415 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

sl_usbd_cdc_write_data

```
sl_status_t sl_usbd_cdc_write_data (uint8_t class_nbr, uint8_t data_if_nbr, uint8_t *p_buf, uint32_t buf_len, uint16_t timeout,
uint32_t *p_xfer_len)
```

Send data on the CDC data interface.

Parameters

N/A	class_nbr	Class instance number.
N/A	data_if_nbr	CDC data interface number.
N/A	p_buf	Pointer to the buffer of data that will be transmitted.
N/A	buf_len	Number of octets to transmit.
N/A	timeout	Timeout in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will receive transfer length. If no errors are returned, the variable is set to number of octets transmitted, otherwise it is set to 0.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 441 of file `protocol/usb/inc/slUsbd_class_cdc.h`

sl_usbd_cdc_write_data_async

```
sl_status_t sl_usbd_cdc_write_data_async (uint8_t class_nbr, uint8_t data_if_nbr, uint8_t *p_buf, uint32_t buf_len,
sl_usbd_cdc_async_function_t async_fnct, void *p_async_arg)
```

Send data on the CDC data interface asynchronously.

Parameters

N/A	class_nbr	Class instance number.
N/A	data_if_nbr	CDC data interface number.
N/A	p_buf	Pointer to the buffer of data that will be transmitted.
N/A	buf_len	Number of octets to transmit.
N/A	async_fnct	Function that will be invoked upon completion of transfer operation.
N/A	p_async_arg	Pointer to the argument that will be passed as parameter of 'async_fnct'.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 465 of file `protocol/usb/inc/slUsbd_class_cdc.h`

sl_usbd_cdc_notify_host

```
sl_status_t sl_usbd_cdc_notify_host (uint8_t class_nbr, uint8_t notification, uint16_t value, uint8_t *p_buf, uint16_t data_len,
bool *p_result)
```

Send a communication interface class notification to the host.

Parameters

N/A	class_nbr	Class instance number.
N/A	notification	Notification code.
N/A	value	Notification value.
N/A	p_buf	Pointer to the notification buffer (see Note #1).
N/A	data_len	Length of the data portion of the notification.
N/A	p_result	Pointer to a boolean variable that will receive the result of the operation. The variable is set to true if no errors are encountered, otherwise it is set to false.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) The notification buffer MUST contain space for the notification header 'SL_USBD_CDC_NOTIFICATION_HEADER' plus the variable-length data portion.

Definition at line 494 of file protocol/usb/inc/sl_usbd_class_cdc.h

Macro Definition Documentation

SL_USBD_CDC_NBR_NONE

```
#define SL_USBD_CDC_NBR_NONE
```

Value:

```
255u
```

MODULE.

INCLUDE FILES DEFINES CDC INVALID INSTANCE NUMBERS DEFINES

Definition at line 57 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_DATA_IF_NBR_NONE

```
#define SL_USBD_CDC_DATA_IF_NBR_NONE
```

Value:

```
255u
```

Definition at line 58 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_SUBCLASS_RSVD

```
#define SL_USBD_CDC_SUBCLASS_RSVD
```

Value:

```
0x00u
```

COMMUNICATIONS CLASS SUBCLASS CODES DEFINES.

Note(s) : (1) Communication class subclass codes are defined in table 4 from the CDC specification revision 1.2.

Definition at line 67 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_SUBCLASS_DLCM

```
#define SL_USBD_CDC_SUBCLASS_DLCM
```

Value:

```
0x01u
```

Definition at line 68 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_SUBCLASS_ACM

```
#define SL_USBD_CDC_SUBCLASS_ACM
```

Value:

```
0x02u
```

Definition at line 69 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_SUBCLASS_TCM

```
#define SL_USBD_CDC_SUBCLASS_TCM
```

Value:

```
0x03u
```

Definition at line 70 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_SUBCLASS_MCCM

```
#define SL_USBD_CDC_SUBCLASS_MCCM
```

Value:

```
0x04u
```

Definition at line 71 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_SUBCLASS_CAPICM

```
#define SL_USBD_CDC_SUBCLASS_CAPICM
```

Value:

```
0x05u
```

Definition at line 72 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_SUBCLASS_ENCM

```
#define SL_USBD_CDC_SUBCLASS_ENCM
```

Value:

```
0x06u
```

Definition at line 73 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_SUBCLASS_ATM

```
#define SL_USBD_CDC_SUBCLASS_ATM
```

Value:

```
0x07u
```

Definition at line 74 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_SUBCLASS_WHCM

```
#define SL_USBD_CDC_SUBCLASS_WHCM
```

Value:

```
0x08u
```

Definition at line 75 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_SUBCLASS_DEV_MGMT

```
#define SL_USBD_CDC_SUBCLASS_DEV_MGMT
```

Value:

```
0x09u
```

Definition at line 76 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_SUBCLASS_MDLM

```
#define SL_USBD_CDC_SUBCLASS_MDLM
```

Value:

```
0x0Au
```

Definition at line 77 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_SUBCLASS_OBEX

```
#define SL_USBD_CDC_SUBCLASS_OBEX
```

Value:

```
0x0Bu
```

Definition at line 78 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_SUBCLASS_EEM

```
#define SL_USBD_CDC_SUBCLASS_EEM
```

Value:

```
0x0Cu
```

Definition at line 79 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_SUBCLASS_NCM

```
#define SL_USBD_CDC_SUBCLASS_NCM
```

Value:

```
0x0Du
```

Definition at line 80 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_SUBCLASS_VENDOR

```
#define SL_USBD_CDC_SUBCLASS_VENDOR
```

Value:

```
0xFEu
```

Definition at line 81 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_COMM_PROTOCOL_NONE

```
#define SL_USBD_CDC_COMM_PROTOCOL_NONE
```

Value:

```
0x00u
```

COMMUNICATION CLASS PROTOCOL CODES DEFINES.

Note(s) : (1) Communication class protocol codes are defined in table 5 from the CDC specification revision 1.2.

Definition at line 90 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_COMM_PROTOCOL_AT_V250

```
#define SL_USBD_CDC_COMM_PROTOCOL_AT_V250
```

Value:

```
0x01u
```

Definition at line 91 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_COMM_PROTOCOL_AT_PCCA_101

```
#define SL_USBD_CDC_COMM_PROTOCOL_AT_PCCA_101
```

Value:

```
0x02u
```

Definition at line 92 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_COMM_PROTOCOL_AT_PCCA_101_ANNEX

```
#define SL_USBD_CDC_COMM_PROTOCOL_AT_PCCA_101_ANNEX
```

Value:

```
0x03u
```

Definition at line 93 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_COMM_PROTOCOL_AT_GSM_7_07

```
#define SL_USBD_CDC_COMM_PROTOCOL_AT_GSM_7_07
```

Value:

```
0x04u
```

Definition at line 94 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_COMM_PROTOCOL_AT_3GPP_27_07

```
#define SL_USBD_CDC_COMM_PROTOCOL_AT_3GPP_27_07
```

Value:

```
0x05u
```

Definition at line 95 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_COMM_PROTOCOL_AT_TIA_CDMA

```
#define SL_USBD_CDC_COMM_PROTOCOL_AT_TIA_CDMA
```

Value:

```
0x06u
```

Definition at line 96 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_COMM_PROTOCOL_EEM

```
#define SL_USBD_CDC_COMM_PROTOCOL_EEM
```

Value:

```
0x07u
```

Definition at line 97 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_COMM_PROTOCOL_EXT**

```
#define SL_USBD_CDC_COMM_PROTOCOL_EXT
```

Value:

```
0xFEu
```

Definition at line 99 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_COMM_PROTOCOL_VENDOR**

```
#define SL_USBD_CDC_COMM_PROTOCOL_VENDOR
```

Value:

```
0xFFu
```

Definition at line 100 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_DATA_PROTOCOL_NONE**

```
#define SL_USBD_CDC_DATA_PROTOCOL_NONE
```

Value:

```
0x00u
```

DATA INTERFACE CLASS PROTOCOL CODES DEFINES.

Note(s) : (1) Data interface class protocol codes are defined in table 7 from the CDC specification revision 1.2.

Definition at line 109 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_DATA_PROTOCOL NTB**

```
#define SL_USBD_CDC_DATA_PROTOCOL_NTb
```

Value:

```
0x01u
```

Definition at line 110 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DATA_PROTOCOL_PHY

```
#define SL_USBD_CDC_DATA_PROTOCOL_PHY
```

Value:

```
0x30u
```

Definition at line 111 of file `protocol/usb/inc/slUsbd_class_cdc.h`**SL_USBD_CDC_DATA_PROTOCOL_HDLC**

```
#define SL_USBD_CDC_DATA_PROTOCOL_HDLC
```

Value:

```
0x31u
```

Definition at line 112 of file `protocol/usb/inc/slUsbd_class_cdc.h`**SL_USBD_CDC_DATA_PROTOCOL_TRANS**

```
#define SL_USBD_CDC_DATA_PROTOCOL_TRANS
```

Value:

```
0x32u
```

Definition at line 113 of file `protocol/usb/inc/slUsbd_class_cdc.h`**SL_USBD_CDC_DATA_PROTOCOL_Q921M**

```
#define SL_USBD_CDC_DATA_PROTOCOL_Q921M
```

Value:

```
0x50u
```

Definition at line 114 of file `protocol/usb/inc/slUsbd_class_cdc.h`**SL_USBD_CDC_DATA_PROTOCOL_Q921**

```
#define SL_USBD_CDC_DATA_PROTOCOL_Q921
```

Value:

```
0x51u
```

Definition at line 115 of file `protocol/usb/inc/slUsbd_class_cdc.h`**SL_USBD_CDC_DATA_PROTOCOL_Q921TM**

```
#define SL_USBD_CDC_DATA_PROTOCOL_Q921TM
```

Value:

```
0x52u
```

Definition at line 116 of file `protocol/usb/inc/slusbdc_class_cdc.h`

SL_USBD_CDC_DATA_PROTOCOL_COMPRES

```
#define SL_USBD_CDC_DATA_PROTOCOL_COMPRES
```

Value:

```
0x90u
```

Definition at line 117 of file `protocol/usb/inc/slusbdc_class_cdc.h`

SL_USBD_CDC_DATA_PROTOCOL_Q9131

```
#define SL_USBD_CDC_DATA_PROTOCOL_Q9131
```

Value:

```
0x91u
```

Definition at line 118 of file `protocol/usb/inc/slusbdc_class_cdc.h`

SL_USBD_CDC_DATA_PROTOCOL_V24

```
#define SL_USBD_CDC_DATA_PROTOCOL_V24
```

Value:

```
0x92u
```

Definition at line 119 of file `protocol/usb/inc/slusbdc_class_cdc.h`

SL_USBD_CDC_DATA_PROTOCOL_CAPI

```
#define SL_USBD_CDC_DATA_PROTOCOL_CAPI
```

Value:

```
0x93u
```

Definition at line 120 of file `protocol/usb/inc/slusbdc_class_cdc.h`

SL_USBD_CDC_DATA_PROTOCOL_HOST

```
#define SL_USBD_CDC_DATA_PROTOCOL_HOST
```

Value:

OxFDu

Definition at line 121 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_DATA_PROTOCOL_CDC

#define SL_USBD_CDC_DATA_PROTOCOL_CDC

Value:

OxFEu

Definition at line 125 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_DATA_PROTOCOL_VENDOR

#define SL_USBD_CDC_DATA_PROTOCOL_VENDOR

Value:

0xFFu

Definition at line 126 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_REQ_SEND_ENCAP_COMM

#define SL_USBD_CDC_REQ_SEND_ENCAP_COMM

Value:

0x00u

MANAGEMENT ELEMENTS REQUESTS DEFINES.

Note(s) : (1) Management elements requests are defined in table 19 from the CDC specification revision 1.2.

Definition at line 135 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_REQ_GET_ENCAP_RESP

#define SL_USBD_CDC_REQ_GET_ENCAP_RESP

Value:

0x01u

Definition at line 136 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_REQ_SET_COMM_FEATURE

#define SL_USBD_CDC_REQ_SET_COMM_FEATURE

Value:

```
0x02u
```

Definition at line 137 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_COMM_FEATURE

```
#define SL_USBD_CDC_REQ_GET_COMM_FEATURE
```

Value:

```
0x03u
```

Definition at line 138 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_CLR_COMM_FEATURE

```
#define SL_USBD_CDC_REQ_CLR_COMM_FEATURE
```

Value:

```
0x04u
```

Definition at line 139 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SET_AUX_LINE_STATE

```
#define SL_USBD_CDC_REQ_SET_AUX_LINE_STATE
```

Value:

```
0x10u
```

Definition at line 140 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SET_HOOK_STATE

```
#define SL_USBD_CDC_REQ_SET_HOOK_STATE
```

Value:

```
0x11u
```

Definition at line 141 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_PULSE_SETUP

```
#define SL_USBD_CDC_REQ_PULSE_SETUP
```

Value:

```
0x12u
```

Definition at line 142 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SEND_PULSE

```
#define SL_USBD_CDC_REQ_SEND_PULSE
```

Value:

```
0x13u
```

Definition at line 143 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_REQ_SET_PULSE_TIME**

```
#define SL_USBD_CDC_REQ_SET_PULSE_TIME
```

Value:

```
0x14u
```

Definition at line 144 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_REQ_RING_AUX_JACK**

```
#define SL_USBD_CDC_REQ_RING_AUX_JACK
```

Value:

```
0x15u
```

Definition at line 145 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_REQ_SET_LINE_CODING**

```
#define SL_USBD_CDC_REQ_SET_LINE_CODING
```

Value:

```
0x20u
```

Definition at line 146 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_REQ_GET_LINE_CODING**

```
#define SL_USBD_CDC_REQ_GET_LINE_CODING
```

Value:

```
0x21u
```

Definition at line 147 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_REQ_SET_CTRL_LINE_STATE**

```
#define SL_USBD_CDC_REQ_SET_CTRL_LINE_STATE
```

Value:

```
0x22u
```

Definition at line 148 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SEND_BREAK

```
#define SL_USBD_CDC_REQ_SEND_BREAK
```

Value:

```
0x23u
```

Definition at line 149 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SET_RINGER_PAARMS

```
#define SL_USBD_CDC_REQ_SET_RINGER_PAARMS
```

Value:

```
0x30u
```

Definition at line 150 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_RINGER_PARAMS

```
#define SL_USBD_CDC_REQ_GET_RINGER_PARAMS
```

Value:

```
0x31u
```

Definition at line 151 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SET_OPERATION_PARAMS

```
#define SL_USBD_CDC_REQ_SET_OPERATION_PARAMS
```

Value:

```
0x32u
```

Definition at line 152 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_OPERATION_PARAMS

```
#define SL_USBD_CDC_REQ_GET_OPERATION_PARAMS
```

Value:

```
0x33u
```

Definition at line 153 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SET_LINE_PARAMS

```
#define SL_USBD_CDC_REQ_SET_LINE_PARAMS
```

Value:

```
0x34u
```

Definition at line 154 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_LINE_PARAMS

```
#define SL_USBD_CDC_REQ_GET_LINE_PARAMS
```

Value:

```
0x35u
```

Definition at line 155 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_DIAL_DIGITS

```
#define SL_USBD_CDC_REQ_DIAL_DIGITS
```

Value:

```
0x36u
```

Definition at line 156 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SET_UNIT_PARAM

```
#define SL_USBD_CDC_REQ_SET_UNIT_PARAM
```

Value:

```
0x37u
```

Definition at line 157 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_UNIT_PARAM

```
#define SL_USBD_CDC_REQ_GET_UNIT_PARAM
```

Value:

```
0x38u
```

Definition at line 158 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_CLR_UNUT_PARAM

```
#define SL_USBD_CDC_REQ_CLR_UNUT_PARAM
```

Value:

```
0x39u
```

Definition at line 159 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_REQ_GET_PROFILE**

```
#define SL_USBD_CDC_REQ_GET_PROFILE
```

Value:

```
0x3Au
```

Definition at line 160 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_REQ_SET_ETHER_MULTI_FILTER**

```
#define SL_USBD_CDC_REQ_SET_ETHER_MULTI_FILTER
```

Value:

```
0x40u
```

Definition at line 161 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_REQ_SET_ETHER_PWR_MGT_FILTER**

```
#define SL_USBD_CDC_REQ_SET_ETHER_PWR_MGT_FILTER
```

Value:

```
0x41u
```

Definition at line 162 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_REQ_GET_ETHER_PWR_MGT_FILTER**

```
#define SL_USBD_CDC_REQ_GET_ETHER_PWR_MGT_FILTER
```

Value:

```
0x42u
```

Definition at line 163 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_REQ_SET_ETHER_PKT_FILTER**

```
#define SL_USBD_CDC_REQ_SET_ETHER_PKT_FILTER
```

Value:

```
0x43u
```

Definition at line 164 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_ETHER_STAT

```
#define SL_USBD_CDC_REQ_GET_ETHER_STAT
```

Value:

```
0x44u
```

Definition at line 165 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SET_ATM_DATA_FMT

```
#define SL_USBD_CDC_REQ_SET_ATM_DATA_FMT
```

Value:

```
0x50u
```

Definition at line 166 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_ATM_DATA_FMT

```
#define SL_USBD_CDC_REQ_GET_ATM_DATA_FMT
```

Value:

```
0x51u
```

Definition at line 167 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_ATM_DEV_STAT

```
#define SL_USBD_CDC_REQ_GET_ATM_DEV_STAT
```

Value:

```
0x52u
```

Definition at line 168 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SET_ATM_DFLT_VC

```
#define SL_USBD_CDC_REQ_SET_ATM_DFLT_VC
```

Value:

```
0x53u
```

Definition at line 169 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_ATM_VC_STAT

```
#define SL_USBD_CDC_REQ_GET_ATM_VC_STAT
```

Value:

```
0x54u
```

Definition at line 170 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_NTB_PARAM2

```
#define SL_USBD_CDC_REQ_GET_NTB_PARAM2
```

Value:

```
0x80u
```

Definition at line 171 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_NET_ADDR

```
#define SL_USBD_CDC_REQ_GET_NET_ADDR
```

Value:

```
0x81u
```

Definition at line 172 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SET_NET_ADDR

```
#define SL_USBD_CDC_REQ_SET_NET_ADDR
```

Value:

```
0x82u
```

Definition at line 173 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_GET_NTB_FMT

```
#define SL_USBD_CDC_REQ_GET_NTB_FMT
```

Value:

```
0x83u
```

Definition at line 174 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_REQ_SET_NTBFMT

```
#define SL_USBD_CDC_REQ_SET_NTBFMT
```

Value:

```
0x84u
```

Definition at line 175 of file `protocol/usb/inc/slusbdc_class_cdc.h`**SL_USBD_CDC_REQ_GET_NTBFMT**

```
#define SL_USBD_CDC_REQ_GET_NTBFMT
```

Value:

```
0x85u
```

Definition at line 176 of file `protocol/usb/inc/slusbdc_class_cdc.h`**SL_USBD_CDC_REQ_SET_NTBFMT_SIZE**

```
#define SL_USBD_CDC_REQ_SET_NTBFMT_SIZE
```

Value:

```
0x86u
```

Definition at line 177 of file `protocol/usb/inc/slusbdc_class_cdc.h`**SL_USBD_CDC_REQ_GET_MAX_DATAGRAM_SIZE**

```
#define SL_USBD_CDC_REQ_GET_MAX_DATAGRAM_SIZE
```

Value:

```
0x87u
```

Definition at line 178 of file `protocol/usb/inc/slusbdc_class_cdc.h`**SL_USBD_CDC_REQ_SET_MAX_DATAGRAM_SIZE**

```
#define SL_USBD_CDC_REQ_SET_MAX_DATAGRAM_SIZE
```

Value:

```
0x88u
```

Definition at line 179 of file `protocol/usb/inc/slusbdc_class_cdc.h`**SL_USBD_CDC_REQ_GET_CRC_MODE**

```
#define SL_USBD_CDC_REQ_GET_CRC_MODE
```

Value:

```
0x89u
```

Definition at line 180 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_REQ_SET_CRC_MODE

```
#define SL_USBD_CDC_REQ_SET_CRC_MODE
```

Value:

```
0x8Au
```

Definition at line 181 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_DESC_TYPE_CS_IF

```
#define SL_USBD_CDC_DESC_TYPE_CS_IF
```

Value:

```
0x24u
```

CDC FUNCTIONAL DESCRIPTOR TYPE DEFINES.

Note(s) : (1) Functional descriptors types are defined in table 12 from the CDC specification revision 1.2.

Definition at line 190 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_DESC_TYPE_CS_EP

```
#define SL_USBD_CDC_DESC_TYPE_CS_EP
```

Value:

```
0x25u
```

Definition at line 191 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_DESC_SUBTYPE_HEADER

```
#define SL_USBD_CDC_DESC_SUBTYPE_HEADER
```

Value:

```
0x00u
```

CDC FUNCTIONAL DESCRIPTOR SUBTYPE DEFINES.

Note(s) : (1) Functional descriptors subtypes are defined in table 13 from the CDC specification revision 1.2.

Definition at line 202 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_CALL_MGMT

```
#define SL_USBD_CDC_DESC_SUBTYPE_CALL_MGMT
```

Value:

```
0x01u
```

Definition at line 203 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_ACM

```
#define SL_USBD_CDC_DESC_SUBTYPE_ACM
```

Value:

```
0x02u
```

Definition at line 204 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_DLM

```
#define SL_USBD_CDC_DESC_SUBTYPE_DLM
```

Value:

```
0x03u
```

Definition at line 205 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_TEL_RINGER

```
#define SL_USBD_CDC_DESC_SUBTYPE_TEL_RINGER
```

Value:

```
0x04u
```

Definition at line 206 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_TEL_CALL_LINE

```
#define SL_USBD_CDC_DESC_SUBTYPE_TEL_CALL_LINE
```

Value:

```
0x05u
```

Definition at line 209 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_UNION

```
#define SL_USBD_CDC_DESC_SUBTYPE_UNION
```

Value:

```
0x06u
```

Definition at line 210 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_COUNTRY_SEL

```
#define SL_USBD_CDC_DESC_SUBTYPE_COUNTRY_SEL
```

Value:

```
0x07u
```

Definition at line 211 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_TEL_OPERA

```
#define SL_USBD_CDC_DESC_SUBTYPE_TEL_OPERA
```

Value:

```
0x08u
```

Definition at line 212 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_USB_TERM

```
#define SL_USBD_CDC_DESC_SUBTYPE_USB_TERM
```

Value:

```
0x09u
```

Definition at line 213 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_NET_CH_TERM

```
#define SL_USBD_CDC_DESC_SUBTYPE_NET_CH_TERM
```

Value:

```
0x0Au
```

Definition at line 214 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_PROTOCOL_UNIT

```
#define SL_USBD_CDC_DESC_SUBTYPE_PROTOCOL_UNIT
```

Value:

```
0x0Bu
```

Definition at line 215 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_EXT_UNIT

```
#define SL_USBD_CDC_DESC_SUBTYPE_EXT_UNIT
```

Value:

```
0x0Cu
```

Definition at line 216 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_MULTI_CH_MGMT

```
#define SL_USBD_CDC_DESC_SUBTYPE_MULTI_CH_MGMT
```

Value:

```
0x0Du
```

Definition at line 217 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_CAPI_CTRL

```
#define SL_USBD_CDC_DESC_SUBTYPE_CAPI_CTRL
```

Value:

```
0x0Eu
```

Definition at line 218 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_ETHER_NET

```
#define SL_USBD_CDC_DESC_SUBTYPE_ETHER_NET
```

Value:

```
0x0Fu
```

Definition at line 219 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_ATM_NET

```
#define SL_USBD_CDC_DESC_SUBTYPE_ATM_NET
```

Value:

```
0x10u
```

Definition at line 220 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_WIRELESS_CTRL

```
#define SL_USBD_CDC_DESC_SUBTYPE_WIRELESS_CTRL
```

Value:

```
0x11u
```

Definition at line 221 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_DESC_SUBTYPE_MOBILE**

```
#define SL_USBD_CDC_DESC_SUBTYPE_MOBILE
```

Value:

```
DLM      0x12u
```

Definition at line 222 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_DESC_SUBTYPE_DETAIL**

```
#define SL_USBD_CDC_DESC_SUBTYPE_DETAIL
```

Value:

```
0x13u
```

Definition at line 223 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_DESC_SUBTYPE_DEV_MGMT**

```
#define SL_USBD_CDC_DESC_SUBTYPE_DEV_MGMT
```

Value:

```
0x14u
```

Definition at line 224 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_DESC_SUBTYPE_OBEX**

```
#define SL_USBD_CDC_DESC_SUBTYPE_OBEX
```

Value:

```
0x15u
```

Definition at line 225 of file `protocol/usb/inc/sl_usbd_class_cdc.h`**SL_USBD_CDC_DESC_SUBTYPE_CMD_SET**

```
#define SL_USBD_CDC_DESC_SUBTYPE_CMD_SET
```

Value:

```
0x16u
```

Definition at line 226 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_CMD_SET_DETAIL

```
#define SL_USBD_CDC_DESC_SUBTYPE_CMD_SET_DETAIL
```

Value:

```
0x17u
```

Definition at line 227 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_TCM

```
#define SL_USBD_CDC_DESC_SUBTYPE_TCM
```

Value:

```
0x18u
```

Definition at line 228 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_OBEX_SERV

```
#define SL_USBD_CDC_DESC_SUBTYPE_OBEX_SERV
```

Value:

```
0x19u
```

Definition at line 229 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_NCM

```
#define SL_USBD_CDC_DESC_SUBTYPE_NCM
```

Value:

```
0x1Au
```

Definition at line 230 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

SL_USBD_CDC_DESC_SUBTYPE_VENDOR

```
#define SL_USBD_CDC_DESC_SUBTYPE_VENDOR
```

Value:

0xFEu

Definition at line 231 of file protocol/usb/inc/sl_usbd_class_cdc.h

SL_USBD_CDC_NOTIFICATION_HEADER

#define SL_USBD_CDC_NOTIFICATION_HEADER

Value:

8u

CDC NOTIFICATION DEFINES.

Definition at line 237 of file protocol/usb/inc/sl_usbd_class_cdc.h

sl_usbd_cdc_subclass_driver_t

DATA TYPES.

CDC Subclass Driver

Public Attributes

void(*	enable	Callback for enable event.
void(*	disable	Callback for disable event.
bool(*	mgmt_req	Callback to handle management requests.
void(*	notify_cmpl	Callback to handle notification completion.
void(*	fnct_descriptor	Callback to build functional desc.
uint16_t(*	fnct_get_descriptor_size	Callback to get the size of the functional desc.

Public Attribute Documentation

enable

```
void(* sl_usbd_cdc_subclass_driver_t::enable) (void *p_subclass_arg)
```

Callback for enable event.

Definition at line 246 of file protocol/usb/inc/sl_usbd_class_cdc.h

disable

```
void(* sl_usbd_cdc_subclass_driver_t::disable) (void *p_subclass_arg)
```

Callback for disable event.

Definition at line 248 of file protocol/usb/inc/sl_usbd_class_cdc.h

mgmt_req

```
bool(* sl_usbd_cdc_subclass_driver_t::mgmt_req) (const sl_usbd_setup_req_t *p_setup_req, void *p_subclass_arg)
```

Callback to handle management requests.

Definition at line 250 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

notify_cmpl

```
void(* sl_usbd_cdc_subclass_driver_t::notify_cmpl) (void *p_subclass_arg)
```

Callback to handle notification completion.

Definition at line 253 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

fnc_t_descriptor

```
void(* sl_usbd_cdc_subclass_driver_t::fnc_t_descriptor) (void *p_subclass_arg, uint8_t first_dcif_nbr)
```

Callback to build functional desc.

Definition at line 255 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

fnc_t_get_descriptor_size

```
uint16_t(* sl_usbd_cdc_subclass_driver_t::fnc_t_get_descriptor_size) (void *p_subclass_arg)
```

Callback to get the size of the functional desc.

Definition at line 258 of file `protocol/usb/inc/sl_usbd_class_cdc.h`

USB Device Core API

USB Device Core API

USB Device Core API.

Modules

[sl_usbd_device_config_t](#)

[sl_usbd_setup_req_t](#)

[sl_usbd_class_driver_t](#)

Enumerations

```
enum    sl\_usbd\_device\_speed\_t {  
        SL_USBD_DEVICE_SPEED_INVALID = 0u  
        SL_USBD_DEVICE_SPEED_LOW  
        SL_USBD_DEVICE_SPEED_FULL  
        SL_USBD_DEVICE_SPEED_HIGH  
    }  
    DATA TYPES.  
  
enum    sl\_usbd\_device\_state\_t {  
        SL_USBD_DEVICE_STATE_NONE = 0  
        SL_USBD_DEVICE_STATE_INIT  
        SL_USBD_DEVICE_STATE_ATTACHED  
        SL_USBD_DEVICE_STATE_DEFAULT  
        SL_USBD_DEVICE_STATE_ADDRESSED  
        SL_USBD_DEVICE_STATE_CONFIGURED  
        SL_USBD_DEVICE_STATE_SUSPENDED  
    }  
  
enum    sl\_usbd\_bus\_event\_t {  
        SL_USBD_EVENT_BUS_CONNECT = 0  
        SL_USBD_EVENT_BUS_DISCONNECT  
        SL_USBD_EVENT_BUS_RESET  
        SL_USBD_EVENT_BUS_SUSPEND  
        SL_USBD_EVENT_BUS_RESUME  
    }  
    BUS EVENT DATA TYPE.  
  
enum    sl\_usbd\_config\_event\_t {  
        SL_USBD_EVENT_CONFIG_SET = 0  
        SL_USBD_EVENT_CONFIG_UNSET  
    }  
    CONFIG EVENT DATA TYPE.
```

Typedefs

```
typedef void(* sl\_usbd\_async\_function\_t)(uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void *p_arg,
sl_status_t status)
EP ASYNCHRONOUS CALLBACK API.
```

Functions

sl_status_t	sl_usbd_core_init (void) FUNCTION PROTOTYPES.
sl_status_t	sl_usbd_core_start_device (void) Start the device stack.
sl_status_t	sl_usbd_core_stop_device (void) Stop the device stack.
sl_status_t	sl_usbd_core_get_device_state (sl_usbd_device_state_t *p_dev_state) Get the current device state.
sl_status_t	sl_usbd_core_get_device_speed (sl_usbd_device_speed_t *p_dev_speed) Get device speed.
sl_status_t	sl_usbd_core_set_device_self_power (bool self_pwr) Set the device's current power source.
sl_status_t	sl_usbd_core_get_device_configuration (sl_usbd_device_config_t **p_dev_cfg) Set the device's Microsoft vendor code.
sl_status_t	sl_usbd_core_get_device_frame_number (uint16_t *p_frame_nbr) Get the last frame number from the driver.
sl_status_t	sl_usbd_core_add_configuration (uint8_t attrib, uint16_t max_pwr, sl_usbd_device_speed_t spd, const char *p_name, uint8_t *p_cfg_nbr) Add a configuration attribute to the device.
sl_status_t	sl_usbd_core_add_interface (uint8_t cfg_nbr, sl_usbd_class_driver_t *p_class_drv, void *p_if_class_arg, void *p_if_alt_class_arg, uint8_t class_code, uint8_t class_sub_code, uint8_t class_protocol_code, const char *p_name, uint8_t *p_if_nbr) Associate a configuration with its alternative-speed counterpart.
sl_status_t	sl_usbd_core_add_alt_interface (uint8_t config_nbr, uint8_t if_nbr, void *p_class_arg, const char *p_name, uint8_t *p_if_alt_nbr) Add an alternate setting to a specific interface.
sl_status_t	sl_usbd_core_add_interface_group (uint8_t config_nbr, uint8_t class_code, uint8_t class_sub_code, uint8_t class_protocol_code, uint8_t if_start, uint8_t if_cnt, const char *p_name, uint8_t *p_if_grp_num) Create an interface group.
sl_status_t	sl_usbd_core_get_device_descriptor (uint8_t *p_buf, uint8_t max_len, uint8_t *p_desc_len) Get the device descriptor.
sl_status_t	sl_usbd_core_get_configuration_descriptor (uint8_t *p_buf, uint16_t max_len, uint8_t config_ix, uint16_t *p_desc_len) Get a configuration descriptor.
sl_status_t	sl_usbd_core_get_string_descriptor (uint8_t *p_buf, uint8_t max_len, uint8_t str_ix, uint8_t *p_desc_len) Get a string descriptor.
sl_status_t	sl_usbd_core_add_string (const char *p_str) Add string to USB device.
sl_status_t	sl_usbd_core_get_string_index (const char *p_str, uint8_t *p_str_ix) Get string index corresponding to a given string.

sl_status_t	sl_usbd_core_write_08b_to_descriptor_buf (uint8_t val) Write an 8-bit value to the descriptor buffer.
sl_status_t	sl_usbd_core_write_16b_to_descriptor_buf (uint16_t val) Write a 16-bit value in the descriptor buffer.
sl_status_t	sl_usbd_core_write_24b_to_descriptor_buf (uint32_t val) Write a 24-bit value to the descriptor buffer.
sl_status_t	sl_usbd_core_write_32b_to_descriptor_buf (uint32_t val) Write a 32-bit value to the descriptor buffer.
sl_status_t	sl_usbd_core_write_buf_to_descriptor_buf (const uint8_t *p_buf, uint16_t len) Write a buffer into the descriptor buffer.
sl_status_t	sl_usbd_core_write_control_sync (void *p_buf, uint32_t buf_len, uint16_t timeout_ms, bool end, uint32_t *p_xfer_len) Send data on the Control IN endpoint.
sl_status_t	sl_usbd_core_read_control_sync (void *p_buf, uint32_t buf_len, uint16_t timeout_ms, uint32_t *p_xfer_len) Receive data on the Control OUT endpoint.
sl_status_t	sl_usbd_core_add_bulk_endpoint (uint8_t config_nbr, uint8_t if_nbr, uint8_t if_alt_nbr, bool dir_in, uint16_t max_pkt_len, uint8_t *p_ep_addr) Add a bulk endpoint to alternate setting interface.
sl_status_t	sl_usbd_core_read_bulk_sync (uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint16_t timeout_ms, uint32_t *p_xfer_len) Receive data on Bulk OUT endpoint.
sl_status_t	sl_usbd_core_read_bulk_async (uint8_t ep_addr, void *p_buf, uint32_t buf_len, sl_usbd_async_function_t async_fnct, void *p_async_arg) Receive data on Bulk OUT endpoint asynchronously.
sl_status_t	sl_usbd_core_write_bulk_sync (uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint16_t timeout_ms, bool end, uint32_t *p_xfer_len) Send data on a Bulk IN endpoint.
sl_status_t	sl_usbd_core_write_bulk_async (uint8_t ep_addr, void *p_buf, uint32_t buf_len, sl_usbd_async_function_t async_fnct, void *p_async_arg, bool end) Send data on the Bulk IN endpoint asynchronously.
sl_status_t	sl_usbd_core_add_interrupt_endpoint (uint8_t config_nbr, uint8_t if_nbr, uint8_t if_alt_nbr, bool dir_in, uint16_t max_pkt_len, uint16_t interval, uint8_t *p_ep_addr) Add an interrupt endpoint to an alternate setting interface.
sl_status_t	sl_usbd_core_read_interrupt_sync (uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint16_t timeout_ms, uint32_t *p_xfer_len) Receive data on the Interrupt OUT endpoint.
sl_status_t	sl_usbd_core_read_interrupt_async (uint8_t ep_addr, void *p_buf, uint32_t buf_len, sl_usbd_async_function_t async_fnct, void *p_async_arg) Receive data on Interrupt OUT endpoint asynchronously.
sl_status_t	sl_usbd_core_write_interrupt_sync (uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint16_t timeout_ms, bool end, uint32_t *p_xfer_len) Send data on Interrupt IN endpoint.
sl_status_t	sl_usbd_core_write_interrupt_async (uint8_t ep_addr, void *p_buf, uint32_t buf_len, sl_usbd_async_function_t async_fnct, void *p_async_arg, bool end) Send data on the Interrupt IN endpoint asynchronously.
sl_status_t	sl_usbd_core_endpoint_write_zlp (uint8_t ep_addr, uint16_t timeout_ms) Add an isochronous endpoint to alternate setting interface.

sl_status_t	sl_usbd_core_endpoint_read_zlp (uint8_t ep_addr, uint16_t timeout_ms) Receive zero-length packet from the host.
sl_status_t	sl_usbd_core_abort_endpoint (uint8_t ep_addr) Abort I/O transfer on the endpoint.
sl_status_t	sl_usbd_core_endpoint_read_complete (uint8_t ep_log_nbr) Notify USB stack that packet receive has completed.
sl_status_t	sl_usbd_core_endpoint_write_complete (uint8_t ep_log_nbr) Notify USB stack that packet transmit has completed.
sl_status_t	sl_usbd_core_endpoint_write_complete_extended (uint8_t ep_log_nbr, sl_status_t xfer_err) Notify USB stack that packet transmit has completed (see Note #1).
sl_status_t	sl_usbd_core_stall_endpoint (uint8_t ep_addr, bool state) Stall the non-control endpoint.
sl_status_t	sl_usbd_core_is_endpoint_stalled (uint8_t ep_addr, bool *p_ep_stall) Get the stall status of a non-control endpoint.
sl_status_t	sl_usbd_core_get_max_endpoint_packet_size (uint8_t ep_addr, uint16_t *p_max_size) Retrieve the endpoint maximum packet size.
sl_status_t	sl_usbd_core_get_max_phy_endpoint_number (uint8_t *p_ep_phy_nbr) Get the maximum physical endpoint number.
sl_status_t	sl_usbd_core_get_max_open_endpoint_number (uint8_t *p_nbr_open)
__WEAK void	sl_usbd_on_bus_event (sl_usbd_bus_event_t event)
__WEAK void	sl_usbd_on_config_event (sl_usbd_config_event_t event, uint8_t config_nbr)

Macros

#define	SL_USBD_FRAME_NBR_MAX 2047u MODULE.
#define	SL_USBD_FRAME_NBR_MASK 0b0000011111111111
#define	SL_USBD_MICROFRAME_NBR_MASK 0b0011100000000000
#define	SL_USBD_DEV_ATTRIB_SELF_POWERED 0x01u CONFIGURATION ATTRIBUTES.
#define	SL_USBD_DEV_ATTRIB_REMOTE_WAKEUP 0x02u
#define	SL_USBD_DESC_LEN_DEV 18u DEVICE DESCRIPTOR LENGTH.
#define	SL_USBD_DESC_LEN_DEV_QUAL 10u
#define	SL_USBD_DESC_LEN_HDR 2u
#define	SL_USBD_DESC_LEN_CFG 9u
#define	SL_USBD_DESC_LEN_OTHER_SPD_CFG 9u
#define	SL_USBD_DESC_LEN_IF 9u
#define	SL_USBD_DESC_LEN_IF_ASSOCIATION 8u

```
#define SL_USBD_DESC_LEN_EP 7u

#define SL_USBD_DESC_LEN_OTG 3u

#define SL_USBD_REQ_DIR_MASK 0x80u

#define SL_USBD_REQ_DIR_HOST_TO_DEVICE 0x00u

#define SL_USBD_REQ_DIR_DEVICE_TO_HOST 0x80u

#define SL_USBD_REQ_DIR_BIT 0x80u

#define SL_USBD_REQ_TYPE_MASK 0x60u

#define SL_USBD_REQ_TYPE_STANDARD 0x00u

#define SL_USBD_REQ_TYPE_CLASS 0x20u

#define SL_USBD_REQ_TYPE_VENDOR 0x40u

#define SL_USBD_REQ_TYPE_RESERVED 0x60u

#define SL_USBD_REQ_RECIPIENT_MASK 0x1Fu

#define SL_USBD_REQ_RECIPIENT_DEVICE 0x00u

#define SL_USBD_REQ_RECIPIENT_INTERFACE 0x01u

#define SL_USBD_REQ_RECIPIENT_ENDPOINT 0x02u

#define SL_USBD_REQ_RECIPIENT_OTHER 0x03u

#define SL_USBD_REQ_GET_STATUS 0u

#define SL_USBD_REQ_CLEAR_FEATURE 1u

#define SL_USBD_REQ_SET_FEATURE 3u

#define SL_USBD_REQ_SET_ADDRESS 5u

#define SL_USBD_REQ_GET_DESCRIPTOR 6u

#define SL_USBD_REQ_SET_DESCRIPTOR 7u

#define SL_USBD_REQ_GET_CONFIGURATION 8u

#define SL_USBD_REQ_SET_CONFIGURATION 9u

#define SL_USBD_REQ_GET_INTERFACE 10u

#define SL_USBD_REQ_SET_INTERFACE 11u

#define SL_USBD_REQ_SYNCH_FRAME 12u

#define SL_USBD_DESC_TYPE_DEVICE 1u

#define SL_USBD_DESC_TYPE_CONFIGURATION 2u

#define SL_USBD_DESC_TYPE_STRING 3u

#define SL_USBD_DESC_TYPE_INTERFACE 4u
```

```
#define SL_USBD_DESC_TYPE_ENDPOINT 5u

#define SL_USBD_DESC_TYPE_DEVICE_QUALIFIER 6u

#define SL_USBD_DESC_TYPE_OTHER_SPEED_CONFIGURATION 7u

#define SL_USBD_DESC_TYPE_INTERFACE_POWER 8u

#define SL_USBD_DESC_TYPE_OTG 9u

#define SL_USBD_DESC_TYPE_IAD 11u

#define SL_USBD_LANG_ID_ARABIC_SAUDI_ARABIA 0x0401u

#define SL_USBD_LANG_ID_CHINESE_TAIWAN 0x0404u

#define SL_USBD_LANG_ID_ENGLISH_US 0x0409u

#define SL_USBD_LANG_ID_ENGLISH_UK 0x0809u

#define SL_USBD_LANG_ID_FRENCH 0x040Cu

#define SL_USBD_LANG_ID_GERMAN 0x0407u

#define SL_USBD_LANG_ID_GREEK 0x0408u

#define SL_USBD_LANG_ID_ITALIAN 0x0410u

#define SL_USBD_LANG_ID_PORTUGUESE 0x0816u

#define SL_USBD_LANG_ID_SANSKRIT 0x044Fu

#define SL_USBD_CLASS_CODE_USE_IF_DESC 0x00u

#define SL_USBD_CLASS_CODE_AUDIO 0x01u

#define SL_USBD_CLASS_CODE_CDC_CONTROL 0x02u

#define SL_USBD_CLASS_CODE_HID 0x03u

#define SL_USBD_CLASS_CODE_PHYSICAL 0x05u

#define SL_USBD_CLASS_CODE_IMAGE 0x06u

#define SL_USBD_CLASS_CODE_PRINTER 0x07u

#define SL_USBD_CLASS_CODE_MASS_STORAGE 0x08u

#define SL_USBD_CLASS_CODE_HUB 0x09u

#define SL_USBD_CLASS_CODE_CDC_DATA 0x0Au

#define SL_USBD_CLASS_CODE_SMART_CARD 0x0Bu

#define SL_USBD_CLASS_CODE_CONTENT_SECURITY 0x0Du

#define SL_USBD_CLASS_CODE_VIDEO 0x0Eu

#define SL_USBD_CLASS_CODE_PERSONAL_HEALTHCARE 0x0Fu

#define SL_USBD_CLASS_CODE_DIAGNOSTIC_DEVICE 0xDCu
```



```
#define SL_USBD_CLASS_CODE_WIRELESS_CONTROLLER 0xE0u

#define SL_USBD_CLASS_CODE_MISCELLANEOUS 0xEFu

#define SL_USBD_CLASS_CODE_APPLICATION_SPECIFIC 0xFEu

#define SL_USBD_CLASS_CODE_VENDOR_SPECIFIC 0xFFu

#define SL_USBD_SUBCLASS_CODE_USE_IF_DESC 0x00u
SUB-CLASS CODES.

#define SL_USBD_SUBCLASS_CODE_USE_COMMON_CLASS 0x02u

#define SL_USBD_SUBCLASS_CODE_VENDOR_SPECIFIC 0xFFu

#define SL_USBD_PROTOCOL_CODE_USE_IF_DESC 0x00u
PROTOCOL CODES.

#define SL_USBD_PROTOCOL_CODE_USE_IAD 0x01u

#define SL_USBD_PROTOCOL_CODE_VENDOR_SPECIFIC 0xFFu

#define SL_USBD_FEATURE_SEL_ENDPOINT_HALT 0u
FEATURE SELECTORS.

#define SL_USBD_FEATURE_SEL_DEVICE_REMOTE_WAKEUP 1u

#define SL_USBD_FEATURE_SEL_TEST_MODE 2u

#define SL_USBD_MAX_BUS_PWR_LIMIT_ma 500u
DEVICE POWER CONSTRAINTS.

#define SL_USBD_ENDPOINT_TYPE_MASK 0x03u
ENDPOINT TYPE.

#define SL_USBD_ENDPOINT_TYPE_CTRL 0x00u

#define SL_USBD_ENDPOINT_TYPE_ISOC 0x01u

#define SL_USBD_ENDPOINT_TYPE_BULK 0x02u

#define SL_USBD_ENDPOINT_TYPE_INTR 0x03u

#define SL_USBD_ENDPOINT_TYPE_SYNC_MASK 0x0Cu

#define SL_USBD_ENDPOINT_TYPE_SYNC_NONE 0x00u

#define SL_USBD_ENDPOINT_TYPE_SYNC_ASYNC 0x04u

#define SL_USBD_ENDPOINT_TYPE_SYNC_ADAPTIVE 0x08u

#define SL_USBD_ENDPOINT_TYPE_SYNC_SYNC 0x0Cu

#define SL_USBD_ENDPOINT_TYPE_USAGE_MASK 0x30u

#define SL_USBD_ENDPOINT_TYPE_USAGE_DATA 0x00u

#define SL_USBD_ENDPOINT_TYPE_USAGE_FEEDBACK 0x10u

#define SL_USBD_ENDPOINT_TYPE_USAGE_IMPLICIT_FEEDBACK 0x20u
```

```
#define SL_USBD_ENDPOINT_NBR_MASK 0x0Fu
    ENDPOINT ADDRESS.

#define SL_USBD_ENDPOINT_MAX_NBR 32u

#define SL_USBD_ENDPOINT_DIR_MASK 0x80u

#define SL_USBD_ENDPOINT_DIR_BIT 0x80u

#define SL_USBD_ENDPOINT_DIR_OUT 0x00u

#define SL_USBD_ENDPOINT_DIR_IN 0x80u

#define SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_1 1u

#define SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_2 2u

#define SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_3 3u

#define SL_USBD_ENDPOINT_MAX_INTERVAL_VAL 32768u

#define SL_USBD_MICROSOFT_COMPAT_ID_NULL 0u
    MICROSOFT OS DESCRIPTOR DEFINES.

#define SL_USBD_MICROSOFT_COMPAT_ID_RNDIS 1u

#define SL_USBD_MICROSOFT_COMPAT_ID_PTP 2u

#define SL_USBD_MICROSOFT_COMPAT_ID_MTP 3u

#define SL_USBD_MICROSOFT_COMPAT_ID_XUSB20 4u

#define SL_USBD_MICROSOFT_COMPAT_ID_BLUETOOTH 5u

#define SL_USBD_MICROSOFT_COMPAT_ID_WINUSB 6u

#define SL_USBD_MICROSOFT_COMPAT_ID_NONE 255u

#define SL_USBD_MICROSOFT_SUBCOMPAT_ID_NULL 0u

#define SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_1_1 1u

#define SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_1_2 2u

#define SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_EDR 3u

#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_SZ 1u

#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_EXPAND_SZ 2u

#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_BINARY 3u

#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_DWORD_LITTLE_ENDIAN 4u

#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_DWORD_BIG_ENDIAN 5u

#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_LINK 6u

#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_MULTI_SZ 7u

#define SL_USBD_ENDPOINT_INFO_TYPE_CTRL 0x01u
    USB EP INFORMATION TABLE DEFINES.
```

```

#define SL_USBD_ENDPOINT_INFO_TYPE_ISOC 0x02u

#define SL_USBD_ENDPOINT_INFO_TYPE_BULK 0x04u

#define SL_USBD_ENDPOINT_INFO_TYPE_INTR 0x08u

#define SL_USBD_ENDPOINT_INFO_DIR_OUT 0x10u

#define SL_USBD_ENDPOINT_INFO_DIR_IN 0x20u

#define SL_USBD_CONFIG_NBR_NONE 255u
USB OBJECT NUMBER.

#define SL_USBD_CONFIG_NBR_SPD_BIT 0x80u

#define SL_USBD_INTERFACE_NBR_NONE 255u

#define SL_USBD_ALT_INTERFACE_NBR_NONE 255u

#define SL_USBD_INTERFACE_GROUP_NBR_NONE 255u

#define SL_USBD_ENDPOINT_ADDR_NONE 255u

#define SL_USBD_ENDPOINT_PHY_NONE 255u

#define SL_USBD_ENDPOINT_NBR_NONE 255u

#define SL_USBD_CLASS_NBR_NONE 255u

#define USBD_CFG_STR_EN 1
USB DEFAULT CONFIGS.

#define USBD_CFG_MS_OS_DESC_EN 0

#define USBD_CFG_URB_EXTRA_EN 0

#define SL_USBD_URB_MAX_NBR SL_USBD_OPEN_ENDPOINTS_QUANTITY

#define USBD_CFG_HS_EN 0

#define USBD_CFG_EP_ISOC_EN 0

#define USBD_CFG_OPTIMIZE_SPD 0

#define SL_USBD_CORE_EVENT_BUS_NBR 7u
USB CORE EVENTS.

#define SL_USBD_CORE_EVENT_BUS_NBR_TOTAL (SL_USBD_CORE_EVENT_BUS_NBR *
SL_USBD_DEVICE_QUANTITY)

#define SL_USBD_CORE_EVENT_ASYNC_NBR ((SL_USBD_OPEN_ENDPOINTS_QUANTITY *
SL_USBD_DEVICE_QUANTITY) + SL_USBD_EXTRA_URB_QUANTITY)

#define SL_USBD_CORE_EVENT_NBR_TOTAL (SL_USBD_CORE_EVENT_BUS_NBR_TOTAL +
SL_USBD_CORE_EVENT_ASYNC_NBR)

#define SL_USBD_FRAME_NBR_GET (frame_nbr)
GLOBAL CONSTANTS.

#define SL_USBD_MICROFRAME_NBR_GET (microframe_nbr)

#define SL_USBD_FRAME_NBR_DIFF_GET (frame_nbr1, frame_nbr2)

```

```
#define SL_USBD_ENDPOINT_PHY_TO_ADDR (ep_phy_nbr)
        ENDPOINT CONVERSION MACROS.

#define SL_USBD_ENDPOINT_ADDR_TO_PHY (ep_addr)

#define SL_USBD_ENDPOINT_ADDR_TO_LOG (ep_addr)

#define SL_USBD_ENDPOINT_PHY_TO_LOG (ep_phy_nbr)

#define SL_USBD_ENDPOINT_LOG_TO_ADDR_IN (ep_log_nbr)

#define SL_USBD_ENDPOINT_LOG_TO_ADDR_OUT (ep_log_nbr)

#define SL_USBD_ENDPOINT_IS_IN (ep_addr)
```

Enumeration Documentation

sl_usbd_device_speed_t

```
sl_usbd_device_speed_t
```

DATA TYPES.

USB DEVICE SPEED DATA TYPE

Note

- (1) 'Universal Serial Bus Specification Rev 2.0', section 4.2.1 defines three data rates:
 - (a) The USB Low-Speed (LS) signaling bit rate is 1.5 Mb/s.
 - (b) The USB Full-Speed (FS) signaling bit rate is 12 Mb/s.
 - (c) The USB High-Speed (HS) signaling bit rate is 480 Mb/s.

Enumerator	
SL_USBD_DEVICE_SPEED_INVALID	
SL_USBD_DEVICE_SPEED_LOW	Low-Speed (see Note #1a).
SL_USBD_DEVICE_SPEED_FULL	Full-Speed (see Note #1b).
SL_USBD_DEVICE_SPEED_HIGH	High-Speed (see Note #1c).

Definition at line 414 of file protocol/usb/inc/slUsbd_core.h

sl_usbd_device_state_t

```
sl_usbd_device_state_t
```

USB DEVICE STATES DATA TYPE

Note

- (1) 'Universal Serial Bus Specification Rev 2.0', section 9.1.1 defines seven visible device states:
 - (a) SL_USBD_DEVICE_STATE_INIT : Device is not attached to the USB.
 - (b) SL_USBD_DEVICE_STATE_ATTACHED : Device is attached to the USB, but not powered.
 - (c) USBDEV_STATE_POWERED : Device is attached to the USB and powered, but has not been reset.
 - (d) SL_USBD_DEVICE_STATE_DEFAULT : Device is attached to the USB and powered and has been reset, but has not been assigned a unique address. Device responds at the default address.
 - (e) SL_USBD_DEVICE_STATE_ADDRESSED : Device is attached to the USB, powered, has been reset, and a unique device address has been assigned. Device is not configured.

- (f) SL_USBD_DEVICE_STATE_CONFIGURED : Device is attached to the USB, powered, has been reset, has a unique address, is configured, and is not suspended. The host may now use the function provided by the device.
- (g) SL_USBD_DEVICE_STATE_SUSPENDED : Device is, at minimum, attached to the USB and is powered and has not seen bus activity for 3 ms. It may also have a unique address and be configured for use. However, because the device is suspended, the host may not use the device's function.
- (2) An additional state is added (USBD_DEV_STATE_START) to determine if the device controller has been initialized.

Enumerator	
SL_USBD_DEVICE_STATE_NONE	
SL_USBD_DEVICE_STATE_INIT	
SL_USBD_DEVICE_STATE_ATTACHED	
SL_USBD_DEVICE_STATE_DEFAULT	
SL_USBD_DEVICE_STATE_ADDRESSED	
SL_USBD_DEVICE_STATE_CONFIGURED	
SL_USBD_DEVICE_STATE_SUSPENDED	

Definition at line 448 of file protocol/usb/inc/slUsbd_core.h

sl_usbd_bus_event_t

sl_usbd_bus_event_t

BUS EVENT DATA TYPE.

Enumerator	
SL_USBD_EVENT_BUS_CONNECT	
SL_USBD_EVENT_BUS_DISCONNECT	
SL_USBD_EVENT_BUS_RESET	
SL_USBD_EVENT_BUS_SUSPEND	
SL_USBD_EVENT_BUS_RESUME	

Definition at line 462 of file protocol/usb/inc/slUsbd_core.h

sl_usbd_config_event_t

sl_usbd_config_event_t

CONFIG EVENT DATA TYPE.

Enumerator	
SL_USBD_EVENT_CONFIG_SET	
SL_USBD_EVENT_CONFIG_UNSET	

Definition at line 474 of file protocol/usb/inc/slUsbd_core.h

Typedef Documentation

sl_usbd_async_function_t

typedef void(* sl_usbd_async_function_t) (uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void *p_arg, sl_status_t status))(uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void *p_arg, sl_status_t status)

EP ASYNCHRONOUS CALLBACK API.

Definition at line 588 of file protocol/usb/inc/sl_usbd_core.h

Function Documentation

sl_usbd_core_init

```
sl_status_t sl_usbd_core_init (void)
```

FUNCTION PROTOTYPES.

Parameters

N/A		
-----	--	--

CONFIGURATION OVERRIDE FUNCTIONS USBD CORE FUNCTIONS

Note(s) : (1) USB Spec 2.0, Section 5.5 states "Control transfers allow access to different parts of a device. Control transfers are intended to support configuration/command/status type communication flows between client software and its function".

(a) "Each USB device is required to implement the Default Control Pipe as a message pipe. This pipe is used by the USB System Software. The Default Control Pipe provides access to the USB device's configuration, status, and control information".

The 'sl_usbd_core_ep_XXXX()' functions perform operations in the default endpoint. Class drivers should use 'sl_usbd_core_ep_XXXX()' to send/receive class specific requests.

```
sl_usbd_core_init()
```

Initialize the USB device stack.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) sli_usbd_core_init_internals() MUST be called ... :
 - (a) ONLY ONCE from a product's application;
 - (b) With the following conditions:
 - (1) AFTER the product's OS has been initialized.
 - (2) BEFORE the product's application calls any USB device stack function(s).

Definition at line 685 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_start_device

```
sl_status_t sl_usbd_core_start_device (void)
```

Start the device stack.

Parameters

N/A		
-----	--	--

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Device stack can be only started if the device is in one of the following states:
- SL_USBD_DEVICE_STATE_NONE Device controller has not been initialized.
 - SL_USBD_DEVICE_STATE_INIT Device controller is already initialized.

Definition at line 696 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_stop_device

```
sl_status_t sl_usbd_core_stop_device (void)
```

Stop the device stack.

Parameters

N/A		
-----	--	--

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 703 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_get_device_state

```
sl_status_t sl_usbd_core_get_device_state (sl_usbd_device_state_t *p_dev_state)
```

Get the current device state.

Parameters

N/A	p_dev_state	Pointer to device state to be returned. Set to current device state, if no errors are returned, or SL_USBD_DEVICE_STATE_NONE, if any errors are returned.
-----	-------------	---

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 714 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_get_device_speed

```
sl_status_t sl_usbd_core_get_device_speed (sl_usbd_device_speed_t *p_dev_speed)
```

Get device speed.

Parameters

N/A	p_dev_speed	Pointer to the variable that will be set to device speed upon return. It will be set to current device speed, if successful, or SL_USBD_DEVICE_SPEED_INVALID, if unsuccessful.
-----	-------------	--

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 725 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_set_device_self_power

```
sl_status_t sl_usbd_core_set_device_self_power (bool self_pwr)
```

Set the device's current power source.

Parameters

N/A	self_pwr	The power source of the device : • true device is self-powered. • false device is bus-powered.
-----	----------	---

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 736 of file `protocol/usb/inc/sl_usbd_core.h`

sl_usbd_core_get_device_configuration

```
sl_status_t sl_usbd_core_get_device_configuration (sl_usbd_device_config_t **p_dev_cfg)
```

Set the device's Microsoft vendor code.

Parameters

N/A	p_dev_cfg	Pointer to a variable of type (<code>sl_usbd_device_config_t *</code>) which will receive the memory address of device configuration structure upon return. The variable will be set to point to device configuration, if no errors are returned, or it will be set to NULL if any errors are returned.
-----	-----------	---

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) The vendor code used MUST be different from any vendor bRequest value.

Get the device configuration.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 761 of file `protocol/usb/inc/sl_usbd_core.h`

sl_usbd_core_get_device_frame_number

```
sl_status_t sl_usbd_core_get_device_frame_number (uint16_t *p_frame_nbr)
```

Get the last frame number from the driver.

Parameters

N/A	p_frame_nbr	Pointer to variable that will receive frame number.
-----	-------------	---

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) The frame number will always be in the range of 0-2047 (11 bits).
- (2) Frame number returned to the caller contains the frame and microframe numbers. It is encoded following this 16-bit format:



Caller must use the macros [SL_USBD_FRAME_NBR_GET\(\)](#) or [SL_USBD_MICROFRAME_NBR_GET\(\)](#) to get the frame or microframe number only.

Definition at line 781 of file `protocol/usb/inc/sl_usbd_core.h`

sl_usbd_core_add_configuration

```
sl_status_t sl_usbd_core_add_configuration (uint8_t attrib, uint16_t max_pwr, sl_usbd_device_speed_t spd, const char *p_name, uint8_t *p_cfg_nbr)
```

Add a configuration attribute to the device.

Parameters

N/A	attrib	Available configuration attributes. • SL_USBD_DEV_ATTRIB_SELF_POWERED Power does not come from VBUS. • SL_USBD_DEV_ATTRIB_REMOTE_WAKEUP Remote wakeup feature enabled.
N/A	max_pwr	Bus power required for this device (see Note #1).
N/A	spd	Available configuration speeds. • SL_USBD_DEVICE_SPEED_FULL Configuration is added in the full-speed configuration set.
N/A	p_name	Pointer to string describing the configuration (see Note #2).
N/A	p_cfg_nbr	Pointer to variable that will receive the configuration number. It will be set to the configuration number, if no errors are returned, otherwise it will be set to SL_USBD_CONFIG_NBR_NONE if any errors are returned.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) USB spec 2.0 (section 7.2.1.3/4) defines power constraints for bus-powered devices: "A low-power function draws up to one unit load from the USB cable when operational" "A function is high-power if, when fully powered, draws over one, but no more than five, unit loads from the USB cable." A unit load is defined as 100mA, so 'max_pwr' argument should be between 0 mA and 500mA
- (2) String support is optional: 'p_name' can be a NULL string pointer.
- (3) Configuration can only be added when the device is in the following states:
 - SL_USBD_DEVICE_STATE_NONE Device controller has not been initialized.
 - SL_USBD_DEVICE_STATE_INIT Device controller is already initialized.

Definition at line 816 of file `protocol/usb/inc/sl_usbd_core.h`

sl_usbd_core_add_interface

```
sl_status_t sl_usbd_core_add_interface (uint8_t cfg_nbr, sl_usbd_class_driver_t *p_class_drv, void *p_if_class_arg, void
*p_if_alt_class_arg, uint8_t class_code, uint8_t class_sub_code, uint8_t class_protocol_code, const char *p_name, uint8_t
*p_if_nbr)
```

Associate a configuration with its alternative-speed counterpart.

Parameters

N/A	cfg_nbr	Configuration index to add the interface.
N/A	p_class_drv	Pointer to interface driver.
N/A	p_if_class_arg	Pointer to interface driver argument.
N/A	p_if_alt_class_arg	Pointer to alternate interface argument.
N/A	class_code	Class code assigned by the USB-IF.
N/A	class_sub_code	Subclass code assigned by the USB-IF.
N/A	class_protocol_code	protocol code assigned by the USB-IF.
N/A	p_name	Pointer to string describing the Interface.
N/A	p_if_nbr	Pointer to a variable that will receive interface number after creation. The variable will be set to the interface number, if no errors are returned, otherwise it is set to SL_USBD_INTERFACE_NBR_NONE.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Configurations from high- and full-speed can be associated with each other to provide comparable functionality regardless of speed.
- (2) Configuration can ONLY be associated when the device is in the following states:
 - SL_USBD_DEVICE_STATE_NONE Device controller has not been initialized.
 - SL_USBD_DEVICE_STATE_INIT Device controller is already initialized.

Add an interface to a specific configuration.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) USB Spec 2.0 Interface (section 9.6.5) states: "An interface may include alternate settings that allow the endpoints and/or their characteristics to be varied after the device has been configured. The default setting for an interface is always an alternate setting of zero."

Definition at line 873 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_add_alt_interface

```
sl_status_t sl_usbd_core_add_alt_interface (uint8_t config_nbr, uint8_t if_nbr, void *p_class_arg, const char *p_name, uint8_t
*p_if_alt_nbr)
```

Add an alternate setting to a specific interface.

Parameters

N/A	config_nbr	Configuration number.
N/A	if_nbr	Interface number.

N/A	p_class_arg	Pointer to alternate interface argument.
N/A	p_name	Pointer to alternate setting name.
N/A	p_if_alt_nbr	Pointer to variable that will receive alternate setting number upon return. It will be set to interface alternate setting number if no error(s) are returned, or SL_USBD_ALT_INTERFACE_NBR_NONE if any errors are returned.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 900 of file `protocol/usb/inc/sl_usbd_core.h`

sl_usbd_core_add_interface_group

```
sl_status_t sl_usbd_core_add_interface_group (uint8_t config_nbr, uint8_t class_code, uint8_t class_sub_code, uint8_t class_protocol_code, uint8_t if_start, uint8_t if_cnt, const char *p_name, uint8_t *p_if_grp_num)
```

Create an interface group.

Parameters

N/A	config_nbr	Configuration number.
N/A	class_code	Class code assigned by the USB-IF.
N/A	class_sub_code	Subclass code assigned by the USB-IF.
N/A	class_protocol_code	protocol code assigned by the USB-IF.
N/A	if_start	Interface number of the first interface that is associated with this group.
N/A	if_cnt	Number of consecutive interfaces that are associated with this group.
N/A	p_name	Pointer to the string that describes the interface group.
N/A	p_if_grp_num	Pointer to the variable that will receive interface group number upon return. The variable is set to interface group number if no errors are returned, or SL_USBD_INTERFACE_GROUP_NBR_NONE if any errors are returned.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 929 of file `protocol/usb/inc/sl_usbd_core.h`

sl_usbd_core_get_device_descriptor

```
sl_status_t sl_usbd_core_get_device_descriptor (uint8_t *p_buf, uint8_t max_len, uint8_t *p_desc_len)
```

Get the device descriptor.

Parameters

N/A	p_buf	Pointer to the destination buffer.
N/A	max_len	Maximum number of bytes to write in destination buffer.
N/A	p_desc_len	Pointer to a variable that will receive the actual size of the descriptor. The variable is set to number of bytes actually in the descriptor if no errors are returned, or set to 0 if any errors are returned.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) This function should be used by drivers that supporting the standard requests auto-reply, (during the initialization process).

Definition at line 954 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_get_configuration_descriptor

```
sl_status_t sl_usbd_core_get_configuration_descriptor (uint8_t *p_buf, uint16_t max_len, uint8_t config_ix, uint16_t *p_desc_len)
```

Get a configuration descriptor.

Parameters

N/A	p_buf	Pointer to the destination buffer.
N/A	max_len	Maximum number of bytes to write in the destination buffer.
N/A	config_ix	Index of the desired configuration descriptor.
N/A	p_desc_len	Pointer to a variable that will receive the actual size of the descriptor. The variable is set to number of bytes actually in the descriptor if no errors are returned, or set to 0 if any errors are returned.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) This function should be used by drivers supporting a standard request's auto-reply, during the initialization process.

Definition at line 976 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_get_string_descriptor

```
sl_status_t sl_usbd_core_get_string_descriptor (uint8_t *p_buf, uint8_t max_len, uint8_t str_ix, uint8_t *p_desc_len)
```

Get a string descriptor.

Parameters

N/A	p_buf	Pointer to the destination buffer.
N/A	max_len	Maximum number of bytes to write in destination buffer.
N/A	str_ix	Index of the desired string descriptor.
N/A	p_desc_len	Pointer to a variable that will receive the actual size of the descriptor. The variable is set to number of bytes actually in the descriptor if no errors are returned, or set to 0 if any errors are returned.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) This function should be used by drivers supporting a standard request's auto-reply, during the initialization process.

Definition at line 1000 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_add_string

```
sl_status_t sl_usbd_core_add_string (const char *p_str)
```

Add string to USB device.

Parameters

N/A	p_str	Pointer to string to add (see Note #1).
-----	-------	---

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) USB spec 2.0 chapter 9.5 states: "Where appropriate, descriptors contain references to string descriptors that provide displayable information describing a descriptor in human-readable form. The inclusion of string descriptors is optional. However, the reference fields within descriptors are mandatory. If a device does not support string descriptors, string reference fields must be reset to zero to indicate no string descriptor is available". Since string descriptors are optional, 'p_str' could be a NULL pointer.

Definition at line 1022 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_get_string_index

```
sl_status_t sl_usbd_core_get_string_index (const char *p_str, uint8_t *p_str_ix)
```

Get string index corresponding to a given string.

Parameters

N/A	p_str	Pointer to string.
N/A	p_str_ix	Pointer to variable where string index will be returned.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 1035 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_write_08b_to_descriptor_buf

```
sl_status_t sl_usbd_core_write_08b_to_descriptor_buf (uint8_t val)
```

Write an 8-bit value to the descriptor buffer.

Parameters

N/A	val	8-bit value to write in the descriptor buffer.
-----	-----	--

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) USB classes may use this function to append class-specific descriptors to the configuration descriptor.

Definition at line 1049 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_write_16b_to_descriptor_buf

```
sl_status_t sl_usbd_core_write_16b_to_descriptor_buf (uint16_t val)
```

Write a 16-bit value in the descriptor buffer.

Parameters

N/A	val	16-bit value.
-----	-----	---------------

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) USB classes may use this function to append class-specific descriptors to the configuration descriptor.
- (2) USB descriptors are in little-endian format.

Definition at line 1063 of file `protocol/usb/inc/slUsbd_core.h`

sl_usbd_core_write_24b_to_descriptor_buf

```
sl_status_t sl_usbd_core_write_24b_to_descriptor_buf (uint32_t val)
```

Write a 24-bit value to the descriptor buffer.

Parameters

N/A	val	32-bit value containing 24 useful bits to write in the descriptor buffer.
-----	-----	---

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) USB classes may use this function to append class-specific descriptors to the configuration descriptor.
- (2) USB descriptors are in little-endian format.

Definition at line 1077 of file `protocol/usb/inc/slUsbd_core.h`

sl_usbd_core_write_32b_to_descriptor_buf

```
sl_status_t sl_usbd_core_write_32b_to_descriptor_buf (uint32_t val)
```

Write a 32-bit value to the descriptor buffer.

Parameters

N/A	val	32-bit value to write in the descriptor buffer.
-----	-----	---

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) USB classes may use this function to append class-specific descriptors to the configuration descriptor.
- (2) USB descriptors are in little-endian format.

Definition at line 1091 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_write_buf_to_descriptor_buf

```
sl_status_t sl_usbd_core_write_buf_to_descriptor_buf (const uint8_t *p_buf, uint16_t len)
```

Write a buffer into the descriptor buffer.

Parameters

N/A	p_buf	Pointer to the buffer to write into the descriptor buffer.
N/A	len	Length of the buffer.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) USB classes may use this function to append class-specific descriptors to the configuration descriptor.

Definition at line 1105 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_write_control_sync

```
sl_status_t sl_usbd_core_write_control_sync (void *p_buf, uint32_t buf_len, uint16_t timeout_ms, bool end, uint32_t *p_xfer_len)
```

Send data on the Control IN endpoint.

Parameters

N/A	p_buf	Pointer to the buffer of data that will be sent (see Note #2).
N/A	buf_len	Number of octets to transmit.
N/A	timeout_ms	Timeout in milliseconds.
N/A	end	End-of-transfer flag (see Note #1).
N/A	p_xfer_len	Pointer to a variable that will be set to the number of octets transmitted. The variable is set to the number of octets transmitted if no errors are returned, otherwise it is set to 0.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) If end-of-transfer is set and transfer length is multiple of maximum packet size, a zero-length packet is transferred to indicate a short transfer to the host.
- (2) Transmit buffer must be 32 bits aligned.

Definition at line 1130 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_read_control_sync

```
sl_status_t sl_usbd_core_read_control_sync (void *p_buf, uint32_t buf_len, uint16_t timeout_ms, uint32_t *p_xfer_len)
```

Receive data on the Control OUT endpoint.

Parameters

N/A	p_buf	Pointer to the destination buffer to receive data (see Note #1).
N/A	buf_len	Number of octets to receive.
N/A	timeout_ms	Timeout in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will be set to the number of octets received. The variable is set to the number of octets received if no errors are returned, otherwise it is set to 0.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Receive buffer must be 32 bits aligned.

Definition at line 1153 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_add_bulk_endpoint

```
sl_status_t sl_usbd_core_add_bulk_endpoint (uint8_t config_nbr, uint8_t if_nbr, uint8_t if_alt_nbr, bool dir_in, uint16_t max_pkt_len, uint8_t *p_ep_addr)
```

Add a bulk endpoint to alternate setting interface.

Parameters

N/A	config_nbr	Configuration number.
N/A	if_nbr	Interface number.
N/A	if_alt_nbr	Interface alternate setting number.
N/A	dir_in	Endpoint direction. • true IN direction. • false OUT direction.
N/A	max_pkt_len	Endpoint maximum packet length (see Note #1)
N/A	p_ep_addr	Pointer to a variable that will receive the new endpoint address. The variable will be set to endpoint address if no errors are returned, otherwise it will be set to SL_USBD_ENDPOINT_ADDR_NONE.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) If the 'max_pkt_len' argument is '0', the stack will allocate the first available BULK endpoint, regardless its maximum packet size.

Definition at line 1182 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_read_bulk_sync

```
sl_status_t sl_usbd_core_read_bulk_sync (uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint16_t timeout_ms, uint32_t *p_xfer_len)
```

Receive data on Bulk OUT endpoint.

Parameters

N/A	ep_addr	Endpoint address.
N/A	p_buf	Pointer to the destination buffer to receive data (see Note #1).
N/A	buf_len	Number of octets to receive.
N/A	timeout_ms	Timeout in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will be set to the number of octets received. The variable is set to the number of octets received if no errors are returned, otherwise it is set to 0.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Receive buffer must be 32 bits aligned.

Definition at line 1208 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_read_bulk_async

```
sl_status_t sl_usbd_core_read_bulk_async (uint8_t ep_addr, void *p_buf, uint32_t buf_len, sl_usbd_async_function_t
async_fnct, void *p_async_arg)
```

Receive data on Bulk OUT endpoint asynchronously.

Parameters

N/A	ep_addr	Endpoint address.
N/A	p_buf	Pointer to the destination buffer to receive data (see Note #1).
N/A	buf_len	Number of octets to receive.
N/A	async_fnct	Function that will be invoked upon completion of receive operation.
N/A	p_async_arg	Pointer to the argument that will be passed as parameter of 'async_fnct'.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Receive buffer must be 32 bits aligned.

Definition at line 1231 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_write_bulk_sync

```
sl_status_t sl_usbd_core_write_bulk_sync (uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint16_t timeout_ms, bool end,
uint32_t *p_xfer_len)
```

Send data on a Bulk IN endpoint.

Parameters

N/A	ep_addr	Endpoint address.
N/A	p_buf	Pointer to buffer of data that will be transmitted (see Note #2).
N/A	buf_len	Number of octets to transmit.
N/A	timeout_ms	Timeout in milliseconds.

N/A	end	End-of-transfer flag (see Note #3).
N/A	p_xfer_len	Pointer to a variable that will receive the number of octets transmitted. The variable is set to the number of octets transmitted if no errors are returned, otherwise it is set to 0.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) This function SHOULD NOT be called from interrupt service routine (ISR).
- (2) Transmit buffer must be 32 bits aligned.
- (3) If end-of-transfer is set and transfer length is multiple of maximum packet sizes, a zero-length packet is transferred to indicate a short transfer to the host.

Definition at line 1263 of file protocol/usb/inc/slUsbd_core.h

sl_usbd_core_write_bulk_async

```
sl_status_t sl_usbd_core_write_bulk_async (uint8_t ep_addr, void *p_buf, uint32_t buf_len, sl_usbd_async_function_t
async_fnct, void *p_async_arg, bool end)
```

Send data on the Bulk IN endpoint asynchronously.

Parameters

N/A	ep_addr	Endpoint address.
N/A	p_buf	Pointer to the buffer of data that will be transmitted (see Note #1).
N/A	buf_len	Number of octets to transmit.
N/A	async_fnct	Function that will be invoked upon completion of transmit operation.
N/A	p_async_arg	Pointer to the argument that will be passed as parameter of 'async_fnct'.
N/A	end	End-of-transfer flag (see Note #2).

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Transmit buffer must be 32 bits aligned.
- (2) If end-of-transfer is set and transfer length is multiple of maximum packet size, a zero-length packet is transferred to indicate a short transfer to the host.

Definition at line 1292 of file protocol/usb/inc/slUsbd_core.h

sl_usbd_core_add_interrupt_endpoint

```
sl_status_t sl_usbd_core_add_interrupt_endpoint (uint8_t config_nbr, uint8_t if_nbr, uint8_t if_alt_nbr, bool dir_in, uint16_t
max_pkt_len, uint16_t interval, uint8_t *p_ep_addr)
```

Add an interrupt endpoint to an alternate setting interface.

Parameters

N/A	config_nbr	Configuration number.
N/A	if_nbr	Interface number.
N/A	if_alt_nbr	Interface alternate setting number.

N/A	dir_in	Endpoint Direction. - true IN direction. - false OUT direction.
N/A	max_pkt_len	Endpoint maximum packet length. (see Note #1)
N/A	interval	Endpoint interval in frames or microframes.
N/A	p_ep_addr	Pointer to a variable that will receive the new endpoint address. The variable will be set to endpoint address if no errors are returned, otherwise it will be set to SL_USBD_ENDPOINT_ADDR_NONE.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) If the 'max_pkt_len' argument is '0', the stack will allocate the first available INTERRUPT endpoint, regardless its maximum packet size.

Definition at line 1325 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_read_interrupt_sync

```
sl_status_t sl_usbd_core_read_interrupt_sync (uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint16_t timeout_ms, uint32_t *p_xfer_len)
```

Receive data on the Interrupt OUT endpoint.

Parameters

N/A	ep_addr	Endpoint address.
N/A	p_buf	Pointer to the destination buffer to receive data (see Note #2).
N/A	buf_len	Number of octets to receive.
N/A	timeout_ms	Timeout in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will be set to the number of octets received. The variable is set to the number of octets received if no errors are returned, otherwise it is set to 0.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) This function SHOULD NOT be called from interrupt service routine (ISR).
- (2) Receive buffer must be 32 bits aligned.

Definition at line 1354 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_read_interrupt_async

```
sl_status_t sl_usbd_core_read_interrupt_async (uint8_t ep_addr, void *p_buf, uint32_t buf_len, sl_usbd_async_function_t async_fnct, void *p_async_arg)
```

Receive data on Interrupt OUT endpoint asynchronously.

Parameters

N/A	ep_addr	Endpoint address.
-----	---------	-------------------

N/A	p_buf	Pointer to the destination buffer to receive data (see Note #1).
N/A	buf_len	Number of octets to receive.
N/A	async_fnct	Function that will be invoked upon completion of receive operation.
N/A	p_async_arg	Pointer to argument that will be passed as parameter of 'async_fnct'.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Receive buffer must be 32 bits aligned.

Definition at line 1377 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_write_interrupt_sync

```
sl_status_t sl_usbd_core_write_interrupt_sync (uint8_t ep_addr, void *p_buf, uint32_t buf_len, uint16_t timeout_ms, bool end,
uint32_t *p_xfer_len)
```

Send data on Interrupt IN endpoint.

Parameters

N/A	ep_addr	Endpoint address.
N/A	p_buf	Pointer to the buffer of data that will be transmitted (see Note #2).
N/A	buf_len	Number of octets to transmit.
N/A	timeout_ms	Timeout in milliseconds.
N/A	end	End-of-transfer flag (see Note #3).
N/A	p_xfer_len	Pointer to a variable that will be set to the number of octets transmitted. The variable is set to the number of octets transmitted if no errors are returned, otherwise it is set to 0.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) This function SHOULD NOT be called from interrupt service routine (ISR).
- (2) Transmit buffer must be 32 bits aligned.
- (3) If end-of-transfer is set and transfer length is multiple of maximum packet size, a zero-length packet is transferred to indicate a short transfer to the host.

Definition at line 1409 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_write_interrupt_async

```
sl_status_t sl_usbd_core_write_interrupt_async (uint8_t ep_addr, void *p_buf, uint32_t buf_len, sl_usbd_async_function_t
async_fnct, void *p_async_arg, bool end)
```

Send data on the Interrupt IN endpoint asynchronously.

Parameters

N/A	ep_addr	Endpoint address.
N/A	p_buf	Pointer to the buffer of data that will be transmitted (see Note #1).

N/A	buf_len	Number of octets to transmit.
N/A	async_fnct	Function that will be invoked upon completion of transmit operation.
N/A	p_async_arg	Pointer to the argument that will be passed as parameter of 'async_fnct'.
N/A	end	End-of-transfer flag (see Note #2).

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Transmit buffer must be 32 bits aligned.
- (2) If end-of-transfer is set and transfer length is multiple of maximum packet size, a zero-length packet is transferred to indicate a short transfer to the host.

Definition at line 1438 of file protocol/usb/inc/slusb_core.h

slusb_core_endpoint_write_zlp

```
sl_status_t slusb_core_endpoint_write_zlp (uint8_t ep_addr, uint16_t timeout_ms)
```

Add an isochronous endpoint to alternate setting interface.

Parameters

N/A	ep_addr	Endpoint address.
N/A	timeout_ms	Timeout in milliseconds.
N/A	if_alt_nbr	Interface alternate setting number.
N/A	dir_in	Data endpoint address.
N/A	attrib	Associated synchronization endpoint.
N/A	max_pkt_len	Endpoint maximum packet length (see Note #1).
N/A	transaction_frame	Endpoint transactions per (micro)frame (see Note #2).
N/A	interval	Endpoint interval in frames or microframes.
N/A	p_ep_addr	Pointer to a variable that will receive the new endpoint address. The variable will be set to endpoint address if no errors are returned, otherwise it will be set to SL_USBD_ENDPOINT_ADDR_NONE.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) If the 'max_pkt_len' argument is '0', the stack allocates the first available ISOCHRONOUS endpoint regardless of its maximum packet size.
- (2) For full-speed endpoints, 'transaction_frame' must be set to 1 since there is no support for high-bandwidth endpoints.
- (3) For full-/high-speed isochronous endpoints, bInterval value must be in the range from 1 to 16. The bInterval value is used as the exponent for a $2^{(bInterval-1)}$ value. Maximum polling interval value is $2^{(16-1)} = 32768$ frames in full-speed and 32768 microframes (i.e., 4096 frames) in high-speed.

Receive data on an isochronous OUT endpoint asynchronously.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

(1) Receive buffer must be 32 bits aligned.

Send data on an isochronous IN endpoint asynchronously.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Transmit buffer must be 32 bits aligned.

Set synchronization feedback rate on synchronization isochronous endpoint.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Synchronization endpoints can ONLY be associated when the device is in the following states:
 - SL_USBD_DEVICE_STATE_NONE Device controller has not been initialized.
 - SL_USBD_DEVICE_STATE_INIT Device controller is already initialized.
- (2) For audio class 1.0, the interface class code must be SL_USBD_CLASS_CODE_AUDIO and protocol 'zero'
- (3) If explicit synchronization mechanism is needed to maintain synchronization during transfers, the information carried over the synchronization path must be available every $2^{(10 - P)}$ frames, with P ranging from 1 to 9 (512 ms down to 2 ms).
- (4) Table 4-22 "Standard AS Isochronous Synch Endpoint Descriptor" of Audio 1.0 specification indicates for bmAttributes field no usage type for bits 5..4. But USB 2.0 specification, Table 9-13 "Standard Endpoint Descriptor" indicates several types of usage. When an explicit feedback is defined for a asynchronous isochronous endpoint, the associated synch feedback should use the Usage type 'Feedback endpoint'.

Associate synchronization endpoint to isochronous endpoint.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Synchronization endpoints can ONLY be associated when the device is in the following states:
 - SL_USBD_DEVICE_STATE_NONE Device controller has not been initialized.
 - SL_USBD_DEVICE_STATE_INIT Device controller is already initialized.
- (2) For audio class 1.0, interface class code must be SL_USBD_CLASS_CODE_AUDIO and protocol 'zero'.

Send zero-length packet to the host.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) This function should only be called during a synchronous transfer.

Definition at line 1630 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_endpoint_read_zlp

```
sl_status_t sl_usbd_core_endpoint_read_zlp (uint8_t ep_addr, uint16_t timeout_ms)
```

Receive zero-length packet from the host.

Parameters

N/A	ep_addr	Endpoint address.
-----	---------	-------------------

N/A	timeout_ms	Timeout in milliseconds.
-----	------------	--------------------------

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) This function should only be called during a synchronous transfer.

Definition at line 1644 of file protocol/usb/inc/sLusbd_core.h

sl_usbd_core_abort_endpoint

sl_status_t sl_usbd_core_abort_endpoint (uint8_t ep_addr)

Abort I/O transfer on the endpoint.

Parameters

N/A	ep_addr	Endpoint address.
-----	---------	-------------------

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 1654 of file protocol/usb/inc/sLusbd_core.h

sl_usbd_core_endpoint_read_complete

sl_status_t sl_usbd_core_endpoint_read_complete (uint8_t ep_log_nbr)

Notify USB stack that packet receive has completed.

Parameters

N/A	ep_log_nbr	Endpoint logical number.
-----	------------	--------------------------

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 1663 of file protocol/usb/inc/sLusbd_core.h

sl_usbd_core_endpoint_write_complete

sl_status_t sl_usbd_core_endpoint_write_complete (uint8_t ep_log_nbr)

Notify USB stack that packet transmit has completed.

Parameters

N/A	ep_log_nbr	Endpoint logical number.
-----	------------	--------------------------

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 1672 of file protocol/usb/inc/sLusbd_core.h

sl_usbd_core_endpoint_write_complete_extended

```
sl_status_t sl_usbd_core_endpoint_write_complete_extended (uint8_t ep_log_nbr, sl_status_t xfer_err)
```

Notify USB stack that packet transmit has completed (see Note #1).

Parameters

N/A	ep_log_nbr	Endpoint logical number.
N/A	xfer_err	Error code returned by the USB driver.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) This function is an alternative to the function [sl_usbd_core_endpoint_write_complete\(\)](#) so that a USB device driver can return to the core an error code upon the Tx transfer completion.

Definition at line 1686 of file `protocol/usb/inc/slUsbd_core.h`

sl_usbd_core_stall_endpoint

```
sl_status_t sl_usbd_core_stall_endpoint (uint8_t ep_addr, bool state)
```

Stall the non-control endpoint.

Parameters

N/A	ep_addr	Endpoint address.
N/A	state	Endpoint stall state.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 1698 of file `protocol/usb/inc/slUsbd_core.h`

sl_usbd_core_is_endpoint_stalled

```
sl_status_t sl_usbd_core_is_endpoint_stalled (uint8_t ep_addr, bool *p_ep_stall)
```

Get the stall status of a non-control endpoint.

Parameters

N/A	ep_addr	Endpoint address.
N/A	p_ep_stall	Pointer to a boolean that will receive the endpoint stalled status. The boolean variable is set to true if endpoint is stalled, or is set to false if endpoint is not stalled.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 1712 of file `protocol/usb/inc/slUsbd_core.h`

sl_usbd_core_get_max_endpoint_packet_size

```
sl_status_t sl_usbd_core_get_max_endpoint_packet_size (uint8_t ep_addr, uint16_t *p_max_size)
```

Retrieve the endpoint maximum packet size.

Parameters		
N/A	ep_addr	Endpoint address.
N/A	p_max_size	Pointer to a variable that will receive the maximum packet size upon return. The variable is set to the maximum packet size if no errors are returned, otherwise it is set to 0.

- Returns
- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 1726 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_get_max_phy_endpoint_number

```
sl_status_t sl_usbd_core_get_max_phy_endpoint_number (uint8_t *p_ep_phy_nbr)
```

Get the maximum physical endpoint number.

Parameters		
N/A	p_ep_phy_nbr	Pointer to variable where maximum physical endpoint number will be returned. If no errors are returned, the variable will be set to maximum physical endpoint number, otherwise it will be set to SL_USBD_ENDPOINT_PHY_NONE.

- Returns
- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 1738 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_core_get_max_open_endpoint_number

```
sl_status_t sl_usbd_core_get_max_open_endpoint_number (uint8_t *p_nbr_open)
```

Parameters		
N/A	p_nbr_open	

Definition at line 1740 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_on_bus_event

```
__WEAK void sl_usbd_on_bus_event (sl_usbd_bus_event_t event)
```

Parameters		
N/A	event	

Definition at line 1743 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_on_config_event

```
__WEAK void sl_usbd_on_config_event (sl_usbd_config_event_t event, uint8_t config_nbr)
```

Parameters			
N/A		event	
N/A		config_nbr	

Definition at line 1745 of file protocol/usb/inc/sl_usbd_core.h

Macro Definition Documentation

SL_USBD_FRAME_NBR_MAX

```
#define SL_USBD_FRAME_NBR_MAX
```

Value:

2047u

MODULE.

INCLUDE FILES MACROS

FRAME

Note(s) : (1) Section 8.3.3 of USB spec 2.0 describes the frame number as: "The frame number field is an 11-bit field that is incremented by the host on a per-frame basis. The frame number field rolls over upon reaching its maximum value of 7FFH and is sent only in SOF tokens at the start of each (micro)frame."

Definition at line 72 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_FRAME_NBR_MASK

```
#define SL_USBD_FRAME_NBR_MASK
```

Value:

0b0000011111111111

Definition at line 73 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_MICROFRAME_NBR_MASK

```
#define SL_USBD_MICROFRAME_NBR_MASK
```

Value:

0b0011100000000000

Definition at line 74 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_DEV_ATTRIB_SELF_POWERED

```
#define SL_USBD_DEV_ATTRIB_SELF_POWERED
```

Value:

```
0x01u
```

CONFIGURATION ATTRIBUTES.

Definition at line 80 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_DEV_ATTRIB_REMOTE_WAKEUP

```
#define SL_USBD_DEV_ATTRIB_REMOTE_WAKEUP
```

Value:

```
0x02u
```

Definition at line 81 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_DESC_LEN_DEV

```
#define SL_USBD_DESC_LEN_DEV
```

Value:

```
18u
```

DEVICE DESCRIPTOR LENGTH.

Definition at line 87 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_DESC_LEN_DEV_QUAL

```
#define SL_USBD_DESC_LEN_DEV_QUAL
```

Value:

```
10u
```

Definition at line 88 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_DESC_LEN_HDR

```
#define SL_USBD_DESC_LEN_HDR
```

Value:

```
2u
```

Definition at line 89 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_DESC_LEN_CFG

```
#define SL_USBD_DESC_LEN_CFG
```

Value:

```
9u
```

Definition at line 90 of file `protocol/usb/inc/slusb_core.h`**SL_USBD_DESC_LEN_OTHER_SPD_CFG**

```
#define SL_USBD_DESC_LEN_OTHER_SPD_CFG
```

Value:

```
9u
```

Definition at line 91 of file `protocol/usb/inc/slusb_core.h`**SL_USBD_DESC_LEN_IF**

```
#define SL_USBD_DESC_LEN_IF
```

Value:

```
9u
```

Definition at line 92 of file `protocol/usb/inc/slusb_core.h`**SL_USBD_DESC_LEN_IF_ASSOCIATION**

```
#define SL_USBD_DESC_LEN_IF_ASSOCIATION
```

Value:

```
8u
```

Definition at line 93 of file `protocol/usb/inc/slusb_core.h`**SL_USBD_DESC_LEN_EP**

```
#define SL_USBD_DESC_LEN_EP
```

Value:

```
7u
```

Definition at line 94 of file `protocol/usb/inc/slusb_core.h`**SL_USBD_DESC_LEN_OTG**

```
#define SL_USBD_DESC_LEN_OTG
```

Value:

```
3u
```

Definition at line 95 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_REQ_DIR_MASK

```
#define SL_USBD_REQ_DIR_MASK
```

Value:

```
0x80u
```

REQUEST CHARACTERISTICS

Note(s) : (1) Request types are defined in USB spec 2.0, section 9.3, Table 9-2, 'RequestType' field.

Definition at line 103 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_REQ_DIR_HOST_TO_DEVICE

```
#define SL_USBD_REQ_DIR_HOST_TO_DEVICE
```

Value:

```
0x00u
```

Definition at line 104 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_REQ_DIR_DEVICE_TO_HOST

```
#define SL_USBD_REQ_DIR_DEVICE_TO_HOST
```

Value:

```
0x80u
```

Definition at line 105 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_REQ_DIR_BIT

```
#define SL_USBD_REQ_DIR_BIT
```

Value:

```
0x80u
```

Definition at line 106 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_REQ_TYPE_MASK

```
#define SL_USBD_REQ_TYPE_MASK
```

Value:

```
0x60u
```

Definition at line 108 of file `protocol/usb/inc/slusb_core.h`**SL_USBD_REQ_TYPE_STANDARD**

```
#define SL_USBD_REQ_TYPE_STANDARD
```

Value:

```
0x00u
```

Definition at line 109 of file `protocol/usb/inc/slusb_core.h`**SL_USBD_REQ_TYPE_CLASS**

```
#define SL_USBD_REQ_TYPE_CLASS
```

Value:

```
0x20u
```

Definition at line 110 of file `protocol/usb/inc/slusb_core.h`**SL_USBD_REQ_TYPE_VENDOR**

```
#define SL_USBD_REQ_TYPE_VENDOR
```

Value:

```
0x40u
```

Definition at line 111 of file `protocol/usb/inc/slusb_core.h`**SL_USBD_REQ_TYPE_RESERVED**

```
#define SL_USBD_REQ_TYPE_RESERVED
```

Value:

```
0x60u
```

Definition at line 112 of file `protocol/usb/inc/slusb_core.h`**SL_USBD_REQ_RECIPIENT_MASK**

```
#define SL_USBD_REQ_RECIPIENT_MASK
```

Value:

```
0x1Fu
```

Definition at line 114 of file `protocol/usb/inc/slusb_core.h`

SL_USBD_REQ_RECIPIENT_DEVICE

```
#define SL_USBD_REQ_RECIPIENT_DEVICE
```

Value:

```
0x00u
```

Definition at line 115 of file `protocol/usb/inc/slusb_core.h`

SL_USBD_REQ_RECIPIENT_INTERFACE

```
#define SL_USBD_REQ_RECIPIENT_INTERFACE
```

Value:

```
0x01u
```

Definition at line 116 of file `protocol/usb/inc/slusb_core.h`

SL_USBD_REQ_RECIPIENT_ENDPOINT

```
#define SL_USBD_REQ_RECIPIENT_ENDPOINT
```

Value:

```
0x02u
```

Definition at line 117 of file `protocol/usb/inc/slusb_core.h`

SL_USBD_REQ_RECIPIENT_OTHER

```
#define SL_USBD_REQ_RECIPIENT_OTHER
```

Value:

```
0x03u
```

Definition at line 118 of file `protocol/usb/inc/slusb_core.h`

SL_USBD_REQ_GET_STATUS

```
#define SL_USBD_REQ_GET_STATUS
```

Value:

0u

STANDARD REQUESTS

Note(s) : (1) Request types are defined in USB spec 2.0, section 9.4, Table 9-4, 'RequestType' field.

Definition at line 126 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_REQ_CLEAR_FEATURE

```
#define SL_USBD_REQ_CLEAR_FEATURE
```

Value:

1u

Definition at line 127 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_REQ_SET_FEATURE

```
#define SL_USBD_REQ_SET_FEATURE
```

Value:

3u

Definition at line 128 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_REQ_SET_ADDRESS

```
#define SL_USBD_REQ_SET_ADDRESS
```

Value:

5u

Definition at line 129 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_REQ_GET_DESCRIPTOR

```
#define SL_USBD_REQ_GET_DESCRIPTOR
```

Value:

6u

Definition at line 130 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_REQ_SET_DESCRIPTOR

```
#define SL_USBD_REQ_SET_DESCRIPTOR
```


Value:

7u

Definition at line 131 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_REQ_GET_CONFIGURATION

```
#define SL_USBD_REQ_GET_CONFIGURATION
```

Value:

8u

Definition at line 132 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_REQ_SET_CONFIGURATION

```
#define SL_USBD_REQ_SET_CONFIGURATION
```

Value:

9u

Definition at line 133 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_REQ_GET_INTERFACE

```
#define SL_USBD_REQ_GET_INTERFACE
```

Value:

10u

Definition at line 134 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_REQ_SET_INTERFACE

```
#define SL_USBD_REQ_SET_INTERFACE
```

Value:

11u

Definition at line 135 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_REQ_SYNCH_FRAME

```
#define SL_USBD_REQ_SYNCH_FRAME
```

Value:

12u

Definition at line 136 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_DESC_TYPE_DEVICE

```
#define SL_USBD_DESC_TYPE_DEVICE
```

Value:

```
1u
```

DESCRIPTOR TYPES

Note(s) : (1) Descriptors types are defined in the USB spec 2.0 section 9.2.6, Table 9-5.

Definition at line 144 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_DESC_TYPE_CONFIGURATION

```
#define SL_USBD_DESC_TYPE_CONFIGURATION
```

Value:

```
2u
```

Definition at line 145 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_DESC_TYPE_STRING

```
#define SL_USBD_DESC_TYPE_STRING
```

Value:

```
3u
```

Definition at line 146 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_DESC_TYPE_INTERFACE

```
#define SL_USBD_DESC_TYPE_INTERFACE
```

Value:

```
4u
```

Definition at line 147 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_DESC_TYPE_ENDPOINT

```
#define SL_USBD_DESC_TYPE_ENDPOINT
```

Value:

```
5u
```

Definition at line 148 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_DESC_TYPE_DEVICE_QUALIFIER

```
#define SL_USBD_DESC_TYPE_DEVICE_QUALIFIER
```

Value:

```
6u
```

Definition at line 149 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_DESC_TYPE_OTHER_SPEED_CONFIGURATION**

```
#define SL_USBD_DESC_TYPE_OTHER_SPEED_CONFIGURATION
```

Value:

```
7u
```

Definition at line 150 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_DESC_TYPE_INTERFACE_POWER**

```
#define SL_USBD_DESC_TYPE_INTERFACE_POWER
```

Value:

```
8u
```

Definition at line 151 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_DESC_TYPE_OTG**

```
#define SL_USBD_DESC_TYPE_OTG
```

Value:

```
9u
```

Definition at line 152 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_DESC_TYPE_IAD**

```
#define SL_USBD_DESC_TYPE_IAD
```

Value:

```
11u
```

Definition at line 153 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_LANG_ID_ARABIC_SAUDI_ARABIA**

```
#define SL_USBD_LANG_ID_ARABIC_SAUDI_ARABIA
```

Value:

```
0x0401u
```

LANGUAGE IDENTIFIERS

Note(s) : (1) Languages identifier are defined in "http://www.usb.org/developers/docs/USB_LANGIDs.pdf".

Definition at line 161 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_LANG_ID_CHINESE_TAIWAN

```
#define SL_USBD_LANG_ID_CHINESE_TAIWAN
```

Value:

```
0x0404u
```

Definition at line 162 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_LANG_ID_ENGLISH_US

```
#define SL_USBD_LANG_ID_ENGLISH_US
```

Value:

```
0x0409u
```

Definition at line 163 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_LANG_ID_ENGLISH_UK

```
#define SL_USBD_LANG_ID_ENGLISH_UK
```

Value:

```
0x0809u
```

Definition at line 164 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_LANG_ID_FRENCH

```
#define SL_USBD_LANG_ID_FRENCH
```

Value:

```
0x040Cu
```

Definition at line 165 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_LANG_ID_GERMAN

```
#define SL_USBD_LANG_ID_GERMAN
```

Value:

```
0x0407u
```

Definition at line 166 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_LANG_ID_GREEK

```
#define SL_USBD_LANG_ID_GREEK
```

Value:

```
0x0408u
```

Definition at line 167 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_LANG_ID_ITALIAN

```
#define SL_USBD_LANG_ID_ITALIAN
```

Value:

```
0x0410u
```

Definition at line 168 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_LANG_ID_PORTUGUESE

```
#define SL_USBD_LANG_ID_PORTUGUESE
```

Value:

```
0x0816u
```

Definition at line 169 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_LANG_ID_SANSKRIT

```
#define SL_USBD_LANG_ID_SANSKRIT
```

Value:

```
0x044Fu
```

Definition at line 170 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_USE_IF_DESC

```
#define SL_USBD_CLASS_CODE_USE_IF_DESC
```

Value:

```
0x00u
```

CLASS CODES

Note(s) : (1) Class codes are defined in "http://www.usb.org/developers/defined_class".

(2) Class code information can be placed [in](#) the Device descriptor or [in](#) the Interface descriptor. Some defined class [code](#) are allowed to be used only [in](#) the Device descriptor, others can be used [in](#) both Device and Interface descriptors and come can only be used [in](#) Interface Descriptors.

(3) Subclass & protocol codes are defined [in](#) the relevant class [drivers](#).

Definition at line 185 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_AUDIO

```
#define SL_USBD_CLASS_CODE_AUDIO
```

Value:

```
0x01u
```

Definition at line 186 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_CDC_CONTROL

```
#define SL_USBD_CLASS_CODE_CDC_CONTROL
```

Value:

```
0x02u
```

Definition at line 187 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_HID

```
#define SL_USBD_CLASS_CODE_HID
```

Value:

```
0x03u
```

Definition at line 188 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_PHYSICAL

```
#define SL_USBD_CLASS_CODE_PHYSICAL
```

Value:

0x05u

Definition at line 189 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_IMAGE

```
#define SL_USBD_CLASS_CODE_IMAGE
```

Value:

0x06u

Definition at line 190 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_PRINTER

```
#define SL_USBD_CLASS_CODE_PRINTER
```

Value:

0x07u

Definition at line 191 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_MASS_STORAGE

```
#define SL_USBD_CLASS_CODE_MASS_STORAGE
```

Value:

0x08u

Definition at line 192 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_HUB

```
#define SL_USBD_CLASS_CODE_HUB
```

Value:

0x09u

Definition at line 193 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_CDC_DATA

```
#define SL_USBD_CLASS_CODE_CDC_DATA
```

Value:

0x0Au

Definition at line 194 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_SMART_CARD

```
#define SL_USBD_CLASS_CODE_SMART_CARD
```

Value:

```
0x0Bu
```

Definition at line 195 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_CLASS_CODE_CONTENT_SECURITY**

```
#define SL_USBD_CLASS_CODE_CONTENT_SECURITY
```

Value:

```
0x0Du
```

Definition at line 196 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_CLASS_CODE_VIDEO**

```
#define SL_USBD_CLASS_CODE_VIDEO
```

Value:

```
0x0Eu
```

Definition at line 197 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_CLASS_CODE_PERSONAL_HEALTHCARE**

```
#define SL_USBD_CLASS_CODE_PERSONAL_HEALTHCARE
```

Value:

```
0x0Fu
```

Definition at line 198 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_CLASS_CODE_DIAGNOSTIC_DEVICE**

```
#define SL_USBD_CLASS_CODE_DIAGNOSTIC_DEVICE
```

Value:

```
0xDCu
```

Definition at line 199 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_CLASS_CODE_WIRELESS_CONTROLLER**


```
#define SL_USBD_CLASS_CODE_WIRELESS_CONTROLLER
```

Value:

```
0xE0u
```

Definition at line 200 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_MISCELLANEOUS

```
#define SL_USBD_CLASS_CODE_MISCELLANEOUS
```

Value:

```
0xEFu
```

Definition at line 201 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_APPLICATION_SPECIFIC

```
#define SL_USBD_CLASS_CODE_APPLICATION_SPECIFIC
```

Value:

```
0xFEu
```

Definition at line 202 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_CODE_VENDOR_SPECIFIC

```
#define SL_USBD_CLASS_CODE_VENDOR_SPECIFIC
```

Value:

```
0xFFu
```

Definition at line 203 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_SUBCLASS_CODE_USE_IF_DESC

```
#define SL_USBD_SUBCLASS_CODE_USE_IF_DESC
```

Value:

```
0x00u
```

SUB-CLASS CODES.

Definition at line 209 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_SUBCLASS_CODE_USE_COMMON_CLASS

```
#define SL_USBD_SUBCLASS_CODE_USE_COMMON_CLASS
```

Value:

```
0x02u
```

Definition at line 210 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_SUBCLASS_CODE_VENDOR_SPECIFIC

```
#define SL_USBD_SUBCLASS_CODE_VENDOR_SPECIFIC
```

Value:

```
0xFFu
```

Definition at line 211 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_PROTOCOL_CODE_USE_IF_DESC

```
#define SL_USBD_PROTOCOL_CODE_USE_IF_DESC
```

Value:

```
0x00u
```

PROTOCOL CODES.

Definition at line 217 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_PROTOCOL_CODE_USE_IAD

```
#define SL_USBD_PROTOCOL_CODE_USE_IAD
```

Value:

```
0x01u
```

Definition at line 218 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_PROTOCOL_CODE_VENDOR_SPECIFIC

```
#define SL_USBD_PROTOCOL_CODE_VENDOR_SPECIFIC
```

Value:

```
0xFFu
```

Definition at line 219 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_FEATURE_SEL_ENDPOINT_HALT

```
#define SL_USBD_FEATURE_SEL_ENDPOINT_HALT
```

Value:

0u

FEATURE SELECTORS.

Definition at line 225 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_FEATURE_SEL_DEVICE_REMOTE_WAKEUP

```
#define SL_USBD_FEATURE_SEL_DEVICE_REMOTE_WAKEUP
```

Value:

1u

Definition at line 226 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_FEATURE_SEL_TEST_MODE

```
#define SL_USBD_FEATURE_SEL_TEST_MODE
```

Value:

2u

Definition at line 227 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MAX_BUS_PWR_LIMIT_mA

```
#define SL_USBD_MAX_BUS_PWR_LIMIT_mA
```

Value:

500u

DEVICE POWER CONSTRAINS.

Definition at line 233 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_MASK

```
#define SL_USBD_ENDPOINT_TYPE_MASK
```

Value:

0x03u

ENDPOINT TYPE.

Note(s) : (1) Endpoint types are defined in the USB spec 2.0 section 9.6.6, Table 9-13, 'bmAttributes' field.

Definition at line 242 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_CTRL

```
#define SL_USBD_ENDPOINT_TYPE_CTRL
```

Value:

```
0x00u
```

Definition at line 243 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_ISOC

```
#define SL_USBD_ENDPOINT_TYPE_ISOC
```

Value:

```
0x01u
```

Definition at line 244 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_BULK

```
#define SL_USBD_ENDPOINT_TYPE_BULK
```

Value:

```
0x02u
```

Definition at line 245 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_INTR

```
#define SL_USBD_ENDPOINT_TYPE_INTR
```

Value:

```
0x03u
```

Definition at line 246 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_SYNC_MASK

```
#define SL_USBD_ENDPOINT_TYPE_SYNC_MASK
```

Value:

```
0x0Cu
```

Definition at line 248 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_SYNC_NONE

```
#define SL_USBD_ENDPOINT_TYPE_SYNC_NONE
```

Value:

```
0x00u
```

Definition at line 249 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_SYNC_ASYNC

```
#define SL_USBD_ENDPOINT_TYPE_SYNC_ASYNC
```

Value:

```
0x04u
```

Definition at line 250 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_SYNC_ADAPTIVE

```
#define SL_USBD_ENDPOINT_TYPE_SYNC_ADAPTIVE
```

Value:

```
0x08u
```

Definition at line 251 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_SYNC_SYNC

```
#define SL_USBD_ENDPOINT_TYPE_SYNC_SYNC
```

Value:

```
0x0Cu
```

Definition at line 252 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_USAGE_MASK

```
#define SL_USBD_ENDPOINT_TYPE_USAGE_MASK
```

Value:

```
0x30u
```

Definition at line 254 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_TYPE_USAGE_DATA

```
#define SL_USBD_ENDPOINT_TYPE_USAGE_DATA
```

Value:

0x00u

Definition at line 255 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_TYPE_USAGE_FEEDBACK

#define SL_USBD_ENDPOINT_TYPE_USAGE_FEEDBACK

Value:

0x10u

Definition at line 256 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_TYPE_USAGE_IMPLICIT_FEEDBACK

#define SL_USBD_ENDPOINT_TYPE_USAGE_IMPLICIT_FEEDBACK

Value:

0x20u

Definition at line 257 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_NBR_MASK

#define SL_USBD_ENDPOINT_NBR_MASK

Value:

0x0Fu

ENDPOINT ADDRESS.

Note(s) : (1) Endpoint address is defined in the USB spec 2.0, section 9.6.6, Table 9-13, 'bEndpointAddress' field.

Definition at line 266 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_MAX_NBR

#define SL_USBD_ENDPOINT_MAX_NBR

Value:

32u

Definition at line 267 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_DIR_MASK

#define SL_USBD_ENDPOINT_DIR_MASK

Value:

0x80u

Definition at line 269 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_DIR_BIT

#define SL_USBD_ENDPOINT_DIR_BIT

Value:

0x80u

Definition at line 270 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_DIR_OUT

#define SL_USBD_ENDPOINT_DIR_OUT

Value:

0x00u

Definition at line 271 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_DIR_IN

#define SL_USBD_ENDPOINT_DIR_IN

Value:

0x80u

Definition at line 272 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_1

#define SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_1

Value:

1u

Definition at line 274 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_2

#define SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_2

Value:

2u

Definition at line 275 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_3

```
#define SL_USBD_ENDPOINT_TRANSACTION_PER_UFRAME_3
```

Value:

```
3u
```

Definition at line 276 of file `protocol/usb/inc/slUsbd_core.h`**SL_USBD_ENDPOINT_MAX_INTERVAL_VAL**

```
#define SL_USBD_ENDPOINT_MAX_INTERVAL_VAL
```

Value:

```
32768u
```

Definition at line 278 of file `protocol/usb/inc/slUsbd_core.h`**SL_USBD_MICROSOFT_COMPAT_ID_NULL**

```
#define SL_USBD_MICROSOFT_COMPAT_ID_NULL
```

Value:

```
0u
```

MICROSOFT OS DESCRIPTOR DEFINES.

Note(s) : (1) For more information on Microsoft OS descriptors, see '<http://msdn.microsoft.com/en-us/library/windows/hardware/gg463179.aspx>'.

Definition at line 288 of file `protocol/usb/inc/slUsbd_core.h`**SL_USBD_MICROSOFT_COMPAT_ID_RNDIS**

```
#define SL_USBD_MICROSOFT_COMPAT_ID_RNDIS
```

Value:

```
1u
```

Definition at line 289 of file `protocol/usb/inc/slUsbd_core.h`**SL_USBD_MICROSOFT_COMPAT_ID_PTP**

```
#define SL_USBD_MICROSOFT_COMPAT_ID_PTP
```

Value:

```
2u
```

Definition at line 290 of file `protocol/usb/inc/slUsbd_core.h`

SL_USBD_MICROSOFT_COMPAT_ID_MTP

```
#define SL_USBD_MICROSOFT_COMPAT_ID_MTP
```

Value:

```
3u
```

Definition at line 291 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_MICROSOFT_COMPAT_ID_XUSB20**

```
#define SL_USBD_MICROSOFT_COMPAT_ID_XUSB20
```

Value:

```
4u
```

Definition at line 292 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_MICROSOFT_COMPAT_ID_BLUETOOTH**

```
#define SL_USBD_MICROSOFT_COMPAT_ID_BLUETOOTH
```

Value:

```
5u
```

Definition at line 293 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_MICROSOFT_COMPAT_ID_WINUSB**

```
#define SL_USBD_MICROSOFT_COMPAT_ID_WINUSB
```

Value:

```
6u
```

Definition at line 294 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_MICROSOFT_COMPAT_ID_NONE**

```
#define SL_USBD_MICROSOFT_COMPAT_ID_NONE
```

Value:

```
255u
```

Definition at line 295 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_MICROSOFT_SUBCOMPAT_ID_NULL**

```
#define SL_USBD_MICROSOFT_SUBCOMPAT_ID_NULL
```

Value:

```
0u
```

Definition at line 298 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_1_1

```
#define SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_1_1
```

Value:

```
1u
```

Definition at line 299 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_1_2

```
#define SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_1_2
```

Value:

```
2u
```

Definition at line 300 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_EDR

```
#define SL_USBD_MICROSOFT_SUBCOMPAT_ID_BLUETOOTH_EDR
```

Value:

```
3u
```

Definition at line 301 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_SZ

```
#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_SZ
```

Value:

```
1u
```

Definition at line 304 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_EXPAND_SZ

```
#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_EXPAND_SZ
```

Value:

```
2u
```

Definition at line 305 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_BINARY

```
#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_BINARY
```

Value:

```
3u
```

Definition at line 306 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_DWORD_LITTLE_ENDIAN

```
#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_DWORD_LITTLE_ENDIAN
```

Value:

```
4u
```

Definition at line 307 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_DWORD_BIG_ENDIAN

```
#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_DWORD_BIG_ENDIAN
```

Value:

```
5u
```

Definition at line 308 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_LINK

```
#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_LINK
```

Value:

```
6u
```

Definition at line 309 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_MULTI_SZ

```
#define SL_USBD_MICROSOFT_PROPERTY_TYPE_REG_MULTI_SZ
```

Value:

```
7u
```

Definition at line 310 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_INFO_TYPE_CTRL

```
#define SL_USBD_ENDPOINT_INFO_TYPE_CTRL
```

Value:

```
0x01u
```

USB EP INFORMATION TABLE DEFINES.

Definition at line 317 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_INFO_TYPE_ISOC

```
#define SL_USBD_ENDPOINT_INFO_TYPE_ISOC
```

Value:

```
0x02u
```

Definition at line 318 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_INFO_TYPE_BULK

```
#define SL_USBD_ENDPOINT_INFO_TYPE_BULK
```

Value:

```
0x04u
```

Definition at line 319 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_INFO_TYPE_INTR

```
#define SL_USBD_ENDPOINT_INFO_TYPE_INTR
```

Value:

```
0x08u
```

Definition at line 320 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_INFO_DIR_OUT

```
#define SL_USBD_ENDPOINT_INFO_DIR_OUT
```

Value:

```
0x10u
```

Definition at line 323 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_INFO_DIR_IN

```
#define SL_USBD_ENDPOINT_INFO_DIR_IN
```

Value:

```
0x20u
```

Definition at line 324 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CONFIG_NBR_NONE

```
#define SL_USBD_CONFIG_NBR_NONE
```

Value:

```
255u
```

USB OBJECT NUMBER.

Definition at line 330 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CONFIG_NBR_SPD_BIT

```
#define SL_USBD_CONFIG_NBR_SPD_BIT
```

Value:

```
0x80u
```

Definition at line 331 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_INTERFACE_NBR_NONE

```
#define SL_USBD_INTERFACE_NBR_NONE
```

Value:

```
255u
```

Definition at line 333 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ALT_INTERFACE_NBR_NONE

```
#define SL_USBD_ALT_INTERFACE_NBR_NONE
```

Value:

```
255u
```

Definition at line 335 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_INTERFACE_GROUP_NBR_NONE

```
#define SL_USBD_INTERFACE_GROUP_NBR_NONE
```

Value:

```
255u
```

Definition at line 337 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_ADDR_NONE

```
#define SL_USBD_ENDPOINT_ADDR_NONE
```

Value:

```
255u
```

Definition at line 339 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_PHY_NONE

```
#define SL_USBD_ENDPOINT_PHY_NONE
```

Value:

```
255u
```

Definition at line 340 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_ENDPOINT_NBR_NONE

```
#define SL_USBD_ENDPOINT_NBR_NONE
```

Value:

```
255u
```

Definition at line 342 of file `protocol/usb/inc/sl_usbd_core.h`

SL_USBD_CLASS_NBR_NONE

```
#define SL_USBD_CLASS_NBR_NONE
```

Value:

```
255u
```

Definition at line 344 of file `protocol/usb/inc/sl_usbd_core.h`

USBD_CFG_STR_EN

```
#define USBD_CFG_STR_EN
```

Value:

1

USB DEFAULT CONFIGS.

Definition at line 351 of file `protocol/usb/inc/sl_usbd_core.h`**USBD_CFG_MS_OS_DESC_EN**

#define USBD_CFG_MS_OS_DESC_EN

Value:

0

Definition at line 355 of file `protocol/usb/inc/sl_usbd_core.h`**USBD_CFG_URB_EXTRA_EN**

#define USBD_CFG_URB_EXTRA_EN

Value:

0

Definition at line 359 of file `protocol/usb/inc/sl_usbd_core.h`**SL_USBD_URB_MAX_NBR**

#define SL_USBD_URB_MAX_NBR

Value:

SL_USBD_OPEN_ENDPOINTS_QUANTITY

Definition at line 365 of file `protocol/usb/inc/sl_usbd_core.h`**USBD_CFG_HS_EN**

#define USBD_CFG_HS_EN

Value:

0

Definition at line 369 of file `protocol/usb/inc/sl_usbd_core.h`**USBD_CFG_EP_ISOC_EN**

#define USBD_CFG_EP_ISOC_EN

Value:

0

Definition at line 373 of file protocol/usb/inc/sl_usbd_core.h

USBD_CFG_OPTIMIZE_SPD

```
#define USBD_CFG_OPTIMIZE_SPD
```

Value:

```
0
```

Definition at line 377 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_CORE_EVENT_BUS_NBR

```
#define SL_USBD_CORE_EVENT_BUS_NBR
```

Value:

```
7u
```

USB CORE EVENTS.

Note(s) : (1) There are 7 possible USB callback events:

```
sl_usbd_core_reset_event(),  
sl_usbd_core_suspend_event(),  
sl_usbd_core_resume_event(),  
sl_usbd_core_connect_event(),  
sl_usbd_core_disconnect_event(),  
sl_usbd_core_high_speed_event().
```

Definition at line 393 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_CORE_EVENT_BUS_NBR_TOTAL

```
#define SL_USBD_CORE_EVENT_BUS_NBR_TOTAL
```

Value:

```
(SL_USBD_CORE_EVENT_BUS_NBR * SL_USBD_DEVICE_QUANTITY)
```

Definition at line 395 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_CORE_EVENT_ASYNC_NBR

```
#define SL_USBD_CORE_EVENT_ASYNC_NBR
```

Value:

```
((SL_USBD_OPEN_ENDPOINTS_QUANTITY * SL_USBD_DEVICE_QUANTITY) + SL_USBD_EXTRA_URB_QUANTITY)
```

Definition at line 396 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_CORE_EVENT_NBR_TOTAL


```
#define SL_USBD_CORE_EVENT_NBR_TOTAL
```

Value:

```
(SL_USBD_CORE_EVENT_BUS_NBR_TOTAL + SL_USBD_CORE_EVENT_ASYNC_NBR)
```

Definition at line 397 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_FRAME_NBR_GET

```
#define SL_USBD_FRAME_NBR_GET
```

Value:

```
(frame_nbr)
```

GLOBAL CONSTANTS.

MACROS FRAME MACROS

Note(s) : (1) This macro takes into account when the frame number rolls over upon reaching the maximum value of 2047.

Definition at line 614 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_MICROFRAME_NBR_GET

```
#define SL_USBD_MICROFRAME_NBR_GET
```

Value:

```
(microframe_nbr)
```

Definition at line 616 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_FRAME_NBR_DIFF_GET

```
#define SL_USBD_FRAME_NBR_DIFF_GET
```

Value:

```
0 | (((frame_nbr2) >= (frame_nbr1)) \
0 | ? ((frame_nbr2) - (frame_nbr1)) \
0 | : ((SL_USBD_FRAME_NBR_MAX + 1u + (frame_nbr2)) - (frame_nbr1)))
```

Definition at line 619 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_PHY_TO_ADDR

```
#define SL_USBD_ENDPOINT_PHY_TO_ADDR
```

Value:

```
0 | ((ep_phy_nbr) / 2u) \
0 | (((ep_phy_nbr) % 2u) ? SL_USBD_ENDPOINT_DIR_BIT : 0x00u))
```

ENDPOINT CONVERSION MACROS.

Definition at line 627 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_ADDR_TO_PHY

```
#define SL_USBD_ENDPOINT_ADDR_TO_PHY
```

Value:

```
0 | (((ep_addr) & SL_USBD_ENDPOINT_NBR_MASK) * 2u) \
0 | + (((ep_addr) & SL_USBD_ENDPOINT_DIR_MASK) != 0u) ? 1u : 0u))
```

Definition at line 630 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_ADDR_TO_LOG

```
#define SL_USBD_ENDPOINT_ADDR_TO_LOG
```

Value:

```
(ep_addr)
```

Definition at line 633 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_PHY_TO_LOG

```
#define SL_USBD_ENDPOINT_PHY_TO_LOG
```

Value:

```
(ep_phy_nbr)
```

Definition at line 635 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_LOG_TO_ADDR_IN

```
#define SL_USBD_ENDPOINT_LOG_TO_ADDR_IN
```

Value:

```
(ep_log_nbr)
```

Definition at line 637 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_LOG_TO_ADDR_OUT

```
#define SL_USBD_ENDPOINT_LOG_TO_ADDR_OUT
```

Value:

(ep_log_nbr)

Definition at line 639 of file protocol/usb/inc/sl_usbd_core.h

SL_USBD_ENDPOINT_IS_IN

```
#define SL_USBD_ENDPOINT_IS_IN
```

Value:

(ep_addr)

Definition at line 641 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_device_config_t

USB DEVICE CONFIGURATION DATA TYPE.

Public Attributes

uint16_t	vendor_id	Vendor id.
uint16_t	product_id	Product id.
uint16_t	device_bcd	Device release number.
const char *	manufacturer_str_ptr	Manufacturer string.
const char *	product_str_ptr	Product string.
const char *	serial_nbr_str_ptr	Serial Number id.
uint16_t	lang_id	Language id.

Public Attribute Documentation

vendor_id

```
uint16_t sl_usbd_device_config_t::vendor_id
```

Vendor id.

Definition at line 484 of file `protocol/usb/inc/sl_usbd_core.h`

product_id

```
uint16_t sl_usbd_device_config_t::product_id
```

Product id.

Definition at line 485 of file `protocol/usb/inc/sl_usbd_core.h`

device_bcd

```
uint16_t sl_usbd_device_config_t::device_bcd
```

Device release number.

Definition at line 486 of file protocol/usb/inc/sl_usbd_core.h

manufacturer_str_ptr

```
const char* sl_usbd_device_config_t::manufacturer_str_ptr
```

Manufacturer string.

Definition at line 487 of file protocol/usb/inc/sl_usbd_core.h

product_str_ptr

```
const char* sl_usbd_device_config_t::product_str_ptr
```

Product string.

Definition at line 488 of file protocol/usb/inc/sl_usbd_core.h

serial_nbr_str_ptr

```
const char* sl_usbd_device_config_t::serial_nbr_str_ptr
```

Serial Number id.

Definition at line 489 of file protocol/usb/inc/sl_usbd_core.h

lang_id

```
uint16_t sl_usbd_device_config_t::lang_id
```

Language id.

Definition at line 490 of file protocol/usb/inc/sl_usbd_core.h

sl_usbd_setup_req_t

USB DEVICE MICROSOFT XTENDED PROPERTIES DATA TYPE.

USB SETUP REQUEST

Public Attributes

uint8_t	bmRequestType	Characteristics of request.
uint8_t	bRequest	Specific request.
uint16_t	wValue	Varies according to request.
uint16_t	wIndex	Varies according to request; typically used as index.
uint16_t	wLength	Transfer length if data stage present.

Public Attribute Documentation

bmRequestType

uint8_t sl_usbd_setup_req_t::bmRequestType

Characteristics of request.

Definition at line 513 of file protocol/usb/inc/sl_usbd_core.h

bRequest

uint8_t sl_usbd_setup_req_t::bRequest

Specific request.

Definition at line 514 of file protocol/usb/inc/sl_usbd_core.h

wValue

uint16_t sl_usbd_setup_req_t::wValue

Varies according to request.

Definition at line 515 of file protocol/usb/inc/sl_usbd_core.h

wIndex

```
uint16_t sl_usbd_setup_req_t::wIndex
```

Varies according to request; typically used as index.

Definition at line 516 of file `protocol/usb/inc/sl_usbd_core.h`

wLength

```
uint16_t sl_usbd_setup_req_t::wLength
```

Transfer length if data stage present.

Definition at line 517 of file `protocol/usb/inc/sl_usbd_core.h`

sl_usbd_class_driver_t

INTERFACE CLASS DRIVER.

Public Attributes

void(*)	enable	Notify class that cfg is active.
void(*)	disable	Notify class that cfg is not active.
void(*)	alt_settings_update	Notify class that alt setting has been updated.
void(*)	endpoint_state_update	Notify class that EP state has been updated.
void(*)	interface_descriptor	Construct IF functional descriptor.
uint16_t(*)	interface_get_descriptor_size	Get size of IF functional descriptor.
void(*)	endpoint_descriptor	Construct EP functional descriptor.
uint16_t(*)	endpoint_get_descriptor_size	Get size of EP functional descriptor.
bool(*)	interface_req	Interface request callback.
bool(*)	class_req	Class-specific request callback.
bool(*)	vendor_req	Vendor-specific request callback.

Public Attribute Documentation

enable

```
void(* sl_usbd_class_driver_t::enable) (uint8_t cfg_nbr, void *p_if_class_arg)
```

Notify class that cfg is active.

Definition at line 524 of file `protocol/usb/inc/sl_usbd_core.h`

disable

```
void(* sl_usbd_class_driver_t::disable) (uint8_t cfg_nbr, void *p_if_class_arg)
```


Notify class that cfg is not active.

Definition at line 527 of file `protocol/usb/inc/sl_usbd_core.h`

alt_settings_update

```
void(* sl_usbd_class_driver_t::alt_settings_update) (uint8_t cfg_nbr, uint8_t if_nbr, uint8_t if_alt_nbr, void *p_if_class_arg, void *p_if_alt_class_arg)
```

Notify class that alt setting has been updated.

Definition at line 530 of file `protocol/usb/inc/sl_usbd_core.h`

endpoint_state_update

```
void(* sl_usbd_class_driver_t::endpoint_state_update) (uint8_t cfg_nbr, uint8_t if_nbr, uint8_t if_alt_nbr, uint8_t ep_addr, void *p_if_class_arg, void *p_if_alt_class_arg)
```

Notify class that EP state has been updated.

Definition at line 536 of file `protocol/usb/inc/sl_usbd_core.h`

interface_descriptor

```
void(* sl_usbd_class_driver_t::interface_descriptor) (uint8_t cfg_nbr, uint8_t if_nbr, uint8_t if_alt_nbr, void *p_if_class_arg, void *p_if_alt_class_arg)
```

Construct IF functional descriptor.

Definition at line 543 of file `protocol/usb/inc/sl_usbd_core.h`

interface_get_descriptor_size

```
uint16_t(* sl_usbd_class_driver_t::interface_get_descriptor_size) (uint8_t cfg_nbr, uint8_t if_nbr, uint8_t if_alt_nbr, void *p_if_class_arg, void *p_if_alt_class_arg)
```

Get size of IF functional descriptor.

Definition at line 549 of file `protocol/usb/inc/sl_usbd_core.h`

endpoint_descriptor

```
void(* sl_usbd_class_driver_t::endpoint_descriptor) (uint8_t cfg_nbr, uint8_t if_nbr, uint8_t if_alt_nbr, uint8_t ep_addr, void *p_if_class_arg, void *p_if_alt_class_arg)
```

Construct EP functional descriptor.

Definition at line 555 of file `protocol/usb/inc/sl_usbd_core.h`

endpoint_get_descriptor_size

```
uint16_t(* sl_usbd_class_driver_t::endpoint_get_descriptor_size) (uint8_t cfg_nbr, uint8_t if_nbr, uint8_t if_alt_nbr, uint8_t ep_addr, void *p_if_class_arg, void *p_if_alt_class_arg)
```

Get size of EP functional descriptor.

Definition at line 562 of file `protocol/usb/inc/sl_usbd_core.h`

interface_req

```
bool(* sl_usbd_class_driver_t::interface_req) (const sl_usbd_setup_req_t *p_setup_req, void *p_if_class_arg)
```

Interface request callback.

Definition at line 569 of file `protocol/usb/inc/sl_usbd_core.h`

class_req

```
bool(* sl_usbd_class_driver_t::class_req) (const sl_usbd_setup_req_t *p_setup_req, void *p_if_class_arg)
```

Class-specific request callback.

Definition at line 572 of file `protocol/usb/inc/sl_usbd_core.h`

vendor_req

```
bool(* sl_usbd_class_driver_t::vendor_req) (const sl_usbd_setup_req_t *p_setup_req, void *p_if_class_arg)
```

Vendor-specific request callback.

Definition at line 575 of file `protocol/usb/inc/sl_usbd_core.h`

USB Device HID API

USB Device HID API

USB Device HID API.

Modules

[sl_usbd_hid_callbacks_t](#)

Enumerations

```
enum    sl_usbd_hid_country_code_t {  
    SL_USBD_HID_COUNTRY_CODE_NOT_SUPPORTED = 0u  
    SL_USBD_HID_COUNTRY_CODE_ARABIC = 1u  
    SL_USBD_HID_COUNTRY_CODE_BELGIAN = 2u  
    SL_USBD_HID_COUNTRY_CODE_CANADIAN_BILINGUAL = 3u  
    SL_USBD_HID_COUNTRY_CODE_CANADIAN_FRENCH = 4u  
    SL_USBD_HID_COUNTRY_CODE_CZECH_REPUBLIC = 5u  
    SL_USBD_HID_COUNTRY_CODE_DANISH = 6u  
    SL_USBD_HID_COUNTRY_CODE_FINNISH = 7u  
    SL_USBD_HID_COUNTRY_CODE_FRENCH = 8u  
    SL_USBD_HID_COUNTRY_CODE_GERMAN = 9u  
    SL_USBD_HID_COUNTRY_CODE_GREEK = 10u  
    SL_USBD_HID_COUNTRY_CODE_HEBREW = 11u  
    SL_USBD_HID_COUNTRY_CODE_HUNGARY = 12u  
    SL_USBD_HID_COUNTRY_CODE_INTERNATIONAL = 13u  
    SL_USBD_HID_COUNTRY_CODE_ITALIAN = 14u  
    SL_USBD_HID_COUNTRY_CODE_JAPAN_KATAKANA = 15u  
    SL_USBD_HID_COUNTRY_CODE_KOREAN = 16u  
    SL_USBD_HID_COUNTRY_CODE_LATIN_AMERICAN = 17u  
    SL_USBD_HID_COUNTRY_CODE_NETHERLANDS_DUTCH = 18u  
    SL_USBD_HID_COUNTRY_CODE_NORWEGIAN = 19u  
    SL_USBD_HID_COUNTRY_CODE_PERSIAN_FARSI = 20u  
    SL_USBD_HID_COUNTRY_CODE_POLAND = 21u  
    SL_USBD_HID_COUNTRY_CODE_PORTUGUESE = 22u  
    SL_USBD_HID_COUNTRY_CODE_RUSSIA = 23u  
    SL_USBD_HID_COUNTRY_CODE_SLOVAKIA = 24u  
    SL_USBD_HID_COUNTRY_CODE_SPANISH = 25u  
    SL_USBD_HID_COUNTRY_CODE_SWEDISH = 26u  
    SL_USBD_HID_COUNTRY_CODE_SWISS_FRENCH = 27u  
    SL_USBD_HID_COUNTRY_CODE_SWISS_GERMAN = 28u  
    SL_USBD_HID_COUNTRY_CODE_SWITZERLAND = 29u  
    SL_USBD_HID_COUNTRY_CODE_TAIWAN = 30u  
    SL_USBD_HID_COUNTRY_CODE_TURKISH_Q = 31u  
    SL_USBD_HID_COUNTRY_CODE_UK = 32u  
    SL_USBD_HID_COUNTRY_CODE_US = 33u  
    SL_USBD_HID_COUNTRY_CODE_YUGOSLAVIA = 34u  
    SL_USBD_HID_COUNTRY_CODE_TURKISH_F = 35u  
}  
DEFINES.
```

Typedefs

```
typedef void(* sl_usbd_hid_async_function_t)(uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void
*p_callback_arg, sl_status_t status)
GLOBAL CONSTANTS.
```

Functions

sl_status_t	sl_usbd_hid_init (void) GLOBAL CONSTANTS.
sl_status_t	sl_usbd_hid_create_instance (uint8_t subclass, uint8_t protocol, sl_usbd_hid_country_code_t country_code, uint16_t interval_in, uint16_t interval_out, bool ctrl_rd_en, sl_usbd_hid_callbacks_t *p_hid_callbacks, uint8_t *p_class_nbr) Add a new instance of the HID class.
sl_status_t	sl_usbd_hid_add_to_configuration (uint8_t class_nbr, uint8_t config_nbr) Add the HID class instance to the USB device configuration (see Note #1).
sl_status_t	sl_usbd_hid_is_enabled (uint8_t class_nbr, bool *p_enabled) Get the HID class enable state.
sl_status_t	sl_usbd_hid_write_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len) Send data to the host through the Interrupt IN endpoint.
sl_status_t	sl_usbd_hid_read_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len) Receive data from the host through the Interrupt OUT endpoint.
sl_status_t	sl_usbd_hid_read_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len, sl_usbd_hid_async_function_t async_fnct, void *p_async_arg) Receive data from the host through the Interrupt OUT endpoint.
sl_status_t	sl_usbd_hid_write_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len, sl_usbd_hid_async_function_t async_fnct, void *p_async_arg) Send data to the host through the Interrupt IN endpoint.

Macros

#define	SLI_USBD_HID_EXTERN extern MODULE.
#define	SL_USBD_HID_SUBCLASS_NONE 0x00u HUMAN INTERFACE DEVICE CLASS SUBCLASS CODES DEFINES.
#define	SL_USBD_HID_SUBCLASS_BOOT 0x01u
#define	SL_USBD_HID_PROTOCOL_NONE 0x00u HUMAN INTERFACE DEVICE CLASS PROTOCOL CODES DEFINES.
#define	SL_USBD_HID_PROTOCOL_KBD 0x01u
#define	SL_USBD_HID_PROTOCOL_MOUSE 0x02u
#define	SL_USBD_HID_ITEM_LONG 0xFE ITEM TYPES.
#define	SL_USBD_HID_ITEM_TYPE_MAIN 0x00
#define	SL_USBD_HID_ITEM_TYPE_GLOBAL 0x04
#define	SL_USBD_HID_ITEM_TYPE_LOCAL 0x08

```
#define SL_USBD_HID_ITEM_TYPE_RESERVED 0x0C

#define SL_USBD_HID_MAIN_INPUT 0x80

#define SL_USBD_HID_MAIN_OUTPUT 0x90

#define SL_USBD_HID_MAIN_COLLECTION 0xA0

#define SL_USBD_HID_MAIN_FEATURE 0xB0

#define SL_USBD_HID_MAIN_ENDCOLLECTION 0xC0

#define SL_USBD_HID_MAIN_CONSTANT 0x01

#define SL_USBD_HID_MAIN_DATA 0x00

#define SL_USBD_HID_MAIN_VARIABLE 0x02

#define SL_USBD_HID_MAIN_ARRAY 0x00

#define SL_USBD_HID_MAIN_RELATIVE 0x04

#define SL_USBD_HID_MAIN_ABSOLUTE 0x00

#define SL_USBD_HID_MAIN_WRAP 0x08

#define SL_USBD_HID_MAIN_NOWRAP 0x00

#define SL_USBD_HID_MAIN_NONLINEAR 0x10

#define SL_USBD_HID_MAIN_LINEAR 0x00

#define SL_USBD_HID_MAIN_NOPREFERRED 0x20

#define SL_USBD_HID_MAIN_PREFERREDSTATE 0x00

#define SL_USBD_HID_MAIN_NULLSTATE 0x40

#define SL_USBD_HID_MAIN_NONULLPOSITION 0x00

#define SL_USBD_HID_MAIN_VOLATILE 0x80

#define SL_USBD_HID_MAIN_NONVOLATILE 0x00

#define SL_USBD_HID_MAIN_BUFFEREDBYTES 0x0100

#define SL_USBD_HID_MAIN_BITFIELD 0x0000

#define SL_USBD_HID_COLLECTION_PHYSICAL 0x00

#define SL_USBD_HID_COLLECTION_APPLICATION 0x01

#define SL_USBD_HID_COLLECTION_LOGICAL 0x02

#define SL_USBD_HID_COLLECTION_REPORT 0x03

#define SL_USBD_HID_COLLECTION_NAMEDARRAY 0x04

#define SL_USBD_HID_COLLECTION_USAGESWITCH 0x05

#define SL_USBD_HID_COLLECTION_USAGEMODIFIER 0x06
```

```
#define SL_USBD_HID_GLOBAL_USAGE_PAGE 0x04

#define SL_USBD_HID_GLOBAL_LOG_MIN 0x14

#define SL_USBD_HID_GLOBAL_LOG_MAX 0x24

#define SL_USBD_HID_GLOBAL_PHY_MIN 0x34

#define SL_USBD_HID_GLOBAL_PHY_MAX 0x44

#define SL_USBD_HID_GLOBAL_UNIT_EXPONENT 0x54

#define SL_USBD_HID_GLOBAL_UNIT 0x64

#define SL_USBD_HID_GLOBAL_REPORT_SIZE 0x74

#define SL_USBD_HID_GLOBAL_REPORT_ID 0x84

#define SL_USBD_HID_GLOBAL_REPORT_COUNT 0x94

#define SL_USBD_HID_GLOBAL_PUSH 0xA4

#define SL_USBD_HID_GLOBAL_POP 0xB4

#define SL_USBD_HID_LOCAL_USAGE 0x08

#define SL_USBD_HID_LOCAL_USAGE_MIN 0x18

#define SL_USBD_HID_LOCAL_USAGE_MAX 0x28

#define SL_USBD_HID_LOCAL_DESIGNATOR_INDEX 0x38

#define SL_USBD_HID_LOCAL_DESIGNATOR_MIN 0x48

#define SL_USBD_HID_LOCAL_DESIGNATOR_MAX 0x58

#define SL_USBD_HID_LOCAL_STRING_INDEX 0x78

#define SL_USBD_HID_LOCAL_STRING_MIN 0x88

#define SL_USBD_HID_LOCAL_STRING_MAX 0x98

#define SL_USBD_HID_LOCAL_DELIMITER 0xA8

#define SL_USBD_HID_BIAS_NOT_APPLICABLE 0

#define SL_USBD_HID_BIAS_RIGHT_HAND 1

#define SL_USBD_HID_BIAS_LEFT_HAND 2

#define SL_USBD_HID_BIAS_BOTH_HANDS 3

#define SL_USBD_HID_BIAS_EITHER_HAND 4

#define SL_USBD_HID_DESIGNATOR_NONE 0x00

#define SL_USBD_HID_DESIGNATOR_HAND 0x01

#define SL_USBD_HID_DESIGNATOR_EYEBALL 0x02

#define SL_USBD_HID_DESIGNATOR_EYEBROW 0x03
```

```
#define SL_USBD_HID_DESIGNATOR_EYELID 0x04
#define SL_USBD_HID_DESIGNATOR_EAR 0x05
#define SL_USBD_HID_DESIGNATOR_NOSE 0x06
#define SL_USBD_HID_DESIGNATOR_MOUTH 0x07
#define SL_USBD_HID_DESIGNATOR_UPPER_LIP 0x08
#define SL_USBD_HID_DESIGNATOR_LOWER_LIP 0x09
#define SL_USBD_HID_DESIGNATOR_JAW 0x0A
#define SL_USBD_HID_DESIGNATOR_NECK 0x0B
#define SL_USBD_HID_DESIGNATOR_UPPER_ARM 0x0C
#define SL_USBD_HID_DESIGNATOR_ELBOW 0x0D
#define SL_USBD_HID_DESIGNATOR_FOREARM 0x0E
#define SL_USBD_HID_DESIGNATOR_WRIST 0x0F
#define SL_USBD_HID_DESIGNATOR_PALM 0x10
#define SL_USBD_HID_DESIGNATOR_THUMB 0x11
#define SL_USBD_HID_DESIGNATOR_INDEX_FINGER 0x12
#define SL_USBD_HID_DESIGNATOR_MIDDLE_FINGER 0x13
#define SL_USBD_HID_DESIGNATOR_RING_FINGER 0x14
#define SL_USBD_HID_DESIGNATOR_LITTLE_FINGER 0x15
#define SL_USBD_HID_DESIGNATOR_HEAD 0x16
#define SL_USBD_HID_DESIGNATOR_SHOULDER 0x17
#define SL_USBD_HID_DESIGNATOR_HIP 0x18
#define SL_USBD_HID_DESIGNATOR_WAIST 0x19
#define SL_USBD_HID_DESIGNATOR_THIGH 0x1A
#define SL_USBD_HID_DESIGNATOR_KNEE 0x1B
#define SL_USBD_HID_DESIGNATOR_CALF 0x1C
#define SL_USBD_HID_DESIGNATOR_ANKLE 0x1D
#define SL_USBD_HID_DESIGNATOR_FOOT 0x1E
#define SL_USBD_HID_DESIGNATOR_HEEL 0x1F
#define SL_USBD_HID_DESIGNATOR_BALL_OF_FOOT 0x20
#define SL_USBD_HID_DESIGNATOR_BIG_TOE 0x21
#define SL_USBD_HID_DESIGNATOR_SECOND_TOE 0x22
```

```
#define SL_USBD_HID_DESIGNATOR_THIRD_TOE 0x23
#define SL_USBD_HID_DESIGNATOR_FOURTH_TOE 0x24
#define SL_USBD_HID_DESIGNATOR_LITTLE_TOE 0x25
#define SL_USBD_HID_DESIGNATOR_BROW 0x26
#define SL_USBD_HID_DESIGNATOR_CHEEK 0x27
#define SL_USBD_HID_QUALIFIER_NOT_APPLICABLE 0
#define SL_USBD_HID_QUALIFIER_RIGHT 1
#define SL_USBD_HID_QUALIFIER_LEFT 2
#define SL_USBD_HID_QUALIFIER_BOTH 3
#define SL_USBD_HID_QUALIFIER_EITHER 4
#define SL_USBD_HID_QUALIFIER_CENTER 5
#define SL_USBD_HID_USAGE_PAGE_UNDEFINED 0x00
#define SL_USBD_HID_USAGE_PAGE_GENERIC_DESKTOP_CONTROLS 0x01
#define SL_USBD_HID_USAGE_PAGE_SIMULATION_CONTROLS 0x02
#define SL_USBD_HID_USAGE_PAGE_VR_CONTROLS 0x03
#define SL_USBD_HID_USAGE_PAGE_SPORT_CONTROLS 0x04
#define SL_USBD_HID_USAGE_PAGE_GAME_CONTROLS 0x05
#define SL_USBD_HID_USAGE_PAGE_GENERIC_DEVICE_CONTROLS 0x06
#define SL_USBD_HID_USAGE_PAGE_KEYBOARD 0x07
#define SL_USBD_HID_USAGE_PAGE_LEDS 0x08
#define SL_USBD_HID_USAGE_PAGE_BUTTON 0x09
#define SL_USBD_HID_USAGE_PAGE_ORDINAL 0x0A
#define SL_USBD_HID_USAGE_PAGE_TELEPHONY 0x0B
#define SL_USBD_HID_USAGE_PAGE_CONSUMER 0x0C
#define SL_USBD_HID_USAGE_PAGE_DIGITIZER 0x0D
#define SL_USBD_HID_USAGE_PAGE_PID_PAGE 0x0F
#define SL_USBD_HID_USAGE_PAGE_UNICODE 0x10
#define SL_USBD_HID_USAGE_PAGE_ALPHANUMERIC_DISPLAY 0x14
#define SL_USBD_HID_USAGE_PAGE_MEDICAL_INSTRUMENTS 0x40
#define SL_USBD_HID_USAGE_PAGE_MONITOR_0 0x80
#define SL_USBD_HID_USAGE_PAGE_MONITOR_1 0x81
```



```
#define SL_USBD_HID_USAGE_PAGE_MONITOR_2 0x82

#define SL_USBD_HID_USAGE_PAGE_MONITOR_3 0x83

#define SL_USBD_HID_USAGE_PAGE_POWER_0 0x84

#define SL_USBD_HID_USAGE_PAGE_POWER_1 0x85

#define SL_USBD_HID_USAGE_PAGE_POWER_2 0x86

#define SL_USBD_HID_USAGE_PAGE_POWER_3 0x87

#define SL_USBD_HID_USAGE_PAGE_BAR_CODE_SCANNER_PAGE 0x8C

#define SL_USBD_HID_USAGE_PAGE_SCALE_PAGE 0x8D

#define SL_USBD_HID_USAGE_PAGE_MSR_DEVICES 0x8E

#define SL_USBD_HID_USAGE_PAGE_POINT_OF_SALE_PAGES 0x8F

#define SL_USBD_HID_USAGE_PAGE_CAMERA_CONTROL_PAGE 0x90

#define SL_USBD_HID_USAGE_PAGE_ARCADE_PAGE 0x91

#define SL_USBD_HID_CP_POINTER 0x01

#define SL_USBD_HID_CA_MOUSE 0x02

#define SL_USBD_HID_CA_JOYSTICK 0x04

#define SL_USBD_HID_CA_GAME_PAD 0x05

#define SL_USBD_HID_CA_KEYBOARD 0x06

#define SL_USBD_HID_CA_KEYPAD 0x07

#define SL_USBD_HID_CA_MULTI_AXIS_CONTROLLER 0x08

#define SL_USBD_HID_DV_X 0x30

#define SL_USBD_HID_DV_Y 0x31

#define SL_USBD_HID_DV_Z 0x32

#define SL_USBD_HID_DV_WHEEL 0x38

#define SL_USBD_HID_CA_SYSTEM_CONTROL 0x80
```

Enumeration Documentation

sl_usbd_hid_country_code_t

```
sl_usbd_hid_country_code_t
```

DEFINES.

HUMAN INTERFACE DEVICE COUNTRY CODES DEFINES

Note(s) : (1) See 'Device Class Definition for Human Interface Devices (HID), 6/27/01, Version 1.11', section 6.2.1 for more details about HID descriptor country code.

(a) The country code identifies which country the hardware is localized for. Most hardware is not localized and thus this value would be zero (0). However, keyboards may use the field to indicate the language of the key caps.

Enumerator

SL_USBD_HID_COUNTRY_CODE_NOT_SUPPORTED	
SL_USBD_HID_COUNTRY_CODE_ARABIC	
SL_USBD_HID_COUNTRY_CODE_BELGIAN	
SL_USBD_HID_COUNTRY_CODE_CANADIAN_BILINGUAL	
SL_USBD_HID_COUNTRY_CODE_CANADIAN_FRENCH	
SL_USBD_HID_COUNTRY_CODE_CZECH_REPUBLIC	
SL_USBD_HID_COUNTRY_CODE_DANISH	
SL_USBD_HID_COUNTRY_CODE_FINNISH	
SL_USBD_HID_COUNTRY_CODE_FRENCH	
SL_USBD_HID_COUNTRY_CODE_GERMAN	
SL_USBD_HID_COUNTRY_CODE_GREEK	
SL_USBD_HID_COUNTRY_CODE_HEBREW	
SL_USBD_HID_COUNTRY_CODE_HUNGARY	
SL_USBD_HID_COUNTRY_CODE_INTERNATIONAL	
SL_USBD_HID_COUNTRY_CODE_ITALIAN	
SL_USBD_HID_COUNTRY_CODE_JAPAN_KATAKANA	
SL_USBD_HID_COUNTRY_CODE_KOREAN	
SL_USBD_HID_COUNTRY_CODE_LATIN_AMERICAN	
SL_USBD_HID_COUNTRY_CODE_NETHERLANDS_DUTCH	
SL_USBD_HID_COUNTRY_CODE_NORWEGIAN	
SL_USBD_HID_COUNTRY_CODE_PERSIAN_FARSI	
SL_USBD_HID_COUNTRY_CODE_POLAND	
SL_USBD_HID_COUNTRY_CODE_PORTUGUESE	
SL_USBD_HID_COUNTRY_CODE_RUSSIA	
SL_USBD_HID_COUNTRY_CODE_SLOVAKIA	
SL_USBD_HID_COUNTRY_CODE_SPANISH	
SL_USBD_HID_COUNTRY_CODE_SWEDISH	
SL_USBD_HID_COUNTRY_CODE_SWISS_FRENCH	
SL_USBD_HID_COUNTRY_CODE_SWISS_GERMAN	
SL_USBD_HID_COUNTRY_CODE_SWITZERLAND	
SL_USBD_HID_COUNTRY_CODE_TAIWAN	
SL_USBD_HID_COUNTRY_CODE_TURKISH_Q	
SL_USBD_HID_COUNTRY_CODE_UK	
SL_USBD_HID_COUNTRY_CODE_US	
SL_USBD_HID_COUNTRY_CODE_YUGOSLAVIA	
SL_USBD_HID_COUNTRY_CODE_TURKISH_F	

Definition at line 78 of file `protocol/usb/inc/sl_usbd_class_hid.h`

Typedef Documentation

`sl_usbd_hid_async_function_t`

```
typedef void(* sl_usbd_hid_async_function_t) (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void *p_callback_arg, sl_status_t status) )(uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void *p_callback_arg, sl_status_t status)
```

GLOBAL CONSTANTS.

DATA TYPES Async comm callback

Definition at line 340 of file protocol/usb/inc/sl_usbd_class_hid.h

Function Documentation

sl_usbd_hid_init

```
sl_status_t sl_usbd_hid_init (void)
```

GLOBAL CONSTANTS.

Parameters

N/A		
-----	--	--

FUNCTION PROTOTYPES HID FUNCTIONS

Initialize HID class.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 408 of file protocol/usb/inc/sl_usbd_class_hid.h

sl_usbd_hid_create_instance

```
sl_status_t sl_usbd_hid_create_instance (uint8_t subclass, uint8_t protocol, sl_usbd_hid_country_code_t country_code, uint16_t interval_in, uint16_t interval_out, bool ctrl_rd_en, sl_usbd_hid_callbacks_t *p_hid_callbacks, uint8_t *p_class_nbr)
```

Add a new instance of the HID class.

Parameters

N/A	subclass	Subclass code.
N/A	protocol	protocol code.
N/A	country_code	Country code id.
N/A	interval_in	Polling interval for input transfers, in milliseconds. It must be a power of 2.
N/A	interval_out	Polling interval for output transfers, in milliseconds. It must be a power of 2. Used only when read operations are not through control transfers.
N/A	ctrl_rd_en	Enable read operations through the control transfers.
N/A	p_hid_callbacks	Pointer to HID descriptor and request callback structure. [Content MUST be persistent]
N/A	p_class_nbr	Pointer to variable that will receive class instance number. The variable is set to class instance number if no errors are returned, otherwise it is set to SL_USBD_CLASS_NBR_NONE.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 437 of file protocol/usb/inc/sl_usbd_class_hid.h

sl_usbd_hid_add_to_configuration

```
sl_status_t sl_usbd_hid_add_to_configuration (uint8_t class_nbr, uint8_t config_nbr)
```

Add the HID class instance to the USB device configuration (see Note #1).

Parameters

N/A	class_nbr	Class instance number.
N/A	config_nbr	Configuration index to add HID class instance to.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Called several times, it creates multiple instances and configurations. For instance, the following architecture could be created :

```
*      FS
*      |-- Configuration 0 (HID class 0)
*              (HID class 1)
*              (HID class 2)
*      |-- Interface 0
*      |-- Configuration 1 (HID class 0)
*      |-- Interface 0
*
```

- (2) Configuration Descriptor corresponding to a HID device uses the following format :

```
*      Configuration Descriptor
*      |-- Interface Descriptor (HID class)
*              |-- Endpoint Descriptor (Interrupt IN)
*              |-- Endpoint Descriptor (Interrupt OUT) - optional
*
```

Definition at line 475 of file protocol/usb/inc/sl_usbd_class_hid.h

sl_usbd_hid_is_enabled

```
sl_status_t sl_usbd_hid_is_enabled (uint8_t class_nbr, bool *p_enabled)
```

Get the HID class enable state.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_enabled	Pointer to a variable that will receive HID class enable state. The variable is set to true if HID class is enabled, and is set to false if it is NOT enabled.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 489 of file protocol/usb/inc/sl_usbd_class_hid.h

sl_usbd_hid_write_sync

```
sl_status_t sl_usbd_hid_write_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len)
```

Send data to the host through the Interrupt IN endpoint.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_buf	Pointer to the transmit buffer. If more than one input report exists, the first byte must represent the report id.
N/A	buf_len	Transmit buffer length, in octets.
N/A	timeout	Timeout in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will receive transfer length. The variable is set to number of octets received, if no errors are returned, or is set to 0 if any errors are returned.

This function is blocking.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 510 of file `protocol/usb/inc/sl_usbd_class_hid.h`

sl_usbd_hid_read_sync

```
sl_status_t sl_usbd_hid_read_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len)
```

Receive data from the host through the Interrupt OUT endpoint.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_buf	Pointer to the receive buffer.
N/A	buf_len	Receive the buffer length, in octets.
N/A	timeout	Timeout, in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will receive transfer length. The variable is set to number of octets received, if no errors are returned, or is set to 0 if any errors are returned.

This function is blocking.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 533 of file `protocol/usb/inc/sl_usbd_class_hid.h`

sl_usbd_hid_read_async

```
sl_status_t sl_usbd_hid_read_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len, sl_usbd_hid_async_function_t  
async_fnct, void *p_async_arg)
```

Receive data from the host through the Interrupt OUT endpoint.

Parameters

N/A	class_nbr	Class instance number.
-----	-----------	------------------------

N/A	p_buf	Pointer to receive buffer.
N/A	buf_len	Receive buffer length, in octets.
N/A	async_fnct	Receive callback.
N/A	p_async_arg	Additional argument provided by application for receive callback.

This function is non-blocking and returns immediately after transfer preparation. Upon transfer completion, the provided callback is called to notify the application.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 556 of file protocol/usb/inc/sl_usbd_class_hid.h

sl_usbd_hid_write_async

```
sl_status_t sl_usbd_hid_write_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len, sl_usbd_hid_async_function_t
async_fnct, void *p_async_arg)
```

Send data to the host through the Interrupt IN endpoint.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_buf	Pointer to transmit buffer. If more than one input report exists, the first byte must represent the report id.
N/A	buf_len	Transmit buffer length, in octets.
N/A	async_fnct	Transmit callback.
N/A	p_async_arg	Additional argument provided by application for transmit callback.

This function is non-blocking, and returns immediately after transfer preparation. Upon transfer completion, the provided callback is called to notify the application.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 580 of file protocol/usb/inc/sl_usbd_class_hid.h

Macro Definition Documentation

SLI_USBD_HID_EXTERN

```
#define SLI_USBD_HID_EXTERN
```

Value:

```
extern
```

MODULE.

INCLUDE FILES EXTERNS

Definition at line 58 of file protocol/usb/inc/sl_usbd_class_hid.h

SL_USBD_HID_SUBCLASS_NONE

```
#define SL_USBD_HID_SUBCLASS_NONE
```

Value:

```
0x00u
```

HUMAN INTERFACE DEVICE CLASS SUBCLASS CODES DEFINES.

Note(s) : (1) Human interface device class subclass codes are defined in section 4.2 of HID specification revision 1.11.

Definition at line 124 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_SUBCLASS_BOOT

```
#define SL_USBD_HID_SUBCLASS_BOOT
```

Value:

```
0x01u
```

Definition at line 125 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_PROTOCOL_NONE

```
#define SL_USBD_HID_PROTOCOL_NONE
```

Value:

```
0x00u
```

HUMAN INTERFACE DEVICE CLASS PROTOCOL CODES DEFINES.

Note(s) : (1) Human interface device class protocol codes are defined in section 4.3 of HID specification revision 1.11.

Definition at line 134 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_PROTOCOL_KBD

```
#define SL_USBD_HID_PROTOCOL_KBD
```

Value:

```
0x01u
```

Definition at line 135 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_PROTOCOL_MOUSE

```
#define SL_USBD_HID_PROTOCOL_MOUSE
```

Value:

```
0x02u
```

Definition at line 136 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_ITEM_LONG

```
#define SL_USBD_HID_ITEM_LONG
```

Value:

0xFE

ITEM TYPES.

Definition at line 143 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_ITEM_TYPE_MAIN

```
#define SL_USBD_HID_ITEM_TYPE_MAIN
```

Value:

0x00

Definition at line 145 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_ITEM_TYPE_GLOBAL

```
#define SL_USBD_HID_ITEM_TYPE_GLOBAL
```

Value:

0x04

Definition at line 146 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_ITEM_TYPE_LOCAL

```
#define SL_USBD_HID_ITEM_TYPE_LOCAL
```

Value:

0x08

Definition at line 147 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_ITEM_TYPE_RESERVED

```
#define SL_USBD_HID_ITEM_TYPE_RESERVED
```

Value:

0x0C

Definition at line 148 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_INPUT

```
#define SL_USBD_HID_MAIN_INPUT
```

Value:

```
0x80
```

Definition at line 150 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_MAIN_OUTPUT**

```
#define SL_USBD_HID_MAIN_OUTPUT
```

Value:

```
0x90
```

Definition at line 151 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_MAIN_COLLECTION**

```
#define SL_USBD_HID_MAIN_COLLECTION
```

Value:

```
0xA0
```

Definition at line 152 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_MAIN_FEATURE**

```
#define SL_USBD_HID_MAIN_FEATURE
```

Value:

```
0xB0
```

Definition at line 153 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_MAIN_ENDCOLLECTION**

```
#define SL_USBD_HID_MAIN_ENDCOLLECTION
```

Value:

```
0xC0
```

Definition at line 154 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_MAIN_CONSTANT**

```
#define SL_USBD_HID_MAIN_CONSTANT
```

Value:

```
0x01
```

Definition at line 156 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_DATA

```
#define SL_USBD_HID_MAIN_DATA
```

Value:

```
0x00
```

Definition at line 157 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_VARIABLE

```
#define SL_USBD_HID_MAIN_VARIABLE
```

Value:

```
0x02
```

Definition at line 158 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_ARRAY

```
#define SL_USBD_HID_MAIN_ARRAY
```

Value:

```
0x00
```

Definition at line 159 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_RELATIVE

```
#define SL_USBD_HID_MAIN_RELATIVE
```

Value:

```
0x04
```

Definition at line 160 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_ABSOLUTE

```
#define SL_USBD_HID_MAIN_ABSOLUTE
```

Value:

```
0x00
```

Definition at line 161 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_WRAP

```
#define SL_USBD_HID_MAIN_WRAP
```

Value:

```
0x08
```

Definition at line 162 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_NOWRAP

```
#define SL_USBD_HID_MAIN_NOWRAP
```

Value:

```
0x00
```

Definition at line 163 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_NONLINEAR

```
#define SL_USBD_HID_MAIN_NONLINEAR
```

Value:

```
0x10
```

Definition at line 164 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_LINEAR

```
#define SL_USBD_HID_MAIN_LINEAR
```

Value:

```
0x00
```

Definition at line 165 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_NOPREFERRED

```
#define SL_USBD_HID_MAIN_NOPREFERRED
```

Value:

```
0x20
```

Definition at line 166 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_PREFERREDSTATE

```
#define SL_USBD_HID_MAIN_PREFERREDSTATE
```

Value:

```
0x00
```

Definition at line 167 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_MAIN_NULLSTATE**

```
#define SL_USBD_HID_MAIN_NULLSTATE
```

Value:

```
0x40
```

Definition at line 168 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_MAIN_NONULLPOSITION**

```
#define SL_USBD_HID_MAIN_NONULLPOSITION
```

Value:

```
0x00
```

Definition at line 169 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_MAIN_VOLATILE**

```
#define SL_USBD_HID_MAIN_VOLATILE
```

Value:

```
0x80
```

Definition at line 170 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_MAIN_NONVOLATILE**

```
#define SL_USBD_HID_MAIN_NONVOLATILE
```

Value:

```
0x00
```

Definition at line 171 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_MAIN_BUFFEREDBYTES**

```
#define SL_USBD_HID_MAIN_BUFFEREDBYTES
```

Value:

```
0x0100
```

Definition at line 172 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_MAIN_BITFIELD

```
#define SL_USBD_HID_MAIN_BITFIELD
```

Value:

```
0x0000
```

Definition at line 173 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_COLLECTION_PHYSICAL

```
#define SL_USBD_HID_COLLECTION_PHYSICAL
```

Value:

```
0x00
```

Definition at line 175 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_COLLECTION_APPLICATION

```
#define SL_USBD_HID_COLLECTION_APPLICATION
```

Value:

```
0x01
```

Definition at line 176 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_COLLECTION_LOGICAL

```
#define SL_USBD_HID_COLLECTION_LOGICAL
```

Value:

```
0x02
```

Definition at line 177 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_COLLECTION_REPORT

```
#define SL_USBD_HID_COLLECTION_REPORT
```

Value:

```
0x03
```

Definition at line 178 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_COLLECTION_NAMEDARRAY

```
#define SL_USBD_HID_COLLECTION_NAMEDARRAY
```

Value:

```
0x04
```

Definition at line 179 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_COLLECTION_USAGESWITCH

```
#define SL_USBD_HID_COLLECTION_USAGESWITCH
```

Value:

```
0x05
```

Definition at line 180 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_COLLECTION_USAGEMODIFIER

```
#define SL_USBD_HID_COLLECTION_USAGEMODIFIER
```

Value:

```
0x06
```

Definition at line 181 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_GLOBAL_USAGE_PAGE

```
#define SL_USBD_HID_GLOBAL_USAGE_PAGE
```

Value:

```
0x04
```

Definition at line 183 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_GLOBAL_LOG_MIN

```
#define SL_USBD_HID_GLOBAL_LOG_MIN
```

Value:

```
0x14
```

Definition at line 184 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_GLOBAL_LOG_MAX

```
#define SL_USBD_HID_GLOBAL_LOG_MAX
```

Value:

```
0x24
```

Definition at line 185 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_GLOBAL_PHY_MIN**

```
#define SL_USBD_HID_GLOBAL_PHY_MIN
```

Value:

```
0x34
```

Definition at line 186 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_GLOBAL_PHY_MAX**

```
#define SL_USBD_HID_GLOBAL_PHY_MAX
```

Value:

```
0x44
```

Definition at line 187 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_GLOBAL_UNIT_EXPONENT**

```
#define SL_USBD_HID_GLOBAL_UNIT_EXPONENT
```

Value:

```
0x54
```

Definition at line 188 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_GLOBAL_UNIT**

```
#define SL_USBD_HID_GLOBAL_UNIT
```

Value:

```
0x64
```

Definition at line 189 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_GLOBAL_REPORT_SIZE**

```
#define SL_USBD_HID_GLOBAL_REPORT_SIZE
```

Value:

```
0x74
```

Definition at line 190 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_GLOBAL_REPORT_ID

```
#define SL_USBD_HID_GLOBAL_REPORT_ID
```

Value:

```
0x84
```

Definition at line 191 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_GLOBAL_REPORT_COUNT

```
#define SL_USBD_HID_GLOBAL_REPORT_COUNT
```

Value:

```
0x94
```

Definition at line 192 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_GLOBAL_PUSH

```
#define SL_USBD_HID_GLOBAL_PUSH
```

Value:

```
0xA4
```

Definition at line 193 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_GLOBAL_POP

```
#define SL_USBD_HID_GLOBAL_POP
```

Value:

```
0xB4
```

Definition at line 194 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_LOCAL_USAGE

```
#define SL_USBD_HID_LOCAL_USAGE
```

Value:


```
0x08
```

Definition at line 196 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_LOCAL_USAGE_MIN

```
#define SL_USBD_HID_LOCAL_USAGE_MIN
```

Value:

```
0x18
```

Definition at line 197 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_LOCAL_USAGE_MAX

```
#define SL_USBD_HID_LOCAL_USAGE_MAX
```

Value:

```
0x28
```

Definition at line 198 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_LOCAL_DESIGNATOR_INDEX

```
#define SL_USBD_HID_LOCAL_DESIGNATOR_INDEX
```

Value:

```
0x38
```

Definition at line 199 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_LOCAL_DESIGNATOR_MIN

```
#define SL_USBD_HID_LOCAL_DESIGNATOR_MIN
```

Value:

```
0x48
```

Definition at line 200 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_LOCAL_DESIGNATOR_MAX

```
#define SL_USBD_HID_LOCAL_DESIGNATOR_MAX
```

Value:

```
0x58
```

Definition at line 201 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_LOCAL_STRING_INDEX

```
#define SL_USBD_HID_LOCAL_STRING_INDEX
```

Value:

```
0x78
```

Definition at line 202 of file protocol/usb/inc/slUsbd_class_hid.h

SL_USBD_HID_LOCAL_STRING_MIN

```
#define SL_USBD_HID_LOCAL_STRING_MIN
```

Value:

```
0x88
```

Definition at line 203 of file protocol/usb/inc/slUsbd_class_hid.h

SL_USBD_HID_LOCAL_STRING_MAX

```
#define SL_USBD_HID_LOCAL_STRING_MAX
```

Value:

```
0x98
```

Definition at line 204 of file protocol/usb/inc/slUsbd_class_hid.h

SL_USBD_HID_LOCAL_DELIMITER

```
#define SL_USBD_HID_LOCAL_DELIMITER
```

Value:

```
0xA8
```

Definition at line 205 of file protocol/usb/inc/slUsbd_class_hid.h

SL_USBD_HID_BIAS_NOT_APPLICABLE

```
#define SL_USBD_HID_BIAS_NOT_APPLICABLE
```

Value:

```
0
```

HID PHYSICAL DESCRIPTOR DEFINES

Note(s) : (1) See 'Device Class Definition for Human Interface Devices Version 1.11', Section 6.2.3.

Definition at line 215 of file protocol/usb/inc/slUsbd_class_hid.h

SL_USBD_HID_BIAS_RIGHT_HAND

```
#define SL_USBD_HID_BIAS_RIGHT_HAND
```

Value:

1

Definition at line 216 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_BIAS_LEFT_HAND**

```
#define SL_USBD_HID_BIAS_LEFT_HAND
```

Value:

2

Definition at line 217 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_BIAS_BOTH_HANDS**

```
#define SL_USBD_HID_BIAS_BOTH_HANDS
```

Value:

3

Definition at line 218 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_BIAS_EITHER_HAND**

```
#define SL_USBD_HID_BIAS_EITHER_HAND
```

Value:

4

Definition at line 219 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_NONE**

```
#define SL_USBD_HID_DESIGNATOR_NONE
```

Value:

0x00

Definition at line 221 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_HAND**

```
#define SL_USBD_HID_DESIGNATOR_HAND
```

Value:

```
0x01
```

Definition at line 222 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_EYEBALL

```
#define SL_USBD_HID_DESIGNATOR_EYEBALL
```

Value:

```
0x02
```

Definition at line 223 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_EYEBROW

```
#define SL_USBD_HID_DESIGNATOR_EYEBROW
```

Value:

```
0x03
```

Definition at line 224 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_EYELID

```
#define SL_USBD_HID_DESIGNATOR_EYELID
```

Value:

```
0x04
```

Definition at line 225 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_EAR

```
#define SL_USBD_HID_DESIGNATOR_EAR
```

Value:

```
0x05
```

Definition at line 226 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_NOSE

```
#define SL_USBD_HID_DESIGNATOR_NOSE
```

Value:

```
0x06
```

Definition at line 227 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_MOUTH

```
#define SL_USBD_HID_DESIGNATOR_MOUTH
```

Value:

```
0x07
```

Definition at line 228 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_UPPER_LIP

```
#define SL_USBD_HID_DESIGNATOR_UPPER_LIP
```

Value:

```
0x08
```

Definition at line 229 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_LOWER_LIP

```
#define SL_USBD_HID_DESIGNATOR_LOWER_LIP
```

Value:

```
0x09
```

Definition at line 230 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_JAW

```
#define SL_USBD_HID_DESIGNATOR_JAW
```

Value:

```
0x0A
```

Definition at line 231 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_NECK

```
#define SL_USBD_HID_DESIGNATOR_NECK
```

Value:

```
0x0B
```

Definition at line 232 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_UPPER_ARM

```
#define SL_USBD_HID_DESIGNATOR_UPPER_ARM
```

Value:

```
0x0C
```

Definition at line 233 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_ELBOW**

```
#define SL_USBD_HID_DESIGNATOR_ELBOW
```

Value:

```
0x0D
```

Definition at line 234 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_FOREARM**

```
#define SL_USBD_HID_DESIGNATOR_FOREARM
```

Value:

```
0x0E
```

Definition at line 235 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_WRIST**

```
#define SL_USBD_HID_DESIGNATOR_WRIST
```

Value:

```
0x0F
```

Definition at line 236 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_PALM**

```
#define SL_USBD_HID_DESIGNATOR_PALM
```

Value:

```
0x10
```

Definition at line 237 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_THUMB**

```
#define SL_USBD_HID_DESIGNATOR_THUMB
```

Value:

```
0x11
```

Definition at line 238 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_INDEX_FINGER

```
#define SL_USBD_HID_DESIGNATOR_INDEX_FINGER
```

Value:

```
0x12
```

Definition at line 239 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_MIDDLE_FINGER

```
#define SL_USBD_HID_DESIGNATOR_MIDDLE_FINGER
```

Value:

```
0x13
```

Definition at line 240 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_RING_FINGER

```
#define SL_USBD_HID_DESIGNATOR_RING_FINGER
```

Value:

```
0x14
```

Definition at line 241 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_LITTLE_FINGER

```
#define SL_USBD_HID_DESIGNATOR_LITTLE_FINGER
```

Value:

```
0x15
```

Definition at line 242 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_HEAD

```
#define SL_USBD_HID_DESIGNATOR_HEAD
```

Value:

```
0x16
```

Definition at line 243 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_SHOULDER

```
#define SL_USBD_HID_DESIGNATOR_SHOULDER
```

Value:

```
0x17
```

Definition at line 244 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_HIP

```
#define SL_USBD_HID_DESIGNATOR_HIP
```

Value:

```
0x18
```

Definition at line 245 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_WAIST

```
#define SL_USBD_HID_DESIGNATOR_WAIST
```

Value:

```
0x19
```

Definition at line 246 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_THIGH

```
#define SL_USBD_HID_DESIGNATOR_THIGH
```

Value:

```
0x1A
```

Definition at line 247 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_KNEE

```
#define SL_USBD_HID_DESIGNATOR_KNEE
```

Value:

```
0x1B
```

Definition at line 248 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_CALF

```
#define SL_USBD_HID_DESIGNATOR_CALF
```

Value:

```
0x1C
```

Definition at line 249 of file `protocol/usb/inc/slUsbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_ANKLE**

```
#define SL_USBD_HID_DESIGNATOR_ANKLE
```

Value:

```
0x1D
```

Definition at line 250 of file `protocol/usb/inc/slUsbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_FOOT**

```
#define SL_USBD_HID_DESIGNATOR_FOOT
```

Value:

```
0x1E
```

Definition at line 251 of file `protocol/usb/inc/slUsbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_HEEL**

```
#define SL_USBD_HID_DESIGNATOR_HEEL
```

Value:

```
0x1F
```

Definition at line 252 of file `protocol/usb/inc/slUsbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_BALL_OF_FOOT**

```
#define SL_USBD_HID_DESIGNATOR_BALL_OF_FOOT
```

Value:

```
0x20
```

Definition at line 253 of file `protocol/usb/inc/slUsbd_class_hid.h`**SL_USBD_HID_DESIGNATOR_BIG_TOE**

```
#define SL_USBD_HID_DESIGNATOR_BIG_TOE
```

Value:

```
0x21
```

Definition at line 254 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_SECOND_TOE

```
#define SL_USBD_HID_DESIGNATOR_SECOND_TOE
```

Value:

```
0x22
```

Definition at line 255 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_THIRD_TOE

```
#define SL_USBD_HID_DESIGNATOR_THIRD_TOE
```

Value:

```
0x23
```

Definition at line 256 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_FOURTH_TOE

```
#define SL_USBD_HID_DESIGNATOR_FOURTH_TOE
```

Value:

```
0x24
```

Definition at line 257 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_LITTLE_TOE

```
#define SL_USBD_HID_DESIGNATOR_LITTLE_TOE
```

Value:

```
0x25
```

Definition at line 258 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_BROW

```
#define SL_USBD_HID_DESIGNATOR_BROW
```

Value:

```
0x26
```

Definition at line 259 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DESIGNATOR_CHEEK

```
#define SL_USBD_HID_DESIGNATOR_CHEEK
```

Value:

```
0x27
```

Definition at line 260 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_QUALIFIER_NOT_APPLICABLE

```
#define SL_USBD_HID_QUALIFIER_NOT_APPLICABLE
```

Value:

```
0
```

Definition at line 262 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_QUALIFIER_RIGHT

```
#define SL_USBD_HID_QUALIFIER_RIGHT
```

Value:

```
1
```

Definition at line 263 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_QUALIFIER_LEFT

```
#define SL_USBD_HID_QUALIFIER_LEFT
```

Value:

```
2
```

Definition at line 264 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_QUALIFIER_BOTH

```
#define SL_USBD_HID_QUALIFIER_BOTH
```

Value:

```
3
```

Definition at line 265 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_QUALIFIER_EITHER

```
#define SL_USBD_HID_QUALIFIER_EITHER
```

Value:

4

Definition at line 266 of file protocol/usb/inc/sl_usbd_class_hid.h

SL_USBD_HID_QUALIFIER_CENTER

```
#define SL_USBD_HID_QUALIFIER_CENTER
```

Value:

5

Definition at line 267 of file protocol/usb/inc/sl_usbd_class_hid.h

SL_USBD_HID_USAGE_PAGE_UNDEFINED

```
#define SL_USBD_HID_USAGE_PAGE_UNDEFINED
```

Value:

0x00

HID USAGE PAGES

Note(s) : (1) See 'Universal Serial Bus HID Usage Tables', Version 1.12, Section 3 for more details about Usage Pages.

(2) See 'Universal Serial Bus HID Usage Tables', Version 1.12, Section 4 for more details about Generic Desktop Page usages and controls.

Definition at line 279 of file protocol/usb/inc/sl_usbd_class_hid.h

SL_USBD_HID_USAGE_PAGE_GENERIC_DESKTOP_CONTROLS

```
#define SL_USBD_HID_USAGE_PAGE_GENERIC_DESKTOP_CONTROLS
```

Value:

0x01

Definition at line 280 of file protocol/usb/inc/sl_usbd_class_hid.h

SL_USBD_HID_USAGE_PAGE_SIMULATION_CONTROLS

```
#define SL_USBD_HID_USAGE_PAGE_SIMULATION_CONTROLS
```

Value:

0x02

Definition at line 281 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_VR_CONTROLS

```
#define SL_USBD_HID_USAGE_PAGE_VR_CONTROLS
```

Value:

```
0x03
```

Definition at line 282 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_SPORT_CONTROLS

```
#define SL_USBD_HID_USAGE_PAGE_SPORT_CONTROLS
```

Value:

```
0x04
```

Definition at line 283 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_GAME_CONTROLS

```
#define SL_USBD_HID_USAGE_PAGE_GAME_CONTROLS
```

Value:

```
0x05
```

Definition at line 284 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_GENERIC_DEVICE_CONTROLS

```
#define SL_USBD_HID_USAGE_PAGE_GENERIC_DEVICE_CONTROLS
```

Value:

```
0x06
```

Definition at line 285 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_KEYBOARD

```
#define SL_USBD_HID_USAGE_PAGE_KEYBOARD
```

Value:

```
0x07
```

Definition at line 286 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_LEDS

```
#define SL_USBD_HID_USAGE_PAGE_LEDS
```

Value:

```
0x08
```

Definition at line 287 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_BUTTON

```
#define SL_USBD_HID_USAGE_PAGE_BUTTON
```

Value:

```
0x09
```

Definition at line 288 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_ORDINAL

```
#define SL_USBD_HID_USAGE_PAGE_ORDINAL
```

Value:

```
0x0A
```

Definition at line 289 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_TELEPHONY

```
#define SL_USBD_HID_USAGE_PAGE_TELEPHONY
```

Value:

```
0x0B
```

Definition at line 290 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_CONSUMER

```
#define SL_USBD_HID_USAGE_PAGE_CONSUMER
```

Value:

```
0x0C
```

Definition at line 291 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_DIGITIZER

```
#define SL_USBD_HID_USAGE_PAGE_DIGITIZER
```

Value:

```
0x0D
```

Definition at line 292 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_PID_PAGE

```
#define SL_USBD_HID_USAGE_PAGE_PID_PAGE
```

Value:

```
0x0F
```

Definition at line 293 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_UNICODE

```
#define SL_USBD_HID_USAGE_PAGE_UNICODE
```

Value:

```
0x10
```

Definition at line 294 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_ALPHANUMERIC_DISPLAY

```
#define SL_USBD_HID_USAGE_PAGE_ALPHANUMERIC_DISPLAY
```

Value:

```
0x14
```

Definition at line 295 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_MEDICAL_INSTRUMENTS

```
#define SL_USBD_HID_USAGE_PAGE_MEDICAL_INSTRUMENTS
```

Value:

```
0x40
```

Definition at line 296 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_MONITOR_0

```
#define SL_USBD_HID_USAGE_PAGE_MONITOR_0
```

Value:

```
0x80
```

Definition at line 297 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_MONITOR_1

```
#define SL_USBD_HID_USAGE_PAGE_MONITOR_1
```

Value:

```
0x81
```

Definition at line 298 of file protocol/usb/inc/sl_usbd_class_hid.h

SL_USBD_HID_USAGE_PAGE_MONITOR_2

```
#define SL_USBD_HID_USAGE_PAGE_MONITOR_2
```

Value:

```
0x82
```

Definition at line 299 of file protocol/usb/inc/sl_usbd_class_hid.h

SL_USBD_HID_USAGE_PAGE_MONITOR_3

```
#define SL_USBD_HID_USAGE_PAGE_MONITOR_3
```

Value:

```
0x83
```

Definition at line 300 of file protocol/usb/inc/sl_usbd_class_hid.h

SL_USBD_HID_USAGE_PAGE_POWER_0

```
#define SL_USBD_HID_USAGE_PAGE_POWER_0
```

Value:

```
0x84
```

Definition at line 301 of file protocol/usb/inc/sl_usbd_class_hid.h

SL_USBD_HID_USAGE_PAGE_POWER_1

```
#define SL_USBD_HID_USAGE_PAGE_POWER_1
```

Value:

```
0x85
```

Definition at line 302 of file protocol/usb/inc/sl_usbd_class_hid.h

SL_USBD_HID_USAGE_PAGE_POWER_2


```
#define SL_USBD_HID_USAGE_PAGE_POWER_2
```

Value:

```
0x86
```

Definition at line 303 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_POWER_3

```
#define SL_USBD_HID_USAGE_PAGE_POWER_3
```

Value:

```
0x87
```

Definition at line 304 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_BAR_CODE_SCANNER_PAGE

```
#define SL_USBD_HID_USAGE_PAGE_BAR_CODE_SCANNER_PAGE
```

Value:

```
0x8C
```

Definition at line 305 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_SCALE_PAGE

```
#define SL_USBD_HID_USAGE_PAGE_SCALE_PAGE
```

Value:

```
0x8D
```

Definition at line 306 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_MSR_DEVICES

```
#define SL_USBD_HID_USAGE_PAGE_MSR_DEVICES
```

Value:

```
0x8E
```

Definition at line 307 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_POINT_OF_SALE_PAGES

```
#define SL_USBD_HID_USAGE_PAGE_POINT_OF_SALE_PAGES
```

Value:

```
0x8F
```

Definition at line 308 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_CAMERA_CONTROL_PAGE

```
#define SL_USBD_HID_USAGE_PAGE_CAMERA_CONTROL_PAGE
```

Value:

```
0x90
```

Definition at line 309 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_USAGE_PAGE_ARCADE_PAGE

```
#define SL_USBD_HID_USAGE_PAGE_ARCADE_PAGE
```

Value:

```
0x91
```

Definition at line 310 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_CP_POINTER

```
#define SL_USBD_HID_CP_POINTER
```

Value:

```
0x01
```

Definition at line 314 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_CA_MOUSE

```
#define SL_USBD_HID_CA_MOUSE
```

Value:

```
0x02
```

Definition at line 315 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_CA_JOYSTICK

```
#define SL_USBD_HID_CA_JOYSTICK
```

Value:

```
0x04
```

Definition at line 316 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_CA_GAME_PAD

```
#define SL_USBD_HID_CA_GAME_PAD
```

Value:

```
0x05
```

Definition at line 317 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_CA_KEYBOARD**

```
#define SL_USBD_HID_CA_KEYBOARD
```

Value:

```
0x06
```

Definition at line 318 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_CA_KEYPAD**

```
#define SL_USBD_HID_CA_KEYPAD
```

Value:

```
0x07
```

Definition at line 319 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_CA_MULTI_AXIS_CONTROLLER**

```
#define SL_USBD_HID_CA_MULTI_AXIS_CONTROLLER
```

Value:

```
0x08
```

Definition at line 320 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_DV_X**

```
#define SL_USBD_HID_DV_X
```

Value:

```
0x30
```

Definition at line 321 of file `protocol/usb/inc/sl_usbd_class_hid.h`**SL_USBD_HID_DV_Y**

```
#define SL_USBD_HID_DV_Y
```

Value:

```
0x31
```

Definition at line 322 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DV_Z

```
#define SL_USBD_HID_DV_Z
```

Value:

```
0x32
```

Definition at line 323 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_DV_WHEEL

```
#define SL_USBD_HID_DV_WHEEL
```

Value:

```
0x38
```

Definition at line 324 of file `protocol/usb/inc/sl_usbd_class_hid.h`

SL_USBD_HID_CA_SYSTEM_CONTROL

```
#define SL_USBD_HID_CA_SYSTEM_CONTROL
```

Value:

```
0x80
```

Definition at line 325 of file `protocol/usb/inc/sl_usbd_class_hid.h`

sl_usbd_hid_callbacks_t

HID callbacks.

Public Attributes

void(*	enable	Callback for enable event.
void(*	disable	Callback for disable event.
void(*	get_report_desc	Callback for get report descriptor event.
void(*	get_phy_desc	Callback for get physical descriptor event.
void(*	set_output_report	Callback for set output report event.
void(*	get_feature_report	Callback for get feature report event.
void(*	set_feature_report	Callback for set feature report event.
void(*	get_protocol	Callback for get protocol event.
void(*	set_protocol	Callback for set protocol event.

Public Attribute Documentation

enable

```
void(* sl_usbd_hid_callbacks_t::enable) (uint8_t class_nbr)
```

Callback for enable event.

Definition at line 349 of file protocol/usb/inc/sl_usbd_class_hid.h

disable

```
void(* sl_usbd_hid_callbacks_t::disable) (uint8_t class_nbr)
```

Callback for disable event.

Definition at line 351 of file protocol/usb/inc/sl_usbd_class_hid.h

get_report_desc

```
void(* sl_usbd_hid_callbacks_t::get_report_desc) (uint8_t class_nbr, const uint8_t **p_report_ptr, uint16_t *p_report_len)
```

Callback for get report descriptor event.

Definition at line 353 of file `protocol/usb/inc/sl_usbd_class_hid.h`

get_phy_desc

```
void(* sl_usbd_hid_callbacks_t::get_phy_desc) (uint8_t class_nbr, const uint8_t **p_report_ptr, uint16_t *p_report_len)
```

Callback for get physical descriptor event.

Definition at line 357 of file `protocol/usb/inc/sl_usbd_class_hid.h`

set_output_report

```
void(* sl_usbd_hid_callbacks_t::set_output_report) (uint8_t class_nbr, uint8_t report_id, uint8_t *p_report_buf, uint16_t report_len)
```

Callback for set output report event.

Definition at line 361 of file `protocol/usb/inc/sl_usbd_class_hid.h`

get_feature_report

```
void(* sl_usbd_hid_callbacks_t::get_feature_report) (uint8_t class_nbr, uint8_t report_id, uint8_t *p_report_buf, uint16_t report_len)
```

Callback for get feature report event.

Definition at line 366 of file `protocol/usb/inc/sl_usbd_class_hid.h`

set_feature_report

```
void(* sl_usbd_hid_callbacks_t::set_feature_report) (uint8_t class_nbr, uint8_t report_id, uint8_t *p_report_buf, uint16_t report_len)
```

Callback for set feature report event.

Definition at line 371 of file `protocol/usb/inc/sl_usbd_class_hid.h`

get_protocol

```
void(* sl_usbd_hid_callbacks_t::get_protocol) (uint8_t class_nbr, uint8_t *p_protocol)
```

Callback for get protocol event.

Definition at line 376 of file protocol/usb/inc/sl_usbd_class_hid.h

set_protocol

```
void(* sl_usbd_hid_callbacks_t::set_protocol) (uint8_t class_nbr, uint8_t protocol)
```

Callback for set protocol event.

Definition at line 379 of file protocol/usb/inc/sl_usbd_class_hid.h

USB Device MSC API

USB Device MSC API

USB Device MSC API.

Modules

[sl_usbd_msc_subclass_driver_t](#)

Functions

sl_status_t	sl_usbd_msc_init (void) FUNCTION PROTOTYPES.
sl_status_t	sl_usbd_msc_create_instance (uint8_t subclass, uint32_t msc_task_stack_size, osPriority_t msc_task_priority, sl_usbd_msc_subclass_driver_t *p_subclass_drv, void *p_subclass_arg, uint8_t *p_class_nbr) Add a new instance of the Mass Storage Class.
sl_status_t	sl_usbd_msc_add_to_configuration (uint8_t class_nbr, uint8_t config_nbr) Add an existing MSC instance to the specified configuration.
sl_status_t	sl_usbd_msc_lun_add (uint8_t class_nbr, void *p_lu_data, uint8_t *p_lu_nbr) Add an MSC logical unit to the MSC interface.
sl_status_t	sl_usbd_msc_is_enabled (uint8_t class_nbr, bool *p_enabled) Get the MSC enable state of the device.

Macros

#define	SL_USBD_MSC_LU_NBR_INVALID 255u MODULE.
#define	SL_USBD_MSC_SUBCLASS_CODE_RBC 0x01 SUBCLASS CODES.
#define	SL_USBD_MSC_SUBCLASS_CODE_SFF_8020i 0x02
#define	SL_USBD_MSC_SUBCLASS_CODE_MMC_2 0x02
#define	SL_USBD_MSC_SUBCLASS_CODE_QIC_157 0x03
#define	SL_USBD_MSC_SUBCLASS_CODE_UFI 0x04
#define	SL_USBD_MSC_SUBCLASS_CODE_SFF_8070i 0x05
#define	SL_USBD_MSC_SUBCLASS_CODE_SCSI 0x06
#define	SL_USBD_MSC_PROTOCOL_CODE_CTRL_BULK_INTR_CMD_INTR 0x00 PROTOCOL CODES.
#define	SL_USBD_MSC_PROTOCOL_CODE_CTRL_BULK_INTR 0x01


```
#define SL_USBD_MSC_PROTOCOL_CODE_BULK_ONLY 0x50
```

Function Documentation

sl_usbd_msc_init

```
sl_status_t sl_usbd_msc_init (void)
```

FUNCTION PROTOTYPES.

Parameters

N/A		
-----	--	--

CONFIGURATION OVERRIDE FUNCTIONS MSC FUNCTIONS

Initialize internal structures and variables used by the Mass Storage Class Bulk Only Transport.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 150 of file protocol/usb/inc/sl_usbd_class_msc.h

sl_usbd_msc_create_instance

```
sl_status_t sl_usbd_msc_create_instance (uint8_t subclass, uint32_t msc_task_stack_size, osPriority_t msc_task_priority, sl_usbd_msc_subclass_driver_t *p_subclass_drv, void *p_subclass_arg, uint8_t *p_class_nbr)
```

Add a new instance of the Mass Storage Class.

Parameters

N/A	subclass	MSC subclass code.
N/A	msc_task_stack_size	Pointer to data structure of functions that implement the subclass driver.
N/A	msc_task_priority	The USB stack creates a task to handle the events of each MSC instance. This parameter specifies the stack size to be used for the newly created task for this instance.
N/A	p_subclass_drv	The USB stack creates a task to handle the events of each MSC instance. This parameter specifies the CMSIS priority of that task.
N/A	p_subclass_arg	An argument that will be passed to the driver when driver functions are called.
N/A	p_class_nbr	Pointer to a variable that will receive the instance number for the created class instance, if no errors are returned. Otherwise, the variable is set to SL_USBD_CLASS_NBR_NONE.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 174 of file protocol/usb/inc/sl_usbd_class_msc.h

sl_usbd_msc_add_to_configuration

```
sl_status_t sl_usbd_msc_add_to_configuration (uint8_t class_nbr, uint8_t config_nbr)
```

Add an existing MSC instance to the specified configuration.

Parameters

N/A	class_nbr	MSC instance number.
N/A	config_nbr	Configuration index to which to add the existing MSC interface.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) `sl_usbd_msc_add_to_configuration()` adds an Interface descriptor and its associated Endpoint descriptor(s) to the Configuration descriptor. One call to `sl_usbd_msc_add_to_configuration()` builds the Configuration descriptor corresponding to an MSC device with the following format:

```
*      Configuration Descriptor
*      |-- Interface Descriptor (MSC)
*      |-- Endpoint Descriptor (Bulk OUT)
*      |-- Endpoint Descriptor (Bulk IN)
*
```

If `sl_usbd_msc_add_to_configuration()` is called several times from the application, it creates multiple instances and configurations. For example, the following architecture could be created for a high-speed device:

```
*      High-speed
*      |-- Configuration 0
*      |-- Interface 0 (MSC 0)
*      |-- Configuration 1
*      |-- Interface 0 (MSC 0)
*      |-- Interface 1 (MSC 1)
*
```

In this example, there are two instances of MSC: 'MSC 0' and 'MSC 1', and two possible configurations for the device: 'Configuration 0' and 'Configuration 1'. 'Configuration 1' is composed of two interfaces. Each class instance has an association with one of the interfaces. If 'Configuration 1' is activated by the host, it allows the host to access two different functionalities offered by the device.

Definition at line 218 of file `protocol/usb/inc/slUsbd_class_msc.h`

sl_usbd_msc_lun_add

```
sl_status_t sl_usbd_msc_lun_add (uint8_t class_nbr, void *p_lu_data, uint8_t *p_lu_nbr)
```

Add an MSC logical unit to the MSC interface.

Parameters

N/A	class_nbr	MSC instance number.
N/A	p_lu_data	Pointer to the logical unit data to store in LU data structure.
N/A	p_lu_nbr	Pointer to a variable that will receive Logical Unit Number.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 232 of file `protocol/usb/inc/slUsbd_class_msc.h`

sl_usbd_msc_is_enabled

```
sl_status_t sl_usbd_msc_is_enabled (uint8_t class_nbr, bool *p_enabled)
```

Get the MSC enable state of the device.

Parameters

N/A	class_nbr	MSC instance number.
N/A	p_enabled	Pointer to a boolean variable that will receive enable state. The variable is set to true if MSC class is enabled. The variable is set to false if the MSC class is not enabled.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 247 of file protocol/usb/inc/sl_usbd_class_msc.h

Macro Definition Documentation

SL_USBD_MSC_LU_NBR_INVALID

```
#define SL_USBD_MSC_LU_NBR_INVALID
```

Value:

```
255u
```

MODULE.

INCLUDE FILES MACROS

Definition at line 63 of file protocol/usb/inc/sl_usbd_class_msc.h

SL_USBD_MSC_SUBCLASS_CODE_RBC

```
#define SL_USBD_MSC_SUBCLASS_CODE_RBC
```

Value:

```
0x01
```

SUBCLASS CODES.

Note(s) : (1) See 'USB Mass Storage Class Specification Overview', Revision 1.2, Section 2

Definition at line 71 of file protocol/usb/inc/sl_usbd_class_msc.h

SL_USBD_MSC_SUBCLASS_CODE_SFF_8020i

```
#define SL_USBD_MSC_SUBCLASS_CODE_SFF_8020i
```

Value:

```
0x02
```

Definition at line 72 of file protocol/usb/inc/sl_usbd_class_msc.h

SL_USBD_MSC_SUBCLASS_CODE_MMC_2

```
#define SL_USBD_MSC_SUBCLASS_CODE_MMC_2
```

Value:

```
0x02
```

Definition at line 73 of file `protocol/usb/inc/sl_usbd_class_msc.h`

SL_USBD_MSC_SUBCLASS_CODE_QIC_157

```
#define SL_USBD_MSC_SUBCLASS_CODE_QIC_157
```

Value:

```
0x03
```

Definition at line 74 of file `protocol/usb/inc/sl_usbd_class_msc.h`

SL_USBD_MSC_SUBCLASS_CODE_UFI

```
#define SL_USBD_MSC_SUBCLASS_CODE_UFI
```

Value:

```
0x04
```

Definition at line 75 of file `protocol/usb/inc/sl_usbd_class_msc.h`

SL_USBD_MSC_SUBCLASS_CODE_SFF_8070i

```
#define SL_USBD_MSC_SUBCLASS_CODE_SFF_8070i
```

Value:

```
0x05
```

Definition at line 76 of file `protocol/usb/inc/sl_usbd_class_msc.h`

SL_USBD_MSC_SUBCLASS_CODE_SCSI

```
#define SL_USBD_MSC_SUBCLASS_CODE_SCSI
```

Value:

```
0x06
```

Definition at line 77 of file `protocol/usb/inc/sl_usbd_class_msc.h`

SL_USBD_MSC_PROTOCOL_CODE_CTRL_BULK_INTR_CMD_INTR

```
#define SL_USBD_MSC_PROTOCOL_CODE_CTRL_BULK_INTR_CMD_INTR
```

Value:

0x00

PROTOCOL CODES.

Note(s) : (1) See 'USB Mass Storage Class Specification Overview', Revision 1.2, Section 3

Definition at line 85 of file protocol/usb/inc/sl_usbd_class_msc.h

SL_USBD_MSC_PROTOCOL_CODE_CTRL_BULK_INTR

#define SL_USBD_MSC_PROTOCOL_CODE_CTRL_BULK_INTR

Value:

0x01

Definition at line 86 of file protocol/usb/inc/sl_usbd_class_msc.h

SL_USBD_MSC_PROTOCOL_CODE_BULK_ONLY

#define SL_USBD_MSC_PROTOCOL_CODE_BULK_ONLY

Value:

0x50

Definition at line 87 of file protocol/usb/inc/sl_usbd_class_msc.h

sl_usbd_msc_subclass_driver_t

DATA TYPES.

MSC subclass driver

Public Attributes

- sl_status_t(* enable
- sl_status_t(* disable
- sl_status_t(* process_command
- sl_status_t(* read_data
- sl_status_t(* write_data

Public Attribute Documentation

enable

```
sl_status_t(* sl_usbd_msc_subclass_driver_t::enable) (uint8_t class_nbr, void *p_subclass_arg)
```

Definition at line 97 of file protocol/usb/inc/sl_usbd_class_msc.h

disable

```
sl_status_t(* sl_usbd_msc_subclass_driver_t::disable) (uint8_t class_nbr, void *p_subclass_arg)
```

Definition at line 100 of file protocol/usb/inc/sl_usbd_class_msc.h

process_command

```
sl_status_t(* sl_usbd_msc_subclass_driver_t::process_command) (uint8_t class_nbr, void *p_lun_data, const uint8_t *p_cbwcb, uint8_t *p_data_buf, uint32_t *p_resp_len, uint8_t *p_data_dir, void *p_subclass_arg)
```

Definition at line 103 of file protocol/usb/inc/sl_usbd_class_msc.h

read_data

```
sl_status_t(* sl_usbd_msc_subclass_driver_t::read_data) (uint8_t class_nbr, void *p_lun_data, uint8_t command, uint8_t *p_data_buf, uint32_t data_len, void *p_subclass_arg)
```

Definition at line 111 of file protocol/usb/inc/sl_usbd_class_msc.h

write_data

```
sl_status_t(* sl_usbd_msc_subclass_driver_t::write_data) (uint8_t class_nbr, void *p_lun_data, uint8_t command, void *p_data_buf, uint32_t data_len, void *p_subclass_arg)
```

Definition at line 118 of file `protocol/usb/inc/slUsbd_class_msc.h`

USB Device MSC SCSI API

USB Device MSC SCSI API

USB Device MSC SCSI API.

Modules

[sl_usbd_msc_scsi_callbacks_t](#)

[sl_usbd_msc_scsi_lun_api](#)

[sl_usbd_msc_scsi_lun_info](#)

[sl_usbd_msc_scsi_lun](#)

Typedefs

typedef uint32_t [sl_usbd_msc_scsi_lb_qty_t](#)
MODULE.

typedef uint32_t [sl_usbd_msc_scsi_lba_t](#)

Functions

sl_status_t [sl_usbd_msc_scsi_init](#)(void)
FUNCTION PROTOTYPES.

sl_status_t [sl_usbd_msc_scsi_create_instance](#)(uint32_t msc_task_stack_size, osPriority_t msc_task_priority, sl_usbd_msc_scsi_callbacks_t *p_scsi_callbacks, uint8_t *p_class_nbr)
Add a new instance of SCSI Mass Storage Class.

sl_status_t [sl_usbd_msc_scsi_add_to_configuration](#)(uint8_t class_nbr, uint8_t config_nbr)
Add an existing MSC SCSI instance to the specified configuration.

sl_status_t [sl_usbd_msc_scsi_lun_add](#)(uint8_t class_nbr, sl_usbd_msc_scsi_lun_info_t *p_lu_info, sl_usbd_msc_scsi_lun_t **p_scsi_lun_ptr)
Initialize the specified logical unit.

sl_status_t [sl_usbd_msc_scsi_lun_attach](#)(sl_usbd_msc_scsi_lun_t *p_scsi_lun)
Attach a storage medium to the given SCSI logical unit.

sl_status_t [sl_usbd_msc_scsi_lun_detach](#)(sl_usbd_msc_scsi_lun_t *p_scsi_lun)
Detach a storage medium from the given SCSI logical unit.

sl_status_t [sl_usbd_msc_scsi_lun_get_capacity](#)(sl_usbd_msc_scsi_lun_t *p_scsi_lun, sl_usbd_msc_scsi_lb_qty_t *p_lb_qty, uint32_t *p_blk_size)
Retrieve capacity of logical unit.

sl_status_t [sl_usbd_msc_scsi_is_enabled](#)(uint8_t class_nbr, bool *p_enabled)
Get the MSC enable state of the SCSI interface.

Typedef Documentation

sl_usbd_msc_scsi_lb_qty_t

```
typedef uint32_t sl_usbd_msc_scsi_lb_qty_t
```

MODULE.
INCLUDE FILES DATA TYPES SCSI LBA Types

Definition at line 67 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

sl_usbd_msc_scsi_lba_t

```
typedef uint32_t sl_usbd_msc_scsi_lba_t
```

Definition at line 68 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

Function Documentation

sl_usbd_msc_scsi_init

```
sl_status_t sl_usbd_msc_scsi_init (void)
```

FUNCTION PROTOTYPES.

Parameters

N/A		
-----	--	--

MSC FUNCTIONS

Initialize internal tables, variables and the storage layer.

- Returns
- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 161 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

sl_usbd_msc_scsi_create_instance

```
sl_status_t sl_usbd_msc_scsi_create_instance (uint32_t msc_task_stack_size, osPriority_t msc_task_priority, sl_usbd_msc_scsi_callbacks_t *p_scsi_callbacks, uint8_t *p_class_nbr)
```

Add a new instance of SCSI Mass Storage Class.

Parameters

N/A	msc_task_stack_size	The USB stack creates a task to handle the events of each MSC instance. This parameter specifies the stack size to be used for the newly created task for this instance.
N/A	msc_task_priority	The USB stack creates a task to handle the events of each MSC instance. This parameter specifies the CMSIS priority of that task.
N/A	p_scsi_callbacks	Pointer to MSC SCSI callback structure. [Content MUST be persistent]

N/A	p_class_nbr	Pointer to a variable that will receive the instance number for the created class instance, if no errors are returned. Otherwise, the variable is set to SL_USBD_CLASS_NBR_NONE.
-----	-------------	--

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 182 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

sl_usbd_msc_scsi_add_to_configuration

```
sl_status_t sl_usbd_msc_scsi_add_to_configuration (uint8_t class_nbr, uint8_t config_nbr)
```

Add an existing MSC SCSI instance to the specified configuration.

Parameters

N/A	class_nbr	MSC instance number.
N/A	config_nbr	Configuration index to which to add the existing MSC interface.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 196 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

sl_usbd_msc_scsi_lun_add

```
sl_status_t sl_usbd_msc_scsi_lun_add (uint8_t class_nbr, sl_usbd_msc_scsi_lun_info_t *p_lu_info, sl_usbd_msc_scsi_lun_t **p_scsi_lun_ptr)
```

Initialize the specified logical unit.

Parameters

N/A	class_nbr	MSC SCSI instance number.
N/A	p_lu_info	Pointer to logical unit information structure.
N/A	p_scsi_lun_ptr	Pointer to a void pointer that will be set to point to SCSI-related logical unit data.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 210 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

sl_usbd_msc_scsi_lun_attach

```
sl_status_t sl_usbd_msc_scsi_lun_attach (sl_usbd_msc_scsi_lun_t *p_scsi_lun)
```

Attach a storage medium to the given SCSI logical unit.

Parameters

N/A	p_scsi_lun	Pointer to Logical Unit structure returned by sl_usbd_msc_scsi_lun_add() .
-----	------------	--

Returns

Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) If the logical unit has been ejected from the host, calling `sl_usbd_msc_scsi_lun_detach()` and `sl_usbd_msc_scsi_lun_attach()` will make it re-appear.
- (2) Use this function with care as ejecting a logical unit in the middle of a transfer may corrupt the file system. The only scenarios where the usage of this function is safe are:
 - (a) Host: Read, Embedded app: Read
 - (b) Host: Read, Embedded app: Write The following scenarios are not considered safe. Use at your own risk.
 - (a) Host: Write, Embedded app: Read
 - (b) Host: Write, Embedded app: Write

Definition at line 233 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

sl_usbd_msc_scsi_lun_detach

```
sl_status_t sl_usbd_msc_scsi_lun_detach (sl_usbd_msc_scsi_lun_t *p_scsi_lun)
```

Detach a storage medium from the given SCSI logical unit.

Parameters

N/A	p_scsi_lun	Pointer to Logical Unit structure returned by sl_usbd_msc_scsi_lun_add() .
-----	------------	--

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) After a call to this function, the media will be available to the embedded application. The standard file API can be used.
- (2) Use this function with care as ejecting a logical unit in the middle of a transfer may corrupt the file system. The only scenarios where the usage of this function is considered safe are:
 - (a) Host: Read, Embedded app: Read
 - (b) Host: Read, Embedded app: Write The following scenarios are not considered safe. Use at your own risk.
 - (a) Host: Write, Embedded app: Read
 - (b) Host: Write, Embedded app: Write

Definition at line 254 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

sl_usbd_msc_scsi_lun_get_capacity

```
sl_status_t sl_usbd_msc_scsi_lun_get_capacity (sl_usbd_msc_scsi_lun_t *p_scsi_lun, sl_usbd_msc_scsi_lb_qty_t *p_lb_qty, uint32_t *p_blk_size)
```

Retrieve capacity of logical unit.

Parameters

N/A	p_scsi_lun	Pointer to Logical Unit structure returned by sl_usbd_msc_scsi_lun_add() .
N/A	p_lb_qty	Pointer to a variable that will receive LB quantity.
N/A	p_blk_size	Pointer to a variable that will receive block size.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 267 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

sl_usbd_msc_scsi_is_enabled

```
sl_status_t sl_usbd_msc_scsi_is_enabled (uint8_t class_nbr, bool *p_enabled)
```

Get the MSC enable state of the SCSI interface.

Parameters

N/A	class_nbr	MSC SCSI instance number.
N/A	p_enabled	Pointer to a boolean variable that will receive enable state. The variable is set to true if MSC class is enabled. The variable is set to false if the MSC class is not enabled.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 282 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

sl_usbd_msc_scsi_callbacks_t

SCSI Callback Functions.

Public Attributes

void(*) [enable](#)

void(*) [disable](#)

void(*) [host_eject](#)

Public Attribute Documentation

enable

```
void(*) sl_usbd_msc_scsi_callbacks_t::enable) (uint8_t class_nbr)
```

Definition at line 73 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

disable

```
void(*) sl_usbd_msc_scsi_callbacks_t::disable) (uint8_t class_nbr)
```

Definition at line 75 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

host_eject

```
void(*) sl_usbd_msc_scsi_callbacks_t::host_eject) (uint8_t class_nbr, uint8_t lu_nbr)
```

Definition at line 77 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

sl_usbd_msc_scsi_lun_api

SCSI Logical Unit Driver API.

Public Attributes

sl_status_t(*)	init	Initialize storage driver.
sl_status_t(*)	open	Request exclusive access to media.
sl_status_t(*)	close	Release exclusive access to media.
sl_status_t(*)	get_capacity	Retrieve storage media capacity.
sl_status_t(*)	read	Read data from storage media.
sl_status_t(*)	write	Write data to storage media.

Public Attribute Documentation

init

```
sl_status_t(*) sl_usbd_msc_scsi_lun_api_t::init (sl_usbd_msc_scsi_lun_t *p_scsi_lun)
```

Initialize storage driver.

Definition at line 88 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

open

```
sl_status_t(*) sl_usbd_msc_scsi_lun_api_t::open (sl_usbd_msc_scsi_lun_t *p_scsi_lun)
```

Request exclusive access to media.

Definition at line 90 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

close

```
sl_status_t(*) sl_usbd_msc_scsi_lun_api_t::close (sl_usbd_msc_scsi_lun_t *p_scsi_lun)
```

Release exclusive access to media.

Definition at line 92 of file protocol/usb/inc/sl_usbd_class_msc_scsi.h

get_capacity

```
sl_status_t(* sl_usbd_msc_scsi_lun_api_t::get_capacity) (sl_usbd_msc_scsi_lun_t *p_scsi_lun, sl_usbd_msc_scsi_lb_qty_t *p_nbr_blks, uint32_t *p_blk_size)
```

Retrieve storage media capacity.

Definition at line 94 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

read

```
sl_status_t(* sl_usbd_msc_scsi_lun_api_t::read) (sl_usbd_msc_scsi_lun_t *p_scsi_lun, sl_usbd_msc_scsi_lba_t blk_addr, uint32_t nbr_blks, uint8_t *p_data_buf)
```

Read data from storage media.

Definition at line 98 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

write

```
sl_status_t(* sl_usbd_msc_scsi_lun_api_t::write) (sl_usbd_msc_scsi_lun_t *p_scsi_lun, sl_usbd_msc_scsi_lba_t blk_addr, uint32_t nbr_blks, uint8_t *p_data_buf)
```

Write data to storage media.

Definition at line 103 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

sl_usbd_msc_scsi_lun_info

SCSI Logical Unit Info.

Public Attributes

sl_usbd_msc_scsi_lun_api_t *	scsi_lun_api_ptr
const char *	vendor_id_ptr
const char *	product_id_ptr
uint32_t	product_revision_level
bool	is_read_only

Public Attribute Documentation

scsi_lun_api_ptr

```
sl_usbd_msc_scsi_lun_api_t* sl_usbd_msc_scsi_lun_info_t::scsi_lun_api_ptr
```

Definition at line 111 of file `protocol/usb/inc/slUsbd_class_msc_scsi.h`

vendor_id_ptr

```
const char* sl_usbd_msc_scsi_lun_info_t::vendor_id_ptr
```

Definition at line 112 of file `protocol/usb/inc/slUsbd_class_msc_scsi.h`

product_id_ptr

```
const char* sl_usbd_msc_scsi_lun_info_t::product_id_ptr
```

Definition at line 113 of file `protocol/usb/inc/slUsbd_class_msc_scsi.h`

product_revision_level

```
uint32_t sl_usbd_msc_scsi_lun_info_t::product_revision_level
```

Definition at line 114 of file `protocol/usb/inc/slUsbd_class_msc_scsi.h`

is_read_only


```
bool sl_usbd_msc_scsi_lun_info_t::is_read_only
```

Definition at line 115 of file `protocol/usb/inc/slUsbd_class_msc_scsi.h`

sl_usbd_msc_scsi_lun

SCSI Logical Unit.

Public Attributes

uint8_t	lun_index	Index of this logical unit in the global SCSI lun table.
const char *	vendor_id_str	Dev vendor info.
const char *	product_id_str	Dev prod ID.
uint32_t	product_revision_level	Revision level of product.
bool	is_read_only	write protected or not.
bool	is_removable	Flag that indicates if logical unit is removeable.
uint8_t	class_nbr	The class number of the MSC instance.
uint8_t	lu_nbr	Logical unit nbr inside the class instance.
sl_usbd_msc_scsi_lun_api_t *	scsi_lun_api_ptr	Pointer to logical unit's driver API.
void *	data_ptr	Pointer to storage driver's data.
sl_usbd_msc_scsi_lb_qty_t	blocks_nbr	Quantity of blocks.
uint32_t	block_size	Size of blocks.
sl_usbd_msc_scsi_lba_t	lb_addr	Logical Blk Addr.
uint32_t	lb_count	Nbr of mem blks.
uint8_t	sense_key	Sense key describing an err or exception cond.
uint8_t	asc	Additional Sense Code describing sense key in detail.
bool	opened_flag	Flag indicating logical unit locked or not.

bool [present_flag](#)
Flag indicating logical unit is present or not.

Public Attribute Documentation

lun_index

```
uint8_t sl_usbd_msc_scsi_lun_t::lun_index
```

Index of this logical unit in the global SCSI lun table.

Definition at line 120 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

vendor_id_str

```
const char* sl_usbd_msc_scsi_lun_t::vendor_id_str
```

Dev vendor info.

Definition at line 122 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

product_id_str

```
const char* sl_usbd_msc_scsi_lun_t::product_id_str
```

Dev prod ID.

Definition at line 123 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

product_revision_level

```
uint32_t sl_usbd_msc_scsi_lun_t::product_revision_level
```

Revision level of product.

Definition at line 124 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

is_read_only

```
bool sl_usbd_msc_scsi_lun_t::is_read_only
```

write protected or not.

Definition at line 125 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

is_removable

```
bool sl_usbd_msc_scsi_lun_t::is_removable
```

Flag that indicates if logical unit is removeable.

Definition at line 126 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

class_nbr

```
uint8_t sl_usbd_msc_scsi_lun_t::class_nbr
```

The class number of the MSC instance.

Definition at line 128 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

lu_nbr

```
uint8_t sl_usbd_msc_scsi_lun_t::lu_nbr
```

Logical unit nbr inside the class instance.

Definition at line 129 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

scsi_lun_api_ptr

```
sl_usbd_msc_scsi_lun_api_t* sl_usbd_msc_scsi_lun_t::scsi_lun_api_ptr
```

Pointer to logical unit's driver API.

Definition at line 131 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

data_ptr

```
void* sl_usbd_msc_scsi_lun_t::data_ptr
```

Pointer to storage driver's data.

Definition at line 132 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

blocks_nbr

```
sl_usbd_msc_scsi_lb_qty_t sl_usbd_msc_scsi_lun_t::blocks_nbr
```

Quantity of blocks.

Definition at line 134 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

block_size

```
uint32_t sl_usbd_msc_scsi_lun_t::block_size
```

Size of blocks.

Definition at line 135 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

lb_addr

```
sl_usbd_msc_scsi_lba_t sl_usbd_msc_scsi_lun_t::lb_addr
```

Logical Blk Addr.

Definition at line 137 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

lb_count

```
uint32_t sl_usbd_msc_scsi_lun_t::lb_count
```

Nbr of mem blks.

Definition at line 138 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

sense_key

```
uint8_t sl_usbd_msc_scsi_lun_t::sense_key
```

Sense key describing an err or exception cond.

Definition at line 139 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

asc

```
uint8_t sl_usbd_msc_scsi_lun_t::asc
```

Additional Sense Code describing sense key in detail.

Definition at line 140 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

opened_flag

```
bool sl_usbd_msc_scsi_lun_t::opened_flag
```

Flag indicating logical unit locked or not.

Definition at line 142 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

present_flag

```
bool sl_usbd_msc_scsi_lun_t::present_flag
```

Flag indicating logical unit is present or not.

Definition at line 143 of file `protocol/usb/inc/sl_usbd_class_msc_scsi.h`

USB Device Vendor API

USB Device Vendor API

USB Device Vendor API.

Modules

[sl_usbd_vendor_callbacks_t](#)

Typedefs

```
typedef void(* sl\_usbd\_vendor\_async\_function\_t)(uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void
*p_callback_arg, sl_status_t status)
App callback used for async comm.
```

Functions

sl_status_t	sl_usbd_vendor_init (void) FUNCTION PROTOTYPES.
sl_status_t	sl_usbd_vendor_create_instance (bool intr_en, uint16_t interval, sl_usbd_vendor_callbacks_t *p_vendor_callbacks, uint8_t *p_class_nbr) Add a new instance of the Vendor class.
sl_status_t	sl_usbd_vendor_add_to_configuration (uint8_t class_nbr, uint8_t config_nbr) Add the Vendor class instance into the specified configuration (see Note #1).
sl_status_t	sl_usbd_vendor_is_enabled (uint8_t class_nbr, bool *p_enabled) Get the vendor class enable state.
sl_status_t	sl_usbd_vendor_abort_read_bulk (uint8_t class_nbr) Add a Microsoft OS extended property to this vendor class instance.
sl_status_t	sl_usbd_vendor_abort_write_bulk (uint8_t class_nbr) Abort send the data to the host through the Bulk IN endpoint.
sl_status_t	sl_usbd_vendor_abort_read_interrupt (uint8_t class_nbr) Abort receive the data from the host through the Interrupt OUT endpoint.
sl_status_t	sl_usbd_vendor_abort_write_interrupt (uint8_t class_nbr) Abort send the data to the host through the Interrupt IN endpoint.
sl_status_t	sl_usbd_vendor_read_bulk_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len) Receive the data from the host through the Bulk OUT endpoint.
sl_status_t	sl_usbd_vendor_write_bulk_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, bool end, uint32_t *p_xfer_len) Send data to the host through Bulk IN endpoint.
sl_status_t	sl_usbd_vendor_read_bulk_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len, sl_usbd_vendor_async_function_t async_fnct, void *p_async_arg) Receive data from the host through the Bulk OUT endpoint.

sl_status_t	sl_usbd_vendor_write_bulk_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len, sl_usbd_vendor_async_function_t async_fnct, void *p_async_arg, bool end) Send data to the host through the Bulk IN endpoint.
sl_status_t	sl_usbd_vendor_read_interrupt_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len) Receive data from the the host through the Interrupt OUT endpoint.
sl_status_t	sl_usbd_vendor_write_interrupt_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, bool end, uint32_t *p_xfer_len) Send data to the host through the Interrupt IN endpoint.
sl_status_t	sl_usbd_vendor_read_interrupt_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len, sl_usbd_vendor_async_function_t async_fnct, void *p_async_arg) Receive data from the host through Interrupt OUT endpoint.
sl_status_t	sl_usbd_vendor_write_interrupt_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len, sl_usbd_vendor_async_function_t async_fnct, void *p_async_arg, bool end) Send data to the host through the Interrupt IN endpoint.

Typedef Documentation

sl_usbd_vendor_async_function_t

```
typedef void(* sl_usbd_vendor_async_function_t) (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void *p_callback_arg, sl_status_t status) (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint32_t xfer_len, void *p_callback_arg, sl_status_t status)
```

App callback used for async comm.

Definition at line 63 of file `protocol/usb/inc/slUsbd_class_vendor.h`

Function Documentation

sl_usbd_vendor_init

```
sl_status_t sl_usbd_vendor_init (void)
```

FUNCTION PROTOTYPES.

Parameters

N/A		
-----	--	--

VENDOR FUNCTIONS

Initialize the internal structures and variables used by the Vendor class.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 89 of file `protocol/usb/inc/slUsbd_class_vendor.h`

sl_usbd_vendor_create_instance

```
sl_status_t sl_usbd_vendor_create_instance (bool intr_en, uint16_t interval, sl_usbd_vendor_callbacks_t *p_vendor_callbacks, uint8_t *p_class_nbr)
```


Add a new instance of the Vendor class.

Parameters

N/A	intr_en	Interrupt endpoints IN and OUT flag: • true Pair of interrupt endpoints added to interface. • false Pair of interrupt endpoints not added to interface.
N/A	interval	Endpoint interval in milliseconds (must be a power of 2).
N/A	p_vendor_callbacks	Pointer to vendor callback structure. [Content MUST be persistent]
N/A	p_class_nbr	Pointer to a variable that will receive class instance number, if no errors are returned, or SL_USBD_CLASS_NBR_NONE otherwise.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 108 of file `protocol/usb/inc/sl_usbd_class_vendor.h`

sl_usbd_vendor_add_to_configuration

```
sl_status_t sl_usbd_vendor_add_to_configuration (uint8_t class_nbr, uint8_t config_nbr)
```

Add the Vendor class instance into the specified configuration (see Note #1).

Parameters

N/A	class_nbr	Class instance number.
N/A	config_nbr	Configuration index to which to add the Vendor class instance.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) Called several times, it creates multiple instances and configurations. For instance, the following architecture could be created :

```
*      FS
*      |-- Configuration 0
*      |   |-- Interface 0 (Vendor 0)
*      |-- Configuration 1
*      |   |-- Interface 0 (Vendor 0)
*      |   |-- Interface 1 (Vendor 1)
*
*
*
```

In that example, there are two instances of Vendor class: 'Vendor 0' and '1', and two possible configurations: 'Configuration 0' and '1'. 'Configuration 1' is composed of two interfaces. Each class instance has an association with one of the interfaces. If 'Configuration 1' is activated by the host, it allows the host to access two different functionalities offered by the device.

- (2) Configuration Descriptor corresponding to a Vendor-specific device has the following format :

```
*      Configuration Descriptor
*      |-- Interface Descriptor (Vendor class)
*      |   |-- Endpoint Descriptor (Bulk OUT)
*      |   |-- Endpoint Descriptor (Bulk IN)
*      |   |-- Endpoint Descriptor (Interrupt OUT) - optional
*      |   |-- Endpoint Descriptor (Interrupt IN) - optional
*
*
```

Definition at line 149 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_is_enabled

```
sl_status_t sl_usbd_vendor_is_enabled (uint8_t class_nbr, bool *p_enabled)
```

Get the vendor class enable state.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_enabled	Pointer to a variable that will receive the enable state. The variable is set to true, if the Vendor class is enabled, and is set to false if the Vendor class is NOT enabled.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 163 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_abort_read_bulk

```
sl_status_t sl_usbd_vendor_abort_read_bulk (uint8_t class_nbr)
```

Add a Microsoft OS extended property to this vendor class instance.

Parameters

N/A	class_nbr	Class instance number.
N/A	property_type	Property type (see Note #2). • OS_PROPERTY_TYPE_REG_SZ • OS_PROPERTY_TYPE_REG_EXPAND_SZ • OS_PROPERTY_TYPE_REG_BINARY • OS_PROPERTY_TYPE_REG_DWORD_LITTLE_ENDIAN • OS_PROPERTY_TYPE_REG_DWORD_BIG_ENDIAN • OS_PROPERTY_TYPE_REG_LINK • OS_PROPERTY_TYPE_REG_MULTI_SZ
N/A	p_property_name	Pointer to the buffer that contains the property name. — Buffer assumed to be persistent —
N/A	property_name_len	Length of the property name in octets.
N/A	p_property	Pointer to the buffer that contains the property name. — Buffer assumed to be persistent —
N/A	property_len	Length of the property in octets.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) For more information on Microsoft OS descriptors, see '<http://msdn.microsoft.com/en-us/library/windows/hardware/gg463179.aspx>'.
- (2) For more information on property types, refer to "Table 3. Property Data Types" of "Extended Properties OS Feature Descriptor Specification" document provided by Microsoft available at '<http://msdn.microsoft.com/en-us/library/windows/hardware/gg463179.aspx>'.

Abort receive the data from the host through the Bulk OUT endpoint.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 216 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_abort_write_bulk

```
sl_status_t sl_usbd_vendor_abort_write_bulk (uint8_t class_nbr)
```

Abort send the data to the host through the Bulk IN endpoint.

Parameters

N/A	class_nbr	Class instance number.
-----	-----------	------------------------

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 225 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_abort_read_interrupt

```
sl_status_t sl_usbd_vendor_abort_read_interrupt (uint8_t class_nbr)
```

Abort receive the data from the host through the Interrupt OUT endpoint.

Parameters

N/A	class_nbr	Class instance number.
-----	-----------	------------------------

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 234 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_abort_write_interrupt

```
sl_status_t sl_usbd_vendor_abort_write_interrupt (uint8_t class_nbr)
```

Abort send the data to the host through the Interrupt IN endpoint.

Parameters

N/A	class_nbr	Class instance number.
-----	-----------	------------------------

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 243 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_read_bulk_sync

```
sl_status_t sl_usbd_vendor_read_bulk_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len)
```

Receive the data from the host through the Bulk OUT endpoint.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_buf	Pointer to the receive buffer.
N/A	buf_len	Receive the buffer length in octets.
N/A	timeout	Timeout in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will receive transfer length. The variable is set to number of octets received, if no errors are returned, or is set to 0 if any errors are returned.

This function is blocking.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 262 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_write_bulk_sync

```
sl_status_t sl_usbd_vendor_write_bulk_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, bool end,
uint32_t *p_xfer_len)
```

Send data to the host through Bulk IN endpoint.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_buf	Pointer to the transmit buffer.
N/A	buf_len	Transmit the buffer length in octets.
N/A	timeout	Timeout in milliseconds.
N/A	end	End-of-transfer flag (see Note #1).
N/A	p_xfer_len	Pointer to a variable that will receive transfer length. The variable is set to number of octets received, if no errors are returned, or is set to 0 if any errors are returned.

This function is blocking.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) If the end-of-transfer is set and the transfer length is a multiple of the maximum packet size, a zero-length packet is transferred to signal the end of transfer to the host.

Definition at line 290 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_read_bulk_async

```
sl_status_t sl_usbd_vendor_read_bulk_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len,
sl_usbd_vendor_async_function_t async_fnct, void *p_async_arg)
```

Receive data from the host through the Bulk OUT endpoint.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_buf	Pointer to the receive buffer.
N/A	buf_len	Receive buffer length in octets.
N/A	async_fnct	Receive the the callback.
N/A	p_async_arg	Additional argument provided by the application for the receive callback.

This function is non-blocking and returns immediately after transfer preparation. Upon transfer completion, a callback provided by the application will be called to finalize the transfer.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 314 of file `protocol/usb/inc/sl_usbd_class_vendor.h`

sl_usbd_vendor_write_bulk_async

```
sl_status_t sl_usbd_vendor_write_bulk_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len,
sl_usbd_vendor_async_function_t async_fnct, void *p_async_arg, bool end)
```

Send data to the host through the Bulk IN endpoint.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_buf	Pointer to the transmit buffer.
N/A	buf_len	Transmit buffer length in octets.
N/A	async_fnct	Transmit the callback.
N/A	p_async_arg	Additional argument provided by the application for the transmit callback.
N/A	end	End-of-transfer flag (see Note #1).

This function is non-blocking and returns immediately after transfer preparation. Upon transfer completion, a callback provided by the application will be called to finalize the transfer.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) If the end-of-transfer is set and the transfer length is a multiple of the maximum packet size, a zero-length packet is transferred to signal the end of transfer to the host.

Definition at line 342 of file `protocol/usb/inc/sl_usbd_class_vendor.h`

sl_usbd_vendor_read_interrupt_sync

```
sl_status_t sl_usbd_vendor_read_interrupt_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, uint32_t *p_xfer_len)
```

Receive data from the the host through the Interrupt OUT endpoint.

Parameters

N/A	class_nbr	Class instance number.
-----	-----------	------------------------

N/A	p_buf	Pointer to the receive buffer.
N/A	buf_len	Receive buffer length in octets.
N/A	timeout	Timeout in milliseconds.
N/A	p_xfer_len	Pointer to a variable that will receive transfer length. The variable is set to number of octets received, if no errors are returned, or is set to 0 if any errors are returned.

This function is blocking.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 366 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_write_interrupt_sync

```
sl_status_t sl_usbd_vendor_write_interrupt_sync (uint8_t class_nbr, void *p_buf, uint32_t buf_len, uint16_t timeout, bool end,
uint32_t *p_xfer_len)
```

Send data to the host through the Interrupt IN endpoint.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_buf	Pointer to the transmit buffer.
N/A	buf_len	Transmit buffer length in octets.
N/A	timeout	Timeout in milliseconds.
N/A	end	End-of-transfer flag (see Note #1).
N/A	p_xfer_len	Pointer to a variable that will receive transfer length. The variable is set to number of octets received, if no errors are returned, or is set to 0 if any errors are returned.

This function is blocking.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) If the end-of-transfer is set and the transfer length is a multiple of the maximum packet size, a zero-length packet is transferred to signal the end of transfer to the host.

Definition at line 394 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_read_interrupt_async

```
sl_status_t sl_usbd_vendor_read_interrupt_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len,
sl_usbd_vendor_async_function_t async_fnct, void *p_async_arg)
```

Receive data from the host through Interrupt OUT endpoint.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_buf	Pointer to the receive buffer.
N/A	buf_len	Receive buffer length in octets.

N/A	async_fnct	Receive callback.
N/A	p_async_arg	Additional argument provided by application for the receive callback.

This function is non-blocking and returns immediately after transfer preparation. Upon transfer completion, a callback provided by the application will be called to finalize the transfer.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Definition at line 418 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_write_interrupt_async

```
sl_status_t sl_usbd_vendor_write_interrupt_async (uint8_t class_nbr, void *p_buf, uint32_t buf_len,
sl_usbd_vendor_async_function_t async_fnct, void *p_async_arg, bool end)
```

Send data to the host through the Interrupt IN endpoint.

Parameters

N/A	class_nbr	Class instance number.
N/A	p_buf	Pointer to the transmit buffer.
N/A	buf_len	Transmit buffer length in octets.
N/A	async_fnct	Transmit callback.
N/A	p_async_arg	Additional argument provided by the application for the transmit callback.
N/A	end	End-of-transfer flag (see Note #1).

This function is non-blocking and returns immediately after transfer preparation. Upon transfer completion, a callback provided by the application will be called to finalize the transfer.

Returns

- Returns SL_STATUS_OK on success or another SL_STATUS code on failure.

Note

- (1) If the end-of-transfer is set and the transfer length is a multiple of the maximum packet size, a zero-length packet is transferred to signal the end of transfer to the host.

Definition at line 446 of file protocol/usb/inc/sl_usbd_class_vendor.h

sl_usbd_vendor_callbacks_t

MODULE.

INCLUDE FILES DATA TYPES Vendor callbacks

Public Attributes

- void(*)

[enable](#)

Callback for enable event.
- void(*)

[disable](#)

Callback for disable event.
- void(*)

[setup_req](#)

Callback for setup req event.

Public Attribute Documentation

enable

```
void(* sl_usbd_vendor_callbacks_t::enable) (uint8_t class_nbr)
```

Callback for enable event.

Definition at line 54 of file protocol/usb/inc/sl_usbd_class_vendor.h

disable

```
void(* sl_usbd_vendor_callbacks_t::disable) (uint8_t class_nbr)
```

Callback for disable event.

Definition at line 56 of file protocol/usb/inc/sl_usbd_class_vendor.h

setup_req

```
void(* sl_usbd_vendor_callbacks_t::setup_req) (uint8_t class_nbr, const sl_usbd_setup_req_t *p_setup_req)
```

Callback for setup req event.

Definition at line 58 of file protocol/usb/inc/sl_usbd_class_vendor.h

Overview

USB Device Troubleshooting

The following are some common issues that you might encounter while developing an application using Silicon Labs USB Device.

- [My device is not detected by Windows OS.](#)
- [My device used to work properly, but now that I added/removed/modified a class instance, it behaves weirdly.](#)
- [My CDC or Vendor device is not working properly on Windows OS.](#)
- [My device has a low data transfer rate.](#)
- [My device works on Windows OS but not on Mac OS and/or Linux OS.](#)
- [My composite device \(multi-function device\) is not recognized by Windows OS.](#)
- [When I create a class instance or add a class instance to a configuration, I keep getting an error `SL\_STATUS\_ALLOCATION\_FAILED`. What do I do?](#)

My device is not detected by Windows OS

There can be many causes for this issue. The first thing to verify is that you have properly called the function `slUsbd_core_add_device()` and, if you disabled the auto-start feature `SL_USBD_AUTO_START_USB_DEVICE`, the function `slUsbd_core_start_device()`, and that both functions return with the error code `SL_STATUS_OK`. Also make sure that you have added at least one configuration to your device with one class.

My device used to work properly, but now that I added/removed/modified a class instance, it behaves weirdly

Once Windows associates one or more drivers with a given device, the association cannot be changed. So, if for instance, you connect a device with an HID class instance, and you modify it to be an MSC device and reconnect it, Windows will assume it is still an HID device. In order to force Windows to re-associate its drivers, you can either uninstall the device completely using the device manager or set a new Product ID with configuration `SL_USBD_DEVICE_PRODUCT_ID` in `sl_usbd_device_config.h`.

My CDC or Vendor device is not working properly on Windows OS

Windows OS requires that all the provided .inf files must be certified. See section [Microsoft Windows OS USB Host](#) for more details on .inf files.

My device has a low data transfer rate

There can be many causes for this issue. Here are some tips to increase the throughput:

- Transmit the largest buffer possible

Keep in mind that if your device operates at Full-Speed, you will never get better throughput than ~850 KBytes/sec.

My device works on Windows OS but not on Mac OS and/or Linux OS

The USB Device module has been successfully tested on both Linux and Mac OS. However, some classes may require certain specific configurations to operate correctly under Mac OS and/or Linux. Refer to the user manual of the class you use for more information.

Note that for our Vendor class, a Vendor host application has to be developed as well. Libraries, such as libusb, can be used to develop such an application.

My composite device (multi-function device) is not recognized by Windows OS

If at least one of your device's function requires a .inf file to load a driver (such as CDC ACM and Vendor), the .inf example file is unlikely to work as-is. You will probably have to modify it. Refer to the [About INF Files](#) section for more information.

When I create a class instance or add a class instance to a configuration, I keep getting an error `SL_STATUS_ALLOCATION_FAILED` . What do I do?

Silicon Labs USB Device allocates a pool of each type of resources at initialization. Your application must specify the quantity of resources of each type to allocate. If you get a resource allocation error, you will have to increase the quantity of the given resource to allocate. For more information, refer to [USB Device Run-Time Application-Specific Configuration](#).

Overview

Microsoft Windows OS USB Host

Microsoft offers class drivers for some standard USB classes. These drivers can also be called native drivers. A complete list of the native drivers can be found in the MSDN online documentation on the page titled [USB class drivers included in Windows](#). If a connected device belongs to a class for which a native driver exists, Windows automatically loads the driver without any additional actions from you. If a vendor-specific driver is required for the device, a manufacturer's INF file giving instructions to Windows for loading the vendor-specific driver is required. In some cases, a manufacturer's INF file may also be required to load a native driver.

When the device has been recognized by Windows and is ready for communication, your application may need to use a Globally Unique Identifier (GUID) to retrieve a device handle that allows your application to communicate with the device.

The following sections explain the use of INF files and GUIDs. The table below shows the USB classes to which the information in the following sub-sections applies.

Table - Classes Concerned by Windows USB Device Management

Section	USB Classes
About INF Files	CDC and Vendor
Using GUIDs	HID and Vendor

About INF Files

An INF file is a setup information file that contains information used by Windows to install software and drivers for one or more devices. The INF file also contains information to store in the Windows registry. Each of the drivers provided natively with the operating system has an associated INF file stored in C:\WINDOWS\inf. For instance, when an HID or MSC device is connected to the PC, Windows enumerates the device and implicitly finds an INF file associated to an HID or MSC class that permits loading the proper driver. INF files for native drivers are called system INF files. Any new INF files provided by manufacturers for vendor-specific devices are copied into the folder C:\WINDOWS\inf. These INF files can be called vendor-specific INF files. An INF file allows Windows to load one or more drivers for a device. A driver can be native or provided by the device manufacturer. The table below shows the Windows driver(s) loaded for each USB class

Table - Windows Drivers Loaded for each USB Class

Class	Windows Driver	Driver Type	INF file type
CDC ACM	usbser.sys	Native	Vendor-specific INF file
HID	Hidclass.sys & Hidusb.sys	Native	System INF file
MSC	Usbstor.sys	Native	System INF file
Vendor	winusb.sys	Native	Vendor-specific INF file

When a device is first connected, Windows searches for a match between the information contained in system INF files and the information retrieved from device descriptors. If there is no match, Windows asks you to provide an INF file for the connected device.

An INF file is arranged in sections whose names are surrounded by square brackets []. Each section contains one or several entries. If the entry has a predefined keyword such as "Class", "Signature", etc, the entry is called a directive. [Example - INF File Structure](#) presents an example of an INF file structure.

Example - INF File Structure

```

;===== Version section =====
[Version]                                     (1)
Signature = "$Windows NT$"
Class     = Ports
ClassGuid = {4D36E978-E325-11CE-BFC1-08002BE10318}

Provider=%ProviderName%
DriverVer=01/01/2012,1.0.0.0

;===== Manufacturer/Models sections =====

[Manufacturer]                               (2)
%ProviderName% = DeviceList, NTx86, NTamd64

[DeviceList.NTx86]                           (3)
%PROVIDER_CDC% = DriverInstall, USB\VID_ffe&PID_1234&ML_00
[DeviceList.NTamd64]                         (3)
%PROVIDER_CDC% = DriverInstall, USB\VID_ffe&PID_1234&ML_00

;===== Installation sections =====      (4)

[DriverInstall]
include = mdmcpq.inf
CopyFiles = FakeModemCopyFileSection
AddReg = LowerFilterAddReg, SerialPropPageAddReg

[DriverInstall.Services]
include = mdmcpq.inf
AddService = usbser, 0x00000002, LowerFilter_Service_Inst

[SerialPropPageAddReg]
HKR,,EnumPropPages32,, "MsPorts.dll,SerialPortPropPageProvider"

;===== Strings section =====
[Strings]                                     (5)
ProviderName = "Silicon Labs"
PROVIDER_CDC = "Silicon Labs CDC Device"

```

(1) The section [Version] is mandatory and informs Windows about the provider, the version and other descriptive information about the driver package.

(2) The section [Manufacturer] is mandatory. It identifies the device's manufacturer.

(3) The following two sections are called Models sections and are defined on a per-manufacturer basis. They give more detailed instructions on the driver(s) to install for the device(s). A section name can use extensions to specify OSes and/or CPUs the entries apply to. In this example, .NTx86 and .NTamd64 indicate that the driver can be installed on an NT-based Windows (that is Windows 2000 and later), on x86- and x64-based PC respectively.

(4) The installation sections actually install the driver(s) for each device described in the Model section(s). The driver installation may involve reading existing information from the Windows registry, modifying existing entries of the registry or creating new entries into the registry.

(5) The section [Strings] is mandatory and it is used to define each string key token indicated by %string name% in the INF file.

Refer to the MSDN online documentation on [this](#) web page for more details about INF sections and directives.

You will be able to modify some sections in order to match the INF file to your device characteristics, such as Vendor ID, Product ID and human-readable strings describing the device. The sections are:

- Models section
- [Strings] section

To identify possible drivers for a device, Windows looks in the Models section for a device identification string that matches a string created from information in the device's descriptors. Every USB device has a device ID, that is a hardware ID

created by the Windows USB host stack from information contained in the Device descriptor. A device ID has the following form:

USB\Vid_xxxx&Pid_yyyy

xxxx, yyyy, represent the value of the Device descriptor fields "idVendor" and "idProduct" respectively (refer to the Universal Serial Bus Specification, revision 2.0, section 9.6.1 for more details about the Device descriptor fields). This string allows Windows to load a driver for the device. You can modify xxxx and yyyy to match your device's Vendor and Product IDs.

Composite devices, formed of several functions, can specify a driver for each function. In this case, the device has a device ID for each interface that represents a function. A device ID for an interface has the following form:

USB\Vid_xxxx&Pid_yyyy&MI_ww

ww is equal to the "bInterfaceNumber" field in the Interface descriptor (refer to the Universal Serial Bus Specification, revision 2.0, section 9.6.5 for more details on the Interface descriptor fields). You can modify ww to match the position of the interface in the Configuration descriptor. If the interface has the position #2 in the Configuration descriptor, ww is equals to 02.

The [Strings] section contains a description of your device. In [Example - INF File Structure](#), the strings define the name of the device driver package provider and the device name. You can see these device description strings in the Device Manager.

Using GUIDs

A Globally Unique Identifier (GUID) is a 128-bit value that uniquely identifies a class or other entity. Windows uses GUIDs for identifying two types of device classes:

- Device setup class
- Device interface class

A device setup GUID encompasses devices that Windows installs in the same way and using the same class installer and co-installers. Class installers and co-installers are DLLs that provide functions related to device installation. There is a GUID associated with each device setup class. System-defined setup class GUIDs are defined in devguid.h. The device setup class GUID defines the `..|CurrentControlSet|Control|Class|ClassGuid` registry key under which to create a new subkey for any particular device of a standard setup class. A complete list of system-defined device setup classes offered by Microsoft Windows® is available on [MSDN online documentation](#).

A device interface class GUID provides a mechanism for applications to communicate with a driver assigned to devices in a class. A class or device driver can register one or more device interface classes to enable applications to learn about and communicate with devices that use the driver. Each device interface class has a device interface GUID. Upon a device's first attachment to the PC, the Windows I/O manager associates the device and the device interface class GUID with a symbolic link name, also called a device path. The device path is stored in the Windows registry and persists across system reboot. An application can retrieve all the connected devices within a device interface class. If the application has gotten a device path for a connected device, this device path can be passed to a function that will return a handle. This handle is passed to other functions in order to communicate with the corresponding device.

